

Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования
«Вятский государственный гуманитарный университет»

Е. В. Котельников

Вычислительные машины, системы и сети

Учебное пособие

Киров
2012

УДК 32.973-018.2
ББК 681.3.066
К73

*Печатается по решению
редакционно-издательского совета
Вятского государственного гуманитарного университета*

Рецензенты:

С. М. Окулов, доктор педагогических наук, профессор кафедры информатики и методики обучения информатике;

В. Ю. Мельцов, кандидат технических наук, доцент кафедры электронных вычислительных машин ВятГУ

Котельников, Е. В.

К73 Вычислительные машины, системы и сети [Текст] : учебное пособие / Е. В. Котельников. – Киров: Изд-во ВятГГУ, 2012. – 218 с.

ISBN 978-5-456-00037-8

Учебное пособие посвящено принципам организации и функционирования современной компьютерной техники. Подробно рассматриваются вопросы машинного представления информации, построения различных узлов компьютера на основе логических элементов. Изложение принципов работы процессора ведется с использованием простой учебной модели. Также в пособии представлен базовый материал по периферийным устройствам, вычислительным системам и компьютерным сетям.

Пособие предназначено для студентов высших учебных заведений, обучающихся по направлениям подготовки 230700 Прикладная информатика в экономике и 080500 Бизнес-информатика.

УДК 32.973-018.2
ББК 681.3.066

ISBN 978-5-456-00037-8

© Вятский государственный
гуманитарный университет
(ВятГГУ), 2012

© Е. В. Котельников, 2012

© Чепурных А. Ю., оформление обложки,
2012

Оглавление

ПРЕДИСЛОВИЕ	6
ГЛАВА 1. ВВЕДЕНИЕ	
1.1. Основные понятия	7
1.2. Многоуровневое представление вычислительных машин	7
1.3. Эволюция вычислительной техники	10
1.4. Классификация вычислительных машин	17
1.5. Архитектура фон Неймана	18
Контрольные вопросы и задания	19
ГЛАВА 2.	
ИНФОРМАЦИОННО-ЛОГИЧЕСКИЕ ОСНОВЫ ПОСТРОЕНИЯ	
ВЫЧИСЛИТЕЛЬНЫХ МАШИН	
2.1. Системы счисления	20
2.2. Представление информации в вычислительных машинах	29
2.2.1. Ячейки памяти	29
2.2.2. Целые числа	30
2.2.3. Вещественные числа	39
2.2.4. Представление текстовой информации	48
2.2.5. Представление графической информации	51
2.2.6. Представление звуковой информации	57
2.3. Булева алгебра	59
Контрольные вопросы и задания	62
ГЛАВА 3. ЦИФРОВОЙ ЛОГИЧЕСКИЙ УРОВЕНЬ	
3.1. Аналоговые и дискретные сигналы	64
3.2. Классификация цифровых схем	66
3.3. Комбинационные схемы	67
3.3.1. Логические вентили	67
3.3.2. Реализация булевых функций	72
3.3.3. Использование вентилей AND в качестве ключа	73
3.3.4. Использование вентилей XOR	
в качестве управляемого инвертора	74
3.3.5. Преобразователи кодов	74
3.3.6. Коммутаторы	76
3.3.7. Двоичная арифметика	77
3.3.8. Арифметические схемы	83
3.3.9. Арифметико-логическое устройство	87
3.3.10. Тактовый генератор	88
3.4. Схемы с памятью	90
3.4.1. Иерархия запоминающих устройств	90

3.4.2. Триггеры	91
3.4.3. Регистры.....	95
3.4.4. Счетчики	97
3.4.5. Организация памяти	97
3.4.6. Оперативные запоминающие устройства.....	103
3.4.7. Виды оперативной динамической памяти.....	104
3.4.8. Конструктивное исполнение.....	109
3.4.9. Постоянные запоминающие устройства.....	110
3.5. Шины	111
3.5.1. Состав шины.....	112
3.5.2. Арбитраж шин	114
3.5.3. Протокол шины	114
3.6. Системные платы.....	115
<i>Контрольные вопросы и задания</i>	118

ГЛАВА 4. ОРГАНИЗАЦИЯ ВЫЧИСЛИТЕЛЬНОЙ МАШИНЫ

4.1. Модель процессора.....	120
4.1.1. Регистры.....	120
4.1.2. Арифметико-логическое устройство	123
4.1.3. Флаги	126
4.1.4. Устройство управления	127
4.1.5. Цикл команды.....	128
4.1.6. Сигналы управления	130
4.1.7. Примеры.....	132
4.1.8. Понятия «архитектура» и «микроархитектура»	137
4.2. Повышение производительности процессоров	138
4.2.1. Конвейеризация.....	138
4.2.2. Суперскалярные процессоры.....	143
4.2.3. Архитектуры CISC и RISC.....	145
4.2.4. Кэш-память	146
4.3. Организация ввода-вывода.....	149
4.3.1. Модули ввода-вывода.....	149
4.3.2. Способы взаимодействия с внешними устройствами.....	150
4.4. Пример реализации микроархитектуры процессора	152
<i>Контрольные вопросы и задания</i>	154

ГЛАВА 5. ВНЕШНИЕ ЗАПОМИНАЮЩИЕ И ПЕРИФЕРИЙНЫЕ УСТРОЙСТВА

5.1. Внешние запоминающие устройства	156
5.1.1. Классификация устройств внешней памяти.....	156
5.1.2. Магнитные диски	157
5.1.3. Оптические диски	162
5.2. Периферийные устройства	166
5.2.1. Мониторы	166

5.2.2. Принтеры	169
5.2.3. Сканеры.....	170
<i>Контрольные вопросы и задания.....</i>	<i>171</i>
 ГЛАВА 6. ВЫЧИСЛИТЕЛЬНЫЕ СИСТЕМЫ	
6.1. Виды вычислительных систем.....	173
6.2. Симметричные многопроцессорные системы (SMP)	174
6.3. Системы с неоднородным доступом к памяти (NUMA)	175
6.4. Кластерные системы	176
6.5. Системы с массовой параллельной обработкой (MPP)	177
6.6. Измерение производительности вычислительных систем	178
<i>Контрольные вопросы и задания.....</i>	<i>178</i>
 ГЛАВА 7. КОМПЬЮТЕРНЫЕ СЕТИ	
7.1. Введение в компьютерные сети.....	179
7.1.1. Основные понятия	179
7.1.2. Классификация компьютерных сетей.....	180
7.1.3. Основные характеристики сети.....	186
7.2. Модель сетевого взаимодействия	188
7.2.1. Модель OSI	188
7.2.2. Функции уровней модели OSI	191
7.3. Линии связи.....	196
7.3.1. Классификация линий связи	196
7.3.2. Коаксиальный кабель	197
7.3.3. Витая пара.....	198
7.3.4. Оптоволоконный кабель.....	200
7.4. Технология Ethernet.....	203
7.4.1. История Ethernet.....	203
7.4.2. Формат кадра	203
7.4.3. Fast Ethernet	205
7.5. Сетевые устройства	206
7.5.1. Сетевые карты	206
7.5.2. Концентраторы.....	207
7.5.3. Коммутаторы	207
7.5.4. Маршрутизаторы.....	208
7.6. стек протоколов TCP/IP	209
7.6.1. История создания TCP/IP	209
7.6.2. Структура TCP/IP	210
7.6.3. Документы RFC.....	210
7.6.4. Обзор основных протоколов.....	212
7.6.5. Утилиты диагностики TCP/IP	213
<i>Контрольные вопросы и задания.....</i>	<i>215</i>
 БИБЛИОГРАФИЧЕСКИЙ СПИСОК	217

Предисловие

В пособии рассматриваются основные принципы построения компьютерной техники – вычислительных машин и систем, а также сетей передачи данных.

В первой главе предлагаются базовые понятия, иерархическое представление вычислительной машины, краткая история вычислительной техники, классификация ЭВМ, архитектура фон Неймана.

Вторая глава посвящена представлению различной информации в памяти компьютера. Подробно обсуждаются системы счисления, способы представления в памяти целых и вещественных чисел, текстовой, графической и аудиоинформации. Также приводятся основные понятия булевой алгебры.

В третьей главе внимание сосредоточено на построении различных по сложности узлов ЭВМ из простых логических элементов – вентилях И, ИЛИ, НЕ. Также рассматриваются шины и системные платы.

В четвертой главе на примере учебной модели описываются принципы работы процессоров; предлагаются способы повышения их производительности.

В пятой главе представлены устройства внешней памяти и периферийные устройства.

Шестая глава посвящена высокопроизводительным вычислительным системам: изучаются системы с общей памятью и распределенные системы.

В седьмой главе рассматриваются компьютерные сети – классификация, модель OSI, линии связи, технология Ethernet, сетевые устройства, стек протоколов TCP/IP.

Вопросы, приведенные в конце каждой главы, разделены на две группы. Первая группа содержит вопросы на теоретическое знание материала. На них можно ответить, просто прочитав главу и выучив основные положения. Другая группа вопросов (они отмечены звездочкой) нацелена на поиск информации в дополнительных источниках и углубленное понимание материала.

ГЛАВА 1. ВВЕДЕНИЕ

1.1. Основные понятия

Вычислительная машина (сокращенно *ВМ*), или *компьютер*¹, – комплекс технических и программных средств, предназначенный для решения задач пользователя путем автоматической обработки информации.

Из определения видно, **для чего** нужна ВМ (для решения задач пользователя), **каким образом** она это делает (автоматически, то есть без участия человека, обрабатывая информацию) и **из чего** состоит (из двух основных компонентов – аппаратных средств, *hardware*, и программных средств, *software*).

Вычислительная система (ВС) – совокупность взаимодействующих процессоров или вычислительных машин, периферийного оборудования и программного обеспечения, предназначенная для решения задач пользователя.

Таким образом, понятие вычислительной машины подразумевает наличие одного вычислителя, а понятие вычислительной системы – нескольких вычислителей. Хотя в современных компьютерах такое разделение достаточно условно. Например, многоядерный процессор представляет собой несколько вычислительных ядер (независимых вычислителей), которые используют общую оперативную память. Поэтому, с одной стороны, такой процессор имеет отношение к вычислительным системам, с другой стороны, компьютер на основе такого процессора традиционно относится к классу вычислительных машин.

1.2. Многоуровневое представление вычислительных машин

Вычислительная машина является сложным объектом, поэтому для простоты изучения в ней выделяют несколько уровней, рассматриваемых отдельно (рис. 1.1).

¹ *Computer* (англ.) – вычислитель.

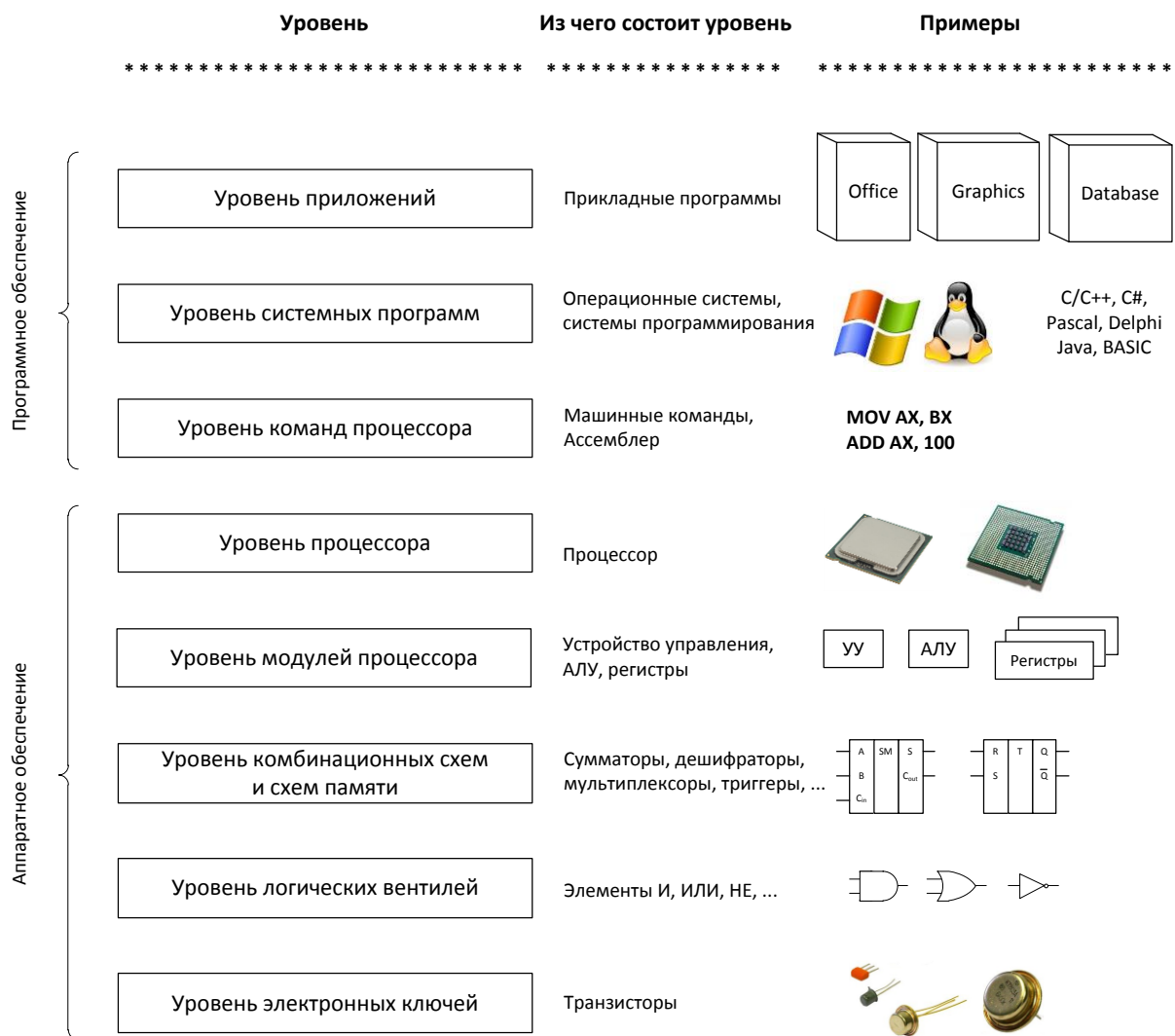


Рис. 1.1. Уровни вычислительной машины

Каждый верхний уровень основан на предыдущем нижнем, т. е. верхний уровень либо состоит из элементов нижнего уровня (это справедливо для уровней аппаратного обеспечения), либо использует предоставляемые им функции (для уровней программного обеспечения).

Рассмотрим кратко принцип построения вычислительной машины, пройдя по рис. 1.1 снизу вверх. На *уровне электронных ключей* работают транзисторы – самые простые элементы компьютера, которые замыкают или размыкают электрическую цепь (как выключатель света в комнате) в зависимости от приложенного к ним управляющего сигнала, поэтому транзисторы называют *электронными переключателями*, или *ключами*.

Такая простая функция оказывается полезной для построения из транзисторов *логических вентилях* – электронных схем, реализующих логические (булевы) операции, например И, ИЛИ, НЕ. Из логических вентилях составляются два вида более сложных схем – *комбинационных*, не умеющих запоминать прошлые сигналы, и *схемы с памятью*. Примером комбинационной схемы может служить сумматор, который выполняет арифметическое сло-

жение двух бит информации, а примером схемы памяти – триггер, который может хранить один бит.

На уровне модулей процессора схемы с памятью и комбинационные схемы служат для построения основных блоков процессора – *устройства управления, арифметико-логического устройства (АЛУ) и регистров*¹. Совокупность нескольких таких блоков образует одно из сложнейших в мире технических устройств – *процессор*.

У любого процессора есть набор команд, которые он умеет исполнять. Такой набор называется *системой команд*, а сами команды – *машинными*, или *процессорными*. Как правило, машинные команды позволяют выполнять довольно простые действия, например загрузить в регистр процессора данные из оперативной памяти; сложить содержимое двух регистров, а сумму записать в один из них; перемножить регистр и содержимое ячейки оперативной памяти. Записываются машинные команды при помощи двоичного или шестнадцатеричного кода, поэтому составлять программы непосредственно на машинном языке неудобно и был придуман специальный язык обозначений для машинных команд, который называется Ассемблер. Например, машинная команда для загрузки в регистр АХ числа 10 в шестнадцатеричном коде запишется так:

8В 00 0А

а на Ассемблере та же команда будет выглядеть следующим образом (*move* – перемещать):

MOV AX, 10

Поскольку система команд у каждого процессора своя, то и Ассемблер для каждого процессора будет особенный, хотя многие общие черты сохраняются.

Уровень системных программ отвечает за взаимодействие пользователя и прикладных программ с аппаратным обеспечением компьютера. К этому уровню относят операционные системы и системы программирования. Наконец, уровень приложений² необходим непосредственно для выполнения основной функции компьютеров – решения задач пользователя. Спектр прикладных программ на этом уровне очень велик – текстовые и графические редакторы, электронные таблицы, системы управления базами данных, системы автоматизированного проектирования, бухгалтерские программы, системы электронного документооборота и многие другие.

Таким образом, каждый вышележащий уровень строится, как из кирпичиков, из элементов нижележащего уровня. Одна из целей данного курса – за любым действием пользователя (например, движением указателя мышки

¹ *Регистр* – многоразрядная ячейка памяти.

² *Приложение* (от англ. application) – прикладная программа.

по экрану) или приложения (например, вычисление формулы в электронной таблице) суметь, хотя бы в общем, представить процесс взаимодействия между всеми уровнями вычислительной машины.

В курсе рассматриваются пять нижних уровней; операционные системы, языки программирования и прикладные программы являются объектом изучения в других дисциплинах.

1.3. Эволюция вычислительной техники

При рассмотрении истории развития вычислительной техники принято выделять несколько поколений вычислительных машин (в разных источниках от 3 до 6). В основу деления на поколения положен технологический подход – поколения меняются со сменой технологий [1, 2]. Следует заметить, что временные рамки поколений указаны довольно условно.

Нулевое поколение (до 1940-х годов)

Технология – механические вычислительные устройства на основе шестеренок – зубчатых колес.

3000 лет до н. э. – абак (Древний Вавилон).

1492 год. Леонардо да Винчи – рисунок тринадцатиразрядного десятичного суммирующего устройства на основе зубчатых колес.

1623 год. Вильгельм Шиккард – эскиз устройства для сложения и вычитания шестиразрядных чисел на основе механических часов (рис. 1.2).

1642 год. Блез Паскаль – «Паскалин» – первое реально осуществленное устройство (суммирование и вычитание 5-разрядных десятичных чисел).

1673 год. Готфрид Вильгельм Лейбниц – «пошаговый вычислитель» (четыре арифметические операции над 12-разрядными десятичными числами).

1822 год. Чарльз Бэббидж – разностная машина (вычисление конечных разностей) (рис. 1.3).

1836 год. Чарльз Бэббидж – проект «Аналитической машины», у которой был процессор («мельница»), память («склад»), ввод программ и данных с перфокарт.

1937 год. Алан Тьюринг предлагает концепцию теоретической вычислительной машины (*машина Тьюринга*).



Рис. 1.2. Механический калькулятор В. Шиккарда



Рис. 1.3. Часть разностной машины Ч. Бэббиджа

До появления первых *электронных* вычислительных машин некоторое время применялась переходная технология, основанная на *электромеханических реле*. Реле – электрический прибор, размыкающий или замыкающий цепь под действием управляющего сигнала. Отличие от транзистора в том, что в реле замыкающий контакт является движущейся механической пластиной, а в транзисторе движущиеся части отсутствуют, замыкание/размыкание происходит внутри транзистора, на уровне электронов. Наличие движущихся частей в реле является причиной следующих недостатков по сравнению с транзисторами:

- 1) малый ресурс переключений – реле быстро изнашиваются;
- 2) низкая частота переключений приводит к невысокой скорости работы компьютера.

Компьютеры на электромеханических реле:

- 1938–1945 – немецкий инженер Конрад Цузе создал серию программируемых вычислительных машин Z1-Z4. Самая известная машина серии – Z3 – состояла из 2200 реле, имела частоту 5–10 Гц;
- 1941–1944 – группа ученых Гарвардского университета во главе с Говардом Айкеном разрабатывает вычислитель на электромеханических реле Mark I. Длина устройства – 18 м, вес – 5 т. Вычислитель обрабатывал 23-разрядные числа, при этом сложение занимало 0,3 с, умножение – 4 с, а деление – 10 с.

Первое поколение (1940–1955)

Базовая технология – электронно-вакуумные лампы, представляющие собой стеклянные колбы, из которых откачивался воздух (создавался вакуум). К колбе присоединялись два электрода (анод и катод), а также управляющий электрод (сетка).

Движущиеся части отсутствовали, поэтому скорость переключений была гораздо выше, чем у реле, однако лампы часто выходили из строя и требовали большого количества электроэнергии.

Компьютеры, построенные на электронно-вакуумных лампах, стали называться *электронные вычислительные машины* (ЭВМ).

Первые ЭВМ:

- ABC (Atanasoff-Berry Computer), 1942 год, Джон Атанасов и Клиффорд Берри (США);
- COLOSSUS, 1943 год, Макс Ньюмен, Томми Флауэрс, Алан Тьюринг (Великобритания) (рис. 1.4);
- ENIAC (Electronic Numerical Integrator and Computer), 1946 год, Джон Мочли, Преспер Эккерт, Джон фон Нейман (США, 30 тонн, 140 кВт, 18000 электронных ламп, 20 десятиразрядных регистров) (рис. 1.5).

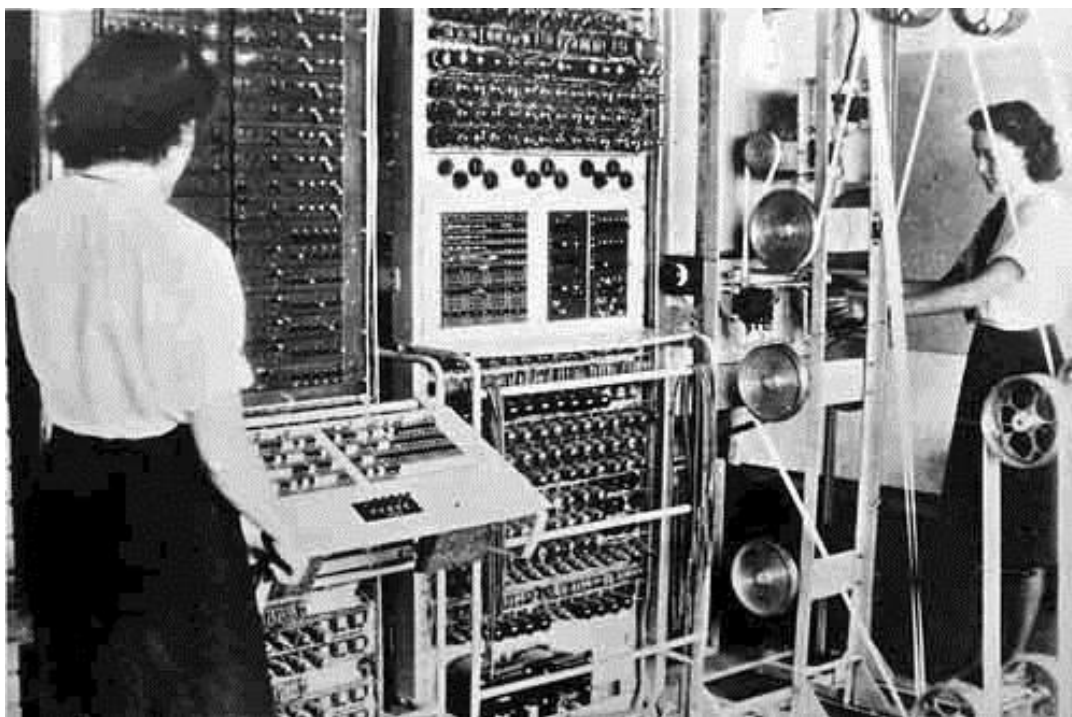


Рис. 1.4. Colossus

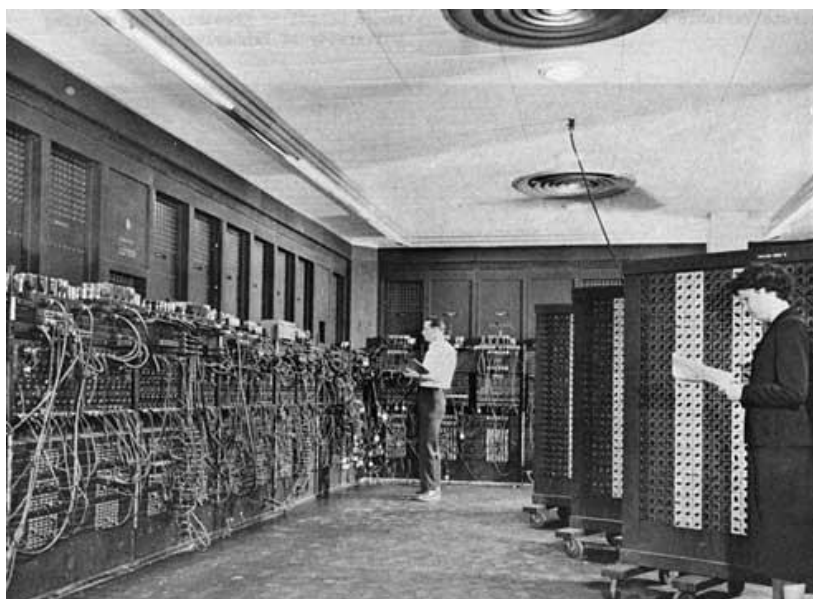


Рис. 1.5. ENIAC

1951 год – первая советская ЭВМ – МЭСМ (Малая Электронная Счетная Машина), Сергей Алексеевич Лебедев.

1953 год – БЭСМ (Большая Электронная Счетная Машина, 10000 опер./с), С. А. Лебедев (рис. 1.6).



Рис. 1.6. БЭСМ-1

Программирование ВМ первого поколения осуществлялось на машинном языке, только в 50-е годы появляется Ассемблер.

Второе поколение (1955–1965)

Технология второго поколения – полупроводниковые транзисторы (Джон Бардин, Уолтер Браттейн, Уильям Шокли в 1956 году получили нобелевскую премию за создание транзистора). С появлением транзисторов на несколько порядков возросли по сравнению с электронно-вакуумными лампами быстродействие и надежность, существенно уменьшились энергопотребление и габариты ВМ.

Для удобства программирования были разработаны первые языки высокого уровня – Фортран (1956), Алгол (1958), Кобол (1959).

Первые транзисторные ЭВМ – TX-0 (Transistorized eXperimental computer), TRADIC (TRAnsistor Digital Computer).

Наибольшее распространение получили два семейства машин:

- IBM 7090, IBM 7094, IBM 1401 (большие ЭВМ, стоимость – несколько миллионов долларов);
- PDP-1 (120 тысяч долларов), PDP-8 (16 тысяч долларов) – малые ЭВМ, компания DEC.

Третье поколение (1965–1980)

Технология третьего поколения – интегральные микросхемы (изобрели Роберт Нойс и Джек Килби независимо друг от друга в 1958 году). *Интегральная микросхема* (chip) – электронное устройство, все элементы которого нераздельно связаны конструктивно. До появления интегральных схем у

каждого элемента был свой корпус; для построения сложного устройства все элементы требовалось разместить на плате и провести между ними проводники. При этом из-за достаточно большого размера элементов не получалось поместить на одной плате требуемое их количество. В интегральных схемах размер элементов можно было уменьшать и одновременно увеличивать плотность размещения. Вследствие миниатюризации увеличивалась частота работы, уменьшались энергопотребление и тепловыделение.

Таким образом, компьютеры на интегральных микросхемах были меньшего размера, работали быстрее и стоили дешевле, чем их предшественники на транзисторах.

Самые популярные ЭВМ третьего поколения:

- семейство больших компьютеров IBM-360 (рис. 1.7);

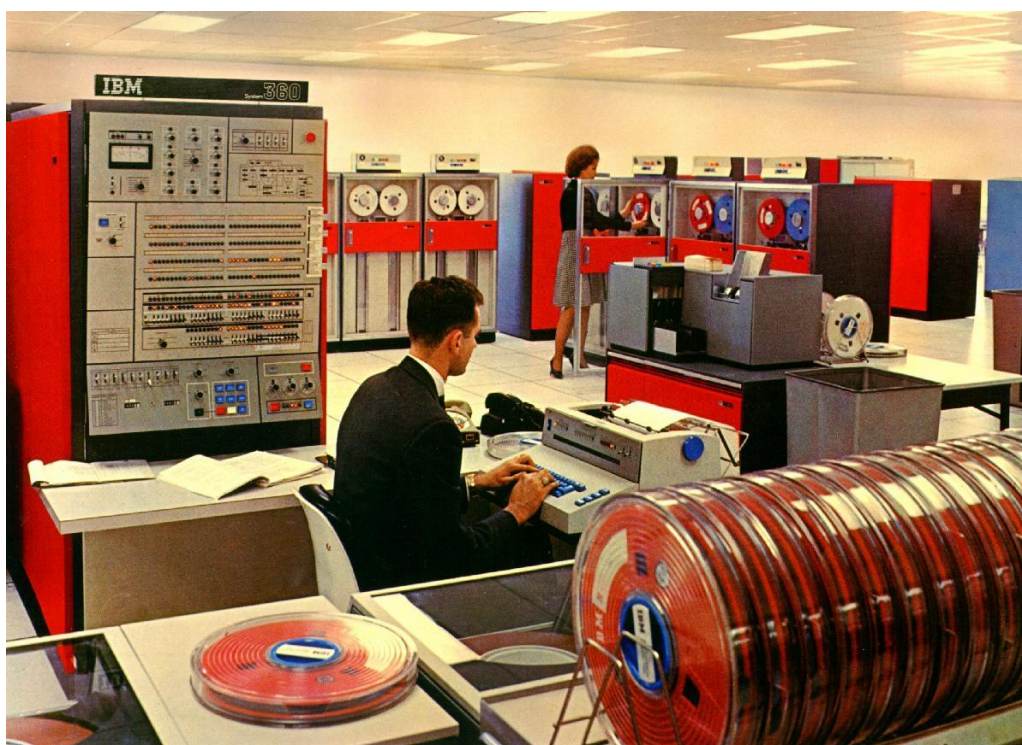


Рис. 1.7. IBM-360

- малая ЭВМ PDP-11;
- первые суперкомпьютеры: CDC-6600 (1964), CDC-7600 (1969), Cray-1 (Сеймур Крей).

Создатель первых суперкомпьютеров Сеймур Крей разработал также алгоритм покупки автомобиля: «Вы идете в магазин, ближайший к вашему дому, показываете на машину, ближайшую к двери, и говорите: «Я беру эту». Этот алгоритм позволяет тратить минимум времени на не очень важные дела (покупку автомобиля) и оставляет большую часть времени на важные (разработку суперкомпьютеров)» [1].

Во время третьего поколения появляются микропроцессоры – процессоры, размещенные на одной или нескольких интегральных схемах. Первым микропроцессором стал Intel 4004, выпущенный в 1971 году на одной микросхеме.

Четвертое поколение (с 1980 года)

Технология четвертого поколения – большие и сверхбольшие интегральные микросхемы (БИС и СБИС).

По количеству элементов выделяют следующие типы интегральных схем:

- малая интегральная схема (МИС) – до 100 элементов;
- средняя интегральная схема (СИС) – от 100 до 1000 элементов;
- большая интегральная схема (БИС) – от 1000 до 10000 элементов;
- сверхбольшая интегральная схема (СБИС) – более 10000 элементов.

Важными чертами четвертого поколения стало повсеместное распространение персональных компьютеров (т. е. компьютеров, предназначенных для личного использования).

1977 год – первый массовый и коммерчески успешный персональный компьютер Apple II (1 МГц, до 48 Кбайт ОЗУ) (рис. 1.8).



Рис. 1.8. Apple II

1981 год – персональный компьютер IBM PC (процессор Intel 8088, 4,77 МГц, до 256 Кбайт ОЗУ, операционная система MS-DOS, руководитель разработки Филипп Эстридж) (рис. 1.9).



Рис. 1.9. IBM PC

История развития вычислительной техники в хороших источниках читается как интересная повесть. Для более подробного ознакомления см. [1, 2, 16, 19].

1.4. Классификация вычислительных машин

Существует множество видов классификаций ВМ – по принципу действия, по этапам создания и элементной базе, по назначению, по способу организации вычислительного процесса, по вычислительной мощности и т. д.

Рассмотрим только один из вариантов [1].

- Встроенные компьютеры:
 - цена – десятки долларов;
 - телевизоры, CD-плееры, микроволновые печи и т. д.
- Игровые и карманные компьютеры, сотовые телефоны:
 - цена – сотни долларов.
- Персональные компьютеры:
 - цена – сотни, тысячи долларов;
 - настольные компьютеры, ноутбуки, планшетные компьютеры, нетбуки.
- Серверы:
 - цена – тысячи, десятки тысяч долларов;
 - сетевые серверы, серверы баз данных.
- Малые компьютеры, или миникомпьютеры (потомки PDP-11):
 - цена – десятки, сотни тысяч долларов;
 - научные и технические ВМ.
- Большие компьютеры, мэйнфреймы (*mainframe*, потомки IBM-360):
 - цена – сотни тысяч, миллионы долларов;

- большие базы данных крупных компаний, сложные научные задачи.
- Суперкомпьютеры:
 - цена – миллионы, десятки и сотни миллионов долларов;
 - ядерная физика, прогноз погоды, космические исследования...

1.5. Архитектура фон Неймана

В 1945 году Джон фон Нейман совместно с другими учеными¹ предложил принципы, положенные в основу большинства современных ВМ.

1. *Принцип двоичного кодирования*: вся информация, как данные, так и команды, кодируется в двоичной форме и разделяется на единицы – слова.
2. *Принцип программного управления*: алгоритм представляется с помощью управляющих слов – команд. Алгоритм, записанный в виде последовательности команд, называется *программой*. Команды выполняются в естественной последовательности, т. е. в порядке их расположения в программе. При необходимости, с помощью специальных команд, эта последовательность может быть изменена.
3. *Принцип однородности памяти*: команды и данные хранятся в одной и той же памяти.
4. *Принцип адресности*: память состоит из пронумерованных ячеек. Номера ячеек называются *адресами*.

Фон Нейман разработал структуру ВМ, показанную на рис. 1.10 (сегодня она стала называться классической). Основным блоком является процессор, состоящий из двух элементов – *устройства управления (УУ)* и *арифметико-логического устройства (АЛУ)*. Обрабатываемые данные хранятся в памяти, а взаимодействие с пользователем происходит через устройства ввода-вывода.

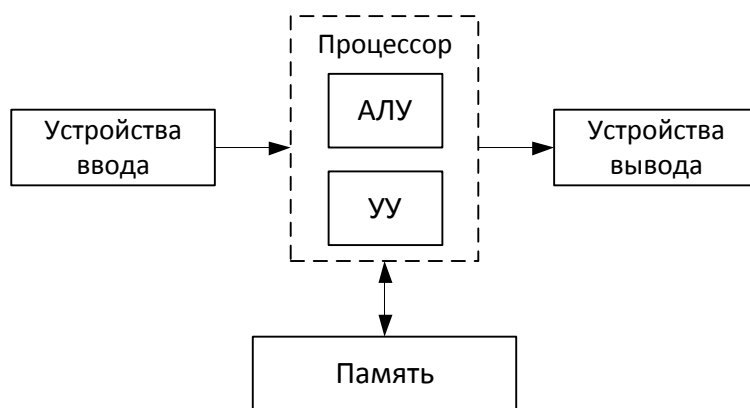


Рис. 1.10. Структура ВМ по фон Нейману

¹ Полунов Ю. Автора!!! // PC Week/Russian Edition. 2006. № 20(530).

Контрольные вопросы и задания

1. В чем ключевое отличие между понятиями вычислительной машины и вычислительной системы?
2. Зачем рассматривать вычислительную машину на нескольких уровнях? Приведите примеры систем, также рассматриваемых по уровням.
3. * Кто был первым в мире программистом?
4. Назовите первый в мире язык высокого уровня. Сохранился ли он до нашего времени?
5. Представьте память, построенную по принципу адресности фон Неймана в графическом виде.
6. * Принцип однородности памяти фон Неймана гласит, что данные и команды хранятся в одной и той же памяти. А как может быть иначе? Ответ проиллюстрируйте примерами из современности и из истории.
7. Почему в ВМ используется двоичное кодирование?
8. * Чем ВМ на основе троичной системы счисления отличается от обычной ВМ, основанной на двоичной? Перечислите преимущества и недостатки обеих.
9. * Найдите информацию о проекте ЭВМ пятого поколения, объявленном в Японии в начале 80-х годов. Почему проект до конца не был реализован? Какие уроки извлекли японцы (и весь мир) из опыта такого проекта?

ГЛАВА 2. ИНФОРМАЦИОННО-ЛОГИЧЕСКИЕ ОСНОВЫ ПОСТРОЕНИЯ ВЫЧИСЛИТЕЛЬНЫХ МАШИН

2.1. Системы счисления

Основные понятия

Система счисления – правило записи чисел с помощью заданного набора специальных символов – *цифр*.

Совокупность цифр, используемых в системе счисления, называется *алфавитом* системы счисления.

Существуют два основных вида систем счисления – непозиционные и позиционные.

В *непозиционных* системах счисления вес цифры (т. е. тот вклад, который она вносит в значение числа) не зависит от ее позиции в записи числа.

В *позиционных* системах счисления вес каждой цифры изменяется в зависимости от ее положения (позиции) в последовательности цифр, изображающей число.

Базис позиционной системы счисления – это возрастающая последовательность чисел, отвечающих за веса цифр.

Если базис образуют члены геометрической прогрессии вида

$$\dots, P^{-2}, P^{-1}, 1, P, P^2, P^3, \dots$$

то такую систему счисления называют *традиционной*.

Основанием традиционной системы счисления называется знаменатель P геометрической прогрессии, члены которой образуют базис системы счисления. Основание соответствует количеству цифр, используемых в системе счисления.

К традиционным относится, например, десятичная система счисления, основание которой равно 10, а базис составляют числа:

$$\dots 0,01 \ 0,1 \ 1 \ 10 \ 100 \ 1000 \ \dots$$

Традиционную систему счисления с основанием P называют *P -ичной системой счисления*.

Если из контекста неясно, в какой системе счисления записано число, основание системы счисления указывают нижним индексом: 125_{10} .

Непозиционные системы счисления

Примерами непозиционных систем счисления являются унарная и римская системы счисления.

В *унарной* системе счисления используется единственная цифра – 1, а число обозначается соответствующим количеством единиц.

Пример. Число 3 в унарной системе счисления обозначается 111, а число 7 – 1111111.

Таким образом, когда при подсчете чего-либо мы используем палочки, или зарубки, или какую-либо другую единственную отметку, оказывается, мы применяем унарную систему счисления.

В *римской* системе счисления цифр намного больше и все они являются буквами латинского алфавита:

$$I = 1, V = 5, X = 10, L = 50, C = 100, D = 500, M = 1000.$$

Число составляется по следующим правилам.

1. Если меньшая цифра стоит слева от большей, то меньшая цифра вычитается из большей. Обычно используют 6 примеров применения этого правила: $IV = 4$, $IX = 9$, $XL = 40$, $XC = 90$, $CD = 400$, $CM = 900$.
2. Если меньшая цифра стоит справа от большей, то обе цифры складываются. Одинаковые цифры также складываются.

Пример. $VIII = 8$, $XXIX = 29$, $LV = 55$, $XLV = 45$, $CCC = 300$, $MCM = 1900$, $MMXII = 2012$.

Как видно из примеров, вес любой цифры, как в унарной, так и в римской системах счисления не зависит от её позиции в числе, в отличие от веса цифр в позиционных системах счисления.

Римская система счисления до сих пор достаточно часто используется на практике, в основном в тех случаях, когда не требуется представлять большие числа и выполнять с ними сложные арифметические операции, например при обозначении веков, времени на циферблатах часов, номеров глав в книгах и т. д.

Непозиционными системами счисления также являются биномиальная система счисления, система остаточных классов, древнеегипетская система счисления и некоторые другие.

Позиционные системы счисления

Общеизвестным примером позиционной системы счисления является *десятичная* система счисления с основанием 10, использующая арабские цифры. Вес цифры в десятичной записи числа связан с местоположением цифры. Позиция цифры в числе называется *разрядом*.

Пример. В десятичной системе счисления в числе 55,5 первая пятерка означает 50, вторая – 5, а третья – 0,5.

Пример. Разряды нумеруются, начиная с запятой, отделяющей целую часть числа от дробной. Цифры целой части нумеруются положительными числами, дробной части – отрицательными. Запишем десятичное число 789,45 с номерами разрядов:

$$\begin{array}{cccccc} 2 & 1 & 0 & & -1 & -2 \\ 7 & 8 & 9 & , & 4 & 5 \end{array}$$

Существуют две формы записи чисел:

1) *свернутая* (обычная форма записи):

$$a_n a_{n-1} \dots a_1, a_0 a_{-1} \dots a_{-m},$$

где a_i – цифры числа.

Пример: $789,45_{10}$;

2) *развернутая*, которая получается с использованием основания системы счисления и номеров разрядов:

$$A = a_n P^n + a_{n-1} P^{n-1} + \dots + a_1 P^1 + a_0 + a_{-1} P^{-1} + \dots + a_{-m} P^{-m}. \quad (2.1)$$

Пример: $789,45_{10} = 7 \cdot 10^2 + 8 \cdot 10^1 + 9 + 4 \cdot 10^{-1} + 5 \cdot 10^{-2}$.

За основание позиционной системы счисления можно принять любое целое число, большее единицы, – 2, 3, 4 и т. д. В информатике кроме десятичной системы счисления широко используются двоичная, восьмеричная и шестнадцатеричная системы счисления.

В *двоичной* системе счисления используются всего 2 цифры – 0 и 1, в *восьмеричной* 8 цифр – от 0 до 7. Наименьшая единица компьютерных данных называется *бит*. Слово *bit* является сокращением английского выражения *binary digit*, т. е. двоичная цифра. Таким образом, *двоичное число* – это *последовательность бит*.

В ВМ используется двоичная система по следующим причинам:

- два состояния легко технически реализуются;
- представление информации двумя состояниями надежно и помехоустойчиво;
- двоичная система экономична в смысле соотношения количества цифр системы счисления и количества разрядов, необходимых для записи числа (более экономична только троичная);
- арифметические операции проще, чем в десятичной системе;
- возможно использование алгебры логики для выполнения логических операций.

В шестнадцатеричной системе счисления должно быть 16 цифр, но поскольку арабских цифр всего 10 (от 0 до 9), то после девятки используются латинские буквы:

$$A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.$$

Шестнадцатеричная система используется в компьютерной технике, потому что позволяет существенно сжать двоичную информацию, делая представление чисел более удобным для человека. Кроме того, перевод из двоичной системы в шестнадцатеричную очень прост – одна шестнадцатеричная цифра соответствует четырем двоичным (*tetраde*). Таким образом, шестнадцатеричная система счисления является «разумным компромиссом между тем, что удобно компьютеру, и тем, что удобно человеку» [11].

Перевод из P -ичной системы счисления в десятичную

Для перевода числа из P -ичной системы счисления в десятичную нужно записать число в развернутой форме (2.1), при этом все цифры a_i , основание P и показатели степеней следует представить в десятичной системе счисления.

Пример. Число $1101010,101_2$ перевести в десятичную систему счисления.

Сначала пронумеруем все разряды двоичного числа:

$$\begin{array}{ccccccccccc} 6 & 5 & 4 & 3 & 2 & 1 & 0 & -1 & -2 & -3 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & , & 1 & 0 & 1 \end{array}$$

Затем воспользуемся формулой (2.1) для получения десятичного представления:

$$\begin{aligned} 1101010,101_2 &= 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = \\ &= 64 + 32 + 0 + 8 + 0 + 2 + 0 + 0,5 + 0 + 0,125 = 106,625_{10}. \end{aligned}$$

Пример. Число $A9F,5_{16}$ перевести в десятичную систему счисления. Сначала пронумеруем все разряды шестнадцатеричного числа:

$$\begin{array}{cccc} 2 & 1 & 0 & -1 \\ A & 9 & F & , 5 \end{array}$$

Применяем формулу (2.1), не забывая представить все шестнадцатеричные цифры в десятичном эквиваленте:

$$\begin{aligned} A9F,5_{16} &= 10 \cdot 16^2 + 9 \cdot 16^1 + 15 \cdot 16^0 + 5 \cdot 16^{-1} = 10 \cdot 256 + 9 \cdot 16 + 15 \cdot 1 + 5 \cdot 0,0625 = \\ &= 2560 + 144 + 15 + 0,3125 = 2719,3125_{10} \end{aligned}$$

Перевод из десятичной системы счисления в P -ичную

Перевод целой и дробной частей десятичного числа в P -ичную систему счисления следует проводить отдельно, объединив затем результаты.

Для перевода *целой части числа* из десятичной системы в P -ичную нужно делить число на основание P требуемой системы счисления до тех пор, пока частное не станет меньше P , а затем записать последнее частное и все остатки от деления в обратном порядке.

Пример. Перевести число 106_{10} в двоичную систему счисления.

$$\begin{array}{r} 106 \overline{) 2} \\ -106 \\ \hline 0 \end{array} \quad \begin{array}{r} 53 \overline{) 2} \\ -52 \\ \hline 1 \end{array} \quad \begin{array}{r} 26 \overline{) 2} \\ -26 \\ \hline 0 \end{array} \quad \begin{array}{r} 13 \overline{) 2} \\ -12 \\ \hline 1 \end{array} \quad \begin{array}{r} 6 \overline{) 2} \\ -6 \\ \hline 0 \end{array} \quad \begin{array}{r} 3 \overline{) 2} \\ -2 \\ \hline 1 \end{array}$$

Ответ: $106_{10} = 1101010_2$.

Пример. Перевести число 2719_{10} в шестнадцатеричную систему счисления.

$$\begin{array}{r} 2719 \overline{) 16} \\ -2704 \\ \hline 15 \end{array} \quad \begin{array}{r} 169 \overline{) 16} \\ -160 \\ \hline 9 \end{array}$$

Ответ: $2719_{10} = A9F_{16}$.

Для перевода *дробной части десятичного числа* нужно умножать число на основание P системы счисления до тех пор, пока произведение не станет равно нулю или пока не достигнута требуемая разрядность. Дело в том, что десятичная конечная дробь может иметь бесконечное представление в P -ичной системе счисления, поэтому и вводится ограничение на число разрядов. После каждого умножения целая часть отбрасывается, а следующее умножение осуществляется с остатком. По завершении умножения искомое число в P -ичной системе счисления формируется путем выписывания целых частей произведений в порядке их получения.

Пример. Перевести число $0,6875_{10}$ в двоичную систему счисления.

$$\begin{array}{r} 0,6875 \\ \times \quad 2 \\ \hline 1,3750 \\ \times \quad 2 \\ \hline 0,750 \\ \times \quad 2 \\ \hline 1,50 \\ \times \quad 2 \\ \hline 1,0 \end{array}$$

↓

Ответ: $0,6875_{10} = 0,1011_2$.

Пример. Перевести число $0,6875_{10}$ в шестнадцатеричную систему счисления.

$$\begin{array}{r} 0,6875 \\ \times \quad 16 \\ \hline 11,0 \end{array}$$

Ответ: $0,6875_{10} = 0,B_{16}$.

Перевод чисел между двоичной, восьмеричной и шестнадцатеричной системами счисления

Перевод чисел из двоичной системы счисления в восьмеричную и шестнадцатеричную и обратно осуществляется достаточно просто, требуется только знать таблицы перевода.

Таблица 2.1. Перевод чисел между двоичной и восьмеричной системами счисления

Двоичная	Восьмеричная
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Таблица 2.2. Перевод чисел между двоичной и шестнадцатеричной системами счисления

Двоичная	Шестнадцатеричная
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Замечание. В столбце двоичных чисел крайняя правая цифра чередует значение 0 и 1, вторая справа цифра чередует значения через два бита, третья справа – через четыре бита, крайняя левая – через восемь. Такая закономерность позволяет строить таблицу очень быстро, причем произвольной очередности.

Особенность перевода чисел в этих системах счисления заключается в том, что всегда одна восьмеричная цифра соответствует трем двоичным ($2^3 = 8$), а одна шестнадцатеричная цифра соответствует четырем двоичным ($2^4 = 16$). Поэтому алгоритмы перевода из двоичной системы счисления сводятся к разбиению двоичного числа, начиная справа, на триады (по три цифры) в случае восьмеричной системы счисления или тетрады (по четыре цифры) в случае шестнадцатеричной и переводе триад или тетрад отдельно в соответствии с приведенными выше табл. 2.1 и 2.2.

Пример. Перевести число 111100110101011_2 в восьмеричную систему счисления.

Разобьем двоичное число на триады, начиная с младшего разряда:

$$111\ 100\ 110\ 101\ 011_2.$$

Переводим триады отдельно по табл. 2.1:

$$111_2 = 7_8, 100_2 = 4_8, 110_2 = 6_8, 101_2 = 5_8, 011_2 = 3_8.$$

Формируем восьмеричное число: 74653_8 .

Пример. Перевести число 111100110101011_2 в шестнадцатеричную систему счисления.

Решаем аналогично, только разбиваем двоичное число на тетрады. Крайняя левая тетрада будет неполной (количество цифр в двоичном числе не кратно 4), поэтому дописываем незначащий ноль слева:

$$111100110101011_2 = \underline{0}111\ 1001\ 1010\ 1011_2.$$

Переводим по тетрадам, пользуясь табл. 2.2:

$$0111_2 = 7_{16}, 1001_2 = 9_{16}, 1010_2 = A_{16}, 1011_2 = B_{16}.$$

Записываем ответ: $111100110101011_2 = 79AB_{16}$.

Перевод чисел из восьмеричной и шестнадцатеричной систем счисления в двоичную происходит поразрядно: каждая цифра в представлении восьмеричного и шестнадцатеричного числа заменяется на триаду или тетраду в соответствии с табл. 2.1 или 2.2.

Пример. Перевести число 74653_8 в двоичную систему счисления.

Воспользуемся табл. 2.1 – находим для каждой восьмеричной цифры заданного числа двоичный эквивалент:

$$7_8 = 111_2, 4_8 = 100_2, 6_8 = 110_2, 5_8 = 101_2, 3_8 = 011_2 \quad -$$

и записываем полученные двоичные числа подряд:

$$74653_8 = 111\ 100\ 110\ 101\ 011_2.$$

Пример. Перевести число $79AB_{16}$ в двоичную систему счисления.

По табл. 2.2 находим двоичные числа, соответствующие шестнадцатеричным в заданном числе:

$$7_{16} = 0111_2, 9_{16} = 1001_2, A_{16} = 1010_2, B_{16} = 1011_2,$$

и записываем двоичные числа подряд:

$$79AB_{16} = 0111\ 1001\ 1010\ 1011_2.$$

Перевод десятичных чисел в шестнадцатеричную систему счисления

Можно действовать двумя способами:

- 1) воспользоваться правилом перевода из десятичной системы счисления в P -ичную, т. е. целую часть делить на 16, дробную – умножать на 16;
- 2) сначала перевести число в двоичную систему счисления, а затем в шестнадцатеричную.

Перевод шестнадцатеричных чисел в десятичную систему счисления

Перевод можно осуществлять также двумя способами:

- 1) воспользовавшись правилом перевода из P -ичной системы счисления в десятичную;
- 2) сначала переведя число в двоичную систему счисления, а затем из двоичной – в десятичную.

2.2. Представление информации в вычислительных машинах

Основными типами информации в ВМ являются целые числа, вещественные числа, символы, графическая информация, аудио- и видеoinформация. Хотя для каждого типа предусмотрен свой способ представления (*формат*), все они хранятся в двоичном виде в ячейках памяти.

2.2.1. Ячейки памяти

Основными ячейками компьютерной памяти являются бит и байт. *Бит* (*bit*, *binary digit* – двоичная цифра) – это минимальная ячейка памяти, которая может хранить один двоичный разряд (значения «0» или «1»). Последовательность смежных двоичных цифр длиной в 8 бит¹ получила название *байт* (*byte*, *binary term*²). Байт – это наименьшая адресуемая компьютером единица информации.

Максимально возможное число, которое размещается в байте, в двоичном формате будет состоять из восьми единиц, а в шестнадцатеричном – из двух цифр FF:

$$1111\ 1111_2 = FF_{16} = 255_{10}.$$

Слово (*word*) – это совокупность нескольких байт, которую процессор обрабатывает как единое целое. Процессор производит операции именно над словами, например считывает из памяти слово целиком и записывает в память, складывает или умножает два слова и т. д.

Размер слова зависит от разрядности процессора, которая в свою очередь определяется разрядностью основных регистров процессора. Например, у 16-разрядного процессора Intel 8086 основные регистры были 16-разрядные и размер слова был равен двум байтам, а современный 32-разрядный процессор Intel Core имеет 8 основных регистров разрядностью 32 бита, поэтому размер слова у него – четыре байта.

Максимально возможное целое число, которое можно поместить в двухбайтовое слово:

$$1111\ 1111\ 1111\ 1111_2 = FFFF_{16} = 65535_{10}.$$

Для 32-разрядного слова максимальное число будет следующим:

$$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_2 = FFFF\ FFFF_{16} = 4\ 294\ 967\ 295_{10}.$$

¹ Термин «байт» появился в 50-х годах XX века, и сначала размер байта в разных компьютерах отличался (например, были байты в 6, 8, 9, 12 бит и др.). Примерно в 70-е годы под байтом стали понимать всегда 8 бит (иногда такой набор ещё называют «октет»).

² Ещё одна версия происхождения слова «байт» связана с тем, что первоначально последовательность бит называли *bite* – кусок, а затем, чтобы не путать со словом *bit*, заменили *i* на *y*.

Большинство современных процессоров 32- или 64-разрядные.

Следует отметить, что разрядность ячеек памяти, начиная с байта, увеличивается каждый раз вдвое, т. е. используются ячейки размером 1 байт, 2 байта, 4 байта, 8 байт, 16 байт, но не применяются ячейки 7- или 9-байтовые¹.

2.2.2. Целые числа

Беззнаковые типы данных

Машинное представление чисел неразрывно связано с понятием типа данных. Напомним его определение. *Тип данных* – это характеристика данных, определяющая диапазон их значений, набор допустимых операций над ними и внутреннее представление в памяти компьютера.

Типы данных для хранения целых чисел делятся на беззнаковые и знаковые. В *беззнаковых типах данных* можно хранить только неотрицательные целые числа. *Знаковые типы* предназначены для хранения и неотрицательных, и отрицательных чисел. Рассмотрим сначала беззнаковые типы.

Числа, представленные беззнаковыми типами, хранятся в памяти в двоичном виде, поэтому для получения представления беззнакового десятичного числа в памяти компьютера нужно просто перевести это число в двоичную систему счисления и дописать незначащие нули слева до требуемой разрядности.

Пример. Пусть имеется десятичное целое число 100. В каком типе данных его можно представить? Сколько требуется бит для хранения числа и каково будет его внутреннее (машинное) представление?

Решение. Для начала нужно перевести это число в двоичную систему счисления:

$$100_{10} = 110\ 0100_2 = 64_{16}.$$

По количеству полученных бит (7) видно, что для хранения такого числа понадобится минимум 8-разрядная ячейка памяти. Для Паскаля это беззнаковый тип `Byte`.

Чтобы получить внутреннее представление числа в памяти, возьмем двоичный вид и допишем слева незначащий ноль до разрядности 8 бит:

$$100_{10} = \underline{0}110\ 0100_2 = 64_{16}.$$

¹ Это утверждение почти всегда справедливо при представлении целых чисел, но, например, существует 10-байтовая ячейка памяти для вещественных чисел (соответствующий тип данных называется *Extended*).

Можно это число разместить и в 16-битном беззнаковом типе (например, `Word`), при этом придется дописывать уже 9 незначащих нулей:

$$100_{10} = \underline{0000} \underline{0000} \underline{0110} 0100_2 = 0064_{16}.$$

Знаковые типы данных

В *знаковом целом типе* могут быть представлены положительные и отрицательные числа, а также ноль. Положительные числа и ноль в знаковом типе хранятся точно так же, как в беззнаковом, – в двоичном виде. Знаковые отрицательные числа хранятся в компьютере в *дополнительном коде*.

В этом коде за знак числа отвечает старший (крайний левый) бит. Если он равен нулю, то число положительное или ноль, а если единице – число отрицательное. Причем при переводе числа в дополнительный код не нужно приписывать необходимый знаковый разряд – если действовать строго по алгоритмам перевода, рассмотренным далее, правильный знаковый бит получается автоматически.

У дополнительного кода есть важное достоинство – он позволяет осуществлять операции сложения и вычитания одинаковым образом как с беззнаковыми целыми числами, так и со знаковыми.

Рассмотрим алгоритм перевода отрицательного числа в дополнительный код (повторим, положительные числа переводить в дополнительный код не нужно – их представление в знаковом типе совпадает с обычным двоичным видом).

Алгоритм 1. Первый способ перевода отрицательного числа в дополнительный код.

1. Перевести модуль отрицательного числа в двоичный код.
2. Дополнить двоичное число незначащими нулями до требуемой разрядности (в зависимости от используемого типа данных).
3. Проинвертировать полученное двоичное число (т. е. ноли становятся единицами, а единицы – нолями).
4. К полученному двоичному числу арифметически прибавить 1.

Пример. Пусть имеется десятичное отрицательное число -100 . В каком машинном формате его можно представить?

Решение. Наше число отрицательное, значит, его нужно перевести в дополнительный код (алгоритм 1):

1) $|-100| = 100_{10} = 110\ 0100_2.$

- 2) На предыдущем шаге получилось число разрядностью 7 бит. Но такого типа данных нет, поэтому используем 8 бит, добавляя слева ноль:

$$\underline{0}110\ 0100_2.$$

- 3) Инвертируем: $0110\ 0100_2 \rightarrow 1001\ 1011_2$.
4) К полученному числу прибавляем 1:

$$\begin{array}{r} +\quad 1001\ 1011 \\ \quad\quad\quad 1 \\ \hline 1001\ 1100 \end{array}$$

Ответ: $1001\ 1100_2 = 9C_{16}$.

Задание. Самостоятельно найдите машинное представление числа -100 в 16-разрядном знаковом типе и в 32-разрядном знаковом типе.

Ответ: $FF9C_{16}$, $FFFF\ FF9C_{16}$.

Задание. Самостоятельно найдите машинное представление числа -1000 в 16-разрядном знаковом типе.

Ответ: $F828_{16}$.

Существует ещё один алгоритм перевода в дополнительный код, связанный с названием кода: «дополнительный» означает, что число в дополнительном коде дополняет модуль исходного отрицательного числа до 2^n , где n – разрядность используемого типа данных.

Алгоритм 2. Второй способ перевода отрицательного числа в дополнительный код.

Обозначим n – разрядность используемого типа данных, X – отрицательное число.

1. Найдём разность:

$$y = 2^n - |X|.$$

2. Переведем y в двоичную систему счисления.

Пример. Найти дополнительный код числа -100 для 8-разрядного типа данных.

Решение. Воспользуемся алгоритмом 2.

- 1) $y = 2^n - |X| = 2^8 - |-100| = 256 - 100 = 156_{10}$.
- 2) $156_{10} = 1001\ 1100_2$.

Ответ: дополнительный код числа -100 в 8-разрядном типе данных равен $1001\ 1100_2 = 9C_{16}$.

Алгоритм 3. Первый способ перевода числа из дополнительного кода в десятичный вид.

1. Инвертируем двоичное число.
2. Прибавляем арифметически к нему единицу.
3. Переводим полученное двоичное число в десятичную систему счисления и приписываем знак «минус».

Пример. Дано двоичное число в дополнительном коде $1001\ 1100_2$. Требуется перевести его в десятичную систему счисления.

Решение.

- 1) Инвертируем число: $1001\ 1100_2 \rightarrow 0110\ 0011_2$.
- 2) Прибавляем к полученному числу единицу:

$$\begin{array}{r}
 0110\ 0011 \\
 + \quad \quad \quad 1 \\
 \hline
 0110\ 0100
 \end{array}$$

- 3) Переводим число в десятичную систему счисления: $0110\ 0100_2 = 100_{10}$ и приписываем знак минус.

Ответ: -100_{10} .

По аналогии с получением дополнительного кода существует второй способ нахождения десятичного числа по дополнительному коду.

Алгоритм 4. Второй способ перевода числа из дополнительного кода в десятичный вид.

Обозначим n – разрядность используемого типа данных, X – искомое отрицательное число.

1. Переводим двоичное число в десятичную систему счисления. Полученное число обозначаем y .

2. Находим разность: $|X| = 2^n - y$.
3. Приписываем к полученному результату знак «минус».

Пример. Найти десятичное представление двоичного числа в дополнительном коде $1001\ 1100_2$.

Решение. Воспользуемся алгоритмом 4.

- 1) $1001\ 1100_2 = 156_{10}$.
- 2) $|X| = 256 - 156 = 100_{10}$.
- 3) Приписываем знак «минус».

Ответ: -100 .

Определение машинного представления десятичного числа

Сейчас можно обобщить наши сведения и рассмотреть алгоритм нахождения внутреннего представления числа в памяти компьютера.

Алгоритм 5. Определение машинного представления десятичного числа (рис. 2.1).

1. Выбрать тип данных, который будет использоваться для машинного представления.
2. Если выбран беззнаковый тип, то определяем знак десятичного числа. Переход на п. 4.
3. Если выбран знаковый тип, то определяем знак десятичного числа. Переход на п. 6.
4. Если знак десятичного числа «минус», то представление такого числа в беззнаковом типе невозможно. Завершение работы. Иначе переход на п. 5.
5. Если знак десятичного числа «плюс» или число равно 0, то переводим десятичное число в двоичный вид и дополняем нолями до требуемой разрядности. Завершение работы.
6. Если знак десятичного числа «минус», то число должно быть представлено в дополнительном коде и нужно использовать алгоритм 1 или 2 для перевода из десятичной системы счисления в дополнительный код. Завершение работы. Иначе переход на п. 5.

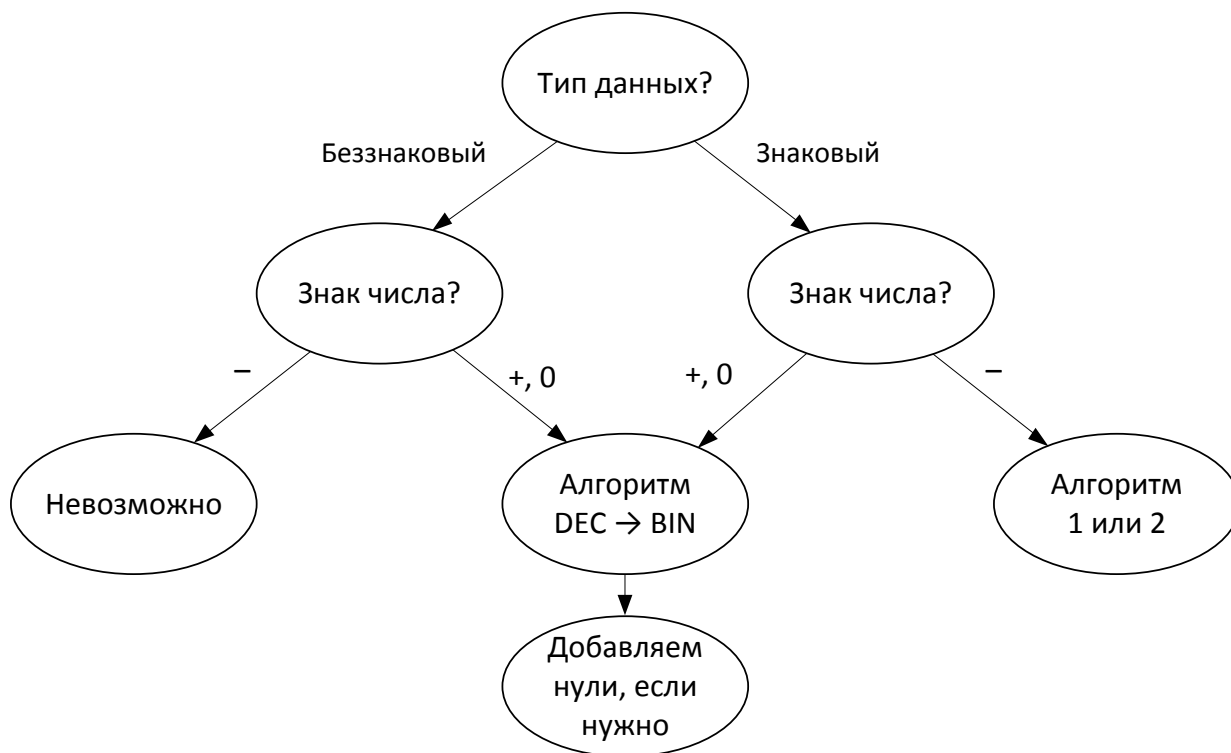


Рис. 2.1. Алгоритм 5 – определение машинного представления десятичного числа

Определение десятичного вида числа по его машинному представлению

Выше рассматривалась проблема представления целых чисел в памяти компьютера. А что, если стоит обратная задача: по машинному представлению числа определить его десятичную форму?

Оказывается, просто по двоичному виду числа в памяти компьютера невозможно определить, какое число там представлено. Мало того, нельзя даже сказать, какого рода данные хранятся – числа, символы, графика или звук. Это является следствием одного из принципов фон Неймана: все данные хранятся в памяти компьютера в двоичном виде.

Чтобы правильно интерпретировать двоичное число, представленное в памяти компьютера, *нужно обязательно знать тип данных*. Так, например, число $FC18_{16}$ в типе `SmallInt` интерпретируется как -1000 , так как тип знаковый. С другой стороны, то же самое число в беззнаковом типе `Word` будет интерпретироваться как 64536 . То есть один и тот же двоичный код может представлять разные числа, в зависимости от заданного типа.

Следует напомнить важное правило: если число представлено в памяти в знаковом типе, то старший (крайний левый) бит отвечает за знак числа: если старший бит равен 0, то число положительное, если 1, то отрицательное. Поэтому в знаковых типах старший бит называется *Signum* (лат. *знак*).

Рассмотрим следующий алгоритм.

Алгоритм 6. Определение десятичного вида числа по его машинному представлению (рис. 2.2).

1. Определить тип данных, который использовался для машинного представления.
2. Если тип беззнаковый, то просто переводим из двоичного вида в десятичный.
3. Если тип знаковый, то проверяем значение старшего бита.
4. Если старший бит равен 0, то просто переводим из двоичного вида в десятичный.
5. Если старший бит равен 1, то число представлено в дополнительном коде и нужно использовать алгоритм 3 или 4 для перевода из дополнительного кода в десятичный вид.

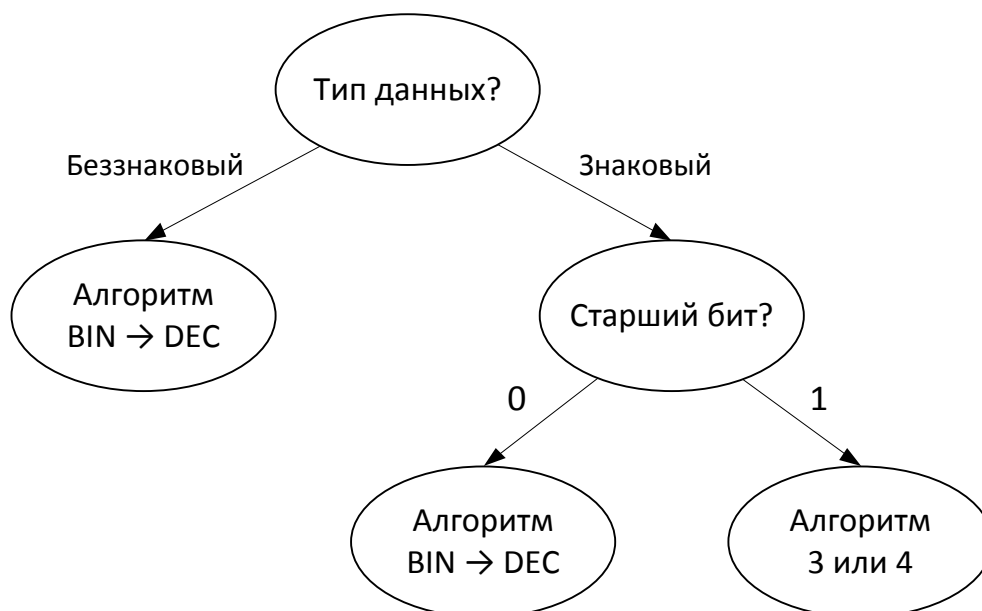


Рис. 2.2. Алгоритм 6 – определение десятичного вида числа по его машинному представлению

Каким образом определить используемый тип данных? Это можно сделать либо на основании априорной информации (например, если программе-отладчику, которой нужно отобразить значение ячейки памяти, доступен исходный код отлаживаемой программы), либо при помощи косвенных данных (например, какие машинные команды используются для работы с интересующей ячейкой памяти – некоторые машинные команды имеют знаковые и беззнаковые варианты).

Диапазоны типов данных

Чтобы уметь правильно работать с типом данных, нужно точно знать его диапазон. Если, например, при сложении двух чисел результат окажется за пределами диапазона используемого типа, значение, которое сохранится в памяти, будет неверным. Такие ситуации надо уметь отслеживать и предупреждать.

Для определения диапазона целого типа данных (т. е. нахождения минимального и максимального значений, представимых в этом типе) достаточно знать его разрядность n и является тип знаковым или беззнаковым.

Независимо от «знаковости» типа, общее количество чисел, представимых в этом типе, равно 2^n :

$$\text{общее количество чисел в типе} = 2^n.$$

Для беззнаковых типов минимальное значение представляется всеми нулями в двоичной записи числа и всегда равно 0.

Чтобы получить максимальное значение, нужно заполнить все разряды единицами и перевести полученное двоичное число в десятичный вид. Например, для 8-разрядного беззнакового типа максимальное число будет равно $1111\ 1111_2 = 255_{10}$. В общем случае, максимальное число, представимое в n разрядах, будет равно $2^n - 1$. Таким образом, получаем:

$$\text{диапазон беззнакового типа: } [0 .. 2^n - 1].$$

Для вывода формулы диапазона знаковых типов рассмотрим некоторый 4-разрядный тип данных. Отложим $2^4 = 16$ двоичных чисел, представимых в этом типе, на числовом круге (см. рис. 2.3). Каждому двоичному числу снаружи круга припишем его десятичный эквивалент в том случае, если бы тип был беззнаковый, и внутри круга, если тип знаковый. Десятичное представление чисел можно находить, пользуясь алгоритмом 6 (см. рис. 2.2).

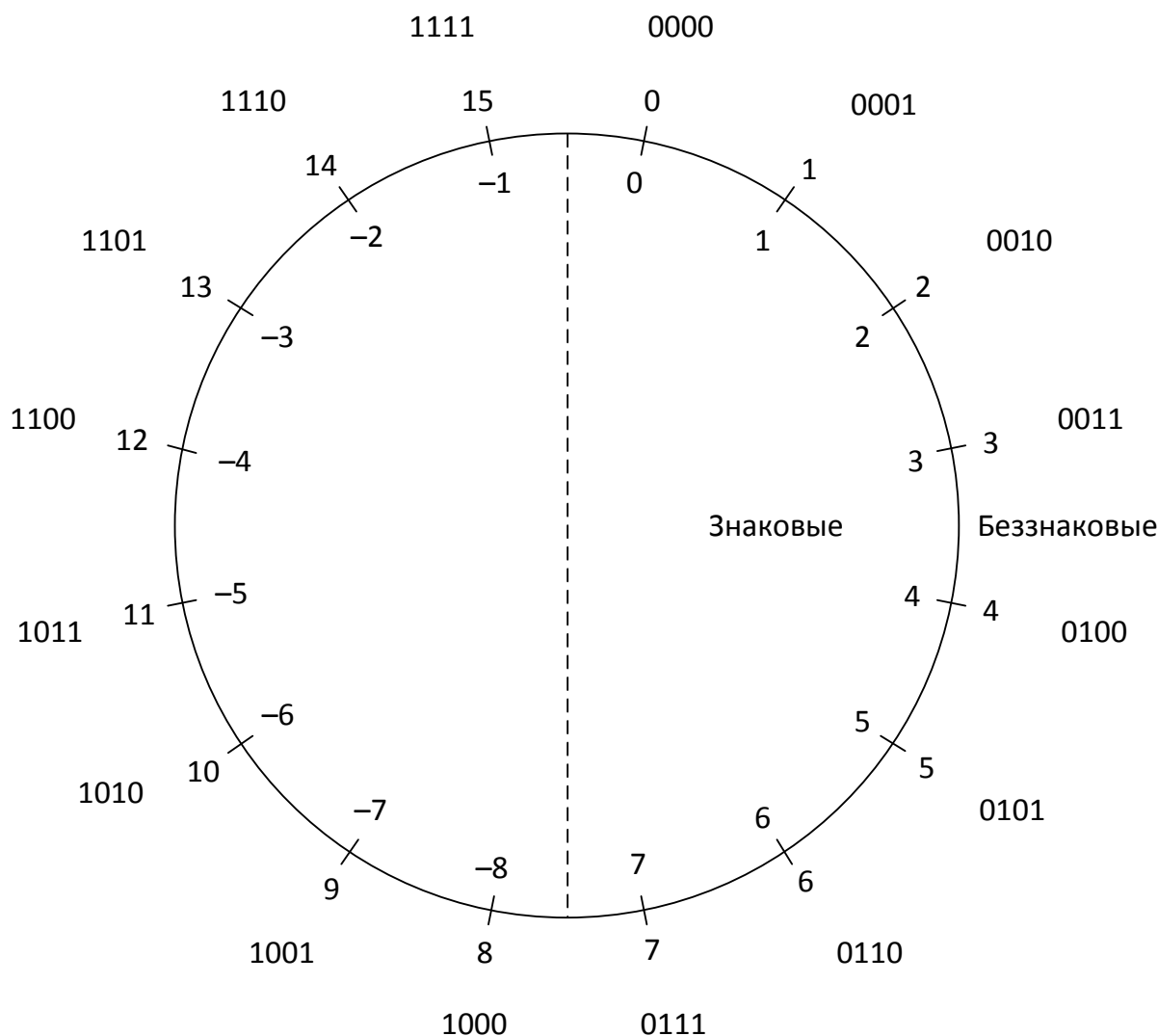


Рис. 2.3. Числовой круг для 4-разрядных типов

На рис. 2.3 можно заметить, что круг для знакового типа поделен на две равные части – неотрицательные числа (справа, там, где старший бит равен 0) и отрицательные числа (слева, старший бит равен 1). Таким образом, диапазон знакового типа делится пополам (количество чисел в каждой $2^n / 2$) – одна половина отводится на положительные числа и ноль, другая – на отрицательные числа:

диапазон знакового типа: $[-2^{n-1} .. 2^{n-1}-1]$.

Половина положительных десятичных чисел из всего диапазона беззнакового типа совпадает по своему внутреннему представлению в памяти компьютера с отрицательными числами.

Целочисленные типы данных в Паскале

В табл. 2.3 представлены целочисленные типы, используемые в средах программирования Borland Delphi и PascalABC.Net. Заметим, что поскольку эти среды программирования 32-разрядные (предназначены для работы на 32-разрядных процессорах), то базовый целый тип Integer является 4-байтовым.

Таблица 2.3. Целые типы данных в Borland Delphi и PascalABC.Net

Тип	Разрядность	Знак	Диапазон значений
Byte	8	Беззнаковый	0..255
ShortInt	8	Знаковый	–128..127
Word	16	Беззнаковый	0..65 535
SmallInt	16	Знаковый	–32 768..32 767
Longword, Cardinal	32	Беззнаковый	0..4 294 967 295
Integer, Longint	32	Знаковый	–2 147 483 648..2 147 483 647
Int64	64	Знаковый	$-2^{63} .. 2^{63}-1$

Задание. Что получится в результате выполнения программы на Паскале? Самостоятельно постройте числовой круг для 8-разрядных типов и объясните на его основе ответ.

```
var x: Byte;  
begin  
  x := 200 + 200;  
  writeln('x=', x);  
end.
```

2.2.3. Вещественные числа

Стандарт IEEE 754-2008

Для представления вещественных чисел в памяти компьютера разработан стандарт IEEE 754-2008¹, в котором описаны форматы представления чисел, алгоритмы округления, операции над числами, обработка исключений (деление на ноль, переполнение и др.).

¹ IEEE (Institute of Electrical and Electronics Engineers) – Институт инженеров по электронике и электротехнике, некоммерческая международная ассоциация, одной из функций которой является разработка стандартов в области информатики.

В этом стандарте используется форма представления вещественных чисел с *плавающей запятой*. В этой форме вещественное число X записывается в следующем виде:

$$X = \pm m \cdot q^{\pm p},$$

где m – мантисса числа (лат. *добавление, прибавка*), q – основание системы счисления, p – порядок.

Пример. Представить число $123,45_{10}$ в форме с плавающей запятой.

$$123,45 = 12,345 \cdot 10^1 = 1,2345 \cdot 10^2.$$

В этом примере числа $12,345$ и $1,2345$ являются мантиссой, а 1 и 2 – порядком.

Из примера видно, что десятичная запятая может перемещаться («плавать») с одновременным изменением порядка: если запятая перемещается влево на n разрядов, то порядок нужно увеличить на n единиц, если запятая перемещается вправо на n разрядов, то порядок нужно уменьшить на n единиц.

Для представления чисел в памяти основание системы счисления принимается равным 2 .

Пример. Представить число $10101,101_2$ в форме с плавающей запятой.

$$10101,101 = 101,01101 \cdot 2^2 = 1010110,1 \cdot 2^{-2}.$$

Для определенности вещественные числа в компьютерах представляются в *нормализованном виде*, т. е. запятая расположена справа от первой ненулевой цифры.

Пример

$0,00101_2$ – ненормализованное двоичное число,

$0,0101 \cdot 2^{-1}$ – ненормализованное двоичное число,

$1,01 \cdot 2^{-3}$ – нормализованное двоичное число.

В двоичном нормализованном представлении числа старший бит, расположенный слева от запятой, всегда равен 1 , поэтому его можно не хранить («скрытая единица»), а диапазон нормализованных мантисс получается следующим: $1 \leq m < 2$.

Рассмотрим основные форматы представления вещественных чисел, описанные в стандарте IEEE 754-2008. Они отличаются размером ячейки памяти, отводимой для одного числа:

- 1) формат одинарной точности (single precision, 4 байта);
- 2) формат двойной точности (double precision, 8 байт);
- 3) формат расширенной точности (extended precision, 10 байт).

В *формате одинарной точности* под число отводится 32 бита, которые распределены следующим образом:

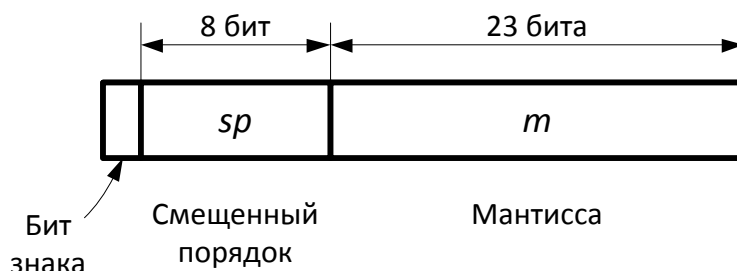


Рис. 2.4. Формат одинарной точности

В этом формате указывается только знак всего числа: если бит знака равен 0, число положительное, если 1 – отрицательное. Знак порядка не указывается, так как применяется *смещенный порядок* sp , который всегда неотрицателен. Смещенный порядок получается прибавлением к исходному порядку константы 127:

$$sp = p + 127. \quad (2.2)$$

При 8-битной ширине поля смещенного порядка диапазон порядков:

$$-127 \leq p \leq 128.$$

Таким образом, для представления вещественного числа X в формате одинарной точности используется следующая формула (в десятичном виде):

$$X = \pm 1, m \cdot 2^{sp-127}. \quad (2.3)$$

Алгоритм 7. Перевод десятичного вещественного числа в формат одинарной точности.

1. Перевести вещественное число в двоичный вид.
2. Нормализовать полученное двоичное число.
3. Отбросить скрытую единицу.

4. Определить мантиссу в двоичном виде.
5. Определить порядок в десятичном виде.
6. Вычислить смещенный порядок по формуле (2.2) и перевести его в двоичный вид.
7. Определить значение старшего бита, соответствующего знаку числа.
8. Записать ответ в соответствии с рис. 2.4.

Пример. Представить в формате одинарной точности число $-27,25_{10}$.

- 1) Переводим десятичное число в двоичный вид: $27,25_{10} = 11011,01_2$.
- 2) Нормализуем двоичное число: $11011,01_2 = 1,101101 \cdot 2^4$.
- 3) Отбрасываем скрытую единицу.
- 4) Мантисса: $m = 101101_2$.
- 5) Порядок: $p = 4_{10}$.
- 6) Находим смещенный порядок: $sp = 4 + 127 = 131_{10} = 1000\ 0011_2$.
- 7) Поскольку знак «минус», то старший бит равен 1.
- 8) Ответ в двоичном виде:

1	1000 0011	101 1010 0000 0000 0000 0000
└─┘	└──────────┘	└────────────────────────────────┘
Бит	Смещенный	Мантисса
знака	порядок	

В шестнадцатеричном виде: $-27,25_{10} = C1DA\ 0000_{16}$.

Алгоритм 8. Перевод машинного представления нормализованного вещественного числа одинарной точности в десятичный вид.

- 1) Отделить старший бит в двоичном представлении и определить знак числа: «плюс», если старший бит равен 0, «минус», если старший бит равен 1.
- 2) Отделить двоичное представление смещенного порядка (8 бит).
- 3) Перевести смещенный порядок в десятичный вид.
- 4) Вычислить порядок искомого числа по формуле:

$$p = sp - 127.$$

- 5) Выделяем мантиссу, прибавляем к ней слева скрытую единицу и двоичную запятую.

- 6) Переводим полученную мантиссу в десятичный вид.
- 7) Записываем ответ по формуле (2.3).

Пример. Найти вещественное число, соответствующее машинному представлению: 1100 0001 1101 1010 0000 0000 0000 0000.

- 1) Старший бит равен 1, следовательно, знак числа «минус».
- 2) Выделяем смещенный порядок: $sp = 1000\ 0011_2$.
- 3) Переводим смещенный порядок в десятичный вид:

$$sp = 1000\ 0011_2 = 131_{10}.$$

- 4) Вычисляем порядок: $p = 131 - 127 = 4$.
- 5) Выделяем мантиссу, прибавляем к ней слева скрытую единицу и двоичную запятую: $m = 1,101101_2$.
- 6) Переводим мантиссу в десятичный вид:

$$m = 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-3} + 1 \cdot 2^{-4} + 1 \cdot 2^{-6} = 1,703125_{10}.$$

- 7) Ответ: $-1,703125 \cdot 2^4 = -27,25_{10}$.

В других форматах стандарта IEEE 754-2008 следующие значения ширины полей:

– *формат двойной точности*: под все число отводится 64 бита (8 байт), из них 1 бит – знак, 11 бит – смещенный порядок, 52 бита – мантисса, смещенный порядок вычисляется по формуле:

$$sp = p + 1023;$$

– *формат расширенной точности*: под все число отводится 80 бит (10 байт), из них 1 бит – знак, 15 бит – смещенный порядок, 64 бита – мантисса, смещенный порядок вычисляется по формуле:

$$sp = p + 16383.$$

Особые числа

В стандарте IEEE 754-2008 определены *особые числа*, которые представляют ноль, бесконечность, неопределенный результат операции и числа, близкие к нулю.

1. Ноль

В отличие от машинного представления целых чисел, при хранении вещественных чисел существует два варианта нуля – положительный и отрицательный (все примеры далее будем приводить только для одинарного формата).

Положительный ноль:

$$\mathbf{0} \quad 0000 \ 0000 \quad 000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000_2 = 0000 \ 0000_{16}$$

1

00000000

000000000000000000000000

Бит
знака

Смещенный
порядок

Мантисса

Отрицательный ноль:

$$\mathbf{1} \quad 0000 \ 0000 \quad 000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000_2 = 8000 \ 0000_{16}$$

1

00000000

000000000000000000000000

Бит
знака

Смещенный
порядок

Мантисса

2. Бесконечность

Вариантов представления бесконечности также два, положительный и отрицательный. В этом представлении в смещенном порядке содержатся все единицы.

Положительная бесконечность:

$$\mathbf{0} \quad 1111 \ 1111 \quad 000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000_2 = 7F80 \ 0000_{16}$$

0

11111111

000000000000000000000000

Бит
знака

Смещенный
порядок

Мантисса

Отрицательная бесконечность:

$$\mathbf{1} \quad 1111 \ 1111 \quad 000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000_2 = FF80 \ 0000_{16}$$

1

11111111

000000000000000000000000

Бит
знака

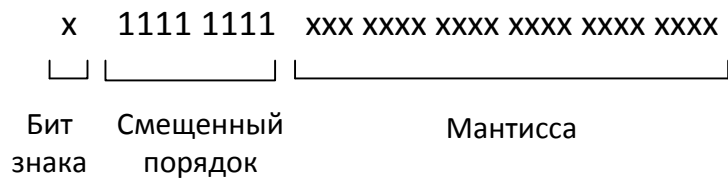
Смещенный
порядок

Мантисса

3. NaN (Not a Number)

Особые числа NaN (дословный перевод «не-числа») используются для представления неопределенных или некорректных результатов, например результатов операций деления нуля на ноль, умножения нуля на бесконечность, извлечения квадратного корня из отрицательного числа.

Число будет являться NaN, если в поле смещенного порядка хранятся все единицы (исключая случай, когда в мантиссе все нули):

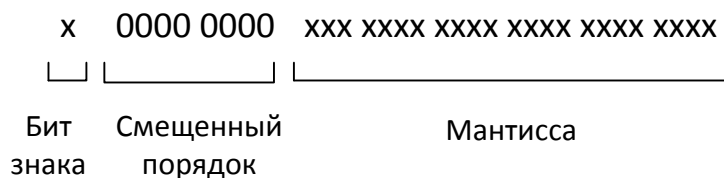


Символ *x* на рисунке говорит о том, что данный бит не имеет значения.

4. Денормализованные числа

Особенность представления вещественных чисел со скрытой единицей в том, что имеется довольно большой разрыв между нулем и ближайшим к нему числом. В некоторых случаях это обстоятельство может приводить к ошибкам при работе с малыми величинами. Чтобы снизить вероятность таких ошибок, были введены *денормализованные числа*, при хранении которых скрытая единица не используется.

Признаком того, что число денормализованное, является наличие всех нулей в поле смещенного порядка (за исключением случая всех нулей в мантиссе):



Алгоритм 9. Перевод машинного представления денормализованного вещественного числа одинарной точности в десятичный вид.

- 1) Отделить старший бит в двоичном представлении и определить знак числа: «плюс», если старший бит равен 1, «минус», если старший бит равен 0.
- 2) Отделить двоичное представление смещенного порядка (8 бит) – для денормализованного числа должен быть равен 0000 0000₂.
- 3) Порядок искомого числа должен быть равен –127.
- 4) Выделяем мантиссу, двоичной запятой отделяем крайний слева бит (ноль или единицу)
- 5) Переводим полученную мантиссу в десятичный вид.
- 6) Записываем ответ по формуле (2.3).

Заметим, что алгоритм 9 отличается от алгоритма 8 тем, что скрытая единица не добавляется.

Алгоритм 10. Перевод машинного представления произвольного вещественного числа одинарной точности в десятичный вид (см. рис. 2.5).

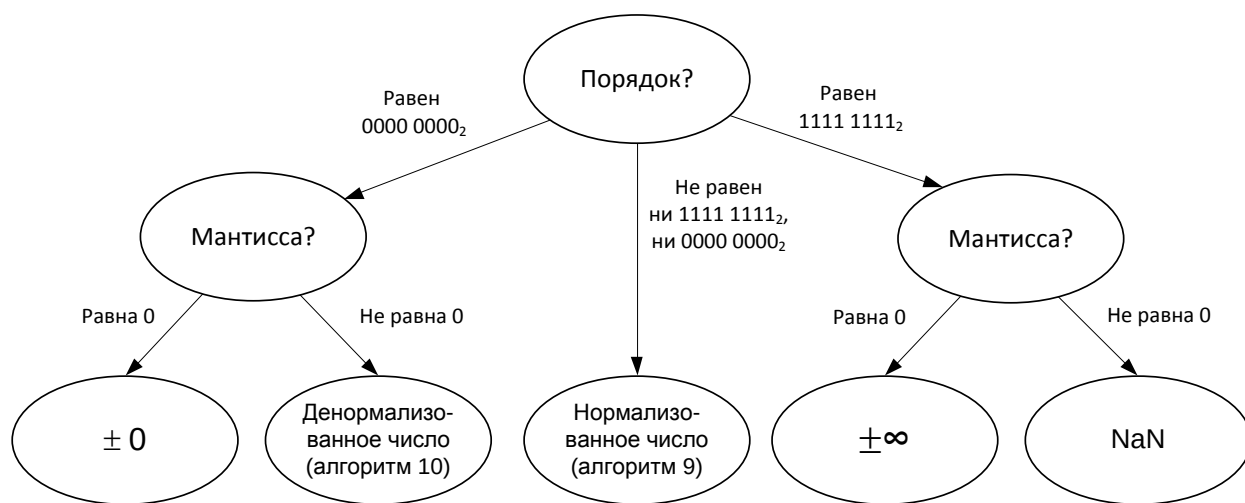


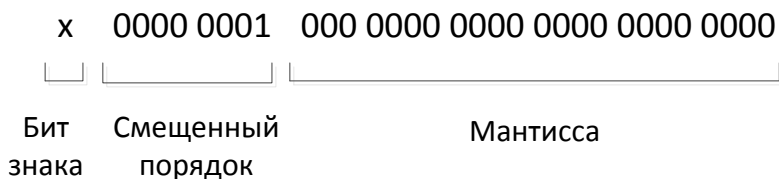
Рис. 2.5. Алгоритм 10 – перевод машинного представления произвольного вещественного числа одинарной точности в десятичный вид

Диапазоны типов данных

Сразу же выделим принципиальное отличие машинного представления целых и вещественных чисел: если мы указываем диапазон целых чисел, то подразумеваем, что *все* числа из данного диапазона имеют свое машинное представление. В то же время, как известно, любой диапазон вещественных чисел содержит бесконечное множество значений, тогда как любая ячейка памяти – конечна и поэтому не может представлять бесконечное множество значений. Таким образом, в машинный диапазон вещественных чисел входят *не все* числа, а только их конечное подмножество.

Для определения диапазона вещественных чисел нужно найти минимальные и максимальные (по модулю) нормализованные и денормализованные числа.

1) Минимальное нормализованное число:

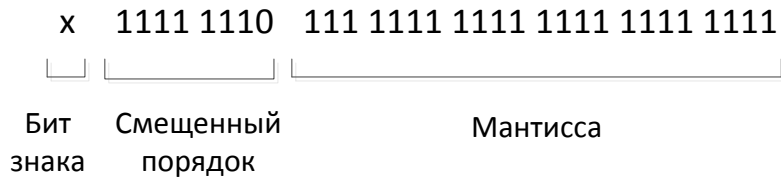


В шестнадцатеричном виде (знак «+»): 0080 0000.

Находим значение этого числа по алгоритму 8:

$$1,0 \cdot 2^{1-127} = 1,0 \cdot 2^{-126} = 1,17549435 \cdot 10^{-38}.$$

2) Максимальное нормализованное число:



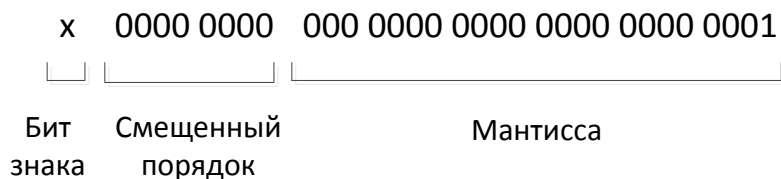
В шестнадцатеричном виде (знак «+»): 7F7F FFFF.

Значение этого числа:

$$1,111\ 1111\ 1111\ 1111\ 1111\ 1111_2 \cdot 2^{254-127} =$$

$$= 1,99999988_{10} \cdot 2^{127} = 3,40282347 \cdot 10^{+38}.$$

3) Минимальное денормализованное число:

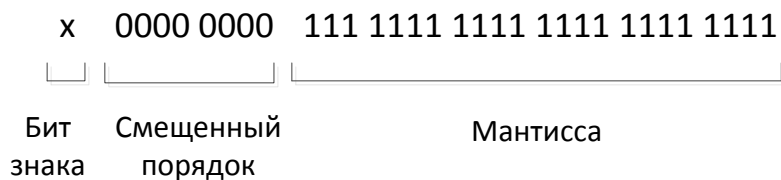


В шестнадцатеричном виде (знак «+»): 0000 0001.

Находим значение этого числа по алгоритму 9 для денормализованных чисел:

$$0,00\ 0000\ 0000\ 0000\ 0000\ 0001_2 \cdot 2^{-127} = 1,0 \cdot 2^{-22} \cdot 2^{-127} = 1.40129846 \cdot 10^{-45}.$$

4) Максимальное денормализованное число:



В шестнадцатеричном виде (знак «+»): 007F FFFF.

Находим значение числа:

$$1,11\ 1111\ 1111\ 1111\ 1111\ 1111_2 \cdot 2^{-127} = 1,99999976 \cdot 2^{-127} = 1.17549421 \cdot 10^{-38}.$$

Таким образом, диапазон вещественных чисел одинарной точности можно представить следующей схемой:

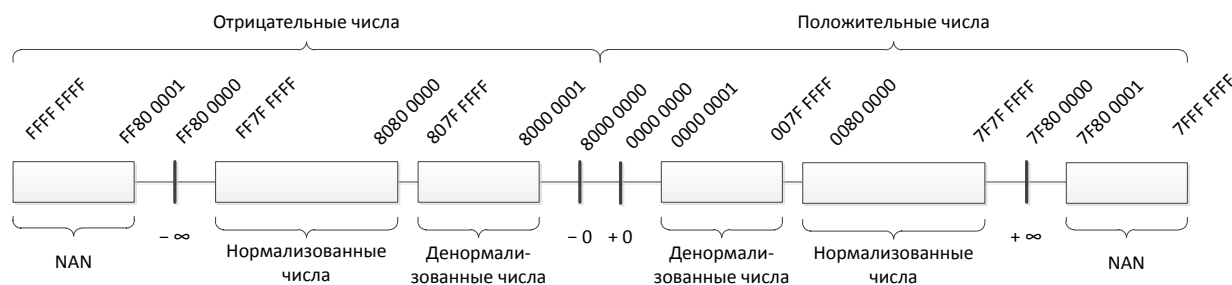


Рис. 2.6. Диапазон вещественных чисел с одинарной точностью

На рис. 2.6 можно увидеть, что если результат некоторой операции получится по модулю меньше, чем минимальное денормализованное число (но больше нуля), то в машинном формате представить его будет невозможно и вместо правильного ответа будет ноль. Такая ситуация называется *потерей значимости*. При операциях с целыми типами такая ситуация невозможна.

Отметим ещё одну особенность формата вещественных чисел – *неравномерность расположения* на числовой прямой. Числа, которые могут быть представлены в машинном формате, меньше по модулю, располагаются чаще, чем большие по модулю.

В заключение приведем диапазоны (только положительные числа) для основных вещественных типов, используемых в Borland Delphi и PascalABC.Net (табл. 2.4).

Таблица 2.4. Вещественные типы данных в Borland Delphi и PascalABC.Net

Тип данных	Разрядность, байт	Диапазон
Single	4	$1,4 \cdot 10^{-45} \dots 3,4 \cdot 10^{+38}$
Real, Double	8	$5,0 \cdot 10^{-324} \dots 1,7 \cdot 10^{+308}$

2.2.4. Представление текстовой информации

ASCII

В ячейки компьютерной памяти можно записывать только двоичные числа, следовательно, для хранения текстовой информации (как и любой другой) нужен способ, позволяющий преобразовывать (кодировать) символы в двоичное представление и обратно. Такой способ называется *кодировкой*.

Обычный подход в этом случае – составить таблицу кодировки, в которой назначить каждому символу определенный код. Тогда компьютер при

необходимости преобразования «символ → код» и обратно обращается к этой таблице.

Самая известная таблица кодировки, реализованная в большинстве компьютеров, называется *ASCII* (*American Standard Code for Information Interchange* – американский стандартный код для обмена информацией).

Это 7-битовый двоичный код, в котором представляются алфавитные, цифровые и специальные символы. Общее количество представимых символов:

$$2^7 = 128.$$

Символы в коде ASCII делятся на *управляющие* (десятичные коды 0..31, 127) и *печатаемые* (десятичные коды 32..126). Управляющие символы не отображаются на экране, а управляют выводом текста, например, символ с кодом 9 обозначает табуляцию, а символ с кодом 10 – перевод строки. Печатаемые символы (табл. 2.5) отображаются при выводе на экран или принтер.

Таблица 2.5. Печатаемые символы в кодировке ASCII (коды 32..127)
(Dec – десятичный код, Hex – шестнадцатеричный код)

Dec	Hex	Символ	Dec	Hex	Символ	Dec	Hex	Символ
32	20	пробел	64	40	@	96	60	'
33	21	!	65	41	A	97	61	a
34	22	“	66	42	B	98	62	b
35	23	#	67	43	C	99	63	c
36	24	\$	68	44	D	100	64	d
37	25	%	69	45	E	101	65	e
38	26	&	70	46	F	102	66	f
39	27	`	71	47	G	103	67	g
40	28	(	72	48	H	104	68	h
41	29	)	73	49	I	105	69	i
42	2A	*	74	4A	J	106	6A	j
43	2B	+	75	4B	K	107	6B	к
44	2C	,	76	4C	L	108	6C	l
45	2D	–	77	4D	M	109	6D	m
46	2E	.	78	4E	N	110	6E	n
47	2F	/	79	4F	O	111	6F	o
48	30	0	80	50	P	112	70	p
49	31	1	81	51	Q	113	71	q
50	32	2	82	52	R	114	72	r
51	33	3	83	53	S	115	73	s
52	34	4	84	54	T	116	74	t
53	35	5	85	55	U	117	75	u
54	36	6	86	56	V	118	76	v
55	37	7	87	57	W	119	77	w
56	38	8	88	58	X	120	78	x
57	39	9	89	59	Y	121	79	y

Dec	Hex	Символ	Dec	Hex	Символ	Dec	Hex	Символ
58	3A	:	90	5A	Z	122	7A	z
59	3B	;	91	5B	[	123	7B	{
60	3C	<	92	5C	\	124	7C	
61	3D	=	93	5D	]	125	7D	}
62	3E	>	94	5E	^	126	7E	~
63	3F	?	95	5F	_	127	7F	

Поскольку таблица ASCII 7-битовая, а минимальная адресуемая ячейка памяти имеет размер 1 байт, остается незадействованным 1 бит (или 128 символов из 256 доступных в байте). Поэтому коды 128..255 были выделены для национальных символов и алфавитов, а также для символов псевдографики. В этом случае таблица ASCII называется *расширенной*.

Например, для кириллицы существуют несколько вариантов расширенных таблиц – KOI-8R, Windows-1251, CP-866, ISO 8859-5. В каждой из них первые 128 символов совпадают (см. табл. 2.5).

Unicode

Таблицы кодировки, использующие один байт, не вмещают все необходимые символы (например, иероглифы, арабский алфавит, математические символы и др.). В 1991 году был опубликован Unicode – стандарт кодировки, использующий для представления одного символа более одного байта и позволяющий закодировать все широко используемые алфавиты в мире. В 2011 году вышла 6-я версия стандарта¹.

В настоящее время символ в Unicode может быть представлен ячейкой памяти от 1 до 4 байт. Например, если это буква латинского алфавита или цифра, то на её представление отводится 1 байт, как и в ASCII-кодировке. Для кириллической буквы будет выделено два байта, для китайского иероглифа – 3 байта, а музыкальные символы будут представлены 4 байтами. Всего в действующем стандарте Unicode может быть представлено 1 112 064 символов².

Обозначаются коды Unicode следующим образом:

U+<шестнадцатеричный код символа>.

Например, код U+0041 обозначает заглавную латинскую букву А (десятичный код 65, совпадает с ASCII), а код U+0410 отвечает за кириллическую букву А. По адресу <http://www.unicode.org/charts/> можно найти код любого символа, представленного в Unicode и, наоборот, по коду определить символ. Также в операционной системе Windows имеется стандартное приложение «Таблица символов», которое позволяет просмотреть и скопировать широко используемые Unicode-символы.

¹ См. <http://unicode.org/>

² Это количество ($2^{20} + 2^{16} - 2048$) можно представить в одном из форматов Unicode – UTF-16.

2.2.5. Представление графической информации

Способы представления графической информации

Существует три основных способа представления графических изображений: растровый, векторный и фрактальный. Растровые форматы хорошо подходят для изображений со сложными гаммами цветов, оттенков и форм (фотографии, рисунки). Векторные форматы более приспособлены для схем, чертежей и простых изображений. Фрактальная графика применяется для создания сложных изображений, структура которых описывается при помощи математических выражений.

В *растровых форматах* изображение представляется прямоугольной матрицей точек – *пикселей*¹ (*pixel* – picture element), положение которых в матрице соответствует координатам точек на экране. Помимо координат каждый пиксель характеризуется своим цветом, цветом фона или градацией яркости. Количество битов, выделяемых для указания цвета пикселя, изменяется в зависимости от формата.

Пример. В высококачественных изображениях (*TrueColor*) цвет пикселя описывают 24 битами, что дает $2^{24} = 16\,777\,216$ цветов.



Рис. 2.7. Пример исходного (32×32 пикселя) и увеличенного в 7 раз растрового изображения

Таким образом, растровое изображение характеризуется следующими параметрами:

- размер матрицы пикселей – *разрешение*;
- количество битов для хранения цвета или количество представимых цветов – *глубина цвета*;
- используемая *цветовая модель*, например, RGB или CMYK (см. следующий параграф).

¹ Существует вариант произношения без мягкого знака – «пиксел». По поводу того, как правильнее, см. обзор <http://yuriy-apostol.livejournal.com/10097.html>.

Основным достоинством растровой графики является высокая точность и возможность отображения малейших нюансов любого изображения – фото, художественной картины, чертежа, графика и т. д.

Недостатки растровой графики заключаются, во-первых, в большой емкости памяти, требуемой для хранения изображения, во-вторых, при масштабировании (увеличении) может проявляться ухудшение четкости, размытость изображения.

Чтобы преодолеть первый недостаток, часто применяют сжатие растровой графики, которое бывает двух видов – без потерь и с потерями. При сжатии без потерь применяются специальные алгоритмы, которые позволяют при хранении уменьшать размер графического файла, а для представления на экране – восстанавливать изображение в исходном виде без искажений (без потерь). В случае сжатия с потерями возможно гораздо значительнее уменьшить размер файла, но при отображении часть информации будет потеряна и качество изображения несколько ухудшится.

В настоящее время существует множество форматов графических файлов, различающихся алгоритмами сжатия и способами представления растровых изображений, а также сферой применения. Некоторые из распространенных форматов растровых графических файлов перечислены в табл. 2.6.

Таблица 2.6. Форматы файлов растровой графики

Формат	Полное название	Глубина цвета	Сжатие	Применение
BMP	Bitmap Picture	1–6 байт	Без потерь	ОС Windows
GIF	Graphics Interchange Format	1 байт (256 цветов)	Без потерь	Интернет, анимация
PNG	Portable Network Graphics	1–6 байт	Без потерь	Интернет
JPEG	Joint Photographic Experts Group	1–3 байта	С потерями	Фотографии
TIFF	Tagged Image File Format	1–8 байт	Без потерь и с потерями	Фотографии, сканеры, издательские системы
PSD	Photoshop Document	1–6 байт	Без потерь	Adobe Photoshop

Для создания и редактирования растровых изображений используются специальные программы – растровые графические редакторы. В качестве примеров можно назвать: Microsoft Paint (входящий в стандартную поставку Windows), Adobe Photoshop, GIMP, Paint.NET, Corel PaintShop Photo и др.

Векторное представление, в отличие от растровой графики, определяет описание изображения не пикселями, а *графическими примитивами* – прямыми, окружностями (эллипсами), многоугольниками, сплайнами.

Сплайн – это гладкая кривая, которая проходит через две или более опорные точки, управляющие формой сплайна. В векторной графике наиболее распространены сплайны на основе кривых Безье. Суть сплайна: любую элементарную кривую можно построить, зная четыре коэффициента P_0 , P_1 , P_2 и P_3 , соответствующие четырем точкам на плоскости (рис. 2.8). Перемещение этих точек влечет за собой изменение формы кривой.

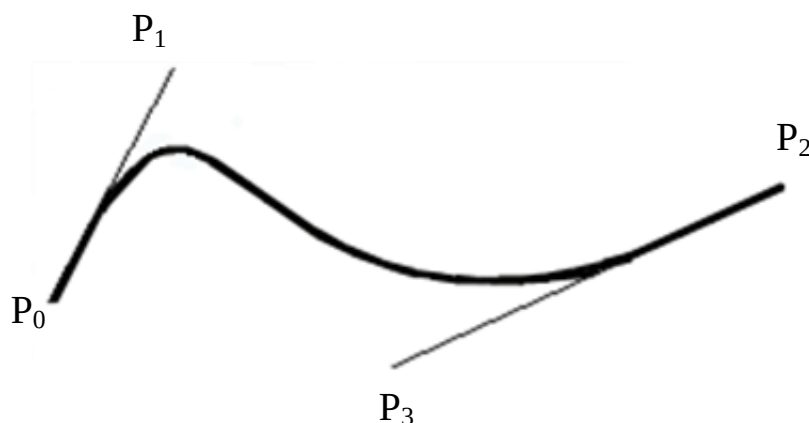


Рис. 2.8. Вариант сплайна

Пример. Для представления в растровом формате отрезка прямой из 100 пикселей с глубиной цвета 3 байта понадобится $100 \cdot 3 = 300$ байт; та же прямая в векторном формате будет описана координатами начала и конца отрезка, параметрами его толщины и цвета. Если каждая координата занимает 2 байта, толщина – 1 байт и цвет – 3 байта, то вся прямая будет храниться в $4 \cdot 2 + 1 + 3 = 12$ байтах.

Достоинства векторной графики:

- описание объекта является простым и занимает мало памяти;
- простота масштабирования изображения без ухудшения его качества.

Недостатком векторного формата является высокая сложность представления сложных изображений, например фотографий, полутоновых рисунков и т. д.

Как и для растровой графики, существует несколько форматов графических векторных файлов (табл. 2.7).

Таблица 2.7. Форматы файлов векторной графики

Формат	Полное название	Применение
AI	Adobe Illustrator	Adobe Illustrator
CDR	Corel Drawing	CorelDRAW
DXF	Drawing Exchange Format	Обмен информацией между САПР ¹
SVG	Scalable Vector Graphics	Открытый формат
VSD	Microsoft Visio format	Microsoft Office Visio

Работа с векторной графикой осуществляется при помощи векторных редакторов: CorelDRAW, Adobe Illustrator, AutoCAD, Microsoft Office Visio, OpenOffice.org Draw, Inkscape и др.

Фрактал – это бесконечно самоподобная геометрическая фигура, каждый фрагмент которой повторяется при уменьшении масштаба. Изображения во фрактальной графике создаются путем математических расчетов, но в отличие от векторной графики в памяти компьютера геометрические объекты не хранятся, а изображения строятся исключительно на основе уравнений. Поэтому ничего, кроме формулы, хранить не надо. Изменив коэффициенты в уравнении, можно получить совершенно другую картину (рис. 2.9).



Рис. 2.9. Пример фрактала – множество Жюлиа

Фракталы применяются в различных областях компьютерной графики: для изображения облаков, снега, береговых линий, деревьев и кустов, для создания текстур и фоновых изображений, в компьютерных играх.

Построенные фрактальные изображения обычно сохраняются в одном из форматов растровой графики – BMP, JPG или PNG.

¹ САПР – системы автоматизированного проектирования (AutoCAD, T-FLEX CAD, OrCAD, КОМПАС).

Цветовые модели

Цветовая модель – это способ представления информации о цвете. Цветовые модели основаны на законах Грассмана¹.

Закон трехмерности: любой цвет может быть выражен посредством трех линейно независимых цветов. Линейная независимость цвета означает, что его нельзя представить с помощью двух других цветов.

Закон непрерывности: при непрерывном изменении составляющих цвет меняется также непрерывно. Для любого цвета существует бесконечно близкий к нему цвет.

По закону трехмерности для построения цветовой модели нужно выбрать три независимых или базовых цвета. Рассмотрим две наиболее часто применяющиеся в информатике цветовые модели, в которых выбор базовых цветов осуществлен по-разному.

Модель RGB (Red, Green, Blue) – в качестве базовых выбраны красный, зеленый и синий цвета. В этой модели любой цвет получается смешением трех базовых цветов. Используется модель RGB при выводе изображений на экран монитора, а также в большинстве форматов графических файлов.

Для хранения в компьютерной памяти каждого цвета может быть использовано разное количество бит (глубина цвета). Рассмотрим глубину цвета TrueColor, в которой каждый основной цвет представлен одним байтом (диапазон 0..255) (табл. 2.8). В модели RGB цвет обозначается набором шестнадцатеричных цифр, например если оранжевый цвет имеет красную составляющую 255, зеленую – 128 и синюю – 0, то его представление в шестнадцатеричном виде будет следующим: FF 80 00.

Таблица 2.8. Представление разных цветов в модели RGB

Цвет	Основные цвета			Обозначение (HEX)
	Red	Green	Blue	
Черный (Black)	0	0	0	00 00 00
Красный (Red)	255	0	0	FF 00 00
Оранжевый (Orange)	255	128	0	FF 80 00
Желтый (Yellow)	255	255	0	FF FF 00
Зеленый (Green)	0	255	0	00 FF 00
Голубой (Sky Blue)	0	191	255	00 BF FF
Циан, или сине-зеленый (Cyan)	0	255	255	00 FF FF
Синий (Blue)	0	0	255	00 00 FF

¹ Грассман, Герман Гюнтер (1809–1877) – немецкий физик, математик и филолог.

Цвет	Основные цвета			Обозначение (HEX)
	Red	Green	Blue	
Фиолетовый (Violet)	128	0	255	80 00 FF
Пурпурный (Magenta)	255	0	255	FF 00 FF
Белый (White)	255	255	255	FF FF FF

В модели *СМУК* (Cyan, Magenta, Yellow, black) в качестве основных используются цвета циан (сине-зеленый), пурпурный, желтый и черный. Применяется модель *СМУК* в цветных принтерах, а также в издательской деятельности.

Вывод цветного изображения на экран монитора и печать на принтере принципиально отличаются. Дело в том, что в мониторе за вывод каждого пикселя отвечают три элемента красного, зеленого и синего цветов, которые излучают собственный свет; таким образом, чтобы получить некоторый цвет, нужно включить эти элементы с соответствующей интенсивностью. Поэтому модель *RGB* называют *аддитивной* (англ. *add* – добавлять).

С другой стороны, при печати на бумаге краска не излучает сама, а поглощает часть света (обычно белого, т. е. включающего все остальные цвета), и мы видим отраженную часть света. Таким образом, чтобы напечатать красным цветом, краска должна поглотить зеленую и синюю составляющие, поэтому модель *СМУК* называется *субтрактивной* (англ. *subtract* – вычитать). Базовые цвета в модели *СМУК* выбраны так, чтобы поглощать свет определенных цветов – циан поглощает красный свет, пурпурная краска поглощает зеленый, а желтая – синий. Чтобы поглотить свет всех цветов, т. е. получить черный цвет, теоретически достаточно смешать все три краски – циановую, пурпурную и желтую. Однако реальные краски неидеальны, и вместо черного получается темно-серый цвет. Поэтому в модель добавлена черная краска.

Конкретный цвет в модели *СМУК* описывается 4 числами, от 0 до 100, которые означают степень яркости соответствующей краски (табл. 2.9).

Таблица 2.9. Представление разных цветов в модели *СМУК*

Цвет	Основные цвета			
	Cyan	Yellow	Magenta	Black
Черный (Black)	0	0	0	100
Красный (Red)	0	100	100	0
Оранжевый (Orange)	0	65	95	0
Желтый (Yellow)	0	0	100	0

Цвет	Основные цвета			
	Cyan	Yellow	Magenta	Black
Зеленый (Green)	100	50	100	0
Голубой (Sky Blue)	70	0	0	0
Циан или сине-зеленый (Cyan)	100	0	0	0
Синий (Blue)	100	100	0	0
Фиолетовый (Violet)	93	94	0	0
Пурпурный (Magenta)	0	0	100	0
Белый (White)	0	0	0	0

Кроме рассмотренных моделей RGB и CMYK существует ещё несколько моделей: XYZ, HSB, YUV и др.

2.2.6. Представление звуковой информации

Понятие аудио связано со звуками, которые способно воспринимать человеческое ухо. Частоты аудиосигналов лежат в диапазоне от 20 Гц до 20 кГц, а сигналы по своей природе являются непрерывными (аналоговыми). Прежде чем быть представленной в компьютере, аудиоинформация должна быть преобразована в цифровую форму (оцифрована) с использованием временной дискретизации и квантования сигнала (рис. 2.10). Для этого значения звуковых сигналов (*выборки, samples*), взятые через малые интервалы времени (временная дискретизация), с помощью аналого-цифровых преобразователей (АЦП) переводятся в двоичный код (квантование сигнала). Обратное действие выполняется цифро-аналоговыми преобразователями (ЦАП). Чем чаще производятся выборки, т. е. чем больше *частота дискретизации*, тем выше может быть точность последующего воспроизведения исходного сигнала, но тем большая емкость памяти требуется для хранения оцифрованного звука.

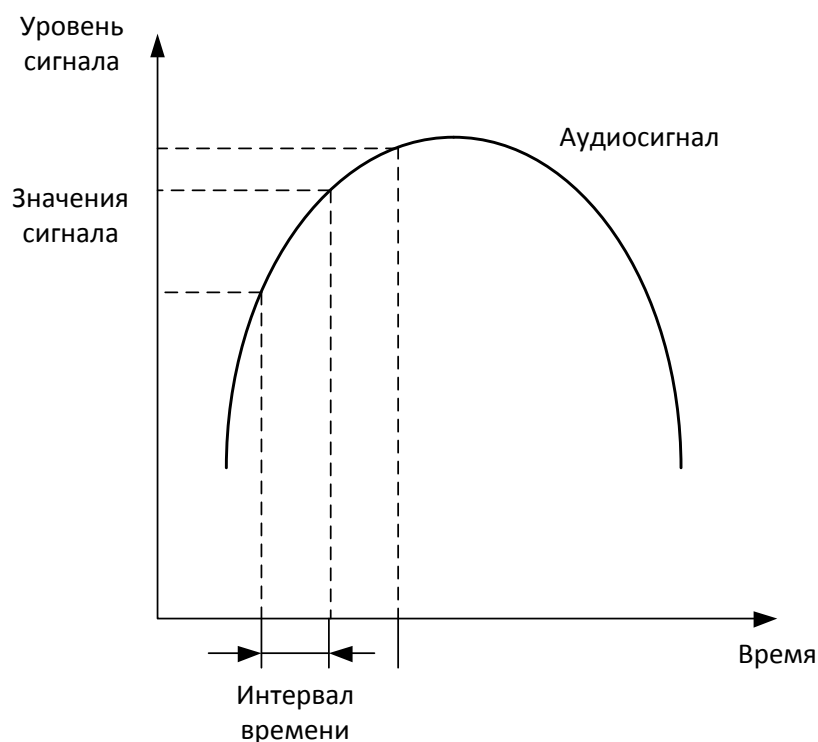


Рис. 2.10. Процесс оцифровки аудиосигнала

Для запоминания уровня каждого сигнала отводится ячейка памяти n бит, размер которой определяет *глубину кодирования звука* (n в разных аудиоформатах меняется от 8 до 32). Тогда количество возможных уровней будет 2^n и это число конечно, в отличие от количества значений уровня реального звука (аудиосигнал непрерывен). Таким образом, вследствие дискретного измерения по времени и по уровню оцифрованный звук всегда имеет худшее качество, чем оригинальный аудиосигнал. Однако на практике стараются выбирать частоту дискретизации и размер ячейки памяти настолько большими, чтобы человек не в состоянии был заметить присутствующие искажения.

По *теореме Найквиста – Котельникова* частоту дискретизации следует выбирать не менее чем в два раза больше, чем максимальная частота измеряемого сигнала.

Пример. Для записи Audio CD (аудио компакт-дисков) используется частота дискретизации 44,1 кГц, поскольку она более чем в два раза превосходит максимальную частоту, воспринимаемую человеческим ухом.

Цифровой эквивалент аудиосигналов обычно хранится в виде файлов, причем широко используются различные методы сжатия такой информации. Как правило, к методам сжатия аудиоинформации предъявляется требование возможности восстановления непрерывного сигнала без заметного ухудшения его качества. В настоящее время распространен целый ряд форматов хранения аудиоинформации. Некоторые из них перечислены в табл. 2.10.

Таблица 2.10. Форматы аудиофайлов

Формат	Полное название	Применение
WAV	WAVE («волна»)	Простое хранение оцифрованного звука без сжатия
WMA	Windows Media Audio	Закрытый формат аудиофайлов от Microsoft; использует сжатие с потерями
AAC	Advanced Audio Coding	Формат аудиофайлов с эффективным сжатием с потерями
OGG	Ogg	Формат аудиофайлов со сжатием с потерями
MIDI	Musical Instrument Digital Interface	Формат для обмена данными между электронными музыкальными инструментами
MP3	Moving Picture Experts Group (MPEG) Layer 3	Один из самых популярных форматов аудиофайлов со сжатием с потерями

2.3. Булева алгебра

Для построения и анализа цифровых схем, в которых используются только два сигнала, применяется особый тип алгебры.

Булева алгебра (алгебра логики) – алгебра, в которой все переменные и функции могут принимать только два значения – 0 и 1.

Её основатель – Джордж Буль (1815–1864). Создавая свою алгебру, он хотел формализовать законы человеческого мышления.

В булевой алгебре есть свои функции, называемые булевыми функциями, которые имеют одну или несколько переменных и значение которых зависит только от значения этих переменных. Булева функция может быть описана формулой или таблицей истинности. Перечислим основные булевы функции от двух переменных с помощью формул и таблиц истинности:

1. *Инверсия, отрицание, НЕ, NOT*: применяется к одной переменной и возвращает её противоположное значение.

Обозначения: $f(x) = \text{NOT } x$; $f(x) = \bar{x}$; $f(x) = \sim x$; $f(x) = \neg x$.

Таблица истинности:

x	$f(x)$
0	1
1	0

2. *Конъюнкция, логическое умножение, И, AND*: применяется к двум переменным, дает в результате 1, только если обе переменные были равны 1.

Обозначения: $f(x, y) = x \text{ AND } y$; $f(x, y) = x \wedge y$; $f(x, y) = x \cdot y$.

Таблица истинности:

x	y	$f(x, y)$
0	0	0
0	1	0
1	0	0
1	1	1

3. *Дизъюнкция, логическое сложение, ИЛИ, OR*: применяется к двум переменным, дает в результате 1, если хотя бы одна переменная была равна 1.

Обозначения: $f(x, y) = x \text{ OR } y$; $f(x, y) = x \vee y$; $f(x, y) = x + y$.

Таблица истинности:

x	y	$f(x, y)$
0	0	0
0	1	1
1	0	1
1	1	1

4. *Сумма по модулю 2, исключающее ИЛИ, XOR (eXclusive OR)*: применяется к двум переменным, дает в результате 1, если переменные не были равны друг другу.

Обозначения: $f(x, y) = x \text{ XOR } y$; $f(x, y) = x \oplus y$.

Таблица истинности:

x	y	$f(x, y)$
0	0	0
0	1	1
1	0	1
1	1	0

Если из контекста ясно, что речь идет только о логических функциях, то для записи логического сложения можно использовать знак «+», а для логического умножения – знак «•». Однако если в задаче участвуют как логические, так и арифметические функции (например, при рассмотрении арифметико-логического устройства), то для записи логических функций используются только символы « $\vee$ », « $\wedge$ », « $\oplus$ » или обозначения AND, OR, XOR (И, ИЛИ, Искл. ИЛИ).

Некоторые соотношения алгебры логики:

Законы одинарных элементов	$x + 0 = x, \quad x + 1 = 1, \quad x \cdot 0 = 0, \quad x \cdot 1 = x$
Законы идемпотентности	$x + x = x, \quad x \cdot x = x$
Законы инверсии	$x + \bar{x} = 1, \quad x \cdot \bar{x} = 0$
Закон двойного отрицания	$\bar{\bar{x}} = x$
Законы де Моргана ¹	$\overline{x + y} = \bar{x} \cdot \bar{y}, \quad \overline{x \cdot y} = \bar{x} + \bar{y}$
Свойство XOR	$x \oplus y = x \cdot \bar{y} + \bar{x} \cdot y$

Представление булевых функций с помощью таблиц истинности становится громоздким при возрастании количества переменных, поэтому часто используется формульный способ задания функции.

Любую булеву функцию можно определить, указав, какие комбинации значений переменных дают значение функции 1.

Пример. Привести формульную запись булевой функции от трех переменных.

x	y	z	$f(x, y, z)$
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

При записи первой единицы в столбце значений функции учтем, что эта единица получается при нулевых значениях переменных. Если записать логическое произведение переменных с инверсией, то результатом будет как раз единица:

$$\bar{x} \cdot \bar{y} \cdot \bar{z} = 1, \text{ при } x = y = z = 0.$$

¹ Август де Морган (1806–1871) – шотландский математик и логик.

Для второй единицы произведение переменных будет иметь вид:

$$\bar{x} \cdot y \cdot \bar{z} = 1, \text{ при } x = z = 0, y = 1.$$

Аналогично, для третьей единицы получаем:

$$x \cdot y \cdot \bar{z} = 1, \text{ при } x = y = 1, z = 0.$$

Чтобы получить окончательную формулу, логически сложим все три полученных произведения:

$$f(x, y, z) = \bar{x} \cdot \bar{y} \cdot \bar{z} + \bar{x} \cdot y \cdot \bar{z} + x \cdot y \cdot \bar{z}.$$

Теперь можно проверить, что при подстановке любой комбинации переменных из таблицы истинности значение функции будет совпадать с табличным.

Контрольные вопросы и задания

1. Переведите число $3D7_{14}$ из 14-ичной системы счисления в 7-ичную. Результат проверьте обратным переводом.
2. Переведите число $8C4_{12}$ из 12-ичной системы счисления в 9-ичную. Результат проверьте обратным переводом.
3. В ячейке памяти ВМ находится число 1000 1100. Каково десятичное представление этого числа?
4. Объясните, чем определяется диапазон представления данных для вещественных чисел.
5. Законы де Моргана имеют вид: $\overline{x + y} = \bar{x} \cdot \bar{y}$, $\overline{x \cdot y} = \bar{x} + \bar{y}$. Верны ли выражения

$$\overline{x + y + z} = \bar{x} \cdot \bar{y} \cdot \bar{z}, \quad \overline{x \cdot y \cdot z} = \bar{x} + \bar{y} + \bar{z}?$$

Как доказать?

6. * Перевод из произвольной q -ичной системы счисления в p -ичную осуществляется через десятичную систему счисления. А как можно обойтись без этого посредника?
7. * Почему в байте именно 8 бит, а, например, не 4, 10, 16?
8. * Преобразуйте логическое выражение из конъюнктивной формы (т. е. конъюнкции дизъюнктов) в дизъюнктивную форму (дизъюнкцию конъюнктов):

$$x \cdot \bar{y} \cdot z + x \cdot \bar{y} \cdot \bar{z} + \bar{x} \cdot y \cdot \bar{z}$$

(указание: воспользуйтесь законами де Моргана).

9. * *Логически полным базисом* называют набор таких функций, из которых можно построить любые сколь угодно сложные функции. Примером функционального базиса является набор {И, ИЛИ, НЕ}. Докажите, что наборы {И, НЕ}, {ИЛИ, НЕ}, {И-НЕ}, {ИЛИ-НЕ} являются логически полными базисами.

ГЛАВА 3. ЦИФРОВОЙ ЛОГИЧЕСКИЙ УРОВЕНЬ

3.1. Аналоговые и дискретные сигналы

В электрических схемах используются аналоговые (непрерывные) и дискретные (цифровые) сигналы.

Сигнал (от лат. *signum* – знак) – это знак, физический процесс или явление, несущий сообщение (информацию) о каком-либо событии, состоянии объекта наблюдения либо передающий команды управления, оповещения и т. д. Сигнал должен изменяться во времени – только в этом случае он может передавать информацию.

Сигналы по своей природе могут быть¹:

- электрическими (изменение напряжения, тока, мощности),
- световыми (вспышки света),
- электромагнитными (радиоволны),
- звуковыми (голос),
- тепловыми (изменение температуры),
- механическими (деформация, изменение давления) и др.

В компьютерной технике используются чаще всего электрические сигналы, реже световые (в оптоволоконных сетях), электромагнитные (в беспроводных сетях), звуковые (например, динамик на материнской плате), механические (сенсорные панели в ноутбуках) и др.

Если в качестве сигнала применяется изменение напряжения, то уровень электрического сигнала измеряется в соответствующих единицах – *вольтах* (В).

Аналоговый сигнал – это сигнал, непрерывно меняющийся во времени (рис. 3.1). Примеры аналоговых величин – движение стрелки часов, система действительных чисел.

Дискретный сигнал – это сигнал, который может принимать конечное число значений в конечное число моментов времени. Примеры – электронные часы, система целых чисел.

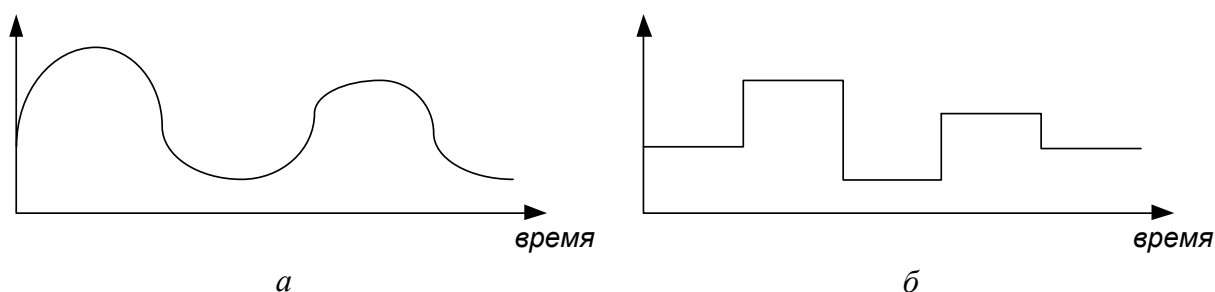


Рис. 3.1. Виды сигналов: *а* – аналоговый сигнал, *б* – дискретный сигнал

¹ Большая советская энциклопедия : в 30 т. / гл. ред. А. М. Прохоров. М. : Сов. энцикл., 1969–1978. Т. 23.

Цифровая схема (логическая схема) – это электрическая схема, в которой есть только два логических значения сигналов – 0 и 1.

Каким образом осуществляется связь логических значений 0 и 1 с реальными значениями уровня сигнала в электрических схемах? Обычно весь диапазон возможных в схеме уровней сигнала (от нуля вольт до напряжения питания) делится на три части (рис. 3.2):

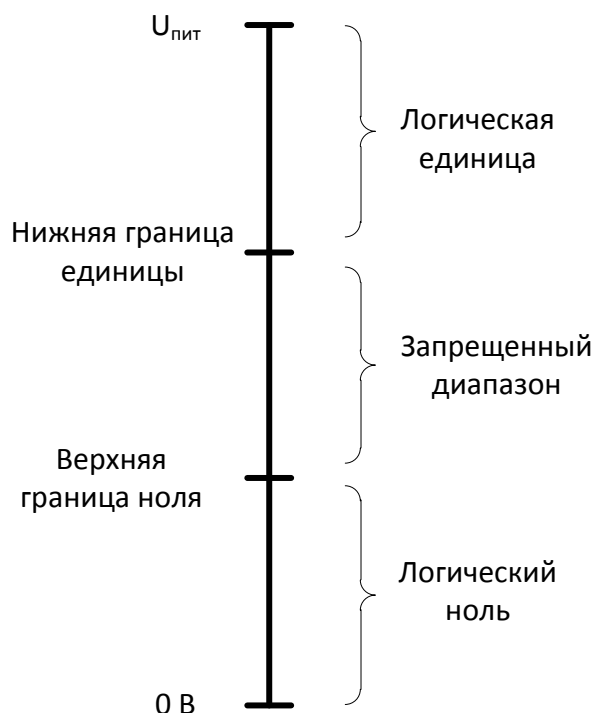


Рис. 3.2. Соответствие логических сигналов реальным электрическим

Нижняя часть диапазона представляет одно логическое значение (например, логический ноль), верхняя часть – другое (например, логическую единицу). Значений напряжения в средней части (в запрещенном диапазоне) стараются избегать. Запрещенный диапазон установлен для четкого разделения логических значений и для исключения влияния помех.

Конкретные значения границ зависят от используемой технологии: например в технологии ТТЛ (транзисторно-транзисторная логика) верхняя граница ноля равна 0,4 В, нижняя граница единицы – 2,4 В. В технологии КМОП (комплементарный металл-оксид-полупроводник) верхняя граница ноля близка к 0 В, нижняя граница единицы близка к напряжению питания.

3.2. Классификация цифровых схем

Все цифровые схемы делятся на два вида – комбинационные схемы (схемы без памяти) и последовательностные схемы (схемы с памятью) (рис. 3.3).



Рис. 3.3. Классификация цифровых схем

Комбинационная схема – схема, выход которой зависит только от входа в текущий момент времени, не зависит от предыдущего состояния и не имеет памяти.

Последовательностная схема – схема, выход которой зависит не только от входа в текущий момент времени, но и от предыдущего состояния, т. е. имеет память. Последовательностные схемы строятся из комбинационных путем введения обратных связей.

Далее будут рассмотрены сначала разные виды комбинационных схем, затем схемы с памятью.

3.3. Комбинационные схемы

3.3.1. Логические вентили

Вся компьютерная техника построена на очень простых элементах, которые называются логическими вентилями. *Логические вентили* – электронные устройства, вычисляющие простейшие булевы функции.

В свою очередь, вентили строятся из транзисторов.

Чтобы понять дальнейший материал, нужно вспомнить из курса физики, что такое проводники, полупроводники и диэлектрики.

Проводники (conductor) – вещества, хорошо проводящие электрический ток благодаря наличию в них большого количества свободных заряженных частиц (пример – металлы).

Диэлектрики (dielectric) – вещества, плохо проводящие электрический ток (примеры – воздух, стекло, резина).

Полупроводники (semiconductor) – вещества, электропроводность которых находится между проводниками и диэлектриками. Важной особенностью полупроводников, является то, что их электропроводность может изменяться в широких пределах под внешним воздействием – электрического поля, света, температуры, деформации и т. д. Именно эта особенность повлияла на повсеместное распространение полупроводников в компьютерной технике. Примеры полупроводников – кремний, германий.

Транзистор – полупроводниковый прибор, имеющий не менее трех выводов, способный изменять выходное напряжение (ток) под действием управляющего напряжения (тока). Транзисторы имеют широкий диапазон применения в электронике, в том числе в качестве усилителей и генераторов сигналов, но в цифровой технике транзисторы применяются чаще всего в качестве *электронных переключателей* или просто *ключей*.

Транзистор имеет три вывода – коллектор, базу и эмиттер (рис. 3.4а). Для упрощения можно рассмотреть схему замещения (рис. 3.4б), на которой

транзистор представляет собой переключатель, подобно выключателю света в комнате, только роль пальца (управляющего воздействия) играет управляющее (входное) напряжение на базе $U_{вх}$, от которого зависит, будет транзистор включен или выключен (рис. 3.5).

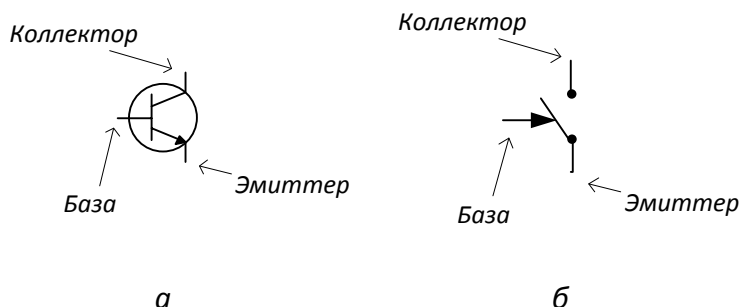


Рис. 3.4. Транзистор: *а* – обозначение на схемах, *б* – схема замещения

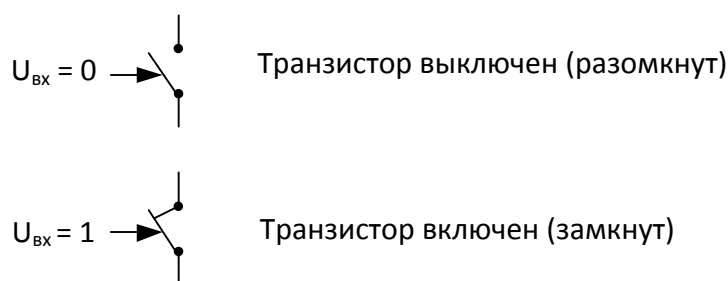


Рис. 3.5. Принцип действия транзистора

Если управляющее напряжение $U_{вх}$ равно логическому нулю, транзистор выключается и действует как очень большое сопротивление. Если $U_{вх}$ становится равно логической единице, транзистор включается и действует как проводник.

Для понимания принципа работы вентилях, рассмотрим рис. 3.6. На нем изображены электронный переключатель и лампочка. При разомкнутом переключателе (рис. 3.6*а*) цепь разомкнута, ток не идет ($I = 0$), лампочка не горит. Все напряжение питания $U_{пит}$ без потерь приходит на выход схемы $U_{вых} = U_{пит} = 1$. Если переключатель замкнуть (рис. 3.6*б*), то пойдет ток ($I = 1$), следовательно, лампочка загорится, и все напряжение будет задействовано на ней. При этом $U_{вых} = 0$.

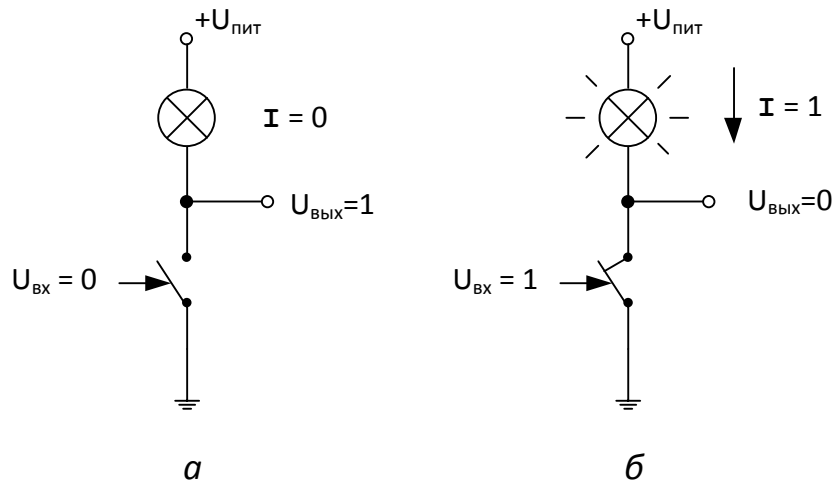


Рис. 3.6. Схема включения лампочки: *а* – переключатель включен; *б* – переключатель выключен

Таким образом, если напряжение $U_{вх}$ высокое, то $U_{вых}$ – низкое, и наоборот, т. е. эта схема реализует булеву функцию НЕ и называется *инвертор*.

Заметим, что выходное напряжение является инверсным не только ко входному напряжению, но и к току: если тока нет, то $U_{вых} = 1$, и наоборот.

На рис. 3.7 вместо переключателя изображен биполярный транзистор, а вместо лампочки – резистор. Резистор нужен для ограничения тока, чтобы транзистор не сгорел. На переключение из одного состояния в другое требуется несколько наносекунд.

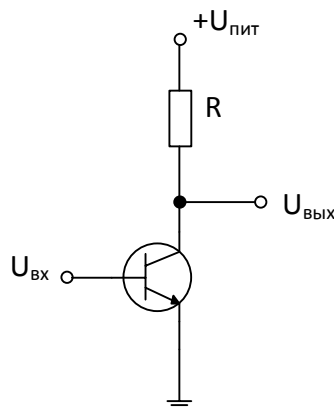


Рис. 3.7. Инвертор

Замечание. Из курса физики известно, что ток может течь только в замкнутой цепи при наличии источника питания. На наших схемах источник питания есть ($+U_{пит}$), но цепь изображена незамкнутой. Почему же ток течет (лампочка горит)?

Дело в том, что при изображении цифровых схем применяют различные упрощения. Например, инвертор на рис. 3.7 можно было бы изобразить так, как показано на рис. 3.8.

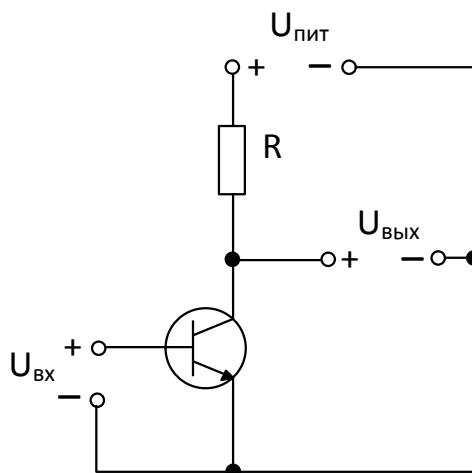


Рис. 3.8. Схема инвертора с обозначением общего провода

Таким образом, обозначения источников питания, входных и выходных сигналов (напряжений) подразумевают наличие у них отрицательных контактов, которые могут быть объединены в *общий провод*. Общий провод (иногда говорят «корпус», «заземление» или «земля») обозначается на схемах так:

Схемы на рис. 3.9 и 3.10 образуют вентили И-НЕ (NOT-AND, или NAND) и ИЛИ-НЕ (NOT-OR, или NOR).

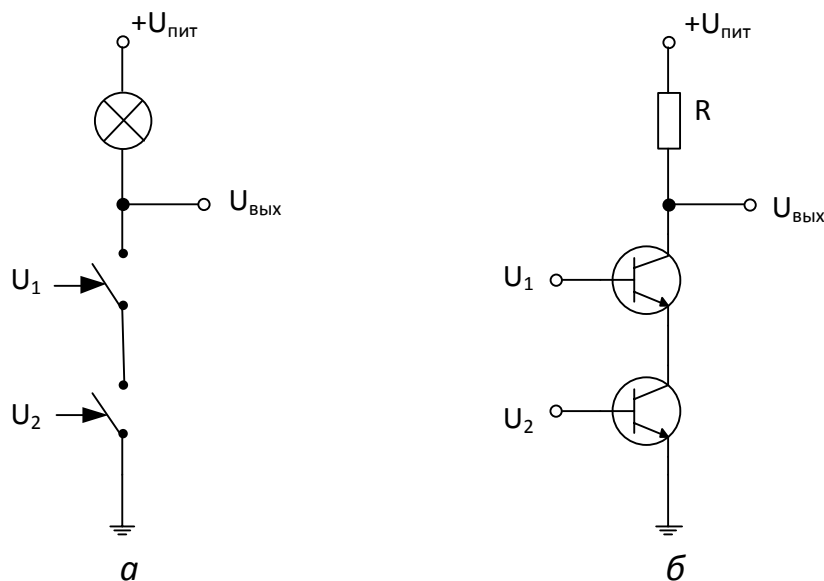


Рис. 3.9. Вентиль И-НЕ: *а* – схема на переключателях;
б – схема на транзисторах

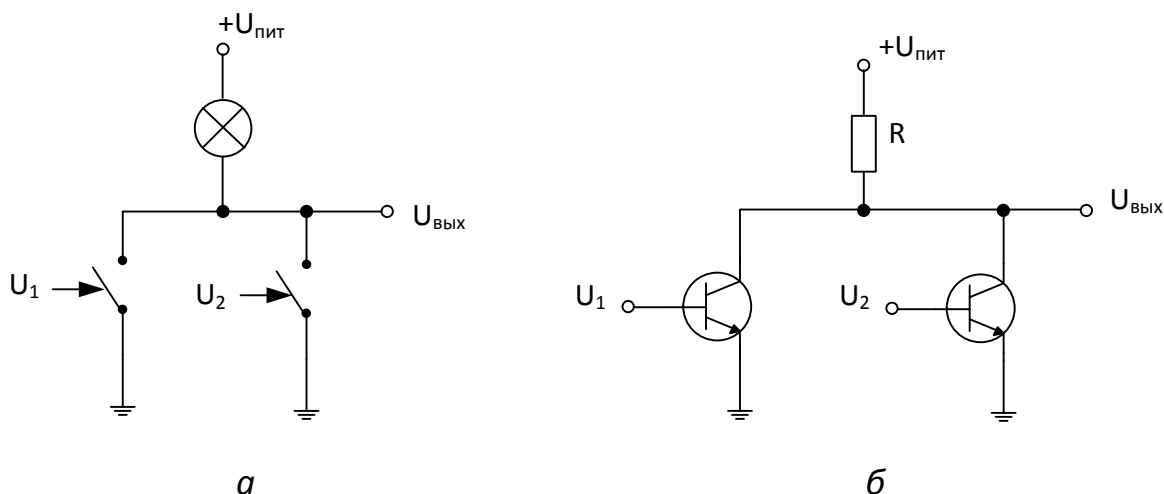


Рис. 3.10. Вентиль ИЛИ-НЕ: *а* – схема на переключателях;
б – схема на транзисторах

Чтобы понять, почему реализуются именно такие функции, можно воспользоваться известным нам правилом, что выходное напряжение является инверсным к току. Проанализируйте, для каких U_1 , U_2 ток течет, а для каких нет.

Задание. Составить таблицы истинности для этих вентилях.

U_1	U_2	$U_{вых}$ (И-НЕ)	$U_{вых}$ (ИЛИ-НЕ)
0	0		
0	1		
1	0		
1	1		

Задание. Нарисовать схемы вентилях AND, OR и XOR.

Обозначения пяти основных вентилях на электрических схемах приведены на рис. 3.11*а* – российский стандарт (ГОСТ 2.743-91) и на рис. 3.11*б* – зарубежный стандарт (ANSI/IEEE Std 91-1984).

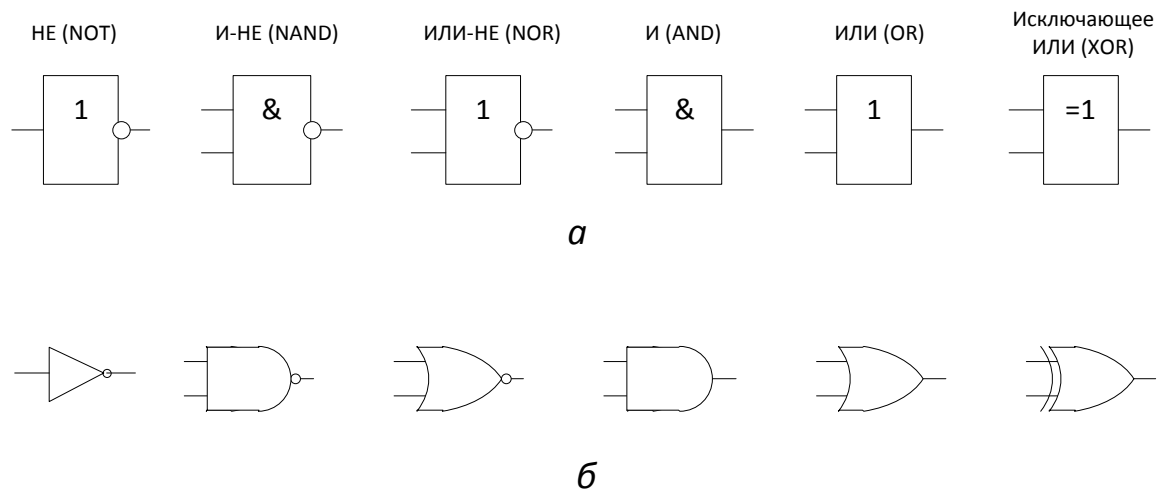


Рис. 3.11. Основные вентили:
 а – российский стандарт; б – зарубежный стандарт

Вентили И и ИЛИ получаются путем подключения инверторов к выходам вентилях И-НЕ и ИЛИ-НЕ. Таким образом, вентили И-НЕ и ИЛИ-НЕ требуют по два транзистора, а вентили И и ИЛИ – по три. Поэтому в большинстве компьютеров используются вентили И-НЕ и ИЛИ-НЕ. (На практике все вентили выполняются несколько сложнее, но вентили И-НЕ и ИЛИ-НЕ все равно проще, чем вентили И и ИЛИ.) Вентили могут иметь более двух входов (до восьми) и в этом случае называются 3И-НЕ, 4И-НЕ и т. д.

3.3.2. Реализация булевых функций

Чтобы реализовать булевы функции на вентилях, используют формульное задание функций.

Пример. Реализовать булеву функцию (см. пример в параграфе 2.3):

$$f(x, y, z) = \bar{x} \cdot \bar{y} \cdot \bar{z} + \bar{x} \cdot y \cdot \bar{z} + x \cdot y \cdot \bar{z}.$$

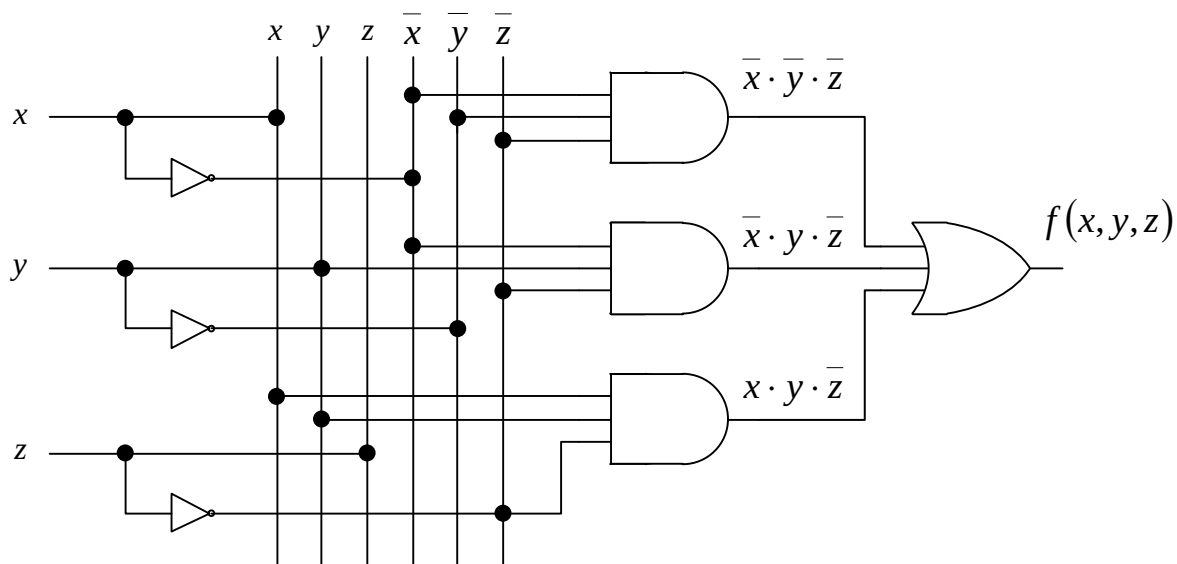


Рис. 3.12. Реализация функции $f(x, y, z)$
(вертикальные линии проведены для удобства изображения)

В примере показана реализация функции с помощью вентилях НЕ, И, ИЛИ. Однако проще строить схемы, используя только один тип вентилях. Вентили И-НЕ и ИЛИ-НЕ считаются *полными*, так как можно построить любую булеву функцию, используя только вентили И-НЕ или только вентили ИЛИ-НЕ.

3.3.3. Использование вентиля AND в качестве ключа

Рассмотрим двухвходовой вентиль AND. Обозначим его входы так, как приведено на рис. 3.13.

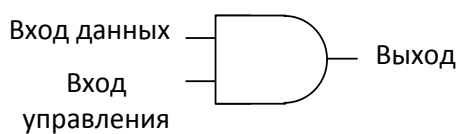


Рис. 3.13. Двухвходовой вентиль AND

Таблица истинности этого элемента:

Вход управления	Вход данных	Выход
0	0	0
0	1	0
1	0	0
1	1	1

Анализируя таблицу, можно заметить, что при *Управлении* = 0 на выходе получается 0, независимо от входа данных, а при *Управлении* = 1 на выходе повторяется вход данных. Таким образом, вентиль AND можно использовать в роли ключа (переключателя), пропуская или не пропуская данные на выход схемы в зависимости от входа управления. Такая возможность окажется очень полезной в нескольких схемах, рассматриваемых далее.

Отметим, что входов у вентиля AND, используемого в качестве ключа, может быть не два, а больше. В этом случае все входы, кроме одного входа данных, будут являться управляющими.

3.3.4. Использование вентиля XOR в качестве управляемого инвертора

По аналогии с предыдущим параграфом рассмотрим двухвходовой вентиль XOR (рис. 3.14).

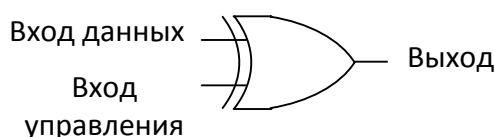


Рис. 3.14. Двухвходовой вентиль XOR

Таблица истинности вентиля XOR:

Вход управления	Вход данных	Выход
0	0	0
0	1	1
1	0	1
1	1	0

При *Управлении* = 0 на выходе повторяются входные данные, а при *Управлении* = 1 выход является инверсным ко входу. Таким образом, элемент XOR при нулевом сигнале управления работает как повторитель, а при единичном – как инвертор, т. е. является *управляемым инвертором*.

3.3.5. Преобразователи кодов

Шифратор (coder, CD) – преобразователь, в котором при подаче сигнала на один из входов на выходе появляется двоичный код этого входа (рис. 3.15).

Если в шифраторе n выходов, то он имеет 2^n входов.

Таблица истинности шифратора:

Входы	Y_1	Y_0
X_0	0	0
X_1	0	1
X_2	1	0
X_3	1	1

По таблице истинности можно вывести логические функции Y_0 и Y_1 :

$$Y_0 = X_1 \vee X_3; \quad Y_1 = X_2 \vee X_3$$

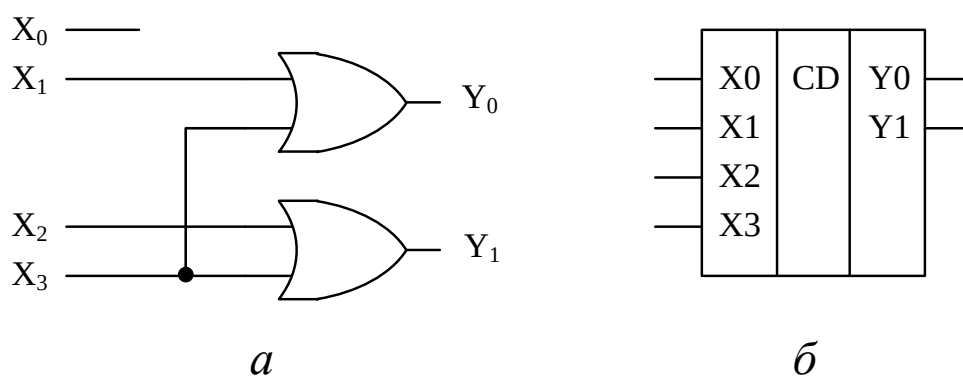


Рис. 3.15. Шифратор 4×2 (4 входа $\times$ 2 выхода):
a – реализация на вентилях; *б* – обозначение на схемах

Дешифратор (decoder, DC) – преобразователь, в котором при подаче двоичного кода на вход на соответствующей выходной линии появляется сигнал (рис. 3.16).

Таблица истинности дешифратора:

Входы		Выходы			
X_1	X_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Логические функции имеют вид:

$$Y_0 = \overline{X_0} \wedge \overline{X_1}; \quad Y_1 = X_0 \wedge \overline{X_1}; \quad Y_2 = \overline{X_0} \wedge X_1; \quad Y_3 = X_0 \wedge X_1.$$

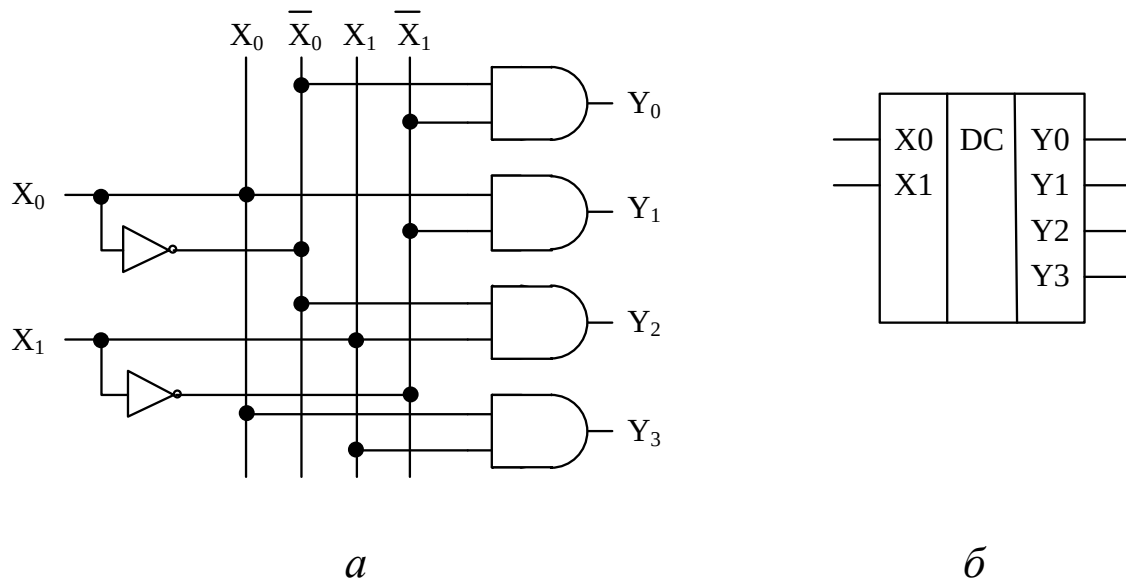


Рис. 3.16. Дешифратор 2×4:
а – реализация на вентилях; б – обозначение на схемах

3.3.6. Коммутаторы

Мультиплексор (multiplexer, MUX) – схема, обеспечивающая передачу информации, поступающей по нескольким входам на один выход. Выбор входной линии осуществляется при помощи адресных входов (рис. 3.17).

Если в мультиплексоре n адресных линий, то максимальное количество входов может быть 2^n .

Четыре вентиля AND используются в этой схеме в качестве ключей. Сигналы на двух нижних входах каждого вентиля AND являются управляющими и определяют, будет ли проходить на выход сигнал данных с верхнего входа. Две адресные линии A_0 и A_1 кодируют 2-битное число (0..3), которое указывает, какая из четырех входных линий данных D_0 – D_3 должна соединяться с выходом F . Каждый вентиль AND запускается определенной комбинацией адресных входов.

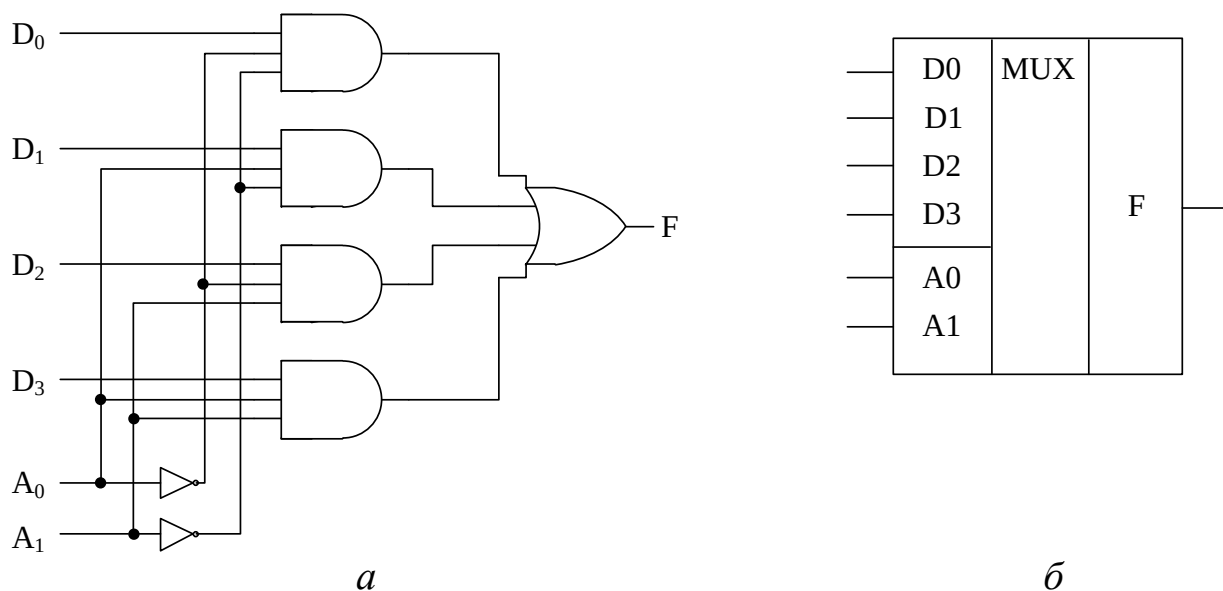


Рис. 3.17. 4-входовой мультиплексор:
a – реализация на вентилях; *б* – обозначение на схемах

Демультимплексор (*demultiplexer, DMX*) – схема, обеспечивающая передачу информации, поступающей по одному входу на один из нескольких выходов. Выбор выходной линии осуществляется при помощи адресных входов (рис. 3.18).

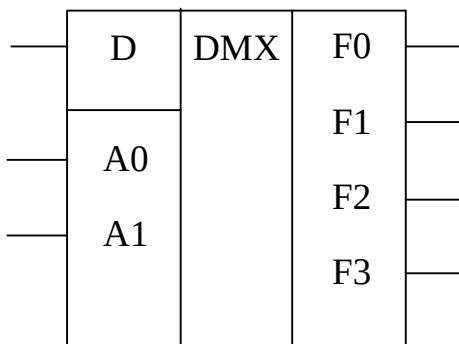


Рис. 3.18. Демультимплексор

Задание. Построить схему демультимплексора из вентилях самостоятельно (аналогично мультиплексору).

3.3.7. Двоичная арифметика

Переполнение и перенос

Основной проблемой при выполнении арифметических операций в компьютере является *переполнение* (*overflow*) – ситуация, когда ответ не помещается в отведенную для него ячейку памяти (выходит за пределы допу-

стимого диапазона). Переполнение нужно строго контролировать, иначе при вычислениях могут появиться неверные ответы.

Ситуации переполнения при арифметических операциях с беззнаковыми и со знаковыми типами данных принципиально отличаются.

За переполнение при операциях с беззнаковыми типами данных в процессоре отвечает специальная ячейка памяти размером 1 бит, которая называется *флаг переноса* (*carry flag, CF*). Если флаг переноса равен 1, значит в результате последней выполненной арифметической операции над беззнаковыми данными произошло переполнение.

Переполнение при операциях со знаковыми типами данных в процессоре контролируется *флагом переполнения* (*overflow flag, OF*)¹. Если флаг переполнения равен 1, то в результате последней выполненной арифметической операции над знаковыми данными произошло переполнение.

Важно понимать, что в случае операций сложения и вычитания для процессора *не имеет значения*, над какими данными производятся эти операции – знаковыми или беззнаковыми. Процессор складывает и вычитает *двоичные числа* по единым правилам, и за счет использования дополнительного кода ответы получаются правильными как в случае беззнаковых типов, так и в случае знаковых.

Поэтому не существует разных машинных команд для сложения/вычитания знаковых и беззнаковых типов, команды в том и другом случае используются одинаковые – ADD (addition) для сложения и SUB (subtract) для вычитания. При этом в процессоре одновременно формируются оба флага CF и OF по разным правилам и программист может контролировать тот или другой флаг в зависимости от того, с каким типом данных в настоящий момент идет работа.

С другой стороны, умножение и деление выполняются по разным правилам для беззнаковых и знаковых данных, поэтому соответствующие машинные команды разные.

Флаг переноса CF отвечает за переполнение при операциях с беззнаковыми типами, *флаг переполнения OF* отвечает за переполнение при операциях со знаковыми типами; вычисляются эти флаги по-разному, одновременно и независимо друг от друга².

¹ Реализуются флаги переноса и переполнения в виде разрядов специального регистра процессора – *регистра флагов* (о регистрах речь в пособии пойдет ниже). Хорошие отладчики в разных средах программирования обычно позволяют просматривать состояния флагов переноса и переполнения.

² Не должно вводить в заблуждение терминологическое несоответствие – флаг переполнения OF, несмотря на название, отвечает только за переполнение при работе со знаковыми типами данных. Флаг переноса CF не всегда равен переносу из старшего разряда и на самом деле отвечает за переполнение при действиях с беззнаковыми типами данных.

Сложение беззнаковых типов данных

Правило 1. При сложении чисел, представленных в беззнаковых типах данных, переполнение возникает в том случае, если есть перенос из старшего разряда результата.

Рассмотрим различные варианты сложения на примере 4-разрядного беззнакового типа с диапазоном [0..15].

Сложение без переполнения (без выхода результата из диапазона):

$$\begin{array}{rcl} (+4) & + & 0\ 1\ 0\ 0 \\ (+3) & + & 0\ 0\ 1\ 1 \\ \hline (+7) & & 0\ 1\ 1\ 1 \\ CF & = & 0 \end{array} \qquad \begin{array}{rcl} (+5) & + & 0\ 1\ 0\ 1 \\ (+10) & + & 1\ 0\ 1\ 0 \\ \hline (+15) & & 1\ 1\ 1\ 1 \\ CF & = & 0 \end{array}$$

Поскольку перенос из старшего разряда результата отсутствует, то флаг переноса CF = 0 и беззнакового переполнения нет, поэтому ответы оказываются верными.

Сложение с переполнением:

$$\begin{array}{rcl} (+4) & + & 0\ 1\ 0\ 0 \\ (+12) & + & 1\ 1\ 0\ 0 \\ \hline (0) & \pm & 0\ 0\ 0\ 0 \\ CF & = & 1 \end{array} \qquad \begin{array}{rcl} (+15) & + & 1\ 1\ 1\ 1 \\ (+15) & + & 1\ 1\ 1\ 1 \\ \hline (+14) & \pm & 1\ 1\ 1\ 0 \\ CF & = & 1 \end{array}$$

В этих примерах перенос из старшего разряда результата есть (зачеркнутая единица), но поскольку мы имеем всего 4 разряда, этот перенос в результате не остается, а уходит во флаг переноса CF. Таким образом, имеет место беззнаковое переполнение, и результаты сложения оказываются неверными (выход результатов за диапазон).

Сложение знаковых типов данных

Правило 2. При сложении чисел, представленных в знаковых типах данных, переполнение возникает в том случае, если складываются два числа с одинаковыми знаками, а знак результата противоположен знаку слагаемых.

Напомним, что за знак отвечает старший бит числа.

Следствие из правила 2: знаковое переполнение не может возникнуть при сложении чисел с разными знаками.

Примеры будем разбирать также в 4-разрядном, но уже знаковом типе данных с диапазоном $[-8..+7]$.

Сложение без переполнения (жирным выделены старшие знаковые разряды чисел):

$$\begin{array}{rclcl}
 (+4) + \mathbf{0}100 & (+7) + \mathbf{0}111 & (-8) + \mathbf{1}000 & (-6) + \mathbf{1}010 \\
 (+3) + \mathbf{0}011 & (-5) + \mathbf{1}011 & (+7) + \mathbf{0}111 & (-2) + \mathbf{1}110 \\
 \hline
 (+7) \mathbf{0}111 & (+2) \mathbf{0}010 & (-1) \mathbf{1}111 & (-8) \mathbf{1}000 \\
 \text{OF} = 0 & \text{OF} = 0 & \text{OF} = 0 & \text{OF} = 0
 \end{array}$$

Заметьте, что, несмотря на наличие во втором и четвертом примерах переноса из старшего разряда, результат знакового сложения оказывается правильным и флаг $\text{OF} = 0$. Это ещё раз подтверждает тот факт, что знаковое и беззнаковое переполнения возникают независимо друг от друга.

Сложение с переполнением:

$$\begin{array}{rclcl}
 (+4) + \mathbf{0}100 & (+7) + \mathbf{0}111 & (-8) + \mathbf{1}000 & (-5) + \mathbf{1}011 \\
 (+4) + \mathbf{0}100 & (+2) + \mathbf{0}010 & (-2) + \mathbf{1}110 & (-4) + \mathbf{1}100 \\
 \hline
 (-8) \mathbf{1}000 & (-7) \mathbf{1}001 & (+6) \mathbf{0}110 & (+7) \mathbf{0}111 \\
 \text{OF} = 1 & \text{OF} = 1 & \text{OF} = 1 & \text{OF} = 1
 \end{array}$$

В каждом из последних примеров срабатывает Правило 2 и имеет место ситуация переполнения, следовательно, ответ получается неверным.

Замечание. При сложении чисел, представленных в знаковых типах данных, может применяться *альтернативное правило переполнения*: переполнение возникает, когда перенос в старший разряд отличается от переноса из старшего разряда.

Примеры (с переполнением):

Перенос из старшего разряда	0	1	Перенос в старший разряд		Перенос из старшего разряда	0	0	Перенос в старший разряд
	(+4)	+	0 1 0 0			(-8)	+	1 0 0 0
	(+4)	+	0 1 0 0			(-2)	+	1 1 1 0
	(-8)		1 0 0 0			(+6)	+	0 1 1 0
			OF = 1					OF = 1

Вычитание беззнаковых типов данных

Для реализации вычитания в компьютере применяется следующее правило: чтобы вычесть одно число (вычитаемое) из другого (уменьшаемого), необходимо найти дополнительный код вычитаемого и прибавить его к уменьшаемому.

Поскольку дополнительный код двоичного числа находится путем инверсии и прибавления единицы, вычитание в компьютере сводится к сложению и инверсии.

Правило 3. При вычитании чисел, представленных в беззнаковых типах данных, переполнение возникает в том случае, если отсутствует перенос из старшего разряда результата.

Таким образом, при вычитании флаг CF равен инверсии переноса.

Снова рассматриваем 4-разрядный беззнаковый тип данных с диапазоном [0..15].

Вычитание без переполнения:

(+4) 0 1 0 0	(+15) 1 1 1 1
(+3) - 0 0 1 1	(+5) - 0 1 0 1
(+4) + 0 1 0 0	(+15) + 1 1 1
(-3) + 1 1 0 1	(-5) + 1 0 1 1
(+1) 1 0 0 0 1	(+10) 1 1 0 1 0
CF = 0	CF = 0

В этих примерах находится дополнительный код для вычитаемого, который прибавляется к уменьшаемому. При сложении образуется перенос из старшего разряда (зачеркнутая единица), и в результате инверсии этого переноса флаг CF получается равным нулю, и ошибок в ответах нет.

Вычитание с переполнением:

(+3) 0 0 1 1	(+5) 0 1 0 1
(+4) ⁻ 0 1 0 0	(+7) ⁻ 0 1 1 1
-----	-----
(+3) 0 0 1 1	(+5) 0 1 0 1
(-4) ⁺ 1 1 0 0	(-7) ⁺ 1 0 0 1
-----	-----
(+15) 1 1 1 1	(+14) 1 1 1 0
CF = 1	CF = 1

Поскольку переноса нет, флаг CF равен единице и имеет место переполнение.

Вычитание знаковых типов данных

Правило 4. При вычитании чисел, представленных в знаковых типах данных, переполнение возникает в том случае, если знаки уменьшаемого и вычитаемого отличаются, а знак разности не совпадает со знаком уменьшаемого.

Следствие из правила 4: знаковое переполнение не может возникнуть при вычитании чисел с одинаковыми знаками.

Рассматриваем 4-разрядный знаковый тип данных с диапазоном [-8..+7].

Вычитание без переполнения (интересующие нас знаковые разряды выделены жирным):

(+3) 0 0 1 1	(+6) 0 1 1 0	(-5) 1 0 1 1	(-2) 1 1 1 0
(+4) ⁻ 0 1 0 0	(-1) ⁻ 1 1 1 1	(+3) ⁻ 0 0 1 1	(-7) ⁻ 1 0 0 1
-----	-----	-----	-----
(+3) 0 0 1 1	(+6) 0 1 1 0	(-5) 1 0 1 1	(-2) 1 1 1 0
(-4) ⁺ 1 1 0 0	(+1) ⁺ 0 0 0 1	(-) ⁺ 1 1 1	(+7) ⁺ 0 1 1 1
-----	-----	-----	-----
(-1) 1 1 1 1	(+7) 0 1 1 1	(-8) [±] 1 0 0 0	(+5) [±] 0 1 0 1
OF = 0	OF = 0	OF = 0	OF = 0

Вычитание с переполнением:

(+7) 0 1 1 1	(+6) 0 1 0	(-5) 1 0 1 1	(-8) 1 0 0 0
(-2) ⁻ 1 1 1 0	(-8) ⁻ 1 0 0 0	(+4) ⁻ 0 1 0 0	(+1) ⁻ 0 0 0 1
-----	-----	-----	-----
(+7) 0 1 1 1	(+6) 0 1 1 0	(-5) 1 0 1 1	(-8) 1 0 0 0
(+2) ⁺ 0 0 1 0	(+8) ⁺ 1 0 0 0	(-4) ⁺ 1 1 0 0	(-1) ⁺ 1 1 1 1
-----	-----	-----	-----
(-7) 1 0 0 1	(-2) 1 1 1 0	(+7) [±] 0 1 1 1	(+7) [±] 0 1 1 1
OF = 1	OF = 1	OF = 1	OF = 1

3.3.8. Арифметические схемы

Полусумматор

Полусумматор (half-adder) предназначен для суммирования одnorазрядных двоичных чисел.

Для построения этой простейшей арифметической схемы из логических вентилей приведем все варианты сложения одnorазрядных двоичных чисел:

$$\begin{array}{r} + 0 \\ 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} + 0 \\ 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} + 1 \\ 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} + 1 \\ 1 \\ \hline 1 \ 0 \end{array}$$

Единица, полученная в последнем примере, называется *единицей переноса*. Таким образом, в нашей схеме будет два входа – для двоичных одnorазрядных чисел A и B и два выхода – разряд суммы S и разряд переноса C_{out} (C – carry, перенос; out – выход).

Таблица истинности полусумматора:

A	B	Сумма (S)	Перенос (C_{out})
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Нетрудно заметить, что столбец суммы соответствует логической функции XOR, а столбец переноса – функции AND. Тогда схема полусумматора получается следующей (рис. 3.19):

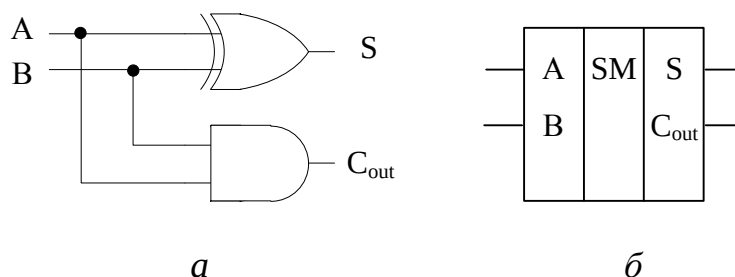


Рис. 3.19. Полусумматор:
 a – реализация на вентилей; b – обозначение на схемах

Полный сумматор

Для суммирования многоразрядных чисел недостаточно соединить несколько полусумматоров, потому что в них не учитывается перенос из предыдущих разрядов. Чтобы просуммировать три одноразрядных двоичных числа (два слагаемых A и B и входной перенос C_{in}), используется *полный сумматор* (full-adder), который строится из двух полусумматоров (рис. 3.20):

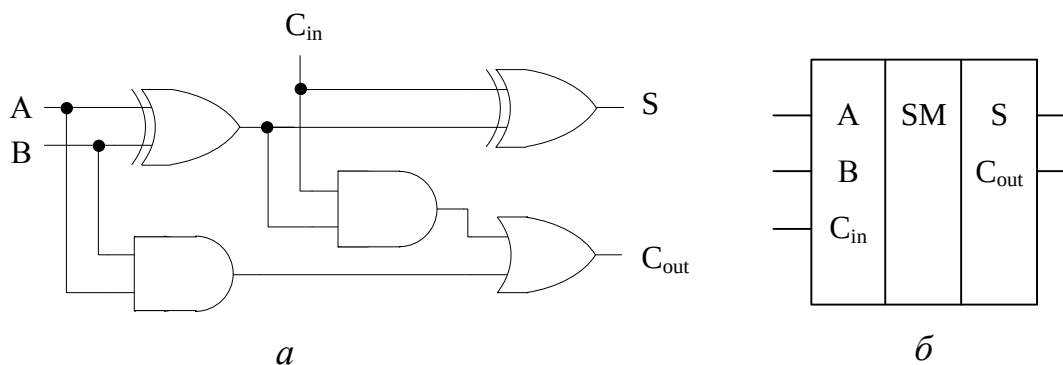


Рис. 3.20. Полный сумматор:
 a – реализация на вентилях; $б$ – обозначение на схемах

Многоразрядный сумматор

Для построения многоразрядных сумматоров нужно соединить требуемое количество полных сумматоров по *каскадной* схеме: выход переноса каждого сумматора связать со входом переноса следующего сумматора (рис. 3.21). Обратите внимание, что для сложения младших разрядов A_0 и B_0 можно использовать полусумматор, однако в схеме на рис. 3.21 изображены полный сумматор и вход переноса для всего 4-разрядного сумматора. Сделано это для того, чтобы иметь возможность построить многоразрядный сумматор из нескольких 4-разрядных.

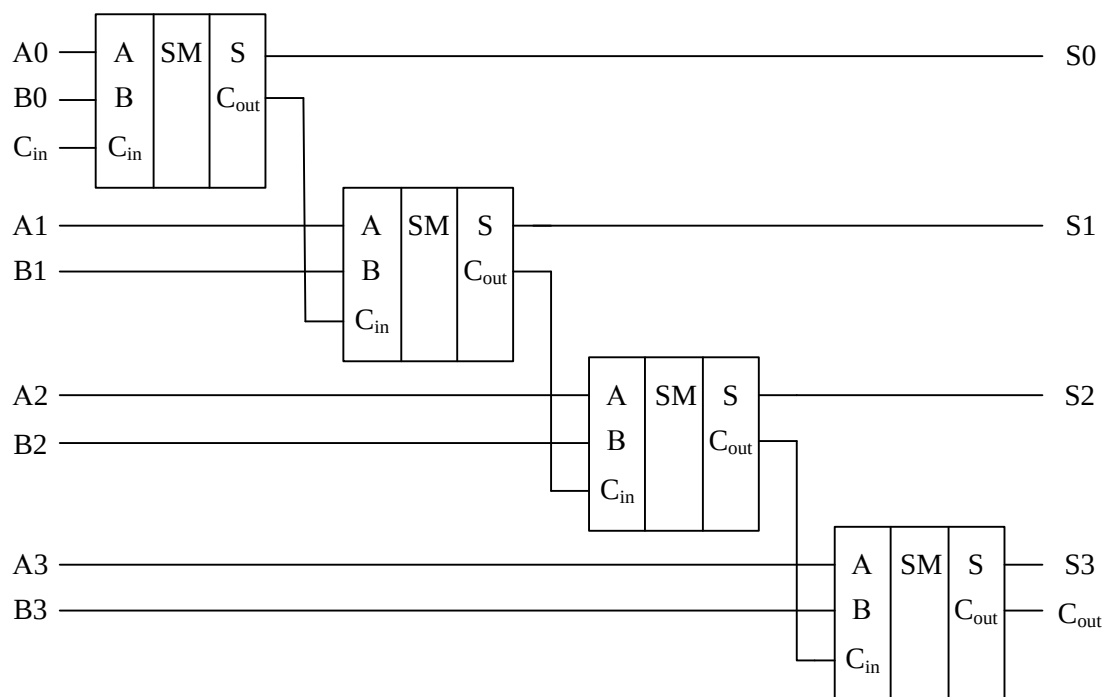


Рис. 3.21. Четырехразрядный сумматор

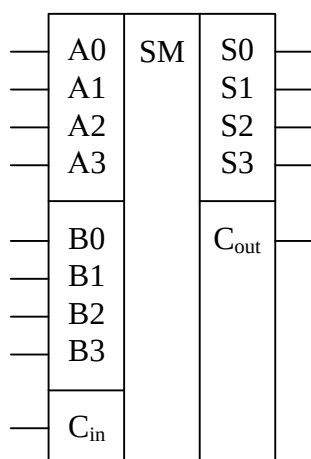


Рис. 3.22. Обозначение 4-разрядного сумматора на схемах

Замечание. Обратите внимание, что для получения полного ответа ($S1-S3$, C_{out}) сигнал переноса должен пройти от самого младшего сумматора ($S1$) к самому старшему ($S3$). Если сумматоров много (16, 32 и т. д.), процесс получения результата может затянуться, поскольку в каждом сумматоре на формирование переноса тратится определенное время. Такая схема называется *схемой с последовательным переносом*. На практике для ускорения вычислений применяется *схема с параллельным переносом* (см., например [3, 20]).

Сумматор-вычитатель

В схеме, которая может складывать и вычитать двоичные числа, должен быть предусмотрен управляющий сигнал «Сложение/вычитание». Обозначим его кратко «*Выч*». Если сигнал «*Выч*» = 0, то схема осуществляет сложение, если «*Выч*» = 1, то вычитание (рис. 3.23).

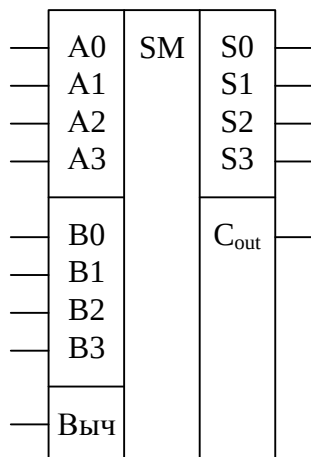


Рис. 3.23. Обозначение на схемах
4-разрядного сумматора-вычитателя

Для построения схемы сумматора-вычитателя следует помнить, что вычитание происходит путем сложения уменьшаемого с дополнительным кодом вычитаемого. Таким образом, как при сложении, так и при вычитании первое число (первое слагаемое или уменьшаемое) не изменяется, а второе число при сложении не изменяется, а при вычитании преобразуется в дополнительный код.

Как известно, дополнительный код – это инверсия и прибавление 1. Чтобы инвертировать число или оставить его без изменения, в зависимости от управляющего сигнала «*Выч*», нам поможет схема управляемого инвертора (см. параграф 3.3.4). Но как прибавлять единицу? Для этого используется простой, но элегантный прием – сигнал «*Выч*» подается на вход переноса младшего сумматора. Таким образом, при сложении («*Выч*» = 0) к итоговой сумме ничего не прибавляется, а при вычитании («*Выч*» = 1) прибавляется требуемая единица (рис. 3.24).

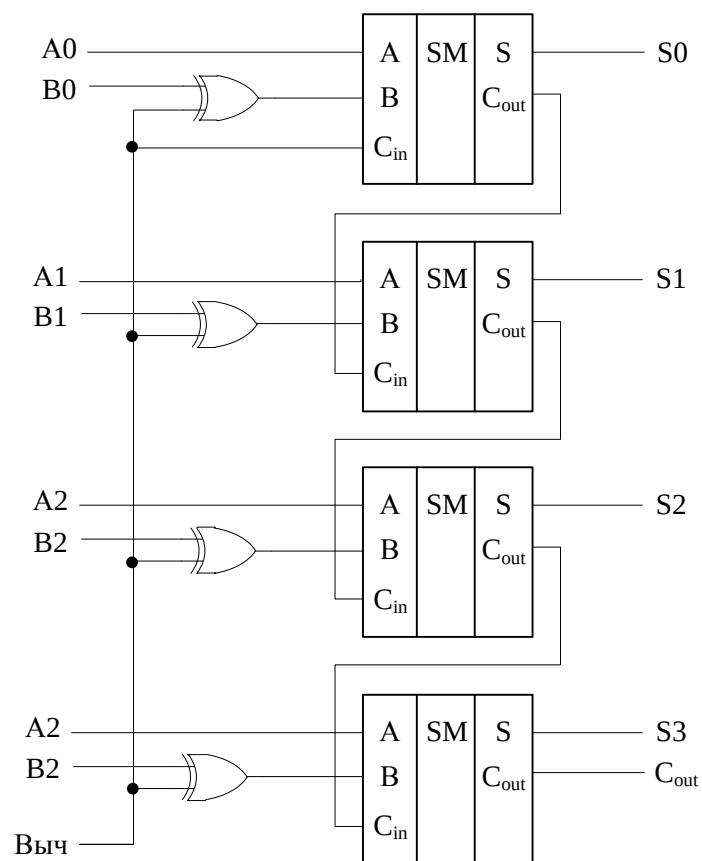


Рис. 3.24. Схема 4-разрядного сумматора-вычитателя

3.3.9. Арифметико-логическое устройство

Арифметико-логическое устройство (АЛУ, Arithmetic Logic Unit) предназначено для выполнения арифметических и логических (НЕ, И, ИЛИ) операций.

Приведем вариант построения одноразрядного АЛУ [1] (рис. 3.25).

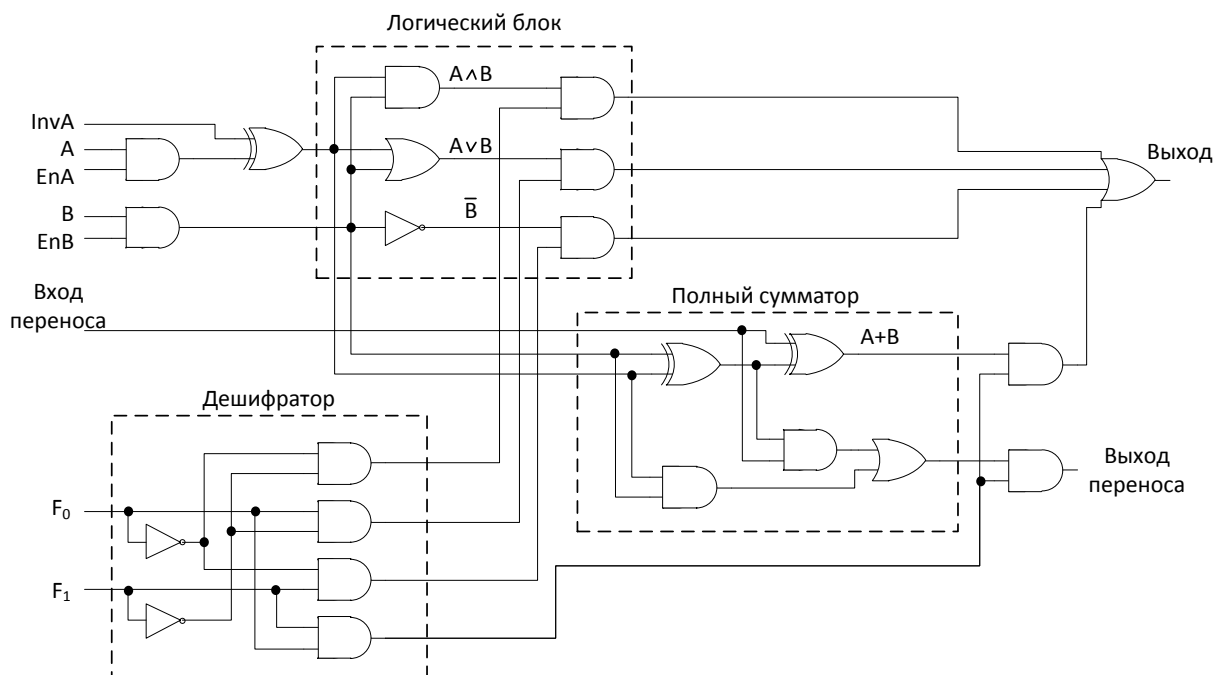


Рис. 3.25. Одноразрядное АЛУ

АЛУ может вычислять следующие функции: $A \wedge B$, $A \vee B$, $\bar{B}$, $A + B$. Выбор функции зависит от сигналов F_0 и F_1 .

В левом нижнем углу схемы находится дешифратор 2×4 , который выдает сигналы управления для четырех операций. Вверху схемы расположен логический блок для вычисления функций AND, OR, NOT. В нижнем правом углу находится полный сумматор для подсчета суммы $A + B$. Таким образом, получается 4 выходных сигнала. Но только один из них проходит через последний вентиль OR на *Выход* в зависимости от того, какую из управляющих линий выбрал дешифратор. Так как ровно один из выходных сигналов дешифратора будет равен 1, то и запускаться будет ровно один из четырех вентилях AND. Остальные три вентиля будут выдавать 0 независимо от значений A и B . Здесь вентили AND работают в качестве ключей.

АЛУ может выполнять не только логические и арифметические операции над A и B , но и делать их равными нулю при сигнале $EnA = 0$ (сигнал разрешения A) или $EnB = 0$ (сигнал разрешения B). Можно также получить $\bar{A}$, установив $InvA = 1$ (при помощи управляемого инвертора). При нормальных условиях EnA и EnB равны 1, а $InvA = 0$.

Наличие входов и выходов переноса позволяет из одноразрядных АЛУ строить АЛУ необходимой разрядности.

3.3.10. Тактовый генератор

Работа цифровых схем зависит от временной согласованности действий её элементов. Например, в рассмотренном АЛУ сигналы логических функций $A \wedge B$, $A \vee B$, $\bar{B}$ и сигналы дешифратора должны приходить на вход

вентилей AND одновременно. В то же время сигналу A нужно до этого вентиля пройти три элемента, а сигналам F_0, F_1 – максимум два. Каждый вентиль задерживает прохождение сигнала, поэтому на вход вентилей AND сигналы будут приходить не одновременно, что может вызвать ошибки в работе АЛУ. Эта проблема называется *проблемой синхронизации*. Причем проблема согласованности, одновременной работы актуальна не только для вычислительных машин, но и для компьютерных сетей, операционных систем и других сложных структур.

Для обеспечения синхронизации в цифровых схемах используются тактовые генераторы.

Тактовый генератор – это схема, которая вырабатывает одинаковые по длительности импульсы через равные промежутки времени (рис. 3.26а). Такой промежуток времени называется *периодом* T .

Формула зависимости частоты от периода: $f = \frac{1}{T}$. Частота f кварцевых тактовых генераторов находится в пределах от нескольких кГц до сотен МГц. Например, при частоте генератора $f = 100$ МГц период $T = \frac{1}{100 \cdot 10^6 \text{ Гц}} = 10^{-8} \text{ с} = 10 \text{ нс}$.

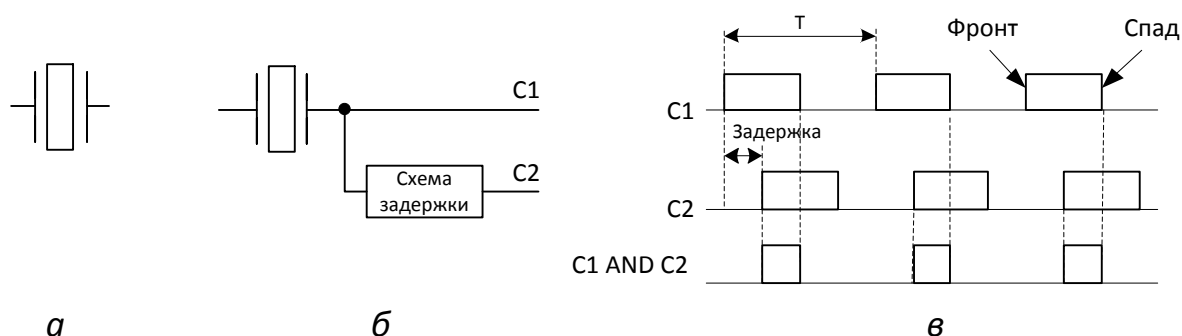


Рис. 3.26. Тактовый генератор:

a – обозначение на схемах; $б$ – применение тактового генератора со схемой задержки;
 $в$ – временные диаграммы

Используя схемы задержки и логические элементы, можно сформировать последовательность импульсов требуемой продолжительности и сдвига относительно основной последовательности (рис. 3.26б и рис. 3.26в).

Многие схемы реагируют на начало (*фронт*, или *передний фронт*) или конец (*спад*, или *задний фронт*) тактовых импульсов. На временной диаграмме (рис. 3.26в) показаны фронт и спад тактового импульса. На рис. 3.27 приведены обозначения на схемах сигналов, работающих по фронту и спаду импульсов.

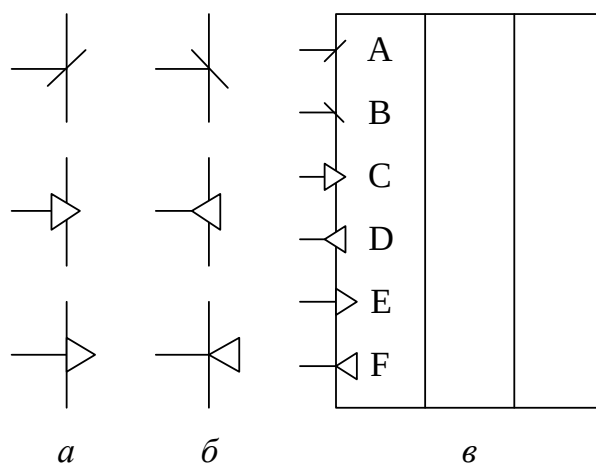


Рис. 3.27. Примеры обозначений на схемах фронта (а, в) и спада (б, в) сигналов

3.4. Схемы с памятью

3.4.1. Иерархия запоминающих устройств

Для современных типов запоминающих устройств характерна следующая закономерность: чем больше емкость, тем ниже стоимость хранения бита, но больше время доступа.

При создании системы памяти ВМ приходится находить компромисс между параметрами стоимости, емкости и быстродействия. Общепринятое решение – построение памяти по иерархическому принципу.

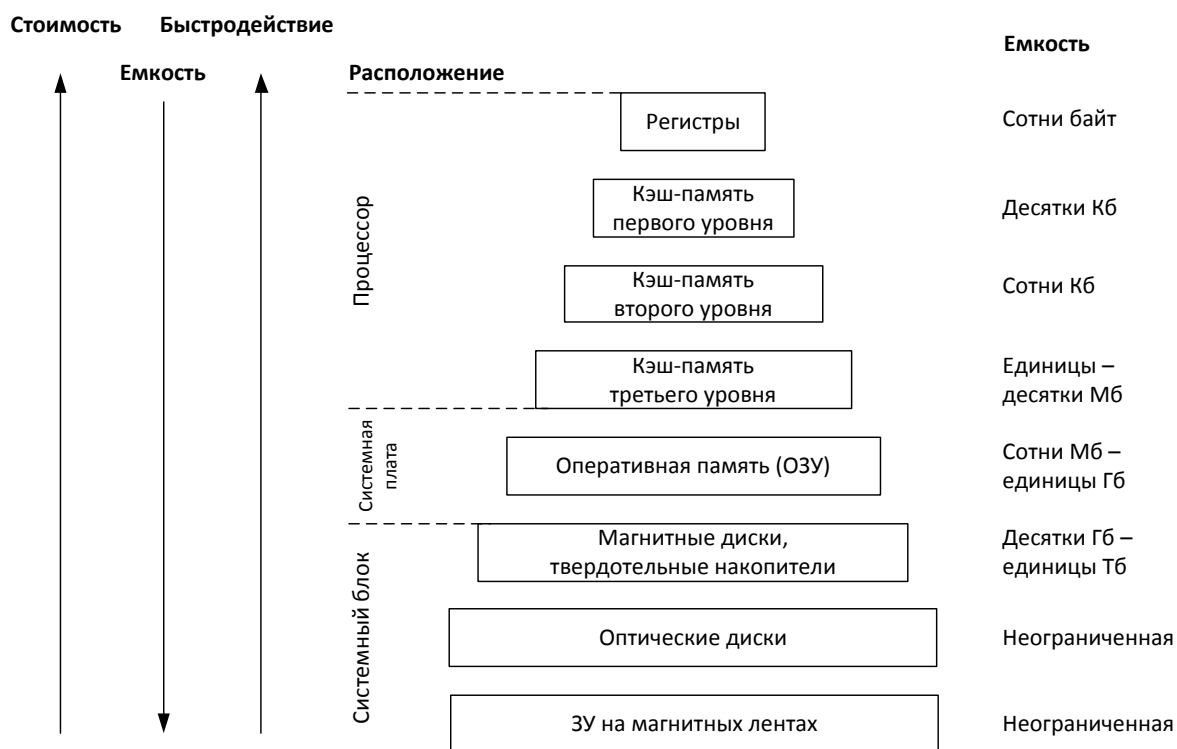


Рис. 3.28. Иерархия запоминающих устройств

3.4.2. Триггеры

RS-триггеры

Триггер – это устройство памяти, способное хранить один бит информации. Триггер можно построить из двух вентилях ИЛИ-НЕ или двух вентилей И-НЕ. На рис. 3.29 изображен *RS-триггер* на основе вентилях ИЛИ-НЕ.

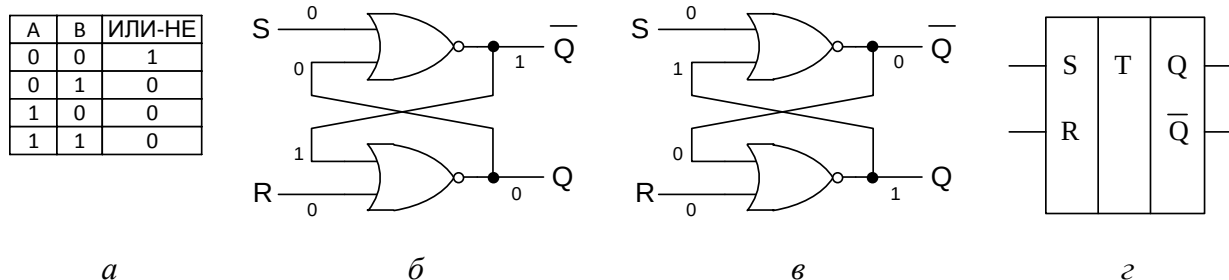


Рис. 3.29. RS-триггер: *а* – таблица истинности вентиля ИЛИ-НЕ; *б, в* – схемы триггера для состояния хранения; *г* – обозначение на схемах

Хранение информации обеспечивается двумя перекрестными обратными связями (соединением выхода со входом). В отличие от комбинационных схем выходные сигналы триггера определяются не только текущими входными сигналами, но и предыдущим состоянием.

Схема имеет два входа: *S* (*set* – установка) и *R* (*reset* – сброс). Также имеется два выхода: *Q* и $\bar{Q}$. Считается, что триггер хранит ноль, если $Q = 0$, $\bar{Q} = 1$, и единицу, если $Q = 1$, $\bar{Q} = 0$. Возможность *хранения* информации следует из способности триггера оставлять выходы *Q* и $\bar{Q}$ неизменными при меняющихся входах *S* и *R*.

Чтобы увидеть, как это осуществляется, предположим, что $S = 0$ и $R = 0$. Пусть *Q* было равно 0, т. е. в триггере был записан ноль. Так как *Q* возвращается в верхний вентиль ИЛИ-НЕ и оба входа этого вентиля равны 0, то его выход $\bar{Q} = 1$. Единица возвращается в нижний вентиль, у которого в итоге один вход равен 0, другой – 1, а на выходе получается $Q = 0$ (рис. 3.29*б*). Такое состояние является устойчивым, в нем триггер может находиться сколько угодно долго (при наличии питания и входах $S = R = 0$).

Теперь предположим, что *Q* было равно 1 (в триггере была записана единица), а на входы *S* и *R* подается 0. Верхний вентиль имеет входы 0 и 1 и выход $\bar{Q} = 0$, который возвращается в нижний вентиль и совместно с входом *R* дает $Q = 1$ (рис. 3.29*в*). Полученное состояние также является устойчивым.

Следовательно, при $S = R = 0$ триггер имеет два стабильных состояния, когда триггер сохраняет значение, которое присутствовало на выходе *Q*.

Рассмотрим, каким образом в триггер *записывается* информация. Предположим, что *S* принимает значение 1, в то время как $R = 0$ и $Q = 0$. То-

гда входные сигналы верхнего вентиля будут 1 и 0, что приведет к выходному сигналу $\bar{Q} = 0$. Это изменение делает оба входа в нижний вентиль равными 0, и, следовательно, выходной сигнал – равным 1. Таким образом, установка S на значение 1 переключает состояние триггера с 0 на 1. Такой переход называется «Установка единицы». Установка R на значение 1 при $S = 0$, когда триггер находится в состоянии 0, не вызывает изменений (проверьте).

Аналогично рассуждая, можно понять, что установка S на значение 1 при состоянии триггера 1 (т. е. при $Q = 1$) не вызывает изменений, но установка $R = 1$ при $S = 0$ приводит к записи в триггер нуля («Сброс в ноль»).

Таким образом, при $S = 1$, $R = 0$ в триггер записывается единица, а при $S = 0$, $R = 1$ записывается ноль, независимо от того, в каком состоянии находился триггер.

Если $S = R = 1$, то $Q = \bar{Q} = 0$. При одновременном возврате входных сигналов к нулю (в состояние хранения) невозможно определить, что будет записано в триггере. Если сигнал S вернется к нулю чуть раньше, то в триггере запишется 0 (так как R ещё равен 1 некоторое время), и наоборот. Из-за неопределенности состояния триггера входы $S = R = 1$ считаются *запрещенными*.

Приведем таблицу входов и выходов RS-триггера.

S	R	Q	$\bar{Q}$	Комментарий
0	0	$Q_{пред}$	$\bar{Q}_{пред}$	Хранение
1	0	1	0	Установка 1
0	1	0	1	Сброс в 0
1	1	0	0	Запрещенное состояние

Синхронные RS-триггеры

Рассмотренный RS-триггер может записывать и запоминать информацию. Однако он обладает существенным недостатком – у него отсутствуют средства синхронизации, т. е. если сигналы на входы R и S будут приходить неодновременно, триггер станет работать неправильно. Поэтому в схему добавляют сигнал синхронизации и такой триггер называется *синхронным RS-триггером* (рис. 3.30).

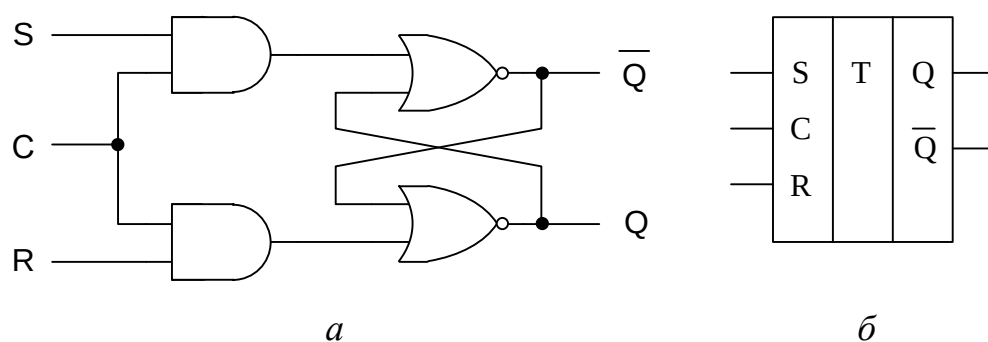


Рис. 3.30. Синхронный RS-триггер:
а – схема из вентилях, *б* – обозначение на схемах

Эта схема имеет дополнительный *синхронизирующий* (он ещё называется *тактовым*) вход *C* (*clock* – тактовый генератор). Два вентиля AND используются в качестве ключей. Если тактовый вход равен 0, то оба выхода вентилях AND равны 0 независимо от *S* и *R* и триггер находится в состоянии хранения. При входе $C = 1$ вентили AND пропускают сигналы *S* и *R* и состояние триггера определяется этими входами подобно обычному RS-триггеру.

Таблица входов и выходов синхронного RS-триггера выглядит следующим образом.

C	S	R	Q	Q̄	Комментарий
0	x	x	$Q_{пред}$	$\bar{Q}_{пред}$	Хранение
1	0	0	$Q_{пред}$	$\bar{Q}_{пред}$	Хранение
1	1	0	1	0	Установка 1
1	0	1	0	1	Сброс в 0
1	1	1	0	0	Запрещенное состояние

x – не имеет значения

Синхронный D-триггер

Чтобы запрещенного состояния RS-триггера не возникало, нужно предотвратить появление $S = R = 1$. На рис. 3.31 изображена схема триггера с одним входом данных *D* (*data* – данные) и входом синхронизации *C*. По сравнению с синхронным RS-триггером в эту схему добавлен только инвертор, роль которого – не допустить запрещенное состояние.

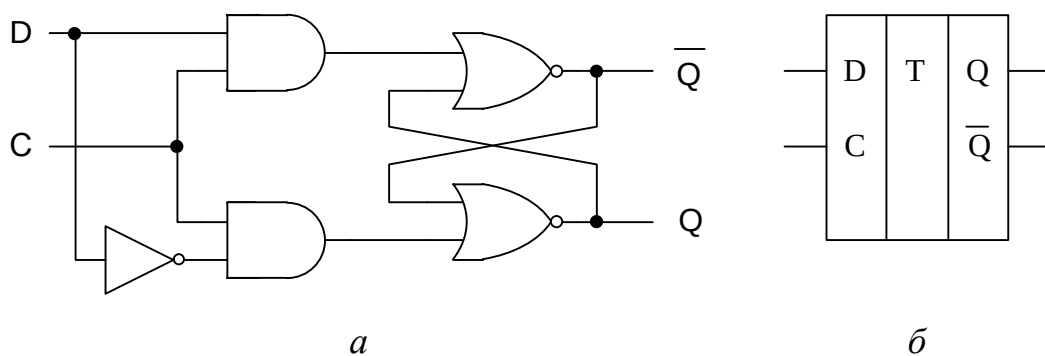


Рис. 3.31. Синхронный D-триггер:
a – схема из вентилях; *б* – обозначение на схемах

Так как входной сигнал в нижний вентиль AND всегда является инверсным к входному сигналу в верхний вентиль AND, ситуация, когда оба входа равны 1, никогда не возникает.

Особенностью D-триггера является то, что в него записывается значение входа данных D (конечно, при $C = 1$). Хранение, как и в синхронном RS-триггере, происходит при $C = 0$.

Задание. Составить таблицу входов и выходов D-триггера.

Часто применяются синхронные D-триггеры с асинхронными входами S и R (рис. 3.32).

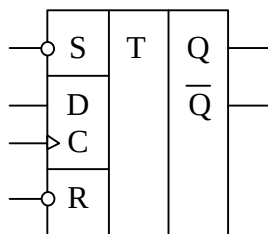


Рис. 3.32. Синхронный D-триггер с асинхронными входами S и R

Смысл входов S и R тот же, что и в RS-триггере – они предназначены для установки/сброса триггера. Входы S и R называются *асинхронными* (не синхронными), потому что работают независимо от входа синхронизации C . Незаштрихованные кружки на входах S и R означают, что входы являются *инверсными*, или активными по низкому уровню сигнала, т. е. установка триггера в единицу происходит при $S = 0$, $R = 1$, а сброс в ноль – при $S = 1$, $R = 0$.

3.4.3. Регистры

Регистр – многоразрядная ячейка памяти, представляющая собой совокупность связанных триггеров, которые действуют как единое устройство. Единство действия обеспечивается подключением синхронизирующих входов всех триггеров к одному тактовому сигналу.

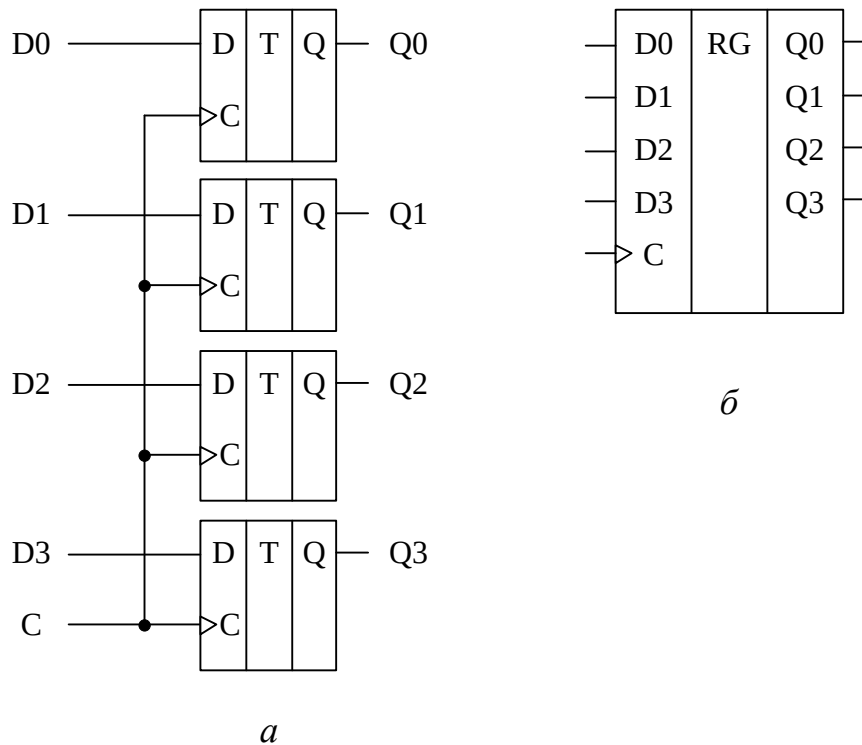


Рис. 3.33. Четырехразрядный регистр хранения:
a – схема из вентилях; *б* – обозначение на схемах

На рис. 3.33 изображен *регистр хранения*, т. е. регистр, предназначенный только для запоминания данных. Также существуют *регистры сдвига* (*сдвиговые регистры*, *shift registers*), данные в которых не только хранятся, но и могут сдвигаться вправо или влево.

Операция сдвига часто используется при обработке данных, например для быстрого умножения и деления на степени двойки, для циклических действий с отдельными битами и др.

Пример. Умножить число 100110_2 на 4. Разделить то же число на 2.

Решение

1) Умножение. Переведем двоичное число в десятичный вид:

$$100110_2 = 38_{10}.$$

Умножим на 4 и переведем обратно в двоичный вид:

$$38 \cdot 4 = 152_{10} = 10011000_2.$$

Заметим, что тот же результат можно получить, просто сдвинув исходное двоичное число на 2 бита влево ($2^2 = 4$).

2) Деление: $38/2 = 19_{10} = 10011_2$.

Тот же результат получается при сдвиге исходного двоичного числа на 1 бит вправо ($2^1 = 2$).

На рис. 3.34 приведен пример реализации сдвигового регистра на D-триггерах.

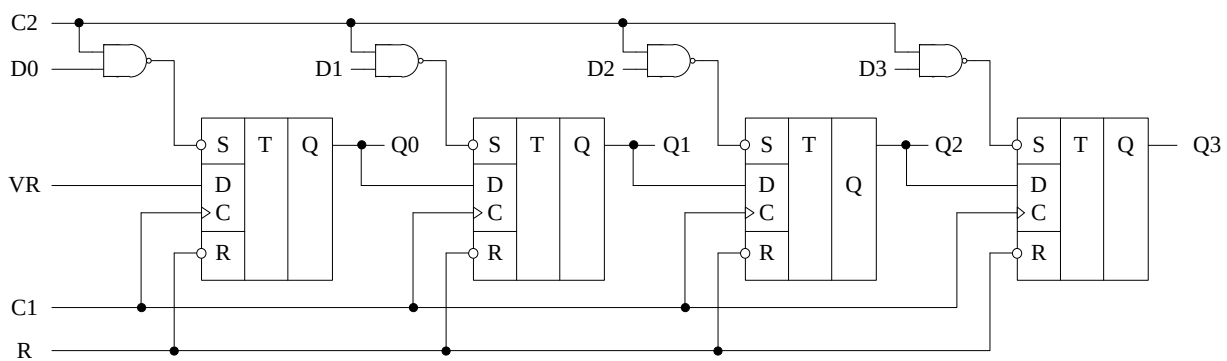


Рис. 3.34. 4-разрядный сдвиговый регистр

Входы S триггеров используются для записи информации ($D0-D3$) в регистр, причем запись осуществляется только при наличии сигнала $C2$. Поскольку входы S триггеров являются инверсными, в качестве ключа применяются вентили И-НЕ. Входы R требуются для сброса всех триггеров одновременно в ноль.

По фронту сигнала $C1$ происходит сдвиг содержимого регистра вправо, при этом в младший разряд регистра $Q0$ записывается сигнал VR (*right* – вправо).

Обозначение сдвигового регистра на схемах приведено на рис. 3.35.

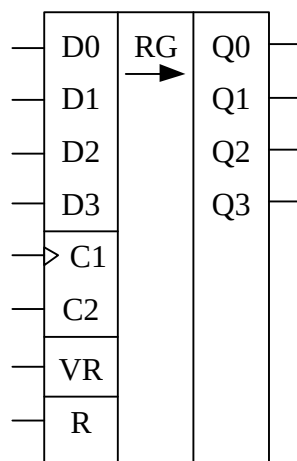


Рис. 3.35. Обозначение 4-разрядного сдвигового регистра на схемах

3.4.4. Счетчики

Счетчик (counter) – это схема, предназначенная для подсчета входных сигналов.

Счетчики, как и регистры, строятся на основе триггеров.

Счетчики по направлению счета делятся на *простые* (суммирующие и вычитающие) и *реверсивные* (которые могут как увеличивать, так и уменьшать).

Подробнее о счетчиках можно прочитать в [20, 21].

3.4.5. Организация памяти

С помощью регистров строится микропроцессорная регистровая память – малая по объему и самая быстросействующая (см. рис. 3.28). Для построения памяти большого объема (например, кэш-памяти или оперативной памяти) используются две основные структуры – с одним дешифратором и с двумя дешифраторами. В качестве запоминающих элементов могут применяться триггеры (кэш-память) или специальные полупроводниковые конденсаторы (оперативная память).

Схема памяти с одним дешифратором

Рассмотрим подробно структуру и принцип действия памяти с одним дешифратором (рис. 3.36). Пусть в качестве запоминающих элементов выступают D-триггеры [1].

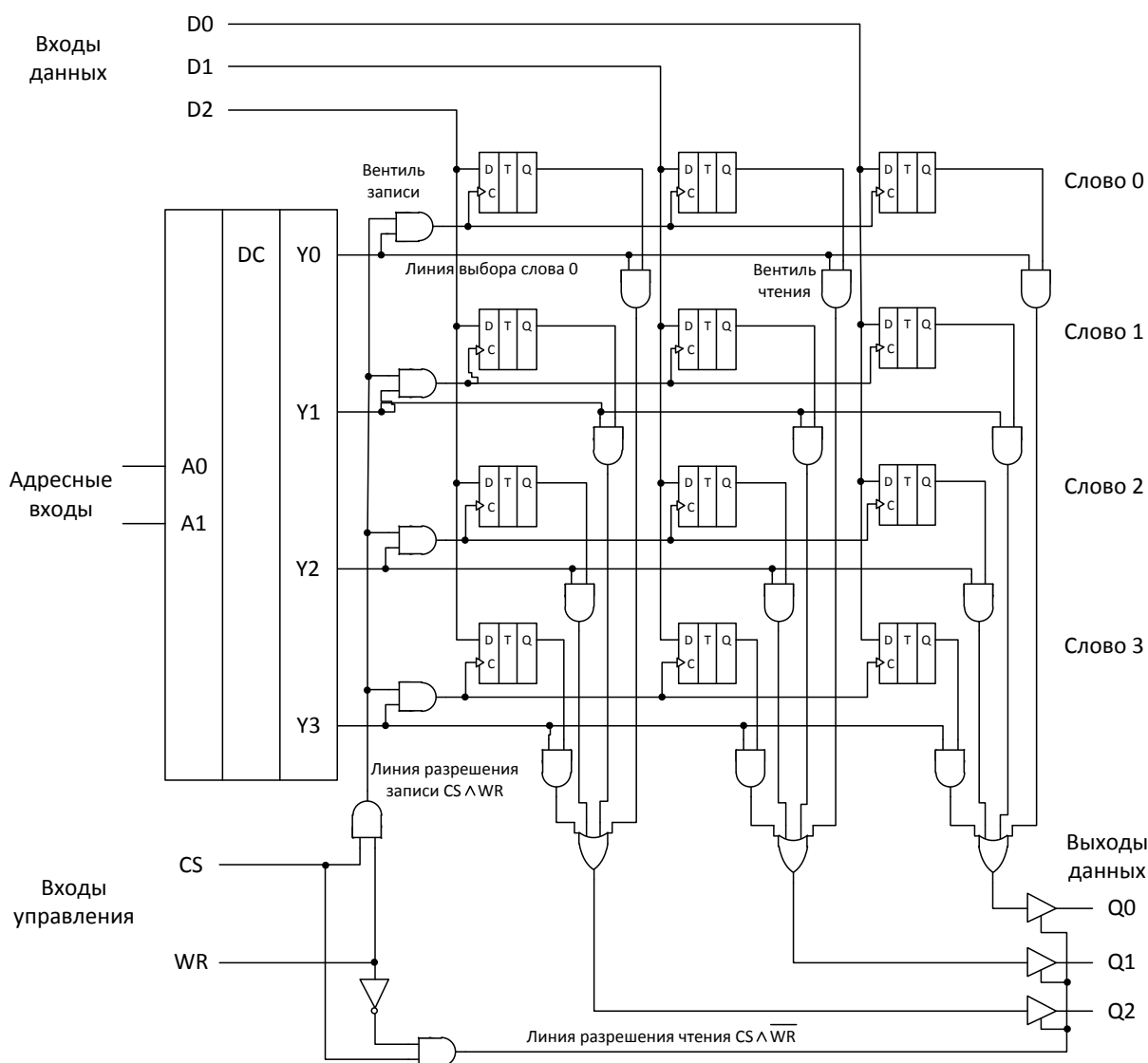


Рис. 3.36. Схема памяти с одним дешифратором

Схема содержит следующие входы и выходы:

- 3 входа данных $D0$ – $D2$.
- 2 адресных входа $A0$ и $A1$ ($A0$ считается младшим, $A1$ – старшим разрядом).
- 2 входа управления:
 - CS (*Chip Select* – выбор микросхемы памяти),
 - WR (*Write/Read* – выбор режима записи или чтения).
- 3 выхода данных: $Q0$ – $Q2$.

Чтение и запись производятся одновременно для совокупности бит, называемой *словом* (*word*, см. параграф 2.2.1). Слово в данной схеме образуют три триггера, расположенных горизонтально в ряд. Всего имеется четыре таких ряда, следовательно, схема памяти содержит 4 слова по три бита (обозначается 4×3).

Выбор нужного слова осуществляется при помощи адресных входов и дешифратора. Почему адресных входов два?

Число ячеек памяти, адресуемых N адресными линиями, равно 2^N .

Чтобы это понять, достаточно построить следующий ряд. На одной адресной линии может быть два сигнала – 0 или 1. Следовательно, она позволяет различать две ячейки памяти с адресами 0 и 1.

Две адресные линии позволяют выдать четыре различных сигнала – 00, 01, 10 и 11, поэтому с их помощью можно выбирать одну из четырех ячеек с адресами 0, 1, 2, 3.

Три адресные линии выдают уже восемь сигналов и могут адресовать 8 ячеек памяти и т. д.

Таким образом, два адресных входа позволяют обращаться (адресовать) к четырем ячейкам памяти.

Обычно память состоит из нескольких микросхем подобной структуры. Поэтому, чтобы выбрать необходимую микросхему, нужно установить сигнал CS в 1, а также предусмотреть $WR = 1$ для записи или $WR = 0$ для чтения.

При *операции записи* биты, находящиеся на входных линиях для данных $D0–D2$, загружаются в выбранное слово памяти; выходные линии не используются.

Вентили записи AND используются в качестве ключей. На их верхние входы подается сигнал разрешения записи $CS \wedge WR = 1$, но на выходе единица появляется только у одного вентиля записи – в зависимости от того, какой из выходов дешифратора активен (т. е. какая ячейка памяти выбирается адресными входами). Эта единица поступает на входы синхронизации S триггеров выбранного слова. Таким образом, запись осуществляется только в одну ячейку, состоящую из трех триггеров.

При *операции чтения* выбирается слово и помещается на выходные линии для данных $Q0–Q2$. При этом входные линии для данных не используются. Линия $CS \wedge WR = 0$ ($CS = 1$, $WR = 0$), поэтому все вентили записи блокируются и ни в один из триггеров запись не происходит.

Линия выбора слов запускает вентили AND (также работающие в качестве ключей), связанные с выходами триггеров Q выбранного слова. Таким образом, выбранное слово передает свои данные в четырехвходовые вентили OR, расположенные в нижней части схемы, а остальные три слова выдают 0. Следовательно, выход вентилей OR идентичен значению, сохраненному в данном слове. Остальные три слова никак не влияют на выходные данные.

Вентили OR соединены с выходными линиями данных не напрямую, а через специальные буферные элементы. Это связано с тем, что обычно для входных и выходных линий используются одни и те же проводники. Поэтому, если подключить вентили OR непосредственно к выходным линиям, то сигналы на них будут мешать в процессе записи.

Буферный элемент – это электронный переключатель, который может устанавливать и разрушать связь за несколько наносекунд под действием сигналов управления (рис. 3.37).



Рис. 3.37. Буферный элемент

Буферный элемент содержит вход для данных, выход для данных и вход управления. Когда вход управления равен 1, буферный элемент работает как обычный проводник и передает данные. Когда вход управления равен 0, буферный элемент размыкает цепь. Соединение может быть восстановлено за несколько наносекунд, если сделать сигнал управления равным 1.

Отличие буферного элемента от вентиля AND заключается в том, что буферный элемент представляет собой *устройство с тремя состояниями*, поскольку он может выдавать 0, 1 или вообще не выдавать сигнала (в случае с разомкнутой цепью).

Когда $CS = 1$, а $WR = 0$, то сигнал разрешения выдачи выходных данных (сигнал управления для буферного элемента) равен 1, в результате чего запускаются буферные элементы и слово передается на выходные линии. Когда сигнал $CS = 0$ или $WR = 1$, то выходы отсоединяются от остальной части схемы.

Схема памяти с двумя дешифраторами

Рассмотрим схему памяти с двумя дешифраторами (рис. 3.38) [2].

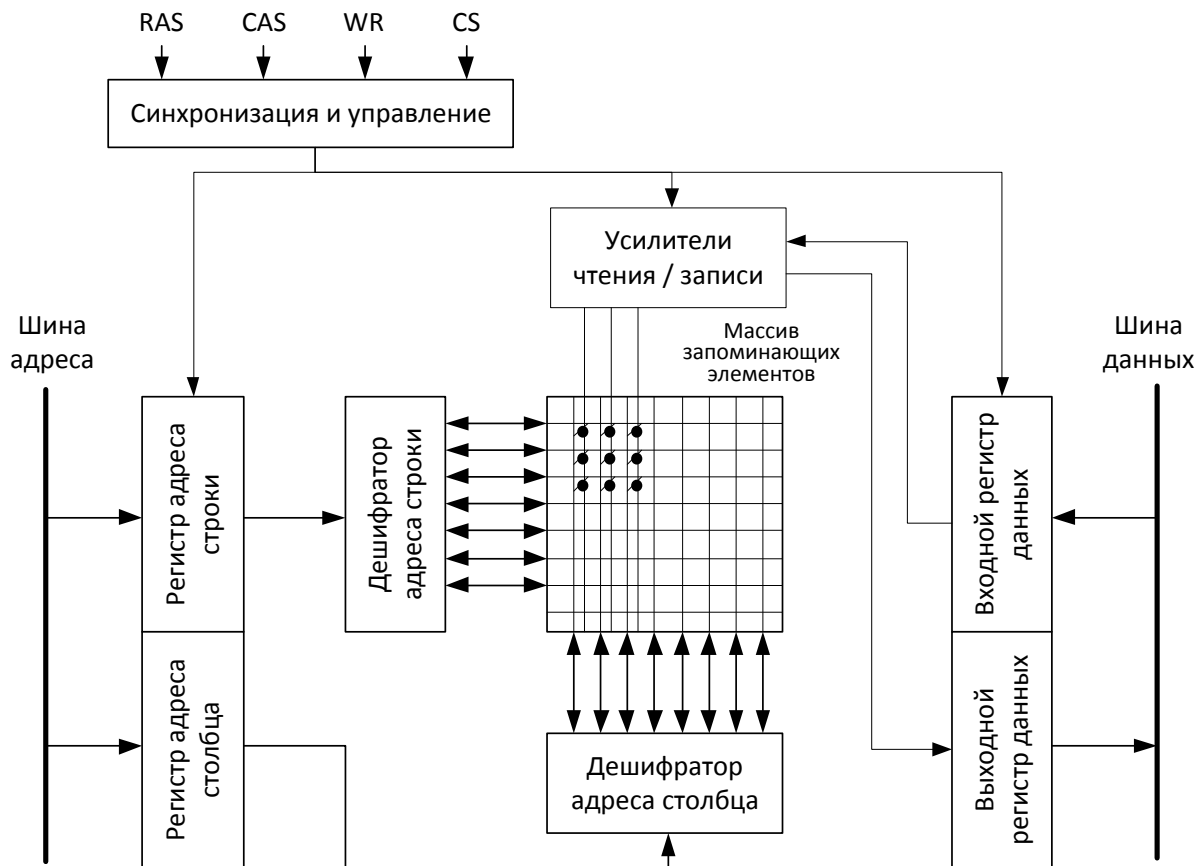


Рис. 3.38. Схема памяти с двумя дешифраторами

Адрес ячейки, поступающий по шине адреса, состоит из адреса строки (row) и адреса столбца (column). Для уменьшения количества выводов интегральной микросхемы и числа проводников адресной шины адреса строки и столбца подаются в микросхему через одни и те же линии и запоминаются в регистрах адреса. Адрес строки сопровождается сигналом *RAS* (Row Access Strobe – сигнал доступа к строке), а адрес столбца – сигналом *CAS* (Column Access Strobe – сигнал доступа к столбцу). Регистры адреса соединены каждый со своим дешифратором. Выходы дешифраторов образуют систему горизонтальных и вертикальных линий, к которым подсоединены запоминающие элементы матрицы. При этом каждый запоминающий элемент расположен на пересечении одной вертикальной и одной горизонтальной линий (рис. 3.39).

При активации этих линий запоминающий элемент, находящийся на их пересечении, выбирается, данные либо считываются из него (при сигнале $WR = 0$), либо записываются (при $WR = 1$) при помощи усилителей чтения/записи. Информация перед выдачей на шину данных запоминается в вы-

ходном регистре данных, точно так же, как перед записью в ячейку заносится с шины данных во входной регистр данных.

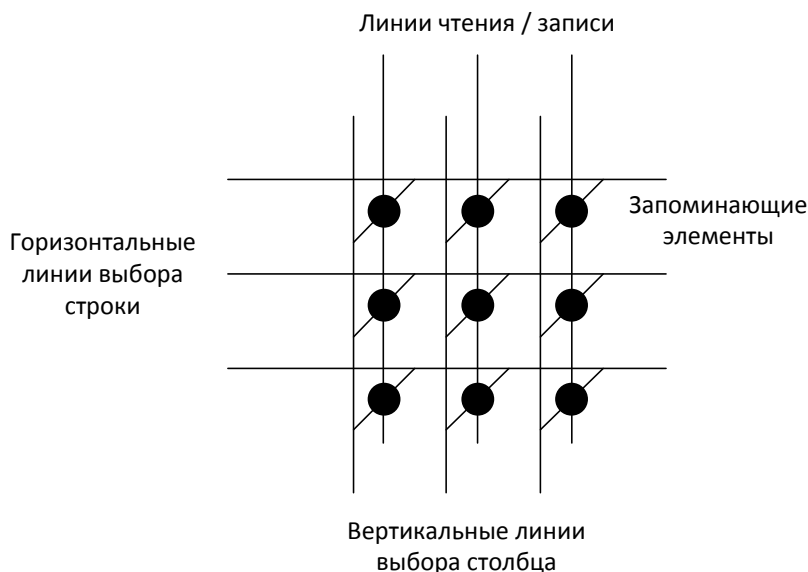


Рис. 3.39. Увеличенное изображение матрицы запоминающих элементов

Структура памяти с одним дешифратором является более простой, однако при росте емкости микросхемы памяти количество адресных входов возрастает быстрее, чем в структуре с двумя дешифраторами. Сравните, на рис. 3.40 показаны микросхемы памяти одинаковой емкости (8 килобит). Однако число адресных входов в микросхеме с одним дешифратором в два раза больше (10 против 5).

Поэтому микросхемы с одним дешифратором применяют для построения кэш-памяти и ПЗУ, а микросхемы с двумя дешифраторами – для ОЗУ.

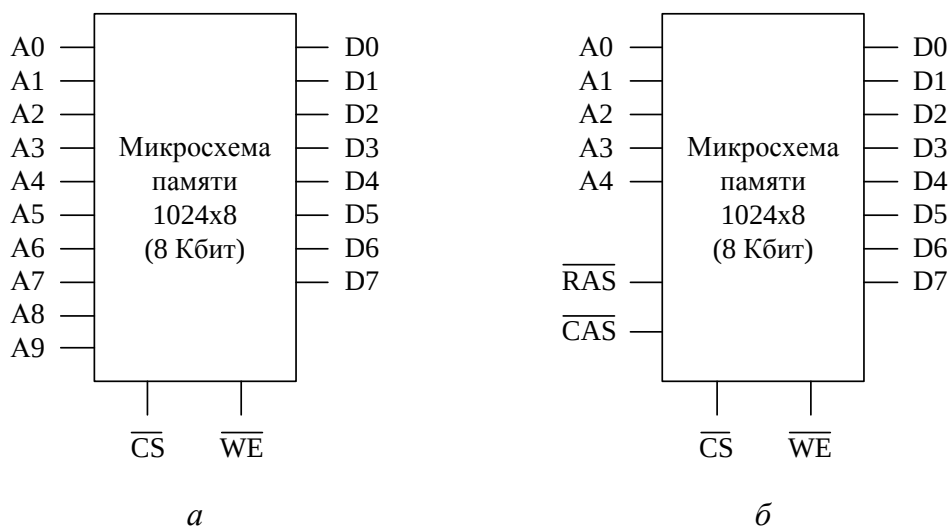


Рис. 3.40. Обозначения микросхем памяти:
 а – с одним дешифратором; б – с двумя дешифраторами

3.4.6. Оперативные запоминающие устройства

ОЗУ (оперативное запоминающее устройство, RAM – *Random Access Memory*) – это память, в которой хранятся данные, необходимые для выполнения текущей программы.

Свойства ОЗУ:

- 1) позволяет как *записывать*, так и *считывать* информацию, причем обе операции выполняются однократно, с одной и той же скоростью;
- 2) процессор может *непосредственно обращаться* к произвольной ячейке ОЗУ (чтобы обратиться к данным, хранящимся во внешней памяти – на винчестере, компакт-диске, флэш-памяти, процессор должен сначала загрузить эти данные в ОЗУ);
- 3) *энергозависимо*, т. е. при выключении питания вся информация теряется.

Существует два типа ОЗУ: *статическое* (*Static RAM, SRAM*) и *динамическое* (*Dynamic RAM, DRAM*). Статическое ОЗУ конструируется с использованием D-триггеров (см. рис. 3.31). Информация в ОЗУ сохраняется на протяжении всего времени, пока к нему подается питание. Статическое ОЗУ работает очень быстро. Обычно время доступа составляет несколько наносекунд. Из-за большой скорости статическое ОЗУ используется в качестве кэш-памяти.

В динамическом ОЗУ, напротив, триггеры не используются. Динамическое ОЗУ представляет собой массив ячеек, каждая из которых содержит транзистор и полупроводниковый конденсатор (рис. 3.41). Конденсатор может быть заряженным и разряженным, что позволяет хранить единицу или ноль.

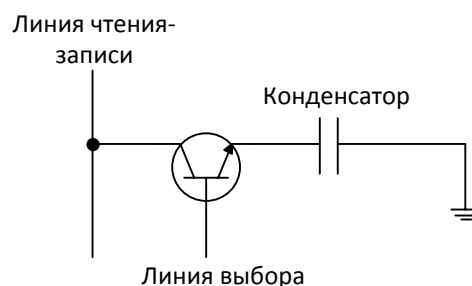


Рис. 3.41. Ячейка динамического ОЗУ

Поскольку электрический заряд конденсатора имеет тенденцию исчезать, каждый бит в динамическом ОЗУ должен обновляться (*регенерироваться*) каждые несколько миллисекунд, чтобы предотвратить утечку данных. Поскольку об обновлении должна заботиться внешняя логика, динамическое ОЗУ требует более сложных схем управления, чем статическое, однако этот недостаток компенсируется большим объемом.

Поскольку динамическому ОЗУ нужен только 1 транзистор и 1 конденсатор на бит (статическому ОЗУ требуется 6–11 транзисторов на бит), динамическое ОЗУ имеет очень высокую плотность записи (много битов на одну микросхему). Однако динамические ОЗУ работают очень медленно (время доступа занимает десятки наносекунд).

3.4.7. Виды оперативной динамической памяти

На начальном этапе это были микросхемы асинхронной памяти, работа которых не привязана жестко к тактовым импульсам системной шины процессор-память.

- *Традиционная DRAM*. Операции чтения-записи требуют 5 тактов:
 - 1) выдача адреса строки;
 - 2) выдача сигнала RAS;
 - 3) выдача адреса столбца;
 - 4) выдача сигнала CAS;
 - 5) выполнение операции чтения-записи.

Традиционная DRAM характеризуется формулой 5-5-5-5, где первая цифра – число тактов системной шины, необходимое для доступа к первой ячейке памяти, а остальные – число тактов, необходимое для доступа к следующим ячейкам. Микросхемы данного типа могли работать на частотах до 40 МГц (время доступа – 120 нс).

Все последующие типы основывались на *принципе локальности адресов* при обращении к ОЗУ. Принцип заключается в том, что при выполнении программы с высокой вероятностью адрес следующей команды либо следует непосредственно за адресом текущей команды, либо находится вблизи него.

- *FPM DRAM (Fast Page Mode – быстрый страничный режим)*. Здесь под «страницей» понимается «строка». Полный адрес строки и столбца передается только при первом обращении к строке. Сигнал RAS остается активным в течение всего периода операций со строкой. Меняется только адрес столбца. Таким образом, при втором и следующих обращениях к строке исключаются первые два такта, и формула работы FPM имеет вид: 5-3-3-3. Время доступа – 60 нс. Однако выигрыш в скорости достигается лишь при передаче блоков данных из одной строки.

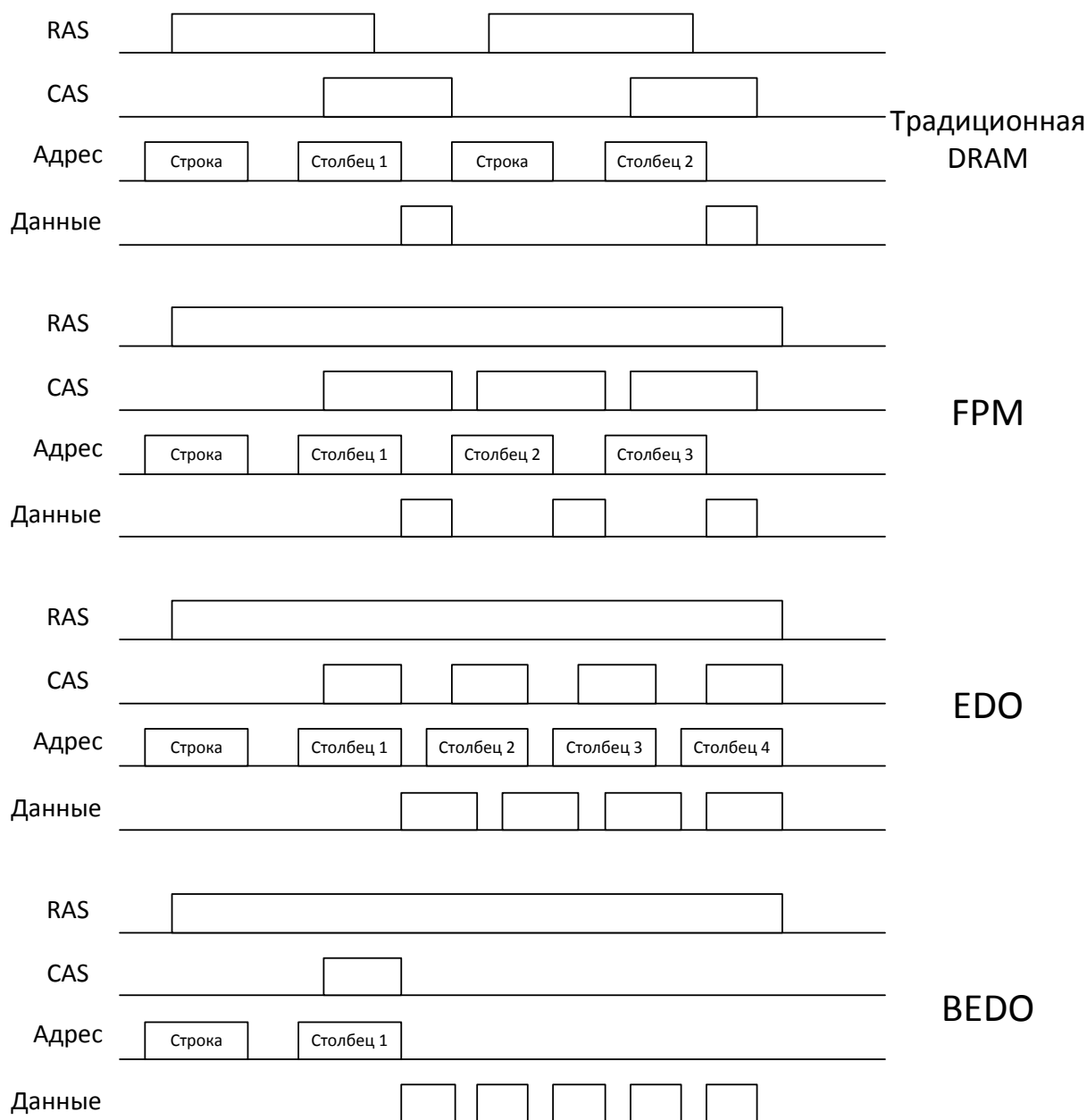


Рис. 3.42. Временные диаграммы различных видов DRAM

- **EDO DRAM (Extended Data Output – ОЗУ с расширенным выводом данных).** Главная особенность – увеличенное время доступности данных на выходе микросхемы. В EDO DRAM по спаду сигнала CAS данные сохраняются во внутреннем регистре. За счет уменьшения длительности CAS формула работы приобретает вид: 5-2-2-2. Частота работы – до 66 МГц.

- **BEDO DRAM (Burst EDO – пакетная EDO).** В микросхеме содержится счетчик адресов столбцов. При обращении к группе слов (пакету) адрес столбца формируется обычным способом только в начале пакетного цикла. Для последующих передач адреса образуются быстро с помощью счетчика. Формула BEDO DRAM: 5-1-1-1. Память BEDO не получила широкого распространения из-за появления SDRAM.

В асинхронной памяти моменты появления данных из памяти на системной шине не синхронизированы с фронтами тактовых импульсов системной шины. Поэтому данные, поступившие на шину сразу после фронта тактового импульса, будут восприняты только при следующем тактовом импульсе, и шина данных простаивает в ожидании данных (рис. 3.43).

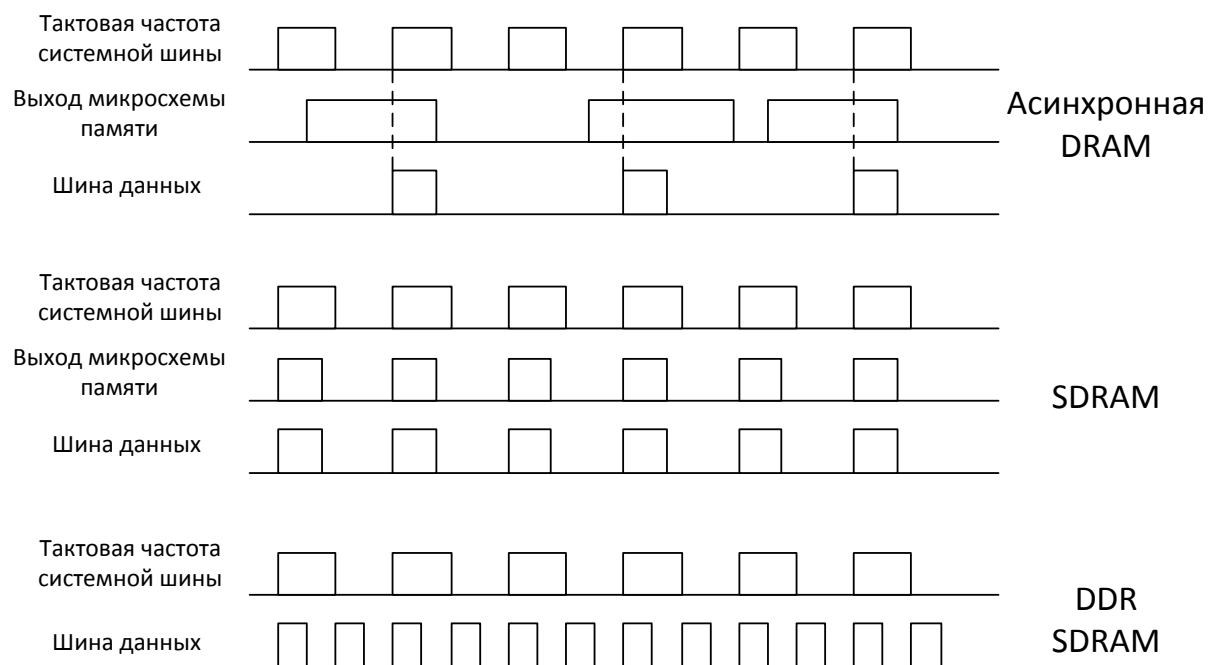


Рис. 3.43. Временные диаграммы для асинхронной и синхронных DRAM

- **SDRAM** (*Synchronous DRAM* – синхронная динамическая память). Предложена в 1994 году, а реализована в конце 1996-го. Основной особенностью SDRAM является синхронизация всех ее операций с тактовыми сигналами системной шины (рис. 3.43). За счет использования страничного режима (FPM), регистра на выходе (EDO) и пакетной передачи с использованием счетчика (BEDO) формула работы SDRAM: 5-1-1-1. Но за счет синхронизации работы с системной шиной SDRAM может работать на частотах до 133 МГц.

Для определения пропускной способности шины используется формула:

$$\text{Пропускная способность (Мбайт/с)} = \text{частота (МГц)} \times \text{разрядность (байт)}.$$

Разрядность шины данных в современных компьютерах равна 64 битам, или 8 байтам.

Пример. Спецификация PC100 памяти SDRAM предполагает частоту 100 МГц. Пропускная способность = 100 МГц · 8 байт = 800 Мбайт/с.

Спецификация PC133 рассчитана на частоту 133 МГц, что дает пропускную способность 1064 Мбайт/с.

- *DDR SDRAM (Double Data Rate SDRAM* – удвоенная скорость передачи данных SDRAM). Вариант памяти SDRAM позволяет при той же тактовой частоте удвоить скорость передачи данных, так как передача информации осуществляется в пакетном режиме по обоим фронтам тактового сигнала (см. рис. 3.43).

Основные спецификации:

Спецификация	Частота шины памяти	Пропускная способность
PC1600 (DDR200)	100 МГц	1600 Мбайт/с
PC2100 (DDR266)	133 МГц	2100 Мбайт/с
PC2700 (DDR333)	166 МГц	2700 Мбайт /с
PC3200 (DDR400)	200 МГц	3200 Мбайт /с

При использовании двухканального доступа к памяти производительность удваивается.

- *DDR2 SDRAM* – стандарт, утвержденный в 2004 году. Он является дальнейшим развитием стандарта DDR SDRAM, передача также происходит по фронту и по спаду тактового импульса, таким образом, скорость передачи данных также удваивается. При одинаковой частоте микросхемы памяти DDR и DDR2 будут иметь одинаковую пропускную способность. Отличия DDR2 от DDR заключаются в техническом усовершенствовании микросхем¹, что позволяет существенно повышать тактовую частоту и снижать энергопотребление.

Основные спецификации DDR2 SDRAM представлены в таблице:

Спецификация	Частота шины памяти	Пропускная способность
PC2-3200 (DDR2-400)	200 МГц	3200 Мбайт/с
PC2-4200 (DDR2-533)	266 МГц	4200 Мбайт/с
PC2-6400 (DDR2-800)	400 МГц	6400 Мбайт /с
PC2-8500 (DDR2-1066)	533 МГц	8500 Мбайт /с

¹ См. стандарт JESD79-2F по DDR3 SDRAM Совета JEDEC (Joint Electronic Device Engineering Council – Объединенный инженерный совет по электронным устройствам) (<http://www.jedec.org/standards-documents/docs/jesd-79-2e>).

В случае двухканального режима доступа к памяти пропускная способность удваивается.

- *DDR3 SDRAM* – новый стандарт памяти SDRAM. Первые микросхемы DDR3 появились на рынке в 2007 году. Дальнейшие технические улучшения (использование более совершенных транзисторов и уменьшение их размеров) привели к возможности повышения тактовых частот. Начальная частота этого вида памяти – 800 МГц.

Основные спецификации DDR3 SDRAM представлены в таблице:

Спецификация	Частота шины памяти	Пропускная способность
PC3-6400 (DDR3-800)	400 МГц	6400 Мбайт/с
PC3-8500 (DDR3-1066)	533 МГц	8500 Мбайт/с
PC3-12800 (DDR3-1600)	800 МГц	12800 Мбайт /с
PC3-17000 (DDR3-2133)	1066 МГц	17000 Мбайт /с

- *DDR4 SDRAM* – разрабатываемый стандарт (выход на рынок ожидается в 2012 году). Характеризуется очередным повышением тактовых частот шины памяти (1066–2133 МГц) и снижением энергопотребления.

- *RDRAM (Rambus DRAM)*. Наиболее очевидные способы повышения эффективности работы процессора с памятью – увеличение тактовой частоты либо ширины шины данных (количества одновременно пересылаемых разрядов). Совмещение обоих вариантов несет технические трудности. В синхронных DRAM применяется широкая шина (64 бита) при ограниченной частоте (хотя, как видно из обзора спецификаций DDR–DDR4, частота все время повышается).

Принципиально отличный подход предложен фирмой Rambus в середине 1990-х годов. Упор сделан на повышение тактовой частоты при уменьшении ширины выборки до 16 бит.

Так же, как и DDR, микросхема RDRAM передает данные по обоим фронтам тактового сигнала. Разработан принципиально новый интерфейс Rambus Channel, который имеет гораздо меньшее число линий, чем SDRAM: 3 линии для адреса строки, 5 линий для адреса столбца, 16 линий для данных. Вместо обычной адресации посылаются пакеты, включающие команды, адреса и данные. Вначале посылается пакет запросов, на который память отвечает пакетом подтверждения, после чего идет пакет данных. Чем больше данных нужно передать за один раз, тем RDRAM эффективнее.

Вследствие высокой стоимости, а затем и отставания по производительности от DDR SDRAM, микросхемы памяти типа RDRAM использовались в компьютерах крайне редко.

В таблице приведены некоторые спецификации:

Спецификация	Ширина шины памяти	Частота шины памяти	Пропускная способность
RIMM 2400	16 бит	600 МГц	2400 Мбайт/с
RIMM 3200	32 бита	400 МГц	3200 Мбайт/с
RIMM 4800	32 бита	600 МГц	4800 Мбайт /с
RIMM 6400	32 бита	800 МГц	6400 Мбайт /с

Фирма Rambus в 2000-х годах на основе RDRAM разработала ещё два вида ОЗУ – XRD DRAM и XDR2 DRAM, однако они также не получили широкого распространения (используются в Sony PlayStation).

3.4.8. Конструктивное исполнение

Конструктивно элементы оперативной памяти выполняются в виде модулей – плат с напаянными на них одной или несколькими микросхемами. Эти модули вставляются в слоты системной платы.

В настоящее время существуют следующие типы модулей:

- SIMM (*Single In line Memory Module* – модуль памяти с однорядным расположением выводов). Название SIMM говорит о том, что контакты модуля располагаются с одной стороны. Выпускались 30- и 72-контактные модули. Это устаревший тип, использовавшийся до появления памяти SDRAM.

- DIMM (*Dual In line Memory Module* – модуль памяти с двухрядным расположением выводов). В этом типе контакты располагаются по обеим сторонам модуля. Имеются следующие основные виды модулей:

- 168-контактные (DIMM-168) – используются в SDRAM;
- 184-контактные (DIMM-184) – используются в DDR SDRAM;
- 240-контактные (DIMM-240) – используются в DDR2 и DDR3.

- RIMM (*Rambus In line Memory Module*). Внешне похож на DIMM. Модули RIMM требуют интенсивного охлаждения. Это связано со значительным энергопотреблением и тепловыделением. Поэтому микросхемы памяти на плате модуля RIMM закрыты радиатором.

3.4.9. Постоянные запоминающие устройства

ПЗУ (постоянное запоминающее устройство, *ROM* – *Read-Only Memory*) – память, используемая для хранения неизменяемой или редко изменяемой информации.

Микросхема ПЗУ, расположенная на материнской плате, содержит BIOS – базовую систему ввода-вывода (Basic Input/Output System) – независимую от операционной системы программу поддержки операций с экраном, клавиатурой и другими периферийными устройствами. При включении компьютера BIOS выполняет следующие функции:

- 1) тестирование и инициализация основных устройств компьютера – процессора, оперативной памяти, чипсета, дисковых устройств;
- 2) начало загрузки операционной системы с одного из устройств внешней памяти – винчестера, CD/DVD-дисководов или флоппи-дисководов.

После того, как загрузочное устройство определено и на нем найдена операционная система, управление передается специальной программе-загрузчику, которая является частью операционной системы.

По технологии записи информации можно выделить следующие типы ПЗУ:

- масочные ПЗУ (ROM) – микросхемы, программируемые только при изготовлении;
- программируемые ПЗУ (ППЗУ), или PROM (Programmable ROM), – микросхемы, программируемые однократно после изготовления с помощью специальных устройств – программаторов;
- перепрограммируемые ПЗУ (EPROM – Erasable PROM) – микросхемы, программируемые многократно. К ним относятся:
 - UV-EPROM (Ultra-Violet EPROM) – микросхемы, стираемые ультрафиолетовым излучением;
 - EEPROM (Electrical EPROM) – микросхемы, стираемые электрически.

В настоящее время в ВМ чаще всего в качестве ПЗУ используют микросхемы *флэш-памяти* (Flash memory), которые можно отнести к классу EEPROM. Первые микросхемы флэш-памяти были предложены фирмой Intel в 1988 году. Благодаря особой технологии построения ячеек флэш-память может перепрограммироваться прямо в компьютере. Время доступа – 35–200 нс. В отличие от ОЗУ флэш-память нельзя переписывать побайтно. Запись осуществляется блоками (от 256 байт до 128 Кбайт). Перед тем как записать информацию в блок, этот блок должен быть стерт.

3.5. Шины

Шина (bus) – группа проводников, соединяющих несколько устройств и передающих сигналы между ними.

Характеристики шин:

- частота;
- разрядность, или ширина, шины;
- пропускная способность = частота × разрядность;
- способ передачи сигнала:
 - *параллельные шины* – одновременно (параллельно) по шине передается несколько (от единиц до сотен) сигналов данных;
 - *последовательные шины* – сигналы данных передаются один за другим (последовательно);
- назначение:
 - *системные шины*, или шины «процессор-память» (*Front-Side Bus, FSB*), – шины между процессором и основной памятью (или северным мостом чипсета – см. параграф 3.6 «Системные платы»);
 - *внутренние шины* – шины для соединения устройств в рамках системной платы (например, PCI, PCI-Express, AGP);
 - *шины накопителей* – шины для подключения устройств внешней памяти (например, ATA, Serial ATA, SCSI);
 - *периферийные шины* – шины для подключения внешних медленных устройств (например, IEEE1284 (LPT), RS-232 (COM), PS/2);
 - *универсальные шины* – шины для подключения широкого класса устройств (например, USB);
- количество подключаемых устройств;
- протокол;
- арбитраж;
- синхронизация.

Операции на шине называют *транзакциями*. Основные виды транзакций – *транзакции чтения* и *транзакции записи*.

Когда два устройства обмениваются информацией по шине, одно из них должно инициировать обмен и управлять им. Такое устройство называют *ведущим (bus master)*. Устройства, не обладающие возможностями инициирования транзакций, называются *ведомыми (bus slave)*.

3.5.1. Состав шины

Шина может включать следующие группы проводников (которые также называются шинами):

- шина адреса;
- шина данных;
- шина управления;
- шина питания.

Например, на рис. 3.36 (схема памяти с одним дешифратором) шину адреса составляют сигналы $A0-A1$, шину данных – $D0-D2$ и $Q0-Q2$, шину управления – CS и WR . Шина питания на этой схеме не показана.

Любая транзакция начинается с выставления ведущим устройством адреса. На *шине адреса* могут выдаваться адреса ячеек памяти, номера регистров процессора, адреса портов ввода-вывода. Число линий, выделенных для передачи адреса, определяет максимально возможный размер адресного пространства.

Как уже отмечалось, размер адресного пространства равен 2^N , где N – число адресных линий.

Пример. Первый персональный компьютер IBM PC с процессором Intel 8088 содержал 20-разрядную шину адреса. Следовательно, процессор мог обращаться к $2^{20} = 1$ Мбайт памяти.

Пример. Процессор Pentium 4 имеет 36-разрядную шину адреса, что позволяет адресовать $2^{36} = 64$ Гбайт памяти.

Для передачи данных служит *шина данных*. Ширина (разрядность) шины данных определяет количество битов, которое может быть передано по шине за одну транзакцию (цикл шины). Цикл шины следует отличать от периода тактовых импульсов – одна транзакция на шине может занимать несколько тактовых импульсов. Пропускная способность определяет скорость передачи данных именно по шине данных.

Пример. Внутренняя шина ISA (компьютер IBM PC) содержит 16-разрядную шину данных, частота шины – 8 МГц.

Пропускная способность = 2 байта $\times$ 8 МГц = 16 Мбайт/с.

Пример. В системной шине FSB процессора Pentium 4 ширина шины данных – 64 разряда, частота – до 800 МГц.

Пропускная способность = 8 байт $\times$ 800 МГц = 6,4 Гбайт/с.

Можно увеличить пропускную способность шин двумя способами:

1) увеличить ширину шины данных. Но в этом случае увеличиваются *взаимные помехи*, наводимые сигналами в одном проводнике на сигналы в другом, и наоборот;

2) увеличить частоту работы шины. При этом усиливается явление *перекоса*. Явление *перекоса* состоит в том, что сигналы, одновременно посланные по разным проводникам шины, достигают пункта назначения в разное время (вследствие отличия характеристик проводников шины и электронных схем, через которые проходят сигналы).

Современная тенденция заключается в преобладании последовательных шин, поскольку оказывается эффективнее увеличивать частоту, а не ширину шины.

Пример. Внутренняя шина PCI – параллельная (32 или 64 разряда данных), а новая шина PCI-Express – последовательная.

Пример. Периферийная шина для подключения внешних накопителей ATA (IDE) – параллельная, в ней 40 или 80 проводов, а новая шина Serial ATA – последовательная, в ней 7 проводов (пара на передачу данных, пара – на прием, 3 – «земля»).

По *шине управления* передаются следующие сигналы:

- управления транзакциями;
- состояния устройств;
- арбитража;
- прерываний;
- синхронизации.

В некоторых шинах линии адреса и данных объединены в единую *мультиплексируемую шину адреса/данных*. Сначала линии шины используются для передачи адреса, а затем те же самые линии – для передачи данных. Объединение линий сокращает ширину и стоимость шины, но замедляет ее работу.

Пример. В шине PCI используются 32 или 64 объединенные линии для данных и адреса.

Рассмотрим последовательность событий, происходящих на шине во время одной транзакции (рис. 3.44). Сначала устройство, которое является ведущим (Bus master), получает управление шиной, после чего оно может выдавать на шину свои данные. Через некоторый интервал времени, определяемый скоростью распространения сигналов по шине, данные достигают ведомого устройства (Bus slave). Скорость распространения сигналов обыч-

но не превышает 70% скорости света (300 000 км/с). После появления данных на ведомом устройстве выдерживается пауза, для того чтобы сигнал стал устойчивым. Затем сигнал может быть считан и удален с шины.

На рис. 3.44 приведены примерные значения интервалов для шины частотой 133 МГц.

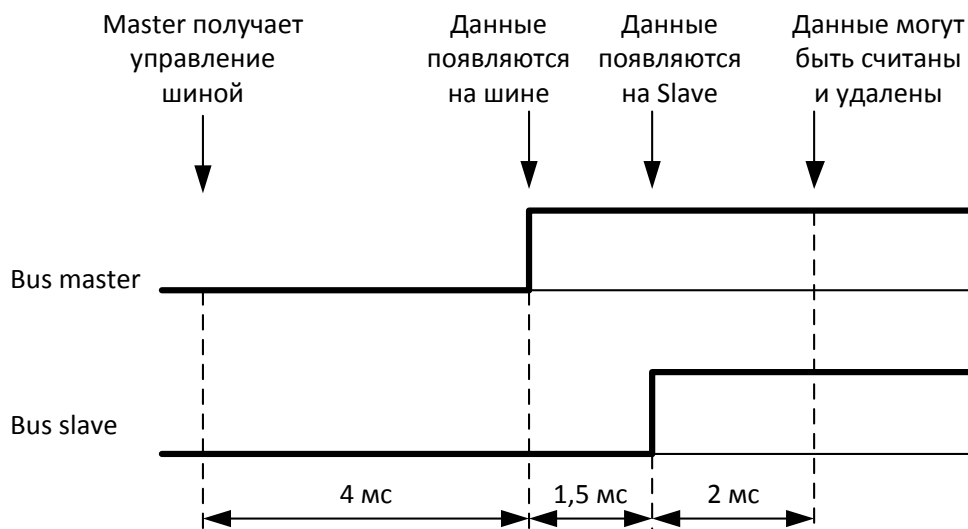


Рис. 3.44. Диаграмма пересылки данных

3.5.2. Арбитраж шин

На роль ведущего могут претендовать одновременно несколько устройств (центральный процессор, сопроцессор, устройства ввода-вывода), однако управлять шиной в каждый момент времени может только одно из них. Чтобы исключить конфликты, вводится механизм *арбитража*.

Арбитраж может быть централизованным и децентрализованным. При централизованном арбитраже в системе имеется специальное устройство – центральный арбитр, который в соответствии с приоритетами каждого устройства предоставляет им доступ к шине. При децентрализованном арбитраже каждое ведущее устройство содержит блок управления доступом к шине. При совместном использовании шины такие блоки взаимодействуют друг с другом, разделяя доступ.

Пример. Внутренняя шина PCI является шиной с централизованным арбитражем. Периферийная шина SCSI – шина с децентрализованным арбитражем.

3.5.3. Протокол шины

Вследствие явления перекоса шины все устройства, использующие шины, должны «знать», когда адреса, данные и управляющие сигналы следует считать достоверными. Метод, выбираемый для информирования о до-

стоверности адресов, данных и сигналов управления шины, называется *протоколом* шины.

Существует два основных класса протоколов:

1) *синхронные* – все сигналы «привязаны» к импульсам тактового генератора. Изменение управляющих сигналов совпадает с фронтом или спадом тактового импульса (см. параграф 3.4.7: SDRAM);

2) *асинхронные* – начало очередного события на шине определяется не тактовым импульсом, а предыдущим событием (см. параграф 3.4.7: традиционная DRAM, FPM, EDO и BEDO).

В прошлом разработчики ВМ предпочитали асинхронные шины, сейчас чаще используются синхронные. Современные синхронные шины несколько быстрее асинхронных, поэтому применяются в канале «процессор-память», а асинхронные – для периферийных устройств.

Пример. Системная шина Pentium 4, так же как внутренняя шина PCI, является синхронной, шина для накопителей ATA (IDE) асинхронная, а шина для накопителей SCSI может работать как по синхронному, так и по асинхронному протоколам.

3.6. Системные платы

Системная, или материнская, плата (motherboard, или mainboard) – это часть компьютера, объединяющая между собой основные электронные компоненты машины. С помощью материнской платы осуществляется физическое соединение и взаимодействие между устройствами компьютера.

Современные системные платы имеют структуру с иерархией шин (рис. 3.45).

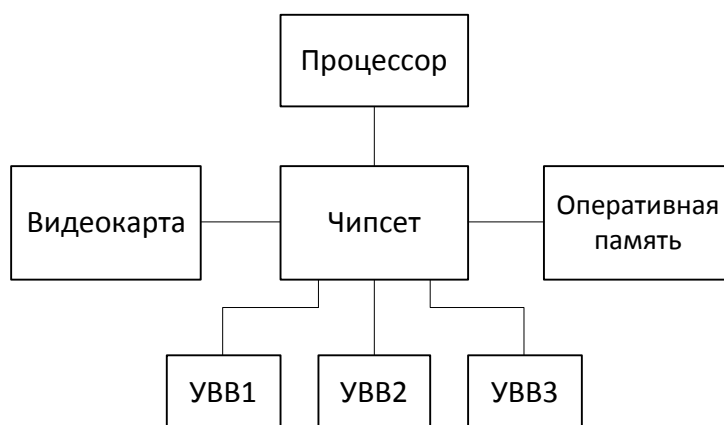


Рис. 3.45. Структура ВМ с иерархией шин

В структуре с иерархией шин помимо системной шины (между процессором и памятью) существует ряд дополнительных шин. Каждая шина имеет свою пропускную способность, достаточную для устройств, которые

она связывает. Наиболее производительные шины должны соединять процессор, оперативную память и видеокарту.

Контролирует взаимодействие всех устройств в такой структуре *чипсет* (*chipset* – набор микросхем). Обычно принято деление чипсета на две части – *северный мост* (*northbridge*), который отвечает за взаимодействие быстродействующих устройств (процессор, оперативную память и видеокарту), и *южный мост* (*southbridge*), соединяющий устройства ввода-вывода (УВВ).

Физически системная плата представляет собой печатную плату, на которой располагаются некоторые микросхемы, остальные компоненты объединяются при помощи системной шины и устанавливаются на дополнительных платах (платах, или картах, расширения), помещаемых в специальные разъемы (слоты), имеющиеся на материнской плате.

Структурная схема типичной системной платы приведена на рис. 3.46.

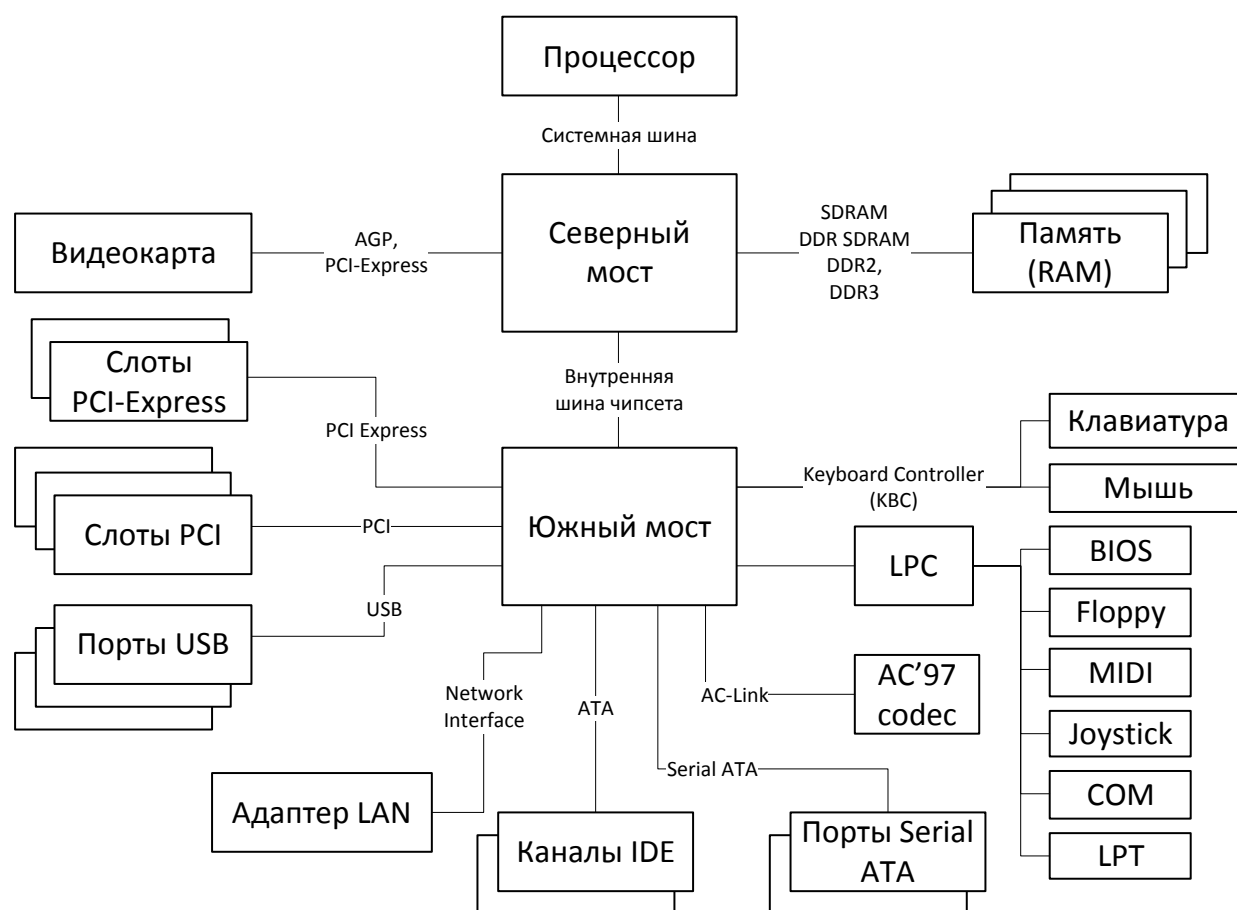


Рис. 3.46. Структурная схема системной платы

Характеристики системных плат:

- *Процессорный разъем* определяет типы поддерживаемых процессоров.

Существуют следующие процессорные разъемы:

- Intel: Socket 370, Socket 423, Socket 478, Socket T (LGA 775), Socket B (LGA 1366), Socket H (LGA 1156), Socket H2 (LGA 1155), Socket H3 (LGA 1150), Socket B2 (LGA 1356), Socket R (LGA 2011) и др.;
- AMD: Socket A, Socket 754, Socket 939, Socket 940, Socket AM2, Socket F, Socket AM3, Socket FM1, Socket FM2 и др.

- *Форм-фактор* (типоразмер) определяет размеры системной платы. Перечислим некоторые популярные форм-факторы:

- ATX (Advanced Technology eXtended) – один из самых распространенных форматов материнских плат для ПК, идеально подходит для построения домашнего компьютера. Платы ATX имеют размеры 30,5×24,4 см и поддерживают семь слотов расширения.

- mATX (micro ATX) – несколько уменьшенный по размерам стандарт ATX. Подходит для построения офисных компьютеров, когда не требуется много слотов для расширения системы. Платы mATX имеют размеры 24,4×24,4 см и поддерживают четыре слота расширения.

- EATX (Extended ATX) материнские платы отличаются от ATX размерами (до 30,5×33,0 см), требуют специальных блоков питания (24 контакта, в отличие от 20 для ATX) и используются в основном для серверов.

- BTX (Balanced Technology Extended) – новый стандарт, который приходит на смену ATX. При разработке этого форм-фактора большое внимание уделялось эффективному охлаждению установленных на плате элементов. BTX идеально подходит для построения миниатюрных компьютеров. Материнские платы BTX имеют размеры 26,7×32,5 см и поддерживают семь слотов расширения.

- mBTX (micro BTX) – уменьшенный вариант BTX. Размеры таких плат составляют 26,7×26,4 см. mBTX поддерживают четыре слота расширения.

- SSI EEB (Server Standards Infrastructure Entry Electronics Bay). Материнские платы этого стандарта обычно служат для построения серверов. Габариты таких плат составляют 30,5×33,0 см.

- SSI CEB (SSI Compact Electronics Bay). Этот стандарт также используется в серверах. Габариты – 30,5×25,9 см.

- *Тип чипсета* определяет количество и виды всех подключаемых к системной плате устройств, а также характеристики шин системной платы.

- *Подсистема оперативной памяти:*

- тип оперативной памяти;
 - тактовая частота шины памяти;
 - количество каналов памяти;
 - количество слотов памяти;
 - максимальная емкость поддерживаемой памяти.

- *Интегрированные устройства:*
 - видеокарта;
 - звуковая карта;
 - адаптер локальной сети;
 - модем.
- *Разъемы (порты) ввода вывода:*
 - разъемы для подключения внешней памяти (ATA, Serial ATA, SCSI);
 - разъемы для универсальной шины USB;
 - сетевые разъемы (Ethernet);
 - разъемы для периферийных устройств (клавиатура, мышь, аудио, COM и LPT-порты).
- *Версия BIOS (Basic Input-Output System)* – определяет возможности системной платы по начальному тестированию устройств компьютера и загрузке операционной системы.
- *CMOS память* – размещается на системной плате или в составе чипсета. Питается от аккумулятора (батарейки), поэтому энергонезависима (сохраняет информацию при отключении компьютера от сети). Память хранит информацию о параметрах многих устройств, входящих в компьютер, а также о системном времени. Информация может изменяться.

Контрольные вопросы и задания

1. Что такое *сигнал*?
2. Что такое *полупроводник*?
3. Постройте на вентилях И-НЕ следующие элементы: инвертор, И, ИЛИ, Исключающее ИЛИ.
4. Приведите примеры схем, в которых вентиль AND используется в качестве ключа.
5. Чем ситуация *переполнения* отличается от ситуации *переноса*?
6. Каким образом из 8 одноразрядных АЛУ построить одно 8-разрядное АЛУ? Какие входы будут общими для всех АЛУ, а какие различными?
7. Зачем нужна иерархия запоминающих устройств?
8. Почему состояние триггера при входах $R = S = 0$ называется *хранением*?
9. Можно ли построить RS-триггер на вентилях И-НЕ? Что при этом изменится?
10. Что такое *синхронизация*?
11. С чем синхронизирует триггер сигнал C ?
12. Постройте сдвиговый регистр на D-триггерах.
13. Чем ОЗУ принципиально отличаются от ПЗУ?

14. Почему ОЗУ на триггерах называют *статическим*, а ОЗУ на полупроводниковых конденсаторах – *динамическим*?
15. Что такое *третье состояние* в буферных элементах? Зачем оно нужно?
16. Поясните роль сигналов CAS и RAS.
17. За счет чего достигается уменьшение времени доступа памяти в схемах FPM, EDO, BEDO по сравнению с традиционной DRAM?
18. С чем синхронизирована память SDRAM?
19. Почему DDR SDRAM в два раза производительнее обычной SDRAM?
20. Назовите способы увеличения пропускной способности шин и возникающие при этом проблемы.
21. Очевидно, что параллельная шина будет работать быстрее, чем последовательная. Почему в таком случае последовательная шина Serial ATA является более производительной, чем параллельная шина Parallel ATA?
22. Почему две основные микросхемы чипсета называют «северным» и «южным» мостами?
23. * Какой минимальный набор операций должно обеспечивать АЛУ для реализации сколько угодно сложных арифметических и логических операций с целочисленными данными? Ответ обоснуйте.
24. * Почему в формулах работы памяти фигурируют четыре числа (например, 5-5-5-5), а, допустим, не два?
25. * Почему память RDRAM не получила прогнозируемого превосходства над памятью типа DDR?
26. * Предложите свой способ повышения производительности памяти.

ГЛАВА 4. ОРГАНИЗАЦИЯ ВЫЧИСЛИТЕЛЬНОЙ МАШИНЫ

4.1. Модель процессора

При рассмотрении принципов работы процессора будем использовать учебную модель, показанную на рис. 4.1.

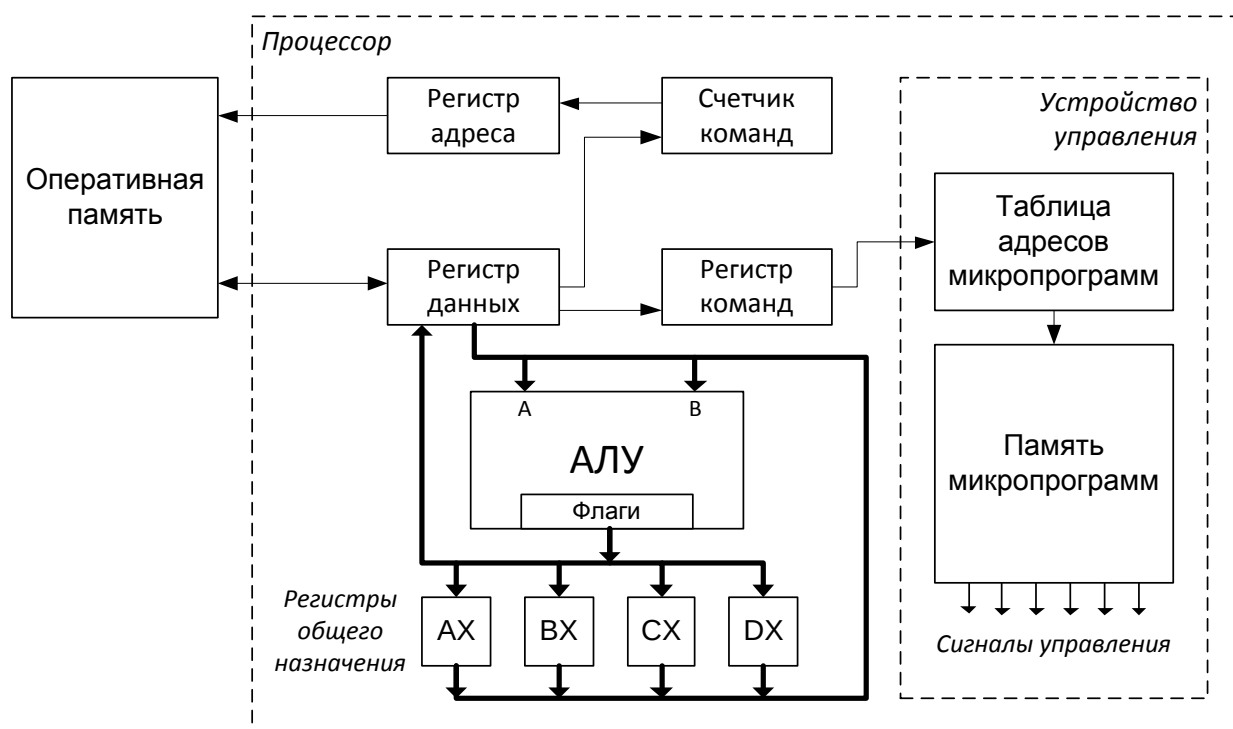


Рис. 4.1. Структура модели процессора

Учебная модель процессора состоит из следующих частей:

- микропроцессорная память (регистры);
- устройство управления (УУ);
- арифметико-логическое устройство (АЛУ).

4.1.1. Регистры

В состав учебной модели входят несколько регистров, которые можно объединить в следующие группы:

- управляющие регистры:
 - счетчик команд,
 - регистр команд;
- регистры работы с оперативной памятью:
 - регистр адреса,
 - регистр данных;

- регистры общего назначения (РОН):
– AX, BX, CX, DX.

Счетчик команд (англ. *Program Counter, PC* – программный счетчик или *Instruction Pointer, IP* – указатель на команду) – регистр, в котором хранится адрес выполняемой команды. Название «счетчик» происходит из-за того, что этот регистр после выполнения команды увеличивает свое содержимое на её длину, переходя, таким образом, к следующей команде (последовательное выполнение команд). Однако в случае выполнения команды перехода (условные и безусловные переходы, циклы) в счетчик команд записывается адрес точки перехода.

Регистр команд (Instruction Register) – регистр, в котором хранится код операции выполняемой команды.

Машинная команда (процессорная команда) состоит из двух частей – кода операции и адресной части. *Код операции* говорит о том, что нужно сделать с операндами, которые (или адреса которых) указаны в адресной части. Операндами могут быть: регистры, константы или адреса ячеек памяти (переменные).

По количеству операндов машинные команды делятся на следующие виды:

- *безадресные* – операнды команде не требуются или жестко заданы заранее;
- *одноадресные* – в команде явно указывается только один операнд, остальные операнды (если они нужны) жестко заданы;
- *двухадресные* – явно указываются два операнда, с которыми выполняется операция, результат операции обычно помещается в первый операнд;
- *трехадресные* – явно задаются операнды, с которыми нужно выполнить операцию и место для записи результата (регистр или адрес ячейки памяти).

Примеры машинных команд приведены на рис. 4.2 (коды операций взяты произвольно, точка с запятой в Ассемблере обозначает комментарий).

Примеры машинных команд (в шестнадцатеричном коде)				Тип	Обозначение на Ассемблере	Действие
Код операции				Безадресная	NOP	; Отсутствие операции
01						
Код операции				Безадресная	RET	; Возврат из процедуры
02						
Код операции		Операнд1		Одноадресная	INC AX	; Увеличить регистр AX на 1
03		01				
Код операции		Операнд1		Одноадресная	NOT X	; Проинвертировать содержимое переменной X, которая хранится по адресу EAh
04		EA				
Код операции		Операнд1	Операнд2	Двухадресная	MOV AX, BX	; Загрузить в регистр AX содержимое регистра BX
05		01	02			
Код операции		Операнд1	Операнд2	Двухадресная	ADD CX, 10	; Сложить содержимое регистра AX с константой 10, результат записать в регистр AX
06		01	0A			
Код операции	Операнд1	Операнд2	Операнд3	Трехадресная	MUL AX, BX, CX	; Умножить содержимое регистра AX на содержимое регистра BX, результат записать в регистр CX
07	01	02	03			

Рис. 4.2. Примеры машинных команд

В табл. 4.1 представлены обозначения на Ассемблере распространенных машинных команд.

Таблица 4.1. Ассемблерные команды

Операция	Обозначение на Ассемблере
Загрузка в регистр или переменную (move)	MOV
Сложение (addition)	ADD
Вычитание (subtraction)	SUB
Инкремент, увеличение на 1 (increment)	INC
Декремент, уменьшение на 1 (decrement)	DEC
Умножение (multiplication)	MUL
Деление (division)	DIV
Логическое сложение	OR

Операция	Обозначение на Ассемблере
Логическое умножение	AND
Логическое отрицание	NOT
Сдвиг влево (shift left)	SHL
Сдвиг вправо (shift right)	SHR
Переход на другую команду (jump – прыжок)	JMP
Сравнение (comparison)	CMP

В модели на рис. 4.1 имеются два регистра работы с оперативной памятью – регистр адреса и регистр данных. *Регистр адреса* предназначен для записи адреса оперативной памяти, с которым в настоящее время нужно работать – считывать или записывать информацию. Для временного хранения информации, считанной из ОЗУ, или той, которую нужно записать в ОЗУ, используется *регистр данных*.

Последняя группа регистров – *регистры общего назначения* AX, BX, CX, DX – предназначены для хранения данных, с которыми сейчас непосредственно работает процессор. Обычно регистров общего назначения немного в процессоре¹, поэтому при окончании работы с текущими данными их записывают из регистров в ОЗУ.

Все регистры учебной модели, как и ячейки памяти ОЗУ, 8-разрядные, поэтому и весь процессор является 8-разрядным. Реальные процессоры имеют гораздо больше регистров разной разрядности (от 8 до 128 разрядов); разрядность самого процессора определяется обычно разрядностью регистров общего назначения.

4.1.2. Арифметико-логическое устройство

Все данные, с которыми в настоящий момент работает процессор, проходят через *арифметико-логическое устройство*, которое при помощи сигналов управления можно настроить на выполнение арифметических или логических операций, а также просто на пересылку данных.

Сигналы управления вырабатываются устройством управления. Всего сигналов 30, обозначаются они Y1–Y30 (подробно они будут рассмотрены в параграфе 4.1.6). Для управления АЛУ используются 4 сигнала Y15–Y18; шестнадцать возможных комбинаций этих сигналов ($2^4 = 16$) могут настраивать АЛУ на выполнение 16 различных функций (табл. 4.2).

¹ Логических регистров немного, количество физических регистров в современных процессорах достаточно велико – см. параграф 4.2.2 «Суперскалярные процессоры».

Таблица 4.2. Функции АЛУ

Y15	Y16	Y17	Y18	Операция	Комментарий
0	0	0	0	Нет операции	
0	0	0	1	$A + B$	Сложение
0	0	1	0	$A + B + CF$	Сложение с флагом переноса
0	0	1	1	$A - B$	Вычитание
0	1	0	0	$A - B - CF$	Вычитание с флагом переноса
0	1	0	1	$A \text{ and } B$	Логическое умножение
0	1	1	0	$A \text{ or } B$	Логическое сложение
0	1	1	1	$A \text{ xor } B$	Исключающее ИЛИ
1	0	0	0	not A	Инверсия входа A
1	0	0	1	not B	Инверсия входа B
1	0	1	0	shl A	Сдвиг влево A
1	0	1	1	shr A	Сдвиг вправо A
1	1	0	0	A	Повторение на выходе входа A
1	1	0	1	B	Повторение на выходе входа B
1	1	1	0	$A + 1$	Инкремент A
1	1	1	1	$A - 1$	Декремент A

Например, при подаче на АЛУ следующей комбинации сигналов управления:

$$Y15 = 0, \quad Y16 = 0, \quad Y17 = 0, \quad Y18 = 1 \quad -$$

на выходе АЛУ появится сумма значений на входах A и B (поскольку все регистры 8-разрядные, то и входы A и B также 8-разрядные)

Каким образом процессор определяет, содержимое какого регистра подавать на входы A и B? И в какой регистр будет записан ответ? За это опять отвечают соответствующие сигналы управления – вход A и выход контролируются сигналами Y7–Y10, а вход B – сигналами Y11–Y14 (табл. 4.3 и 4.4).

Таблица 4.3. Управление входом А и выходом АЛУ

Y7	Y8	Y9	Y10	Вход А, выход
1	0	х	х	Регистр данных
0	1	х	х	Регистр адреса
0	0	0	0	Регистр АХ
0	0	0	1	Регистр ВХ
0	0	1	0	Регистр СХ
0	0	1	1	Регистр ДХ

х – не имеет значения

Таблица 4.4. Управление входом В

Y11	Y12	Y13	Y14	Вход В
1	0	х	х	Регистр данных
0	1	х	х	Регистр адреса
0	0	0	0	Регистр АХ
0	0	0	1	Регистр ВХ
0	0	1	0	Регистр СХ
0	0	1	1	Регистр ДХ

х – не имеет значения

Обратите внимание, что за вход А и выход АЛУ отвечает *одинаковый набор сигналов управления*. Это означает, что после выполнения заданной операции АЛУ результат будет записан в тот же регистр, содержимое которого поступало на вход А. Такое поведение является следствием использования в нашем процессоре двухадресных команд, в которых ответ попадает в один из операндов; при этом экономятся сигналы управления, следовательно, проще устройство процессора.

Пример. Сложить содержимое регистров АХ и ВХ (далее часто будем опускать слово «содержимое» и писать просто «сложить регистры АХ и ВХ»).

Для осуществления операции сложения нужно задать входы А и В (см. табл. 4.3 и 4.4) и настроить АЛУ на выполнение этой операции (см. табл. 4.2). Результат сложения запишется в тот же регистр, который поступал на вход А.

Ниже приведены необходимые сигналы управления.

Вход А и выход (регистр АХ): $Y7 = 0, Y8 = 0, Y9 = 0, Y10 = 0;$
вход В (регистр ВХ): $Y11 = 0, Y12 = 0, Y13 = 0, Y14 = 1;$
операция АЛУ (сложение): $Y15 = 0, Y16 = 0, Y17 = 0, Y18 = 1.$

Такой набор сигналов управления эквивалентен машинной команде:

ADD AX, BX ; AX = AX + BX

4.1.3. Флаги

Помимо выполнения арифметических и логических операций АЛУ отвечает за формирование флагов. *Флаг* – это ячейка памяти размером в один бит, в которой хранится определенная характеристика (признак) только что выполненной в АЛУ операции (мы встречались с флагами в параграфе 3.3.7 «Двоичная арифметика»).

В нашем процессоре пять флагов:

- CF (Carry Flag) – флаг переноса,
 - CF = 1, если в результате операции получился бит переноса, иначе CF = 0;
- OF (Overflow Flag) – флаг переполнения,
 - OF = 1, если есть переполнение, иначе OF = 0;
- ZF (Zero Flag) – флаг нуля,
 - ZF = 1, если результат равен нулю, иначе ZF = 0;
- SF (Signum Flag) – флаг знака,
 - SF = 1, если знаковый (старший) бит результата равен 1, иначе SF = 0;
- PF (Parity Flag) – флаг четности,
 - PF = 1, если количество единиц в получившемся результате четно, иначе PF = 0.

Флаги необходимы для проверки всевозможных условий.

Пример. Каким образом процессор может проверить равенство двух чисел А и В?

Процессор не может, как человек, просто посмотреть на два числа и решить, равны они или нет. Чтобы проверить равенство, процессор действует по следующему алгоритму:

1. Записывает число А в регистр (например, в АХ).
2. Записывает число В в другой регистр (например, в ВХ).
3. Вычитает из содержимого регистра АХ содержимое регистра ВХ, настраивая АЛУ на выполнение операции вычитания и подавая на входы соответствующие регистры.

4. При выполнении вычитания формируются флаги, в том числе флага ZF.

5. Процессор проверяет флаг ZF: если $ZF = 1$, то числа равны, иначе не равны.

4.1.4. Устройство управления

Мы уже знаем, что цифровые устройства, в том числе и процессор, для обеспечения синхронизации работают под действием равномерных тактовых импульсов (см. параграф 3.3.10 «Тактовый генератор»).

В одном такте каждый блок процессора выполняет определенное действие, задаваемое сигналами управления, которые вырабатывает *устройство управления*. Набор конкретных значений всех управляющих сигналов Y_1 – Y_{30} называется *микрокомандой*. Микрокоманда полностью определяет поведение процессора в данном такте.

Какова связь машинных команд, рассмотренных в параграфе 4.1.1 «Регистры», и микрокоманд? Набор машинных команд определяет, что программист может делать на данном процессоре; это функции, предоставляемые процессором во внешний мир, программисту. Набор микрокоманд определяет способ реализации и машинных команд; микрокоманды недоступны обычному программисту, с ними работает только разработчик процессора.

Одна машинная команда может быть реализована одной или несколькими микрокомандами. Естественно, чем машинная команда сложнее, тем больше микрокоманд требуется для её реализации. Набор микрокоманд, необходимых для одной машинной команды, называется *микропрограммой*.

Микропрограммы должны где-то храниться. Оказывается, в процессоре есть внутреннее ПЗУ со своими адресами, в которое при создании процессора записываются микропрограммы. Это ПЗУ называется *памятью микропрограмм*.

Для каждой машинной команды в процессоре записана своя микропрограмма. Каким образом процессор определяет соответствие машинной команды и микропрограммы? За это отвечает таблица адресов микропрограмм. В этой таблице для каждого кода операции, который поддерживается процессором, указывается адрес в памяти микропрограмм, по которому записана соответствующая данному коду операции микропрограмма (рис. 4.3).

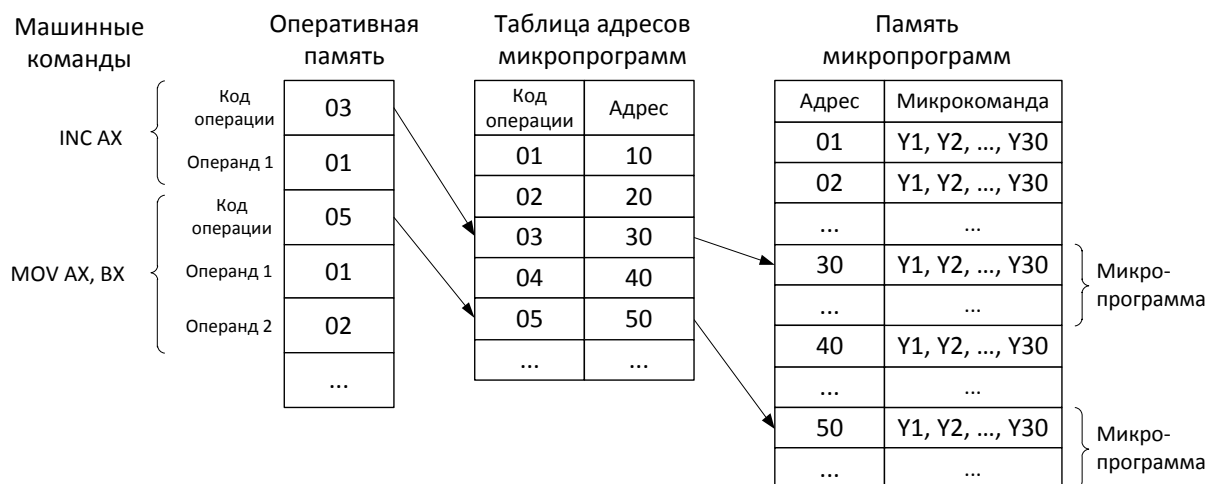


Рис. 4.3. Соответствие кода операции адресу микропрограммы

Кроме микропрограмм для машинных команд нужна общая последовательность микрокоманд, отвечающая за постоянное считывание машинных команд из памяти.

4.1.5. Цикл команды

Совокупность действий процессора, происходящих при чтении и исполнении машинной команды, называется *циклом команды*.

Цикл команды включает следующие этапы:

- 1) выборка команды;
- 2) декодирование команды;
- 3) исполнение команды;
- 4) запись результата;
- 5) формирование адреса следующей команды.

1. *Выборка команды* – это чтение из ОЗУ очередной машинной команды.

Как процессор узнает, откуда брать очередную команду? В параграфе 4.1.1 «Регистры» мы говорили об одном из регистров процессора – *счетчике команд*. Именно в нем хранится адрес команды, которую сейчас необходимо выполнить. Этот адрес из счетчика команд записывается в регистр адреса, оперативная память настраивается на чтение, и в регистр данных считывается код операции текущей команды. Затем код операции из регистра данных переписывается в регистр команд. На этом этап выборки команды завершается.

2. *Декодирование команды* – определение по коду операции адреса в памяти микропрограмм, где располагается соответствующая коду операции микропрограмма.

За декодирование команды отвечает регистр команды, в котором после этапа выборки находится код операции, и таблица адресов микропрограмм. Код операции из регистра команды поступает на таблицу адресов, по которой определяется адрес требуемой микропрограммы.

3. *Исполнение команды* – осуществление необходимых для данной машинной команды действий:

- подготовка входов А и В арифметико-логического устройства, в том числе загрузка при необходимости операнда из ОЗУ в регистр данных;
- настройка АЛУ на нужную операцию;
- выполнение операции с формированием флагов.

4. *Запись результата* – пересылка результата операции в нужный регистр или ячейку памяти.

5. *Формирование адреса следующей команды* – если текущая команда не являлась командой перехода (условного или безусловного), т. е. не нарушала естественной последовательности команд программы, то значение счетчика команд увеличивается на величину размера текущей команды, так что в счетчике команд будет храниться адрес следующей команды.

Если текущая команда была командой перехода, то в счетчик команд записывается значение адреса точки перехода, которое является операндом данной команды.

Цикл команды проиллюстрирован на рис. 4.4.



Рис. 4.4. Цикл команды

4.1.6. Сигналы управления

Цикл команды и микропрограммы, реализующие машинные команды, представляют собой последовательности микрокоманд, которые, в свою очередь, являются наборами тридцати сигналов управления Y1–Y30.

Все сигналы управления делятся на три группы:

- 1) Y1–Y6 – отвечают за управляющие регистры и регистры работы с ОЗУ;
- 2) Y7–Y18 – настраивают АЛУ на выполнение требуемой операции и задают входы АЛУ;
- 3) Y19–Y30 – управляют переходом на следующую команду в памяти микропрограмм.

Сигналы управления первой группы приведены в табл. 4.5.

Таблица 4.5. Сигналы для управляющих регистров и регистров работы с ОЗУ

Сигнал управления	Действие
Y1 = 1	Увеличение счетчика команд на единицу
Y2 = 1	Пересылка содержимого счетчика команд в регистр адреса
Y3 = 1	Пересылка содержимого регистра данных в счетчик команд
Y4 = 1	Пересылка содержимого регистра данных в регистр команд
Y5 = 1	Запись содержимого регистра данных в ячейку ОЗУ, адрес которой указан в регистре адреса
Y6 = 1	Чтение в регистр данных содержимого ячейки ОЗУ, адрес которой указан в регистре адреса

Сигналы управления второй группы уже были рассмотрены в параграфе 4.1.2 «Арифметико-логическое устройство».

Чтобы разобраться с сигналами управления третьей группы, нужно понять один важный момент. Давайте ответим на вопрос: что управляет процессором? Кажется, ответ очевиден – последовательность машинных команд в оперативной памяти. На самом деле все не так просто: если бы внутри процессора не была реализована последовательность микрокоманд, отвечающая за цикл команды, процессор просто не стал бы считывать и выполнять машинные команды.

Таким образом, процессором управляют микрокоманды, хранящиеся в памяти микропрограмм и образующие своего рода внутреннюю программу процессора. Как и в любой компьютерной программе, в этой внутренней программе требуются возможность перехода на любой адрес и продолжение

выполнения с микрокоманды, находящейся по этому адресу. За такую возможность отвечает группа сигналов управления Y19–Y30. При этом сигналы Y19–Y22 определяют вид и условие перехода (табл. 4.6), а сигналы Y23–Y30 кодируют восьмиразрядный адрес перехода.

Таблица 4.6. Сигналы управления ходом вычислительного процесса в памяти микропрограмм

Y19	Y20	Y21	Y22	Условие перехода
0	0	0	0	Переход по следующему адресу
0	0	0	1	Переход, если флаг CF сброшен
0	0	1	0	Переход, если флаг OF сброшен
0	0	1	1	Переход, если флаг ZF сброшен
0	1	0	0	Переход, если флаг SF сброшен
0	1	0	1	Переход, если флаг PF сброшен
0	1	1	0	Переход по следующему адресу
0	1	1	1	Безусловный переход по адресу
1	0	0	0	СТОП
1	0	0	1	Переход, если флаг CF установлен
1	0	1	0	Переход, если флаг OF установлен
1	0	1	1	Переход, если флаг ZF установлен
1	1	0	0	Переход, если флаг SF установлен
1	1	0	1	Переход, если флаг PF установлен
1	1	1	0	Переход по следующему адресу
1	1	1	1	Переход по адресу из таблицы адресов

В табл. 4.6 представлены следующие виды переходов:

- условный переход – выполнение перехода определяется установленным или сброшенным флагом; если условие не выполняется, то происходит переход на следующую по порядку команду;
- безусловный переход – переход выполняется всегда, независимо от значений флагов;
- выполнение команды по следующему за текущей командой адресу (в связи с избыточностью кодирования соответствующих комбинаций сигналов Y19–Y22 несколько);
- переход по адресу из таблицы адресов – реализация этапа декодирования цикла команды.

Кроме того, присутствует команда СТОП – остановка выполнения процессором цикла команды.

Память микропрограмм в нашем процессоре имеет емкость 256 тридцатиразрядных слов, поэтому для определения адреса перехода достаточно 8 бит ($2^8 = 256$). За адрес перехода отвечают сигналы Y23–Y30.

4.1.7. Примеры

Рассмотрим примеры реализации машинных команд в нашей учебной модели процессора.

Пример 1. Реализовать цикл команды.

Для реализации цикла команды нужно записать микрокоманды для каждого его этапа (см. параграф 4.1.5 «Цикл команды»).

1) *Выборка команды.* Требуется по адресу из счетчика команд прочитать из ОЗУ код операции и записать его в регистр команды.

Первая микрокоманда – передадим содержимое счетчика команд в регистр адреса: Y2 = 1;

переход по следующему адресу: Y19–Y22 = 0000.

Вторая микрокоманда – прочитаем из ОЗУ код операции в регистр данных: Y6 = 1;

переход по следующему адресу: Y19–Y22 = 0000.

Третья микрокоманда – передадим содержимое регистра данных в регистр команд: Y4 = 1;

переход по следующему адресу: Y19–Y22 = 0000.

2) *Декодирование команды.* Нужно найти в таблице адресов микропрограмм строку, соответствующую содержимому регистра команд, в котором хранится код операции, и перейти по найденному адресу в памяти микропрограмм. За все эти действия отвечает последовательность сигналов управления Y19–Y22 = 1111.

Первые два этапа цикла команды выполняются одинаково для каждой машинной команды; удобнее всего записать их в первых четырех адресах памяти микропрограмм. После выполнения любой из микропрограмм нужно будет возвращаться на начало цикла команды, т. е. на нулевой адрес.

Оставшиеся три этапа цикла команды зависят от конкретной машинной команды и будут рассмотрены в следующих примерах.

Пример 2. Реализовать машинную команду MOV AX, const (загрузить в регистр AX константу).

Сначала нужно придумать код операции для указанной машинной команды. В нашем процессоре код операции выбирается произвольно, в реальных процессорах при назначении кодов операций стараются минимизировать длину кода и сделать процесс декодирования максимально простым, чтобы уменьшить количество элементов процессора.

Пусть код операции для загрузки в регистр константы будет 01.

Рассматриваемая машинная команда является двухадресной, т. е. в ней присутствуют два операнда – регистр-приемник и сама константа. Давайте для простоты считать, что в нашей команде указывается только один операнд – константа, а загружается она всегда в регистр АХ. Тогда, чтобы загрузить константу в другой регистр, например ВХ, мы должны будем придумать другой код операции, и это будет уже другая машинная команда.

Итак, машинная команда, например, MOV АХ, 10 будет представлена в оперативной памяти двумя байтами: 01 (код операции) и 0А (константа 10 в шестнадцатеричном виде).

Далее нужно определить, с какого адреса в памяти микропрограмм будет располагаться микропрограмма, соответствующая заданной машинной команде. Это делается также произвольно – возьмем, например, адрес 10₁₀.

Поскольку мы уже рассмотрели первые два этапа цикла команды, нам нужно реализовать оставшиеся три этапа. Напомним, что второй этап закончился переходом на микропрограмму, соответствующую прочитанному коду операции. Поскольку мы договорились, что нашу микропрограмму располагаем по адресу 10, то, чтобы перейти на правильный адрес, нужно записать в таблицу адресов микропрограмм напротив кода 01 значение адреса 10 (в десятичном виде).

Далее наступает третий этап цикла команды – исполнение команды. Нужно прочитать константу из памяти и записать её в регистр АХ. Константа располагается в ОЗУ по следующему адресу после кода операции. Адрес кода операции есть в счетчике команд, поэтому увеличиваем счетчик команд на единицу и считываем по этому адресу байт в регистр данных.

Первая микрокоманда: Y1 = 1 (счетчик команд + 1);
Y19–Y22 = 0000 (переход по следующему адресу).

Вторая микрокоманда: Y2 = 1 (счетчик команд записываем в регистр адреса);
Y19–Y22 = 0000 (переход по следующему адресу).

Третья микрокоманда: Y6 = 1 (читаем из памяти в регистр данных);
Y19–Y22 = 0000 (переход по следующему адресу).

Затем нужно записать значение из регистра данных в регистр АХ. Уже говорилось о том, что все операции в процессоре, даже пересылка данных, осуществляются при помощи АЛУ. Нужно настроить АЛУ на по-

вторение на выходе входа В (вход А повторять нельзя, так как он должен совпадать с выходом), на вход В подавать регистр данных, в выход соединить с регистром АХ.

Четвертая микрокоманда:

Y7–Y10 = 0000 (выход – регистр АХ);

Y11–Y14 = 1000 (вход В – регистр данных);

Y15–Y18 = 1101 (операция АЛУ – повторение входа В);

Y19–Y22 = 0000 (переход по следующему адресу).

В результате исполнения четвертой микрокоманды произойдет запись в регистр АХ константы из памяти, поэтому этап цикла команды «Запись результата» окажется выполненным.

Заключительный этап – формирование адреса следующей команды в счетчике команд. В этот момент в счетчике команд находится следующий адрес после кода операции – адрес константы. Поэтому, чтобы перейти на следующую команду, нам нужно просто увеличить счетчик команд на единицу. Одновременно необходимо вернуться на начало цикла команды (нулевой адрес в памяти микропрограмм).

Пятая микрокоманда: Y1 = 1 (счетчик команд + 1);

Y19–Y22 = 0111 (безусловный переход по указанному адресу);

Y23–Y30 = 0000 0000 (адрес перехода).

На этом микропрограмма завершается. Поскольку она состоит из пяти микрокоманд, то будет занимать адреса в памяти микропрограмм с десятого по четырнадцатый. А с пятнадцатого адреса можно уже располагать следующую микропрограмму.

Пример 3. Реализовать машинную команду ADD AX, BX (сложить содержимое регистров АХ и ВХ, результат поместить в первый регистр).

Пусть код операции равен 02, микропрограмму будем размещать с 20-го адреса, соответствующую запись нужно внести в таблицу адресов микропрограмм. Снова упростим задачу, предположив, что код операции, кроме самой операции сложения, определяет и оба операнда. В этом случае, кроме кода операции, в ОЗУ для данной машинной команды ничего не нужно записывать.

Этапы исполнения команды и записи результата заключаются в настройке АЛУ на операцию сложения и подготовке входов и выхода.

Первая микрокоманда:

Y7–Y10 = 0000 (вход А, выход – регистр АХ);

Y11–Y14 = 0001 (вход В – регистр ВХ);

Y15–Y18 = 0001 (операция АЛУ – повторение входа В);
Y19–Y22 = 0000 (переход по следующему адресу).

Формирование адреса следующей команды выполняется аналогично предыдущему примеру.

Вторая микрокоманда: Y1 = 1 (счетчик команд + 1);
Y19–Y22 = 0111 (безусловный переход по указанному адресу);
Y23–Y30 = 0000 0000 (адрес перехода).

Пример 4. Реализовать машинную команду MUL AX, BX (умножить содержимое регистров AX и BX, результат поместить в первый регистр).

Пусть код операции равен 03, микропрограмма будет располагаться с адреса 30.

В нашем АЛУ нет операции умножения, поэтому будем реализовывать его последовательным сложением. Нам потребуется вспомогательный регистр (например, CX), для накопления результата. Чтобы не изменять в процессе умножения содержимое регистра BX, перепишем его в DX и далее будем работать с DX. Умножение реализуем при помощи цикла, в котором будем прибавлять AX к CX и уменьшать DX на единицу. Условие завершения цикла – равенство DX нулю.

Нарисуем схему алгоритма (рис. 4.5). Цифрами обозначены адреса микрокоманд.

Микрокоманда 30: обнуление регистра CX. Можно записать константу ноль в регистр CX, но лучше воспользоваться функцией XOR. Если посмотреть на таблицу истинности исключающего ИЛИ, можно заметить, что при действиях с одинаковыми битами в результате получается всегда ноль:

$$0 \text{ XOR } 0 = 0, \quad 1 \text{ XOR } 1 = 0.$$

Таким образом, если для числа осуществлять операцию XOR с самим собой, получится ноль. Этим свойством XOR часто пользуются для быстрого обнуления регистров.

Y7–Y10 = 0010 (вход А, выход – регистр CX);
Y11–Y14 = 0010 (вход В – регистр CX);
Y15–Y18 = 0111 (операция АЛУ – XOR);
Y19–Y22 = 0000 (переход по следующему адресу).

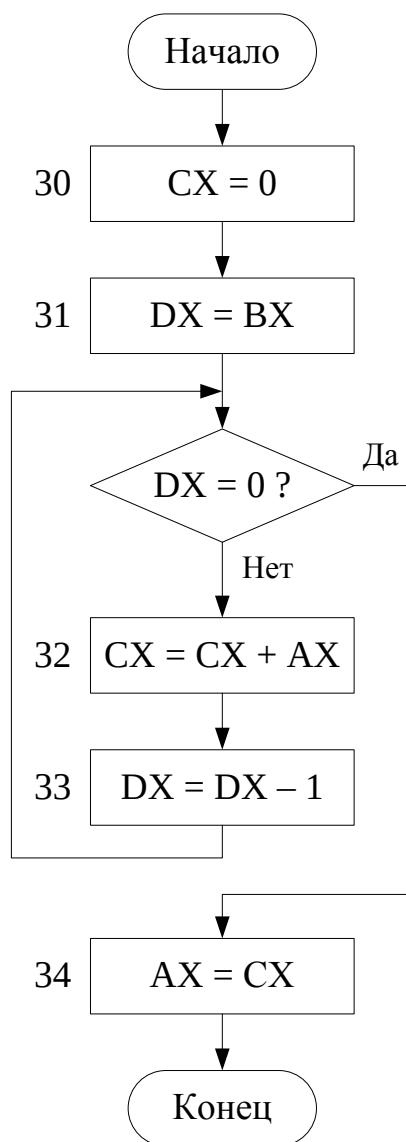


Рис. 4.5. Схема алгоритма умножения

Микрокоманда 31: пересылка содержимого ВХ в DX. Отметим, что сразу после этого идет проверка условия. Мы объединим оба действия в одной микрокоманде.

Пересылка:

Y7–Y10 = 0011 (выход – регистр DX);

Y11–Y14 = 0001 (вход В – регистр ВХ);

Y15–Y18 = 1101 (операция АЛУ – повторение В).

Как проверить, что регистр DX равен нулю? Есть флаг нуля ZF, но он зависит от результата только что выполненной операции. А мы только что переслали содержимое ВХ в DX, т. е. если $DX = 0$, флаг ZF будет установлен.

Y19–Y22 = 1011 (переход по установленному флагу ZF);

Y23–Y30 = 0010 0010 (переход на микрокоманду по адресу 34₁₀).

Микрокоманда 32: прибавление к регистру CX содержимого регистра DX.

Y7–Y10 = 0010 (вход A, выход – регистр CX);

Y11–Y14 = 0011 (вход B – регистр DX);

Y15–Y18 = 0001 (операция АЛУ – сложение);

Y19–Y22 = 0000 (переход по следующему адресу).

Микрокоманда 33: вычитание из регистра DX единицы (декремент) и, одновременно, проверка на равенство DX нулю. В этой микрокоманде нужно переходить на микрокоманду 32 не по установленному флагу ноля, а по сброшенному, потому что переход на следующую по порядку микрокоманду (34-ую) происходит естественным образом.

Y7–Y10 = 0011 (вход A, выход – регистр DX);

Y15–Y18 = 1111 (операция АЛУ – декремент);

Y19–Y22 = 0011 (переход по сброшенному флагу ZF);

Y23–Y30 = 0010 0000 (переход на микрокоманду по адресу 32₁₀).

Микрокоманда 34: пересылка данных из регистра CX в регистр AX. Можно в этой же микрокоманде выполнить формирование адреса следующей команды (инкремент счетчика команд) и вернуться на начало цикла команды.

Y7–Y10 = 0000 (выход – регистр AX);

Y11–Y14 = 0010 (вход B – регистр CX);

Y15–Y18 = 1101 (операция АЛУ – повторение B).

Y1 = 1 (счетчик команд + 1);

Y19–Y22 = 0111 (безусловный переход по указанному адресу);

Y23–Y30 = 0000 0000 (адрес перехода).

4.1.8. Понятия «архитектура» и «микроархитектура»

Чтобы различать внешние и внутренние аспекты поведения процессора, вводят понятия «архитектура» и «микроархитектура».

Архитектура (architecture) – это логическое построение процессора, т. е. то, какой машина представляется программисту. Это понятие охватывает перечень и формат машинных команд (*система команд*), формы представления данных, набор доступных программисту регистров, способы адресации памяти и т. д.

Под *разрядностью* архитектуры обычно понимают разрядность регистров общего назначения.

Пример. Все процессоры Intel, начиная с Intel 386, поддерживают 32-разрядную архитектуру IA-32 (другое название – x86). В поздних версиях процессоров Pentium 4 и во всех последующих процессорах, введена поддержка 64-разрядной архитектуры Intel 64 (другое название – x86-64).

Микроархитектура (microarchitecture) – это структура процессора, отраженная в его составных частях и связях между ними.

Микроархитектура – это конкретный способ реализации архитектуры.

Пример. В процессорах Intel с 2006 года сменилось несколько микроархитектур – Core, Nehalem, Sandy Bridge.

В примерах в предыдущем параграфе мы реализовывали машинные команды (часть архитектуры процессора) при помощи микрокоманд (средствами микроархитектуры).

4.2. Повышение производительности процессоров

Учебная модель процессора, рассмотренная в разделе 4.1, позволяет продемонстрировать базовые принципы работы процессора. Современные процессоры, хоть и построены на таких же принципах, гораздо сложнее устроены. Отличия заключаются в многочисленных способах повышения производительности на основе рассмотренной нами базовой структуры. В этом разделе мы изучим важнейшие приемы повышения производительности, используемые в современных процессорах – конвейеризацию, суперскалярность, кэширование и др.

4.2.1. Конвейеризация

Идея конвейеризации

В параграфе 4.1.5 «Цикл команды» было показано, что выполнение каждой машинной команды состоит из нескольких этапов (цикл команды):

- выборка команды (ВК);
- декодирование кода операции (ДК);
- исполнение команды (ИК);
- запись результата (ЗР);
- формирование адреса следующей команды (ФА).

Поручив реализацию каждого этапа отдельному устройству и соединив их последовательно, получим конвейер команд (рис. 4.6). Блок процессора, реализующий этап цикла команды, называется *ступенью конвейера*.

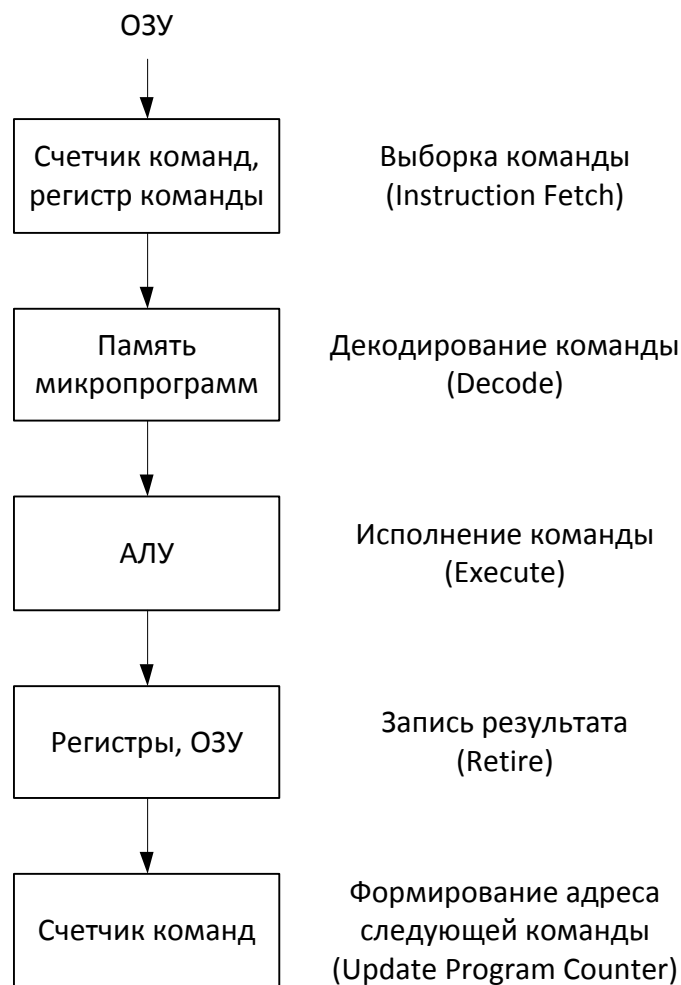


Рис. 4.6. Конвейер команд с пятью ступенями

Идея конвейеризации заключается в том, что при выполнении машинной команды можно не дожидаться её завершения, а по мере освобождения ступеней конвейера запускать следующие команды. Эту идею иллюстрируют временные диаграммы на рис. 4.7.

Из рис. 4.7 видно, что в случае применения конвейера с пятью ступенями скорость выполнения команд теоретически увеличивается в 3 раза: за одно и то же время в процессоре без конвейеризации выполняются две команды, а с конвейеризацией – 6 команд.

Количество ступеней конвейера в современных процессорах от 5 до 20; в процессорах Intel Pentium 4 была 31 ступень конвейера; длинные конвейеры называют «гиперконвейерами».

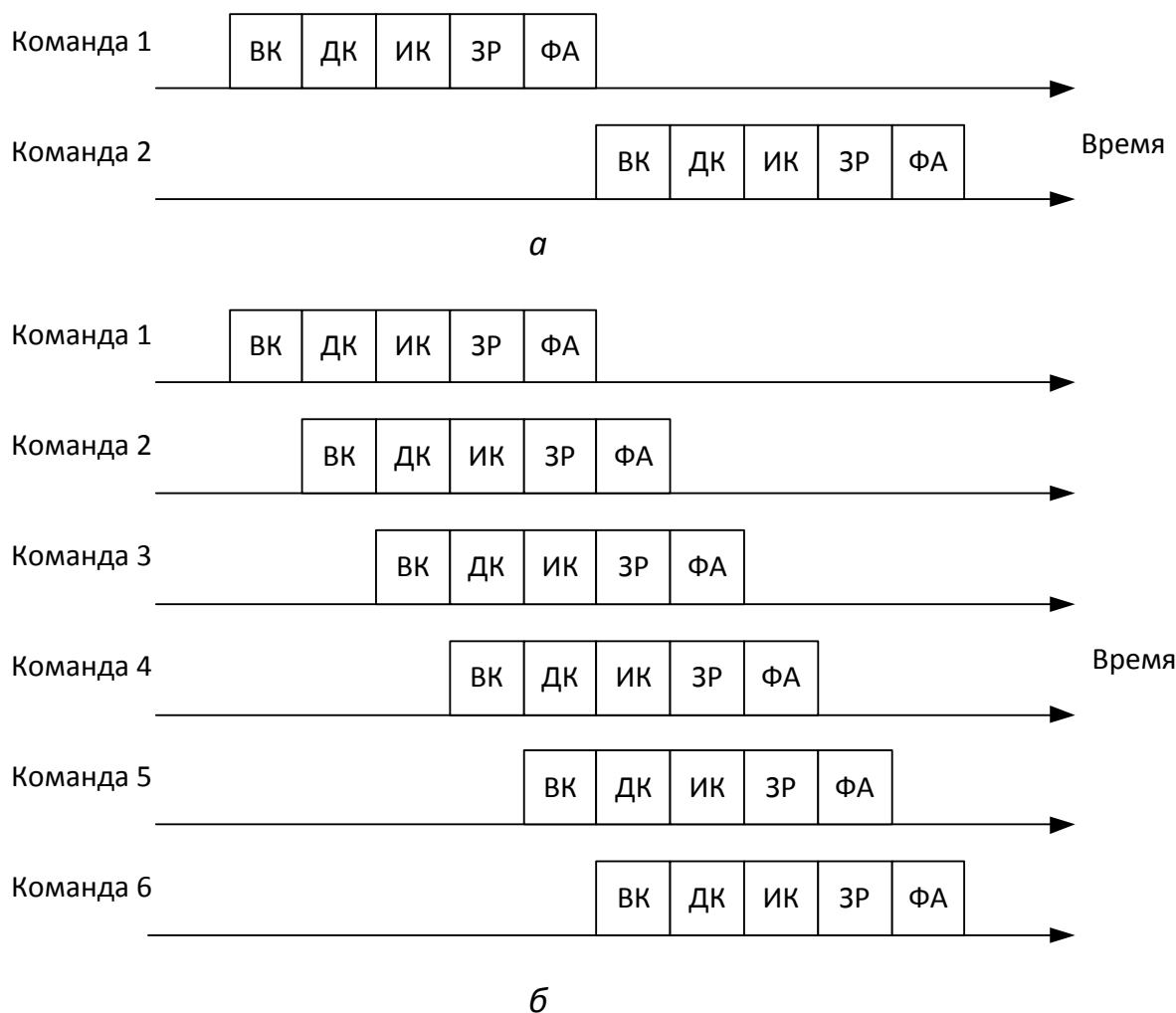


Рис. 4.7. Временные диаграммы процесса выполнения команд:
а – без конвейера команд; б – с конвейером команд

Проблемы конвейеризации

Ничто не совершенно, и у конвейеризации есть свои недостатки. Ускорение при использовании конвейера достигается за счет одновременного (параллельного) выполнения нескольких команд – это и является источником проблем (*рисков*).

Выделяют три группы рисков: структурные риски, риски по данным и риски по управлению.

1. *Структурные риски* – риски одновременного доступа к ресурсам, в основном, к оперативной памяти. Проблема решается за счет физического разделения памяти по отдельным модулям, использования кэш-памяти, разделения кэш-памяти на кэш команд и кэш данных (см. параграф 4.2.4 «Кэш-память»).

2. *Риски по данным* – риски одновременной работы с данными – несколько команд обращаются к одной и той же ячейке памяти в неверной последовательности – возникает конфликт. Самый распространенный вид та-

ких конфликтов – *чтение после записи* – правильно было сначала первой командой записать значение в ячейку памяти, а затем второй командой прочесть, однако конвейеризация нарушила этот порядок.

Пример. Выполняются последовательно две команды:

$a = 10;$
 $b = b + a;$

При использовании конвейера результат может быть неверным (рис. 4.8).

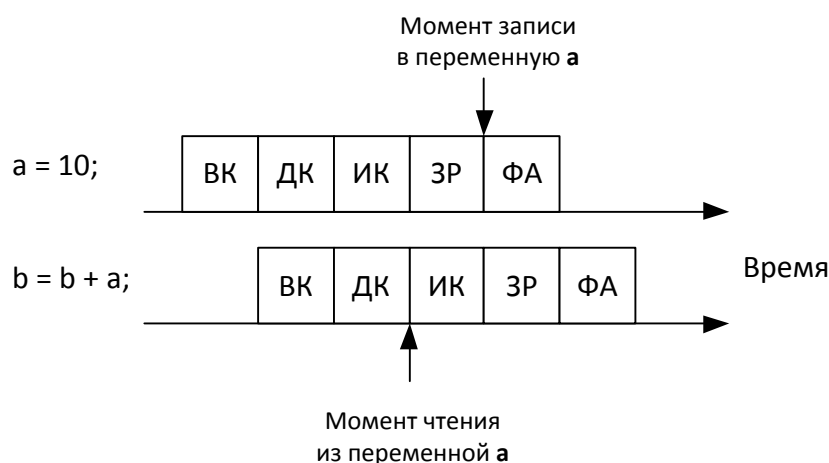


Рис. 4.8. Иллюстрация конфликта «Чтение после записи»

Способы борьбы с конфликтами:

1) *оптимизирующие компиляторы* – компилятор не просто генерирует машинный код по тексту программы на языке высокого уровня, но и анализирует рядом расположенные команды на предмет потенциального возникновения конфликта, и, если такая опасность существует, компилятор вставляет между конфликтующими командами одну или несколько других команд так, чтобы результат вычислений не менялся;

2) *задержка команд* – если при выполнении программы становится ясно, что будет конфликт, процессор задерживает вторую конфликтующую команду на несколько тактов, при этом в конвейере оказывается несколько простаивающих ступеней (говорят, образуется «пузырек»);

3) *ускоренное продвижение данных* (data forwarding) – если прогнозируется конфликт последовательно идущих команд, данные с выхода АЛУ, полученные во время выполнения первой команды, могут быть направлены не в регистр, как обычно, а сразу на вход АЛУ для выполнения второй команды.

3. *Риски по управлению* – если встречается команда перехода (условного или безусловного), то в течение нескольких тактов, пока команда выполняется, процессор не знает, куда нужно осуществлять переход.

Ситуация иллюстрируется на рис. 4.9.

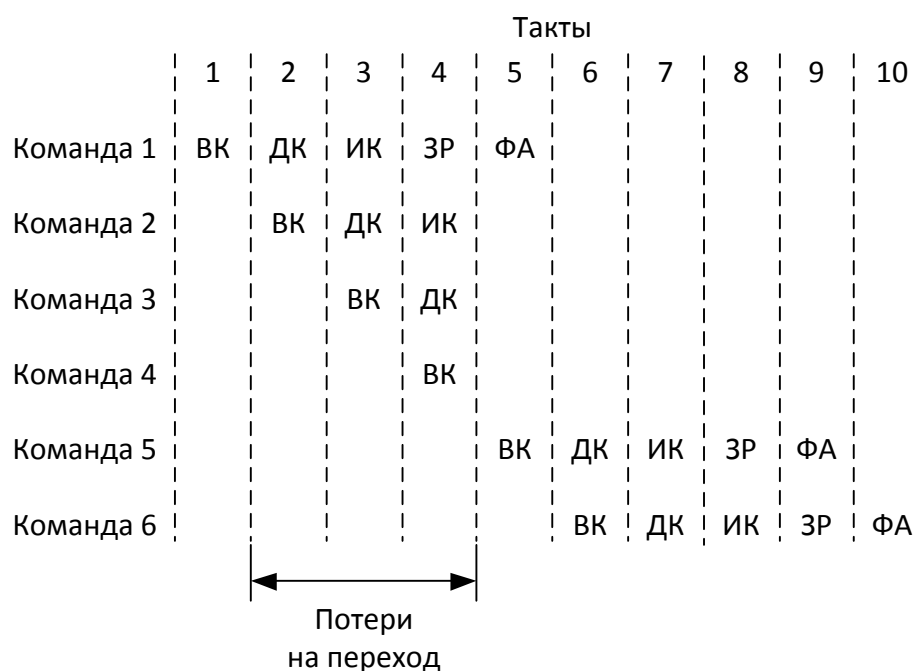


Рис. 4.9. Влияние условного перехода на работу конвейера команд

На рис. 4.9 команда 1 является командой перехода (например, условного, на команду 5). О том, что это команда перехода, процессор не знает до завершения этапа декодирования (2-й такт), а о том, истинно или нет условие перехода (установлен или сброшен определенный флаг), неизвестно до конца этапа записи результата (4-й такт). Поэтому процессор продолжает выполнять следующие команды 2, 3 и 4. Только в конце 5-го такта становится ясно, что произошла ошибка и нужно было выполнять 5-ю команду. Процессор вынужден очищать конвейер, вследствие чего теряется (в нашем примере) 3 такта.

Для решения проблемы рисков по управлению применяется несколько способов. Рассмотрим два из них – использование буферов адресов переходов и предсказание ветвлений.

Буфер адресов переходов (Branch Target Buffer – BTB) – это память небольшой емкости в процессоре, которая представляет собой таблицу; в каждой строке таблицы хранится, во-первых, адрес команды перехода, во-вторых, адрес точки перехода. Таблица заполняется по мере выполнения программы – когда встречается команда перехода и переход осуществляется, для данной команды заносится запись в таблицу. Если таблица заполнена, запись производится на место дольше всех не использованной строки.

Каждый раз при выборке машинной команды осуществляет поиск значения счетчика команд в БТВ, и если совпадение найдено (т. е. текущая команда является командой перехода, она уже встречалась и переход происхо-

дил), то процессор извлекает из таблицы адрес точки перехода и следующие команды считывает, начиная с этого адреса.

Предсказание ветвлений (branch prediction) – механизм автоматического определения того факта, будет выполняться условный переход или нет. Когда встречается команда условного перехода, процессор, используя этот механизм, предсказывает результат выполнения команды ещё до её завершения и считывает дальнейшие команды в соответствии с предсказанием.

Существуют два основных метода предсказания ветвлений. В *статическом методе* результат предсказания заложен в процессор: например, считается, что условные переходы всегда происходят или, наоборот, не происходят. В *динамическом методе* результат предсказания зависит от хода выполнения программы – накапливается статистика, что для одних условий переходы чаще всего происходят, а для других – нет; при появлении новой команды условного перехода предсказание осуществляется на основе собранной статистики.

В современных процессорах оба метода работают совместно – если возможно, используется динамическое предсказание, иначе – статическое, за счет этого обеспечивается точность предсказания свыше 90%.

4.2.2. Суперскалярные процессоры

Суперскалярный процессор – это процессор, содержащий несколько исполнительных блоков, например несколько АЛУ для работы с целыми и вещественными числами, блоки работы с памятью и т. п. (рис. 4.10).

Команды нужно распределять на соответствующие исполнительные устройства, за это в скалярном процессоре отвечает специальный блок, который называется *диспетчером*, или *планировщиком* (scheduler).

Ясно, что если в программе сначала все время идут целочисленные операции, а затем одни вещественные, то от наличия нескольких исполнительных блоков не будет никакого толку. Поэтому в суперскалярных процессорах применяется *механизм внеочередного исполнения команд* (out-of-order execution) – более поздние команды исполняются перед более ранними, чтобы обеспечить максимально эффективную загрузку всех исполнительных блоков.



Рис. 4.10. Суперскалярный процессор с тремя исполнительными блоками

Логические и физические регистры

Процессор обычно предоставляет программисту небольшое число регистров общего назначения.

Пример. В архитектуре IA-32 (Intel Architecture 32) имеется 8 32-разрядных регистров общего назначения – EAX, EBX, ECX, EDX, EBP, ESP, ESI, EDI.

При использовании внеочередного исполнения команд, суперскалярности и конвейеризации часто оказывается, что в один и тот же момент времени несколько команд работают, например, с регистром EAX. Если такой регистр физически всего один, то одновременно такие команды выполнять

невозможно. Поэтому в современных процессорах разделяют понятия логических и физических регистров.

Логические регистры – это регистры процессора, которые доступны программисту, например в IA-32 это EAX, EBX и т. д. *Физические регистры* – это регистры, аппаратно реализованные в процессоре, на которые отображается множество логических регистров.

Пример. На 8 логических регистров IA-32 в микроархитектуре Intel Sandy Bridge приходится 160 физических регистров.

Процесс назначения логическому регистру в конкретной команде номера физического регистра называется *переименованием регистров* (register renaming).

4.2.3. Архитектуры CISC и RISC

В современных процессорах можно выделить два основных вида системы команд:

- CISC (*Complex Instruction Set Computer*) – компьютер с полным набором команд;
- RISC (*Reduced Instruction Set Computer*) – компьютер с сокращенным набором команд.

До 80-х годов все компьютеры имели архитектуру CISC. Для нее характерно:

- большое количество разнообразных команд (несколько сотен) множества форматов;
- неодинаковая длина команд;
- сложное устройство управления (вследствие большого числа неоднородных команд);
- малое число регистров (от 8 до нескольких десятков).

Важное преимущество CISC заключается в том, что для этой архитектуры проще реализовать сложный язык высокого уровня – команды такого языка не сильно отличаются от команд CISC-процессора и легко в них преобразуются. Однако это удобство оборачивается трудностью реализации быстрого выполнения команд – сложное устройство управления работает не слишком быстро, малое число регистров приводит к частым обменам данными с оперативной памятью, конвейер для сложных команд оказывается медленным.

Все эти недостатки привели к разработке в начале 80-х годов нового вида архитектуры – RISC, имеющей следующие особенности:

- небольшое число команд (около 100) и форматов команд;
- одинаковая длина команд, равная разрядности шины данных;

- как следствие, простое устройство управления и быстрый конвейер;
- большое число регистров (от нескольких десятков до нескольких сотен);
- редкое взаимодействие с оперативной памятью (за счет множества регистров).

В компьютерах RISC сложные действия, для которых в CISC имелись специальные команды, разбивались на несколько простых быстрых команд.

Пример RISC-процессоров: Ultra SPARC (Sun Microsystems), Alpha (DEC, с 1998 года – Compaq, затем Hewlett-Packard), PowerPC (IBM).

В общем случае, архитектура RISC является более производительной, чем CISC. Однако повсеместного перехода на архитектуру RISC не произошло по двум причинам. Во-первых, было уже вложено много денег в архитектуру CISC и в программное обеспечение для нее (первые процессоры Intel). Во-вторых, в процессорах CISC (начиная с AMD-K5, а затем и в процессорах Intel) стали применять RISC-технологии – декодировать сложные CISC-команды в простые RISC-микрокоманды, использовать большое количество физических регистров и технологию переименования регистров.

4.2.4. Кэш-память

В параграфе 3.4.1 на рис. 3.28 представлена иерархия видов памяти. Если рассматривать верхушку пирамиды и сравнить регистры (микропроцессорную память) и оперативную память, то окажется, что их быстродействие различается на порядок, т. е. машинная команда пересылки данных в ОЗУ в сравнении с командой пересылки между регистрами работает примерно в 10 раз медленнее.

Причина такой серьезной разницы в скорости заключается в разной элементной базе – регистры строятся из быстродействующих триггеров, а оперативная память – из полупроводниковых конденсаторов, процессы зарядки/разрядки которых занимают относительно большое время.

Если строить всю память из регистров, то она получится очень быстрой, но сверхдорогой; если из конденсаторов, то не будут удовлетворяться требования по скорости. Поэтому в 60-х годах XX века был предложен компромисс – использовать промежуточную (буферную) память средней емкости, по скорости несильно отличающуюся от регистров, в которой будут храниться часто используемые данные. Такую память называли *кэш-памятью*. Термин *кэш* произошел от английского слова *cache* – тайник, поскольку программист обычно не может обращаться к этой памяти напрямую. Впервые кэш-память появились в компьютерах модели 85 семейства IBM 360.

Рассмотрим алгоритм взаимодействия с кэш-памятью (рис. 4.11).

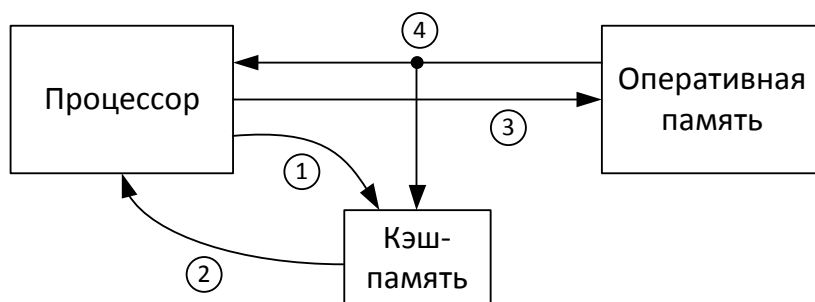


Рис. 4.11. Алгоритм работы с кэш-памятью

В процессе работы программы, когда процессору требуется очередная машинная команда, он запрашивает её сначала не в оперативной памяти (где находится вся программа), а в кэш-памяти (цифра 1 на рис. 4.11). Если необходимая команда присутствует там, то процессор считывает команду из кэш-памяти (цифра 2). Такая ситуация называется *попаданием* (hit). Если команды нет в кэше, то фиксируется *промах* (miss) и процессор обращается в оперативную память (цифра 3). При этом в кэш-память копируется целый блок данных, содержащих эту команду (цифра 4). В дальнейшем, если процессор обращается к какому-либо участку этого блока, данные считываются не из медленной оперативной, а из быстрой кэш-памяти.

При заполнении кэш-памяти новая информация записывается на место тех данных, которые наиболее давно не использовались (*алгоритм LRU – Least Recently Used*).

Высокая вероятность работы именно с кэш-памятью связана с *принципом локальности*, в соответствии с которым примерно 90% времени выполнения программы используются только 10% её команд и данных. Таким образом, если в настоящий момент процессор работает с некоторой командой, то в следующий момент времени ему потребуется команда, расположенная близко от текущей. Такие конструкции языков программирования, как следование и циклы, являются хорошей иллюстрацией принципа локальности. Наоборот, примером нелокального расположения выполняющихся команд является вызов процедур и возврат из них – обычно в этом случае происходит переход на большое количество байт и требуемых команд в кэш-памяти не оказывается.

Смешанная и разделенная кэш-память

По способу хранения информации в памяти различают два вида архитектур – *Принстонскую* (смешанную, или *архитектуру фон Неймана*), в которой команды и данные находятся в одной и той же памяти, и *Гарвардскую* (раздельную), в которой команды и данные хранятся раздельно. Свои имена оба вида получили по названиям университетов, где они были разработаны.

Принстонская архитектура в компьютерной технике получила преимущество – в большинстве видов памяти команды и данные хранятся вместе. В кэш-памяти используются оба вида архитектур.

Преимуществом Принстонской архитектуры для кэш-памяти является естественное определение оптимального соотношения команд и данных — если некоторой программе требуется больше команд, а не данных, в смешанной кэш-памяти будут преобладать команды. В то же время из кэш-памяти, построенной по Гарвардской архитектуре, можно одновременно извлекать команды и данные.

Уровни кэш-памяти

Чем больше емкость кэш-память, тем дольше осуществляется поиск требуемой команды, но зато в ней может храниться больше потенциально нужной информации, т. е. больше вероятность попаданий. Наоборот, меньшая емкость кэша обеспечивает быстрый поиск и малую вероятность попаданий. Опять идут на компромисс — делят кэш-память на уровни (рис. 4.12). Чем выше уровень, тем больше емкость.

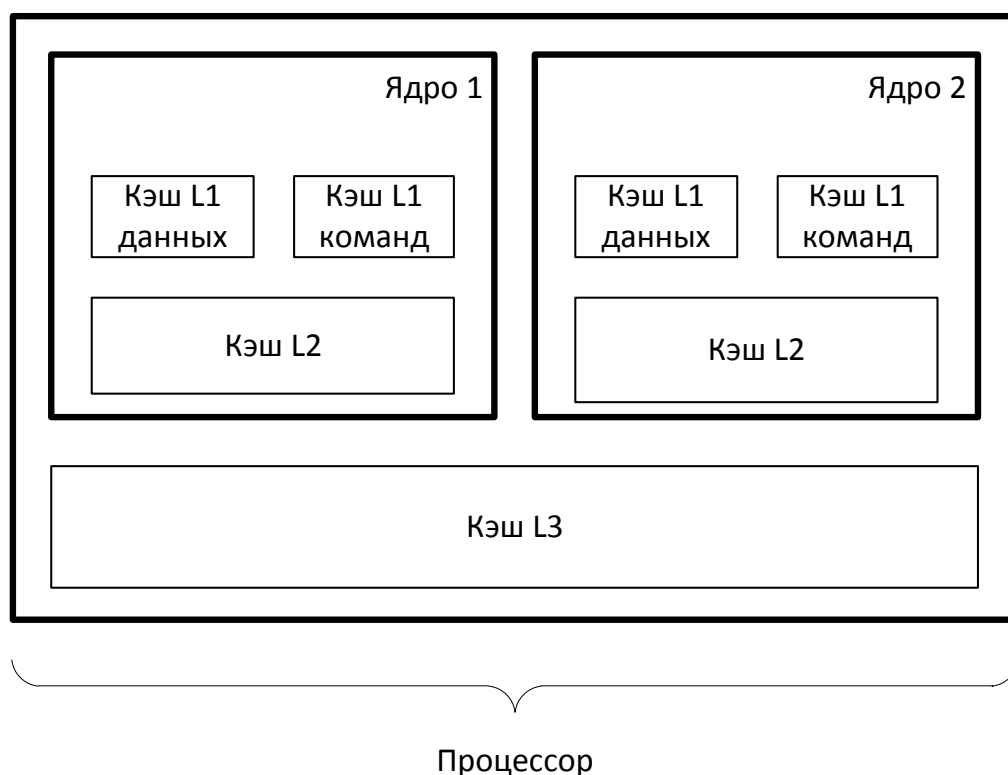


Рис. 4.12. Структура двухъядерного микропроцессора

- Кэш-память первого уровня (L1) — располагается на одном кристалле с процессором (в ядре процессора). Объем: десятки Кбайт. Делится на кэш команд и кэш данных.
- Кэш-память второго уровня (L2) — располагается также в ядре процессора. Объем: сотни Кбайт.
- Кэш-память третьего уровня (L3) — располагается в корпусе процессора, часто разделяется между ядрами. Объем: единицы–десятки Мбайт.

При работе с кэшем процессор обращается последовательно к разным уровням – сначала к L1, затем к L2 (если информации нет в L1), и, наконец, к L3 (если информации нет и в L2). В случае промаха блок данных из оперативной памяти заносится во все уровни.

4.3. Организация ввода-вывода

4.3.1. Модули ввода-вывода

В фон-неймановской модели (см. рис. 1.10), как и в архитектуре с иерархией шин (см. рис. 3.45), процессор кроме оперативной памяти взаимодействует ещё с устройствами ввода и вывода (УВВ), или *периферийными устройствами*. За организацию обмена информацией с ними в ВМ отвечает система ввода-вывода, представляющая собой набор *модулей ввода-вывода* (МВВ) (рис. 4.13).

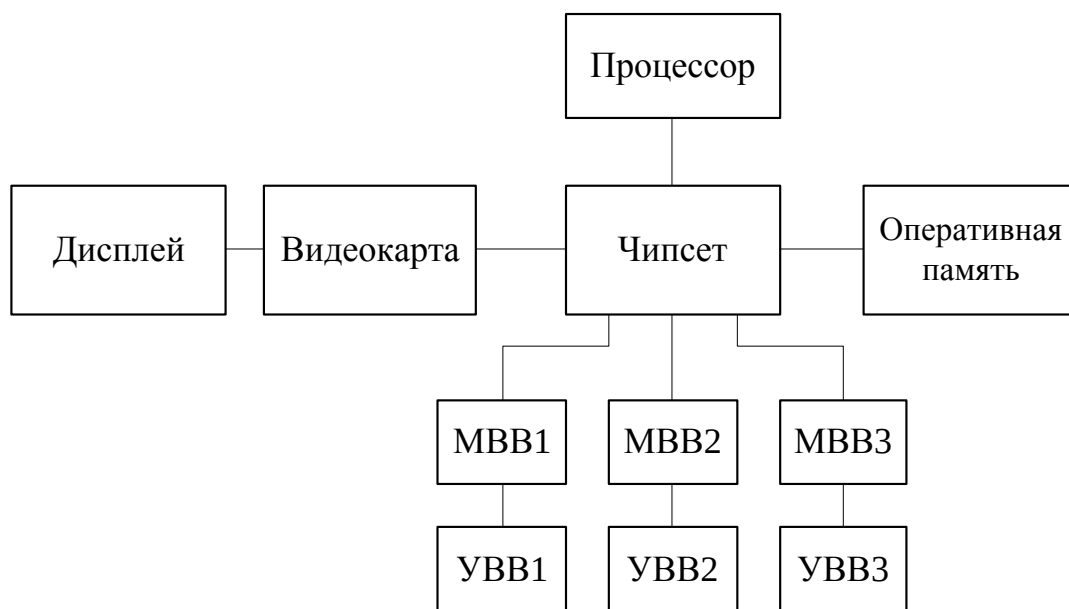


Рис. 4.13. Модули ввода-вывода

Видеокарта также, по сути, является модулем ввода-вывода; на схеме она выделяется, поскольку канал взаимодействия с видеокартой обычно по быстродействию ненамного уступает шине оперативной памяти.

Модули ввода-вывода обычно называются *контроллерами*, но для некоторых из них предусмотрены специальные названия (например, видеокарта). Схема (обычно разъем) для связи периферийного устройства с процессором (или чипсетом) называется *портом ввода-вывода* (I/O ports).

Примеры. Монитор является периферийным устройством, видеокарта – модулем ввода-вывода, а разъем, через который осуществляется связь с чипсетом, называется VGA-портом.

Клавиатура – периферийное устройство, контроллер клавиатуры – MBV, разъем – порт PS/2.

Устройства ввода-вывода делятся на следующие типы:

- 1) устройства ввода – клавиатура, мышь, сенсорная панель (touchpad), сенсорный экран, джойстик, сканер, веб-камера, микрофон;
- 2) устройства вывода – дисплей, проектор, принтер, акустическая система (колонки, динамики, наушники);
- 3) устройства внешней памяти – винчестер (накопитель на жестких магнитных дисках), привод оптических дисков (CD, DVD, Blu-ray), стример;
- 4) сетевые устройства – сетевая карта, модем.

Модули ввода-вывода выполняют следующие функции:

- 1) обмен информацией с процессором;
- 2) обмен информацией с УВВ;
- 3) адресация в УВВ для чтения и записи данных;
- 4) управление УВВ;
- 5) обнаружение ошибок.

4.3.2. Способы взаимодействия с внешними устройствами

Можно выделить три способа взаимодействия процессора с внешними устройствами [2]:

- 1) программно управляемый ввод-вывод;
- 2) ввод-вывод по прерываниям;
- 3) прямой доступ к памяти.

Указанные способы отличаются по степени вовлеченности процессора: в наибольшей степени процессор занят при программно управляемом вводе-выводе, в наименьшей – в случае прямого доступа к памяти.

Программно управляемый ввод-вывод целиком контролируется процессором. При необходимости взаимодействия с УВВ процессор передает команду на MBV, определяющую тип взаимодействия (прием или передача), объем и адрес начала блока данных. Если устройство ввода-вывода готово для такой работы, MBV передает соответствующий сигнал процессору и взаимодействие начинается. В ходе передачи данных процессор считывает очередное слово от MBV и записывает его в оперативную память (или, наоборот, считывает слово из памяти и отправляет его на MBV). После каждого слова процессор проверяет, не закончился ли блок данных.

Таким образом, в ходе программно управляемого ввода-вывода процессор все время занят; при этом, учитывая, что скорости работы процессора и УВВ в большинстве случаев несопоставимы, вычислительная мощность процессора используется неэффективно. Преимуществом подобной схемы является простота её реализации.

Ввод-вывод по прерываниям практически всегда реализуется в ВМ и является стандартным способом взаимодействия с внешними устройствами.

Мы уже выяснили, что из-за несоответствия скоростей невыгодно заставлять процессор все время контролировать ход взаимодействия с УВВ. Лучше дать возможность процессору заниматься выполнением вычислений, а при необходимости отвлекать его на работу с внешним устройством (*прерывать* ход вычислений). При этом внешнее устройство посылает процессору сигнал *запроса на прерывание* по специальным входам.

Реализуется такая схема путем добавления в конец цикла команды (см. параграф 4.1.5 «Цикл команды») ещё одного действия – проверку запросов на прерывание. При получении запроса процессор приостанавливает выполнение текущей программы, определяет источник и цель запроса на прерывание и переходит к специальной служебной программе – *обработчику прерываний*. При завершении обработки прерывания процессор возвращается к выполнению прерванной программы.

Следует отметить, что прерывания от устройств ввода-вывода называются *внешними*; существуют ещё *внутренние прерывания* (исключения), которые возникают при ошибках в ходе выполнения программы – деление на ноль, переполнение, неопределенный код операции, ошибка стека и др. Способ обработки внутренних прерываний совпадает с рассмотренной схемой обработки внешних прерываний.

Преимуществом ввода-вывода по прерываниям является уменьшение степени занятости процессора; однако сама обработка прерываний все ещё требует его непосредственного участия.

Прямой доступ к памяти используется в тех случаях, когда требуется передавать большие блоки данных между оперативной памятью и УВВ. При этом в ВМ должен присутствовать *контроллер прямого доступа к памяти* (DMA – Direct Memory Access). Процессор только инициализирует передачу данных, отдавая команду контроллеру DMA. Всем остальным процессом управляет контроллер, полностью освобождая процессор.

Такая схема взаимодействия наиболее эффективна, если задача заключается просто в пересылке больших объемов данных. При необходимости реализации более сложных алгоритмов взаимодействия используются программно управляемый ввод-вывод или ввод-вывод по прерываниям.

4.4. Пример реализации микроархитектуры процессора

Рассмотрим микроархитектуру Intel Sandy Bridge, процессоры на основе которой появились в 2011 году (Intel Core i3, i5, i7, новые модели Pentium и Celeron).

Процессоры на Sandy Bridge являются 64-разрядными (т. е. основные регистры имеют размер 64 бита), хотя и поддерживают выполнение 16 и 32-разрядных программ (*обратная совместимость*).

Микроархитектура Sandy Bridge основана на 32-нанометровом технологическом процессе (1 нанометр = 10^{-9} м), такой масштаб определяет размеры элементов процессора – транзисторов.

Разные варианты процессоров на Sandy Bridge имеют следующие характеристики:

- Количество ядер – от 2 до 8.
- Частота ядра – 1,1–4,0 ГГц.
- Количество транзисторов – от 504 млн до 2,27 млрд.
- Кэш-память трех уровней:
 - L1 – 32 Кбайт кэш команд и 32 Кбайт кэш данных;
 - L2 – 256 Кбайт;
 - L3 – от 1 до 20 Мбайт.

Структурная схема 4-ядерного процессора на микроархитектуре Sandy Bridge представлена на рис. 4.14.

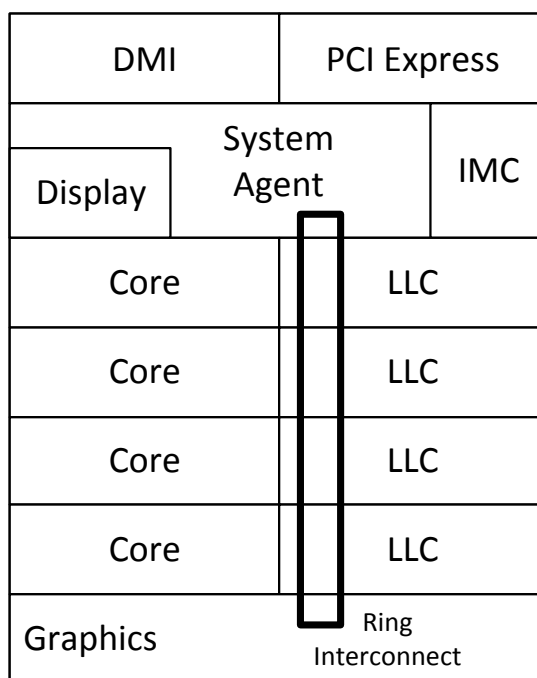


Рис. 4.14. Структура процессора на базе микроархитектуры Sandy Bridge

На рис. 4.14 показаны 4 *процессорных ядра* (Core) (их структура будет рассмотрена ниже), с каждым ядром связан свой блок *кэш-памяти 3-го уровня* (LLC – Last Level Cache, кэш последнего уровня). Также присутствуют *графическое ядро* (Graphics), способное заменить видеокарту начального уровня, и *Системный агент* (System Agent), в состав которого входят *двухканальный контроллер памяти DDR3* (IMC – Integrated Memory Controller), *блок видеовыхода* (Display), *контроллер шины PCI Express*. По сути, Системный агент реализует функции северного моста чипсета, поэтому в чипсете на материнской плате оставлены только функции южного моста (связь с медленными периферийными устройствами); для подключения такого чипсета к процессору используется *интерфейс DMI* (Direct Media Interface).

Особенностью микроархитектуры Sandy Bridge является *кольцевая шина* (Ring Interconnect), соединяющая все блоки процессора: вычислительные ядра, графическое ядро, кэш-память 3-го уровня, Системный агент. Шина состоит из четырех 32-байтовых колец – шины данных (Data Ring), шины запросов (Request Ring), шины мониторинга (Snoop Ring) и шины подтверждения (Acknowledge Ring). На частоте 3 ГГц 32-байтовая шина данных позволяет получить пропускную способность 96 Гбайт/с.

Использование кольцевой шины обеспечивает доступ любого ядра процессора к любому блоку кэш-памяти 3-го уровня, т. е. этот кэш строится по Принстонской архитектуре. В отличие от него модули кэш-памяти 1-го и 2-го уровней входят в состав каждого ядра и не разделяются между ядрами. При этом кэш 2-го уровня также относится к Принстонской архитектуре, а кэш 1-го уровня является отдельным (Гарвардская архитектура).

Упрощенная структура ядра процессора представлена на рис. 4.15.

Цикл команды начинается с выборки текущей команды – сначала из кэша команд L1 (98–99% попаданий), если там требуемая команда отсутствует (промах), то из кэша L2 с одновременным копированием в L1. Если команды нет и в L2, тогда происходит обращение к кэшу L3 и далее к оперативной памяти. Если встречается команда перехода, то срабатывает предсказатель перехода.

Выбранная команда поступает в очередь команд (работает конвейер), а затем в декодер (по мере его освобождения). Наступает этап декодирования команды. Простые команды декодируются при помощи специальных электронных схем, для сложных команд имеется ПЗУ микрокоманд. Декодированная последовательность микрокоманд копируется в специальный кэш микрокоманд (называемый кэшем нулевого уровня). В среднем 80% обращений попадает в этот кэш. В дальнейшем микрокоманды могут браться не после декодера, а сразу из кэша микрокоманд.

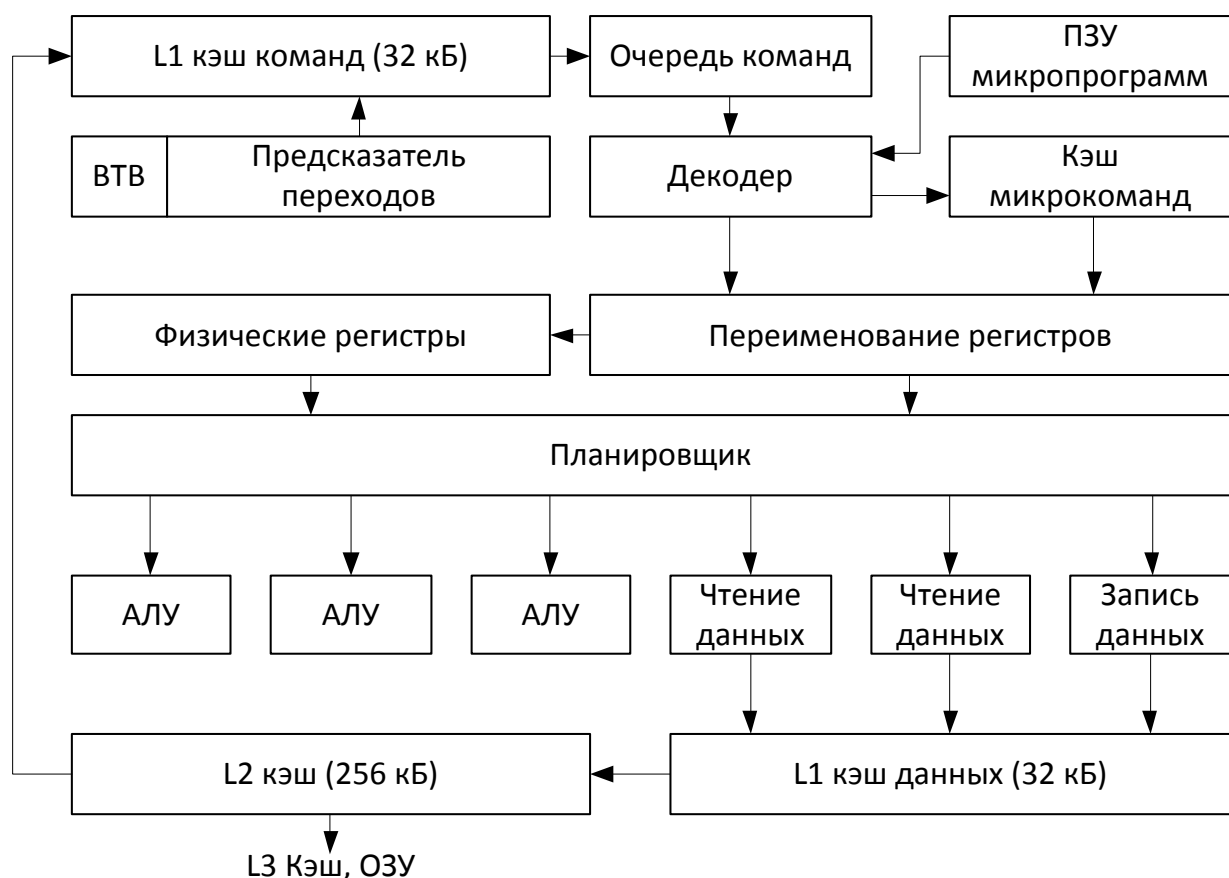


Рис. 4.15. Структура ядра процессора на микроархитектуре Sandy Bridge

На этапе выполнения команды происходит переименование регистров: логические регистры отображаются на 160 64-разрядных целочисленных регистров и 144 256-разрядных регистра для операций с плавающей точкой. Затем микрокоманды поступают в планировщик, который формирует эффективную смесь микрокоманд (внеочередное исполнение).

За непосредственное выполнение микрокоманд отвечают три АЛУ, каждое из которых реализует свой набор операций с целыми и вещественными числами, и три блока работы с памятью. Результаты вычислений сохраняются в кэш-память первого и второго уровней.

Контрольные вопросы и задания

1. Что означает понятие «разрядность процессора»? Чем 16-разрядный процессор отличается от 64-разрядного?
2. Каким образом осуществляются условные переходы в процессоре (как проверяется условие и происходит переход на нужную команду)?
3. Где хранятся микропрограммы?
4. Какие способы повышения производительности процессоров вы знаете?
5. Во сколько раз повышается производительность при применении конвейеризации? Почему?

6. Поясните причины возникновения проблем при использовании конвейеризации.
7. Что такое суперскалярный процессор?
8. Поясните, за счет чего достигается высокая точность в предсказании ветвлений?
9. Зачем нужна кэш-память?
10. Объясните алгоритм работы кэш-памяти.
11. Зачем использовать разделенную кэш-память?
12. Почему процессоры CISC считаются более медленными, чем процессоры RISC?
13. Что такое виртуальная память? Зачем она используется?
14. * Какова зависимость производительности от числа конвейеров? Какова при одном конвейере зависимость производительности от ступеней конвейера?
15. * Предложите свой способ повышения производительности процессоров.

ГЛАВА 5. ВНЕШНИЕ ЗАПОМИНАЮЩИЕ И ПЕРИФЕРИЙНЫЕ УСТРОЙСТВА

5.1. Внешние запоминающие устройства

Процессор может выполнять команды и работать с данными только из оперативной памяти (ОЗУ). Размер информации, хранимой в ОЗУ, относительно невелик – от нескольких сотен мегабайт до нескольких гигабайт. Современные сложные программы могут занимать несколько гигабайт памяти, а кроме них приходится хранить всевозможные данные – тексты, таблицы, рисунки и схемы, аудио- и видеофайлы. Очевидно, в оперативной памяти все это не поместится, следовательно, требуются устройства, имеющие объем на несколько порядков больше, чем ОЗУ, пусть даже менее быстродействующие, зато более дешевые (см. рис. 3.28 «Иерархия запоминающих устройств»).

Для длительного хранения больших объемов данных, которые не требуется обрабатывать в текущий момент, применяются *внешние запоминающие устройства* – винчестеры, CD и DVD-приводы, флеш-накопители и др.

Введем два основных определения.

Носитель – физическая среда для записи, хранения и воспроизведения информации.

Накопитель – устройство, которое обеспечивает запись информации на внешний носитель, её поиск и считывание в оперативную память.

Например, винчестер является накопителем, в котором находится носитель – магнитный диск (диски). CD-привод – это накопитель, а носители – компакт-диски – вставляются в него.

5.1.1. Классификация устройств внешней памяти

Все широко используемые в настоящее время запоминающие устройства можно разделить на три вида в зависимости от используемого принципа записи: полупроводниковые, магнитные и оптические (рис. 5.1).

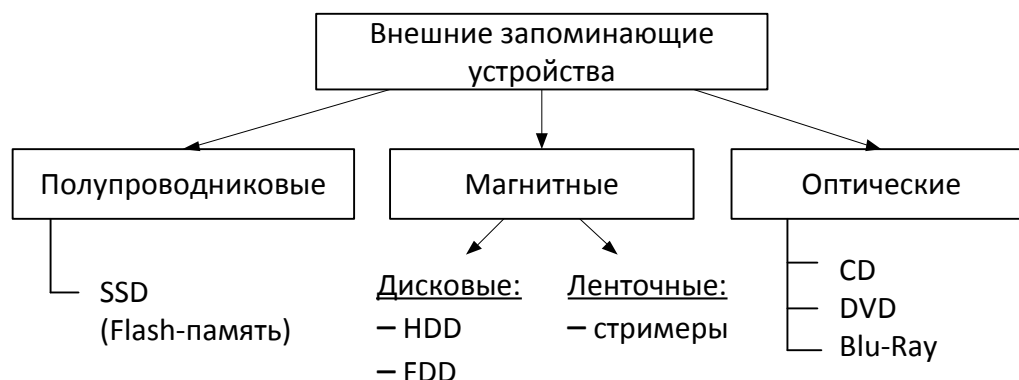


Рис. 5.1. Классификация внешних запоминающих устройств

В полупроводниковых носителях для хранения информации используются специальные транзисторы, которые способны запоминать свое состояние (заряд) в течение длительного времени (см. параграф 3.4.9 «Постоянные запоминающие устройства»). Вид памяти, действующий по этому принципу, – SSD (solid-state drive, твердотельный накопитель), основанный на флеш-памяти.

Магнитный принцип записи основан на способности некоторых материалов (ферритов) сохранять уровень намагниченности, задаваемый внешним магнитным полем. Примеры магнитных накопителей – винчестеры, или HDD (hard-disk drive, накопитель на жестких магнитных дисках), и устаревшие к настоящему времени дискеты, или FDD (floppy-disk drive, накопитель на гибких магнитных дисках).

В основу действия оптических носителей положен тот факт, что свет по-разному отражается от ровных и неровных поверхностей. Таким образом, изменяя форму некоторого материала, затем направляя на него луч света и анализируя отраженный луч, можно использовать материал как носитель информации. Примеры носителей: компакт-диски CD (compact disk), DVD-диски (digital versatile disc – цифровой универсальный диск) и Blu-ray диски.

5.1.2. Магнитные диски

Магнитные диски представляют собой металлические круглые пластины, покрытые слоем из ферромагнитных материалов – различных сплавов железа, никеля, кобальта, марганца и др. Ферромагнетики обладают способностью сохранять намагниченность при отсутствии внешнего магнитного поля, т. е. можно сначала намагнитить носитель, подводя к нему сильный магнит, а затем, когда магнит будет убран, намагниченность сохранится (это явление называется *магнитным гистерезисом*). Запись (как и чтение) производится специальным устройством – *магнитной головкой*. Запись ноля или единицы зависит от направления магнитного поля.

Чтение записанной информации осуществляется на основе либо принципа электромагнитной индукции, либо магниторезистивного эффекта. Если используется электромагнитная индукция, то при вращении диска к нему подводится магнитная головка, переменное магнитное поле создает в электрической цепи головки переменное электрическое поле, по направлению которого можно определить, ноль или единица прочитана. Магниторезистивный эффект заключается в изменении сопротивления электрических цепей головки под действием переменного магнитного поля.

Магнитные диски выпускают разных диаметров. Наибольшее распространение в настольных компьютерах получили диски диаметром 3,5'' (3,5 дюйма, 1 дюйм = 2,54 см), в ноутбуках – диаметром 2,5''.

Физическая структура магнитного диска

Структура диска схематично представлена на рис. 5.2 и 5.3.

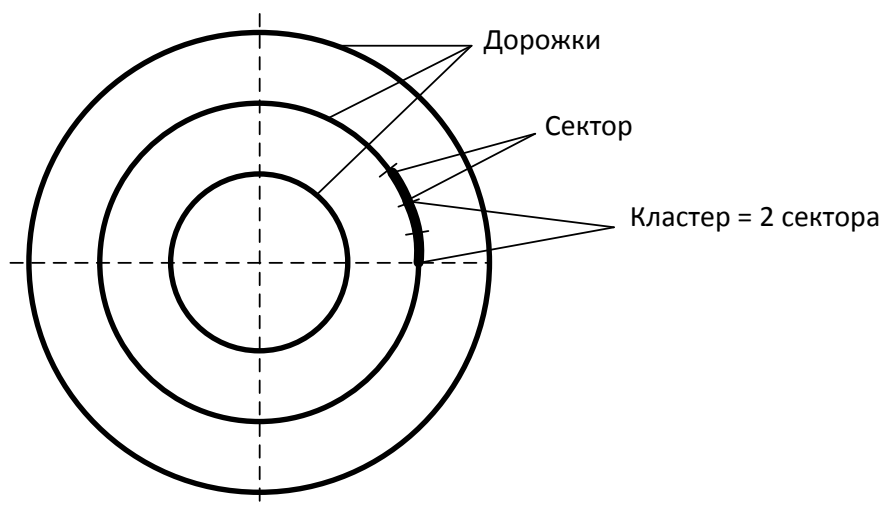


Рис. 5.2. Структура магнитного диска (вид сверху)

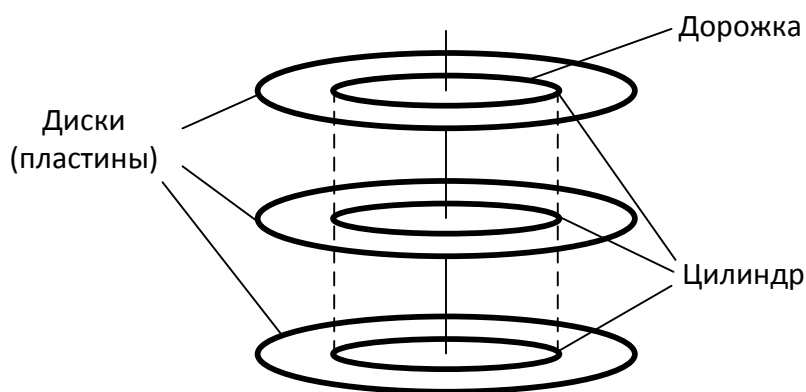


Рис. 5.3. Структура магнитного диска (объемный вид)

Магнитный диск состоит из множества концентрических *дорожек* (*треков*), разделенных на *секторы*. В секторе содержится 512 байт данных; для упрощения схем управления на всех дорожках стараются хранить одинаковое количество информации, поэтому плотность записи и, соответственно, число секторов на дорожке увеличиваются по мере близости к центру диска.

Информация записывается и считывается в винчестере по секторам: нельзя прочитать или записать один байт, только весь сектор целиком. Обмен с оперативной памятью происходит по *кластерам*, каждый из которых содержит количество секторов, кратное степеням двойки (1, 2, 4, 8, ...). Размер кластера зависит от используемой *файловой системы*¹ (FAT32, NTFS, ext4, HFS и др.).

¹ Файловая система – совокупность правил, определяющих способ организации данных на устройствах внешней памяти.

Обычно в накопителях на жестких магнитных дисках содержится несколько пластин и, соответственно, несколько магнитных головок. Совокупность дорожек разных пластин, находящихся на одинаковом расстоянии от центра, называется *цилиндром*. Поскольку магнитные головки объединены в блок и перемещаются одинаково, то работа с информацией на дисках в каждый момент времени происходит на дорожках, входящих в один цилиндр.

Таким образом, положение сектора на диске определяется тремя числами: номером цилиндра, номером головки и номером сектора.

Рассмотренная структура жесткого диска (дорожки и секторы) образуется путем *низкоуровневого форматирования*, которое выполняется на заводе-изготовителе. Разметка под конкретную файловую систему, определяющую способ организации информации на диске, в том числе размер кластера, называется *высокоуровневым форматированием* и может осуществляться на любом компьютере.

Накопители на жестких магнитных дисках

Накопители на жестких магнитных дисках (НЖМД, жесткие диски, винчестеры, Hard Disk Drive – HDD) в настоящее время являются одними из самых распространенных устройств внешней памяти. Практически каждый настольный компьютер или ноутбук оснащаются по меньшей мере одним винчестером. Однако в последнее время конкуренцию жестким дискам стали составлять твердотельные накопители (SSD) на основе флеш-памяти, особенно в ноутбуках; а для планшетных и карманных компьютеров SSD являются основным видом внешней памяти.

Название *винчестер* накопители на жестких магнитных дисках получили в 70-х годах в фирме IBM благодаря случайному совпадению обозначений винтовки Винчестера и жесткого диска IBM. И в том и в другом случае использовалось обозначение 30-30, которое для винтовки означало калибр патрона, а для диска – 2 модуля памяти по 30 Мбайт в каждом.

В НЖМД набор пластин, двигатель для их вращения (типичная скорость 7200 об/мин, но может достигать и 15000 об/мин), блок магнитных головок и управляющая электроника объединены в герметичный корпус, на котором расположены порты ввода-вывода. В процессе вращения пластин создается подъемная сила, которая поддерживает магнитные головки в парящем состоянии над пластинами. За счет минимального расстояния от поверхности пластин ($0,1 \text{ мкм} = 0,1 \cdot 10^{-6} \text{ м}$) обеспечивается большая емкость диска. В 2011 году был выпущен винчестер с пластиной емкостью 1 Тбайт (1 терабайт = 1000 Гбайт), что соответствует 155 Гбит/см^2 . Герметичность корпуса винчестера необходима для поддержания идеальной чистоты внутри него – небольшая пылинка на таких больших скоростях способна повредить пластину или головку.

Для самотестирования в винчестерах имеется технология SMART (Self-Monitoring Analysis and Reporting Technology), позволяющая оценить текущее состояние диска, а также предсказать вероятный выход из строя.

Дисковые массивы RAID

Многие вычислительные машины и системы – серверы, суперкомпьютеры – предъявляют повышенные требования к надежности и производительности дисковой памяти. В таких системах возможностей одиночных винчестеров оказывается недостаточно, поэтому применяются дисковые массивы RAID (Redundant Array of Independent Disk – избыточный массив независимых дисков) – системы, в которых несколько жестких дисков используются совместно под управлением RAID-контроллера.

Использование RAID-массивов преследует следующие цели:

- 1) повышение надежности за счет избыточности данных;
- 2) повышение производительности за счет параллельной работы нескольких дисков.

Существует несколько уровней массивов RAID:

- RAID 0 – дисковый массив без отказоустойчивости (рис. 5.4). Информация разбивается на блоки и записывается на несколько дисков поочередно (расслоение данных). Скорость доступа к данным увеличивается. Избыточности нет. При отказе одного диска отказывает вся система.

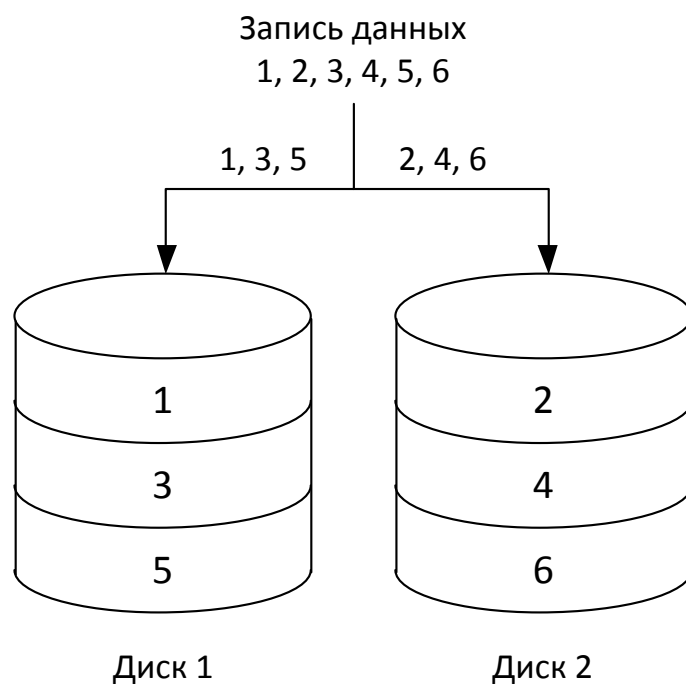


Рис. 5.4. RAID уровня 0

- RAID 1 – «зеркальный» дисковый массив. Общее число дисков делится пополам, вторая половина является зеркальной копией первой (рис. 5.5). Наибольшая избыточность.

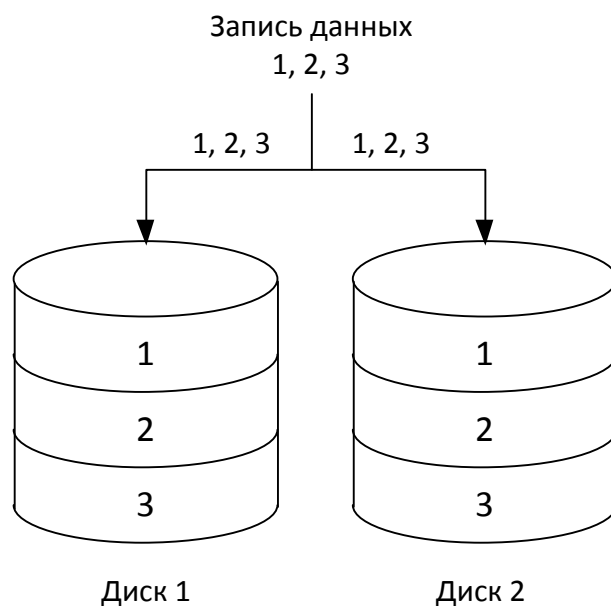


Рис. 5.5. RAID уровня 1

- RAID 2 использует несколько дисков специально для хранения контрольных сумм (коды Хэмминга). Отказоустойчивость ниже, чем у RAID 1, но избыточность меньше. Производительность невысокая. В настоящее время не используется.

- RAID 3 для хранения контрольных сумм всегда использует один диск, независимо от числа дисков с данными. Производительность, отказоустойчивость и эффективность использования пространства довольно высокие.

- RAID 4 подобен RAID 3, но расслоение данных производится по секторам, а не по байтам – эффективнее, но сложнее реализация.

- RAID 5 для хранения контрольных сумм использует некоторое пространство каждого диска. При выходе из строя одного из дисков информация восстанавливается. Самый популярный из уровней, в первую очередь благодаря своей экономичности.

- RAID 6 – сложная схема, устойчивая к отказам сразу двух дисков, но имеющая низкую производительность.

Помимо базовых уровней RAID 0–RAID 5 существуют комбинированные уровни RAID 1+0, RAID 3+0, RAID 5+0, RAID 1+5. Например, уровень RAID 1+0 совпадает с RAID 0, но роль отдельных дисков выполняют массивы, построенные по схеме RAID 1.

Накопители на магнитной ленте

Накопители на магнитной ленте применяются для резервного копирования (создания копий данных, необходимых для оперативного восстановления работоспособности системы в случае аварии) и архивирования (переноса на другой накопитель данных, предназначенных для длительного хранения).

хранения, в целях увеличения надежности и снижения стоимости владения информацией).

В современных компьютерах используются накопители на кассетной магнитной ленте, которая находится в картридже. Лентопротяжные механизмы для картриджей называются *стримеры* (streamer). В настоящее время выпускаются картриджи емкостью до нескольких терабайт со скоростью считывания данных с магнитной ленты – несколько Гбит/с.

5.1.3. Оптические диски

Оптическими называют такие диски, чтение и запись информации в которых происходят при помощи лазерного луча (оптического излучения).

Основные виды оптических дисков: CD, DVD, Blu-ray disk.

CD

CD-диски (Compact Disk, компакт-диск) начали выпускаться в 1982 году компаниями Sony и Philips.

Компакт-диск состоит из нескольких слоев: на основу из поликарбоната наносится слой отражающего покрытия (обычно из алюминия), который для защиты от повреждений покрывается лаком. Сверху диска может быть наклеена этикетка (рис. 5.6).

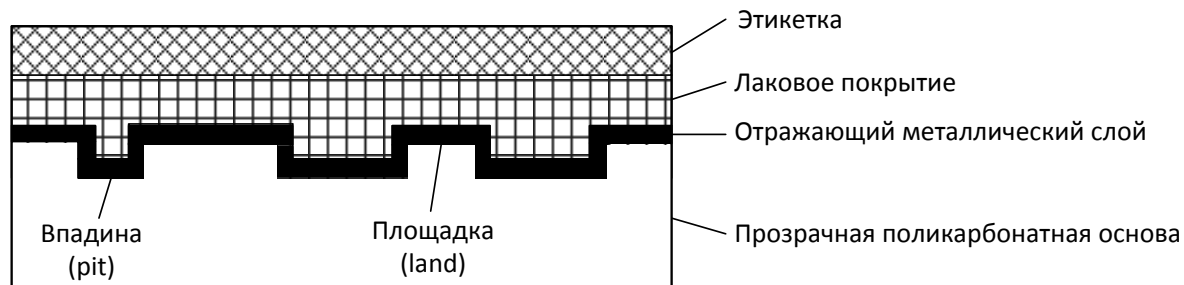


Рис. 5.6. Компакт-диск в разрезе

Информация располагается на единственной дорожке, которая спиралью проходит по всему диску (как в грампластинках, только начинается от центра диска). Данные хранятся за счет *впадин* (pits) и *площадок* (lands), сформированных на поверхности поликарбонатного слоя. При считывании информации луч инфракрасного лазера (длина волны – 780 нм) проходит по дорожке и, отражаясь от металлического слоя, попадает в фотоприемник. Если луч отражается от ровной поверхности (впадины или площадки), это означает двоичный ноль; если луч отражается от перепада, фиксируется двоичная единица (рис. 5.7). Для обнаружения и коррекции ошибок используется избыточное кодирование – 14 прочитанных бит соответствуют одному байту полезных данных.

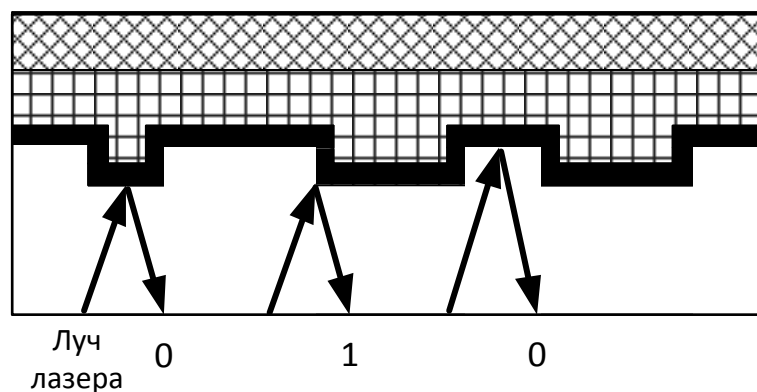


Рис. 5.7. Чтение информации с компакт-диска

Диаметр компакт-диска – 120 мм, толщина – 1,2 мм. Расстояние между витками спиральной дорожки составляет 1,6 мкм; ширина впадин – 0,8 мкм. Общая длина дорожки – 5,6 км.

Компакт-диски имеют емкость 650 Мбайт, 700 Мбайт (чаще всего) и 800 Мбайт.

Для обеспечения постоянной скорости чтения данных компакт-диск вращается с переменной угловой скоростью. В аудио CD скорость чтения 150 Кбайт/с, эта скорость принимается за единицу (1x). Для определения скорости чтения в компьютерных дисках указывается коэффициент: 4x, 10x, 48x, до 56x.

В зависимости от технологии записи информации выделяют следующие виды компакт-дисков:

- CD-ROM (Compact Disk Read Only Memory) – диск, записываемый при изготовлении;
- CD-R (recordable) – диск, записываемый однократно пользователем;
- CD-RW (rewritable) – перезаписываемый диск.

CD-ROM

Диски CD-ROM записываются однократно на заводе-изготовителе и в дальнейшем уже не перезаписываются. В процессе производства таких дисков сначала подготавливается стеклянный мастер-диск – образец, с которого затем путем штамповки делают партию компакт-дисков. Информацию на дорожке мастер-диска прожигают мощным лазером.

CD-R

Для однократной записи компакт-дисков в домашних условиях используются диски CD-R (появились в 1988 году). Пользователю поставляется заготовки, «болванки» (target), которые отличаются от CD-ROM во-первых, тем, что отражающий металлический слой изготавливается из золота или серебра, во-вторых, запись информации происходит в дополнительный

слой, называемый активным, который состоит из красящего органического вещества (цианина или фталоцианина). Лазер при записи не формирует впадины, как в CD-ROM, а нагревает краситель, который становится темным и хуже отражает цвет. Таким образом имитируются перепады обычного компакт-диска.

CD-RW

В 1997 году появились многократно записываемые (перезаписываемые) компакт-диски CD-RW. Активный слой красителя был заменен на сплав серебра, индия, сурьмы и теллура, который может находиться в двух состояниях – кристаллическом и аморфном (стеклообразном). В этих состояниях активный слой по-разному отражает свет: в кристаллическом хорошо (соответствует записанной единице), в аморфном – хуже (ноль).

Лазер в CD-дисковом устройстве имеет три режима работы:

1) в первом режиме (самом мощном по уровню излучения) лазер, подавая сильный короткий импульс, разогревает активный слой до 500 °С. Затем активный слой быстро охлаждается и переходит в аморфное состояние (записывается ноль);

2) во втором режиме (средней мощности) лазер длинным импульсом разогревает активный слой до 200 °С. После этого активный слой медленно остывает и переходит в кристаллическое состояние (записывается единица). Этот режим используется также для стирания информации;

3) в третьем режиме (маломощном) лазер осуществляет чтение, структура активного слоя не меняется.

CD-RW диски имеют около 1000 циклов перезаписи.

DVD

В 1996 году появилось новое поколение оптических дисков – DVD. Эта аббревиатура сначала расшифровывалась как Digital Video Disk – цифровой видеодиск, а затем как Digital Versatile Disk – цифровой универсальный диск.

По виду и размеру DVD похожи на обычные компакт-диски, однако отличаются гораздо большей плотностью записи, которой удалось добиться за счет следующих усовершенствований:

- 1) вдвое уменьшились размеры впадин – с 0,8 мкм до 0,4 мкм;
- 2) соседние витки дорожки «прижались» друг к другу – с расстояния 1,6 мкм до 0,74 мкм;
- 3) лазер красного света (вместо инфракрасного) с длиной волны 650 нм вместо 780 нм;
- 4) снижение избыточной служебной информации для обнаружения и коррекции ошибок за счет более эффективного кодирования.

Емкость стандартного DVD-диска стала равна 4,7 Гбайт вместо 700 Мбайт для CD. Однако разработчики не остановились на этих достиже-

ниях и ввели двустороннюю (с обеих сторон диска, а не с одной) и двуслойную запись. При двуслойном расположении данных на обычный отражающий слой помещается ещё один полупрозражающий, а лазер фокусируется либо на одном, либо на другом. При этом размер впадин приходится делать больше на полупрозражающем слое, поэтому его плотность записи немного меньше.

Базовая скорость (1x) чтения/записи DVD равна 1,32 Мбайт/с. Современные модели обеспечивают скорость до 20x.

Виды DVD:

- DVD-5 – односторонний/однослойный, 4,7 Гбайт;
- DVD-9 – односторонний/двухслойный, 8,54 Гбайт;
- DVD-10 – двухсторонний/однослойный, 9,4 Гбайт;
- DVD-18 – двухсторонний/двухслойный, 17,08 Гбайт.

Как и для компакт-дисков, существуют варианты записываемых/перезаписываемых DVD:

- DVD-R / DVD-RW – однократно записываемые/перезаписываемые диски, разработанные фирмой Pioneer;
- DVD+R / DVD+RW – однократно записываемые/перезаписываемые диски, разработанные фирмами Hewlett-Packard, Philips и Sony;
- DVD-RAM – перезаписываемые диски, разработанные фирмами Hitachi, Panasonic и Toshiba (менее распространены, чем первые два вида).

Представленные варианты несовместимы между собой, но современные DVD-приводы обычно поддерживают все три формата.

Ещё одной особенностью DVD-дисков является наличие регионального кода защиты (Regional Protection Code, RPC), который позволяет контролировать распространение, например, новых фильмов. Весь мир поделен на 6 зон (Россия относится к 5-й зоне), каждому DVD-диску и каждому DVD-проигрывателю присваивается определенный региональный код. Если код диска и код проигрывателя не совпадают, воспроизведение невозможно.

Blu-ray Disk

Очередным этапом развития оптических дисков стал формат *Blu-ray Disk* (BD), появившийся на рынке в 2006 году. Такое название («голубой луч») формат получил от коротковолнового 405-нанометрового «синего» лазера, который позволяет записывать данные с большей плотностью, чем на DVD при тех же физических размерах (буква «е» была сознательно исключена из слова «blue», что дало возможность зарегистрировать торговую марку, так как выражение «blue ray» является часто используемым и не может быть зарегистрировано как торговая марка).

Стандарт Blu-ray – продукт совместной разработки группы компаний по производству бытовой электроники и компьютеров, которые вошли в Ассоциацию Blu-ray дисков (Blu-ray Disc Association, BDA). Емкость одно-

слойного Blu-ray диска – 25 Гбайт, двухслойного – 50 Гбайт, трехслойного – 100 Гбайт, четырехслойного – 128 Гбайт. В настоящее время разрабатываются (и уже имеются образцы) BD-дисков с большим числом слоев и емкостью несколько сот гигабайт.

Увеличение емкости Blu-ray дисков по сравнению с DVD произошло за счет дальнейшего уменьшения длины волны лазера (405 нм) и размеров впадин (0,32 мкм), а также совершенствования лазерной системы.

Базовая скорость (1х) чтения/записи HD-дисков составляет 4,5 Мбайт/с. Выпускаются приводы со скоростями до 12х.

Виды BD-дисков:

- Mini Blu-ray Disc – компактный BD-диск диаметром 8 см, емкостью 7,5 Гбайт;
- BD-R (Blu-ray Disc Recordable) – однократно записываемый Blu-ray диск;
- BD-RE (Blu-ray Disc Rewritable) – многократно записываемый Blu-ray диск.

5.2. Периферийные устройства

5.2.1. Мониторы

Рассмотрим сначала, в чем отличие понятий «дисплей» (display) и «монитор» (monitor). *Дисплей* – это экран, поверхность для отображения информации. *Монитор* – это устройство вывода информации, состоящее из дисплея (экрана), схем управления и питания, находящихся в едином корпусе. Таким образом, монитор всегда имеет дисплей, но дисплей может быть без монитора. Например, сотовые телефоны, карманные компьютеры, калькуляторы имеют дисплей, но не имеют монитора. Тем не менее часто термины «дисплей» и «монитор» используют как синонимы.

Для преобразования компьютерных данных в формат, понятный монитору, и для управления работой монитора служит *видеоконтроллер* (видеокарта, графическая карта).

В настоящее время выпускают следующие виды компьютерных мониторов:

- ЭЛТ (CRT) – мониторы с электронно-лучевой трубкой (cathode ray tube);
- ЖК (LCD) – жидкокристаллические мониторы (liquid crystal display);
- плазменные (PDP) – мониторы на основе газоразрядного (плазменного) экрана (plasma display panel);
- OLED – мониторы на базе органических светоизлучающих диодов (organic light-emitting diode).

ЭЛТ-мониторы

Основным элементом ЭЛТ-мониторов является катодно-лучевая трубка – герметичная металлическая трубка, из которой выкачан воздух (создан вакуум) (рис. 5.8).

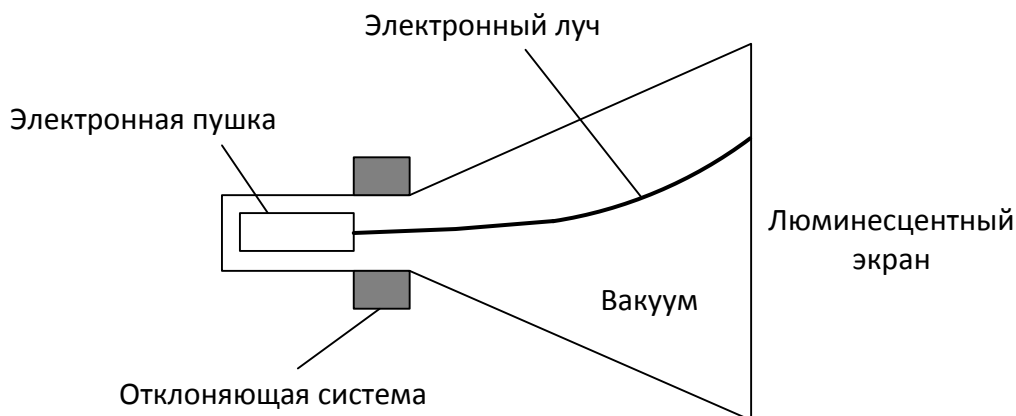


Рис. 5.8. Катодно-лучевая трубка

В трубке находится *электронная пушка*, которая испускает поток электронов (*электронный луч*) на экран монитора (*люминесцентный экран*), покрытый *люминофором* – веществом, которое может светиться под действием электронов. Чтобы луч попадал в заданную точку, на экране применяются электромагниты (*отклоняющая система*). В цветных мониторах используют три электронные пушки – для красного, зеленого и синего цветов (см. параграф 2.2.5 «Представление графической информации»).

Для формирования одного изображения – *кадра* – электронный луч быстро скользит по поверхности экрана по строкам, спускаясь сверху вниз. Чтобы начать следующий кадр, луч должен вернуться в верхний угол экрана. Такой процесс построения кадра называется *разверткой*. Человеческое восприятие таково, что для создания впечатления непрерывного движения на экране требуется формировать не менее 25 кадров в секунду (частота обновления экрана, измеряется в герцах). Современные мониторы и видеокарты могут обеспечивать частоту обновления 100 и более Гц.

Важной характеристикой любого монитора является его *разрешение* – количество пикселей по вертикали и горизонтали. Например, разрешение 800×600 обозначает 800 пикселей по горизонтали и 600 по вертикали. Разрешение зависит как от монитора, так и от графической карты.

ЖК-мониторы

В жидкокристаллических мониторах (LCD) содержатся особые вязкие органические вещества – жидкие кристаллы, которые обладают одновременно свойствами жидкости (текучесть) и твердого тела (кристаллическая

структура). На жидкие кристаллы можно воздействовать электрическим полем, заставляя их менять свое положение, влияя, таким образом, на способность пропускать свет.

ЖК-дисплей представляет собой две прозрачные стеклянные пластины, между которыми находится слой жидких кристаллов. К обеим пластинам присоединена матрица электродов, управляющих ориентацией кристаллов для каждого пикселя. Также в мониторе находится источник света. Молекулы жидких кристаллов, расположенные в разных частях экрана, под воздействием электрического поля от электродов по-разному пропускают свет, формируя изображение на экране. В цветных мониторах каждый пиксель состоит из трех субпикселей с фильтрами основных цветов (красного, зеленого и синего).

Плазменные мониторы

В плазменных мониторах (PDP), так же как и в ЖК-мониторах, имеются две стеклянные пластины, только вместо жидких кристаллов между ними расположена матрица ячеек, в которых находится газ (неон или ксенон). К каждой ячейке подводится пара электродов, и при подаче на них напряжения в газовой ячейке происходит ионизация и образуется плазма, в которой возникает электрический разряд и ультрафиолетовое излучение. Стенка ячейки покрыта люминофором; под действием ультрафиолета люминофор начинает светиться – пиксель на экране загорается. Подобный принцип образования света используется в лампах дневного света (люминесцентных лампах).

OLED-мониторы

В OLED-мониторах используются органические светоизлучающие диоды – полупроводниковые приборы, выполненные из нескольких слоев тонких полимеров, способные испускать свет под действием электрического тока.

Сравнение видов мониторов

В настоящее время основным видом компьютерных мониторов являются ЖК-мониторы. По сравнению с ЭЛТ-мониторами у них меньше размер, масса, энергопотребление, отсутствуют мерцание и геометрические искажения. С другой стороны, ЖК-мониторы имеют меньшую контрастность, их изображение зависит от угла наблюдения, четкость изображения достигается только при одном разрешении, совпадающем с размером матрицы экрана. Кроме того, у ЖК-дисплеев могут отказывать отдельные пиксели («битые пиксели»).

Плазменные панели не могут быть полноценной альтернативой ЖК-дисплеям вследствие большого размера пикселей – не существует плазменных панелей с диагональю менее 32 дюймов (81 см). Однако для про-

смотря крупных изображений на большом расстоянии плазменные панели являются хорошим выбором.

OLED-мониторы иногда рассматриваются в качестве технологии, способной в будущем заменить ЖК-дисплеи. OLED-мониторы имеют меньшие размеры и вес, более высокую контрастность, существенно меньшее время отклика (время формирования изображения), отсутствие искажения картинки при увеличении угла обзора. Недостатками OLED-дисплеев являются постепенное выгорание люминофора и пока ещё высокая стоимость.

5.2.2. Принтеры

Принтер (printer) – это устройство, способное выводить компьютерный текст или изображение на твердый носитель (чаще всего на бумагу).

Основные виды принтеров – матричные, струйные и лазерные.

Характеристики принтеров:

- *цветность* – различают черно-белые и цветные принтеры;
- *разрешающая способность* (разрешение) – измеряется количеством точек (dots) на дюйм длины – *dpi* (dots per inch). Наибольшее разрешение имеют лазерные принтеры;
- *скорость печати* – измеряется в листах (page) в минуту – *ppm* (pages per minute).

Матричные принтеры

Вывод изображения осуществляет печатающая головка, содержащая от 7 до 24 иголок, расположенных вертикально в один или несколько рядов. Иголки ударяют по бумаге через красящую ленту (как в печатных машинках) под действием электромагнитов. Печатающая головка в процессе печати движется слева направо, в нужные моменты иголки наносят отдельные элементы изображения, бумага перемещается в вертикальном направлении – таким образом, головка печатает одну строку за другой.

Для улучшения качества печать может осуществляться в несколько проходов так, что точки накладываются друг на друга; тем не менее максимальное разрешение матричных принтеров – 360 dpi. Некоторые модели матричных принтеров обеспечивают возможность цветной печати.

Несмотря на недостатки – значительный шум, невысокая скорость, низкое качество, в настоящее время матричные принтеры применяются во многих организациях, где не требуется высокое качество печати, выводится в основном текст, нужны неприхотливые и длительно эксплуатируемые устройства – для печати посадочных талонов в самолеты, железнодорожных билетов, товарных и кассовых чеков, торговых этикеток и т. п.

Струйные принтеры

В струйных принтерах также имеется подвижная печатающая головка, но вместо иголок в ней находятся микроскопические сопла, через которые на бумагу выстреливают чернила. Давление, под действием которого происходит выброс чернил, создается либо быстрым нагревом сопла, либо изгибом металлической пластинки при подаче на неё напряжения (*пьезоэффект*).

Основным преимуществом струйных принтеров является их невысокая стоимость при неплохом качестве печати, особенно цветной. Недостатки – низкая скорость работы, высокая стоимость напечатанной страницы и расходных материалов, сложность обслуживания (вследствие засыхания чернил на соплах).

Лазерные принтеры

Принцип действия лазерных принтеров существенно отличается от матричных и струйных. Вместо печатающей головки используется фотобарабан – барабан, поверхность которого покрыта фоточувствительным полупроводником – веществом, способным менять свое сопротивление под действием света. Сначала фотобарабан заряжается равномерно по всей поверхности. Затем лазерный луч проходит по поверхности фотобарабана и разряжает те места, где должны быть элементы изображения. К разряженным частям поверхности притягивается красящий порошок – тонер. В процессе вращения фотобарабана по нему протягивается лист бумаги, на котором остается тонер, формируя требуемое изображение. Далее тонер закрепляется на бумаге посредством разогрева и придавливания двумя валами, а фотобарабан разряжается, и остатки тонера счищаются.

Лазерные принтеры имеют высокую скорость и качество печати, как черно-белой, так и цветной. Однако стоимость лазерных принтеров (особенно цветных) существенно выше, чем струйных.

5.2.3. Сканеры

Сканер (scanner) – это устройство ввода графической информации в компьютер.

При сканировании на изображение падает луч света и, отражаясь, попадает на фотоприемник, который фиксирует яркость данного элемента изображения. Часто используется не один фотоприемник, а несколько, расположенных в линию.

Основные виды сканеров:

- *планшетные* – изображение неподвижно лежит, над ним автоматически проходит сканирующее устройство. Самый распространенный вид;

- *ручные* – изображение неподвижно, сканирующее устройство перемещается вручную. Используются для сканирования нестандартных поверхностей, например штрих-кодов на продуктах;

- *листопротяжные (рулонные)* – сканирующее устройство неподвижно, над ним автоматически протягивается изображение. Удобны для сканирования длинных изображений (показания самописцев);

- *барабанные* – изображение накладывается на барабан и вращается, сканирующее устройство за один оборот считывает вертикальную линию, затем перемещается по горизонтали. Имеют очень высокое качество сканирования;

- *проекционные* – отличаются от планшетных тем, что можно менять расстояние между сканирующим устройством и изображением. Применяются, например, для оцифровки больших книг с толстым переплетом.

Характеристики сканеров:

- *цветность* – сканеры бывают цветные, черно-белые полутоновые (различают множество оттенков серого) и черно-белые (только два цвета);

- *разрешение* – измеряется в dpi; различают следующие виды разрешений в сканерах:

- *оптическое разрешение* – количество фоточувствительных элементов в сканирующем устройстве (горизонтальное разрешение);
- *механическое разрешение* – количество шагов механического передвижения сканирующего устройства (вертикальное разрешение);
- *интерполяционное разрешение* – разрешение, устанавливаемое программной обработкой отсканированного изображения – к сканирующему устройству не имеет отношения.

Обычно указывается оптическое и механическое разрешение, причем оптическое всегда меньше, например разрешение 600×1200 означает оптическое разрешение 600 dpi, механическое – 1200 dpi;

- *глубина цвета* – см. параграф 2.2.5 «Представление графической информации»;

- *скорость сканирования*.

Контрольные вопросы и задания

1. Объясните принцип записи, применяемый в магнитных носителях.
2. Объясните принцип записи, применяемый в оптических носителях.
3. Запишите на свой винчестер файл размером 100 Кбайт. Какой объем памяти на диске будет занимать этот файл? Объясните, почему.
4. Чем отличается архивирование от резервного копирования?
5. За счет чего увеличилась емкость DVD-дисков по сравнению с CD?
6. Объясните принцип действия струйных принтеров.

7. Объясните принцип действия лазерных принтеров.
8. Чем отличается механическое, оптическое и интерполяционное разрешение сканеров?
9. * Каким образом хранятся данные на дисковых массивах RAID 5+0 и RAID 1+5? Для чего они используются?
- 10.* Какой принцип записи станет преобладающим через десять лет? Ответ обоснуйте.
- 11.* Имеется ли электромагнитное излучение у жидкокристаллических дисплеев?
- 12.* Что такое голография? Какие периферийные устройства могут использовать принцип голографии?
- 13.* Какие принципы применяются при создании устройств «виртуальной реальности»?
- 14.* Какие периферийные устройства появятся в ближайшие десять лет?

ГЛАВА 6. ВЫЧИСЛИТЕЛЬНЫЕ СИСТЕМЫ

Вычислительная машина, собранная нами по «кирпичикам» в предыдущих главах, позволяет решать любые задачи, которые в принципе поддаются решению на компьютере. Однако часто встречаются проблемы, объем данных или вычислений которых настолько велик, что одна ВМ не способна справиться с ними за приемлемое время. Например, моделирование Большого взрыва (рождение Вселенной), воспроизведение ядерной реакции, расчет траекторий космических аппаратов, разработка новых лекарств, обработка генетической информации, моделирование глобальных экономических процессов и т. д.

Для решения таких сложных задач создаются *вычислительные системы*.

Напомним, что понятия «вычислительная машина» и «вычислительная система» тесно связаны и отличаются количеством вычислителей (см. параграф 1.1). Наличие нескольких вычислителей позволяет ввести понятия *параллельных вычислений*, т. е. когда в каждый момент времени решается несколько задач¹.

Таким образом, целью создания вычислительных систем является *повышение производительности* за счет *параллельной обработки информации*, возможность которой в свою очередь возникает из-за наличия *нескольких вычислителей*.

6.1. Виды вычислительных систем

Рассмотрим некоторые наиболее распространенные структуры вычислительных систем (рис. 6.1).

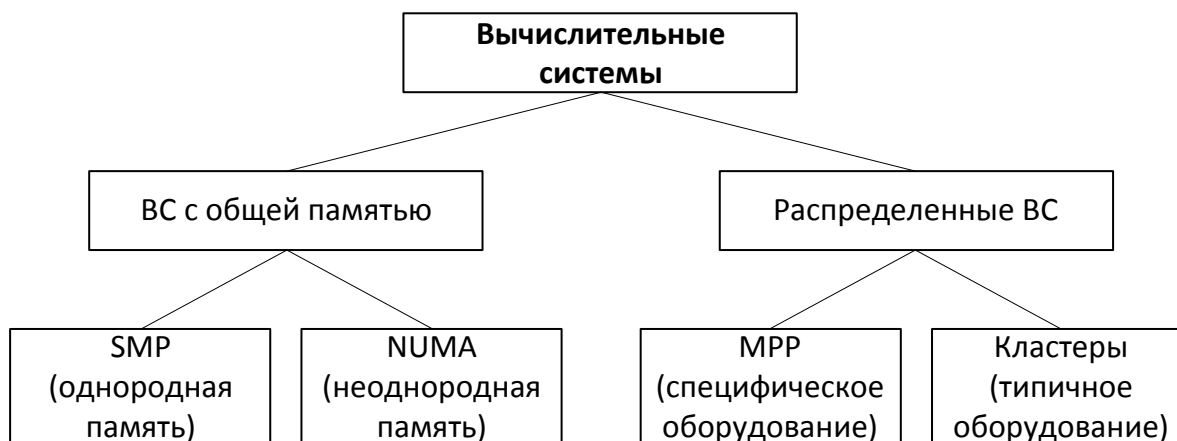


Рис. 6.1. Классификация вычислительных систем

¹ Следует отметить, что даже в однопроцессорных одноядерных ВМ в настоящее время присутствует возможность параллельных вычислений. Примерами могут служить суперскалярные процессоры с несколькими исполнительными блоками.

Главный принцип классификации – способ использования оперативной памяти. В соответствии с этим критерием существуют два основных класса вычислительных систем – с общей памятью и с распределенной памятью.

В *вычислительных системах с общей памятью* все процессоры совместно используют её для хранения и обмена данными. Эта общая память может быть однородной или неоднородной. Однородность памяти означает, что доступ каждого процессора к памяти производится одинаково и занимает одно и то же время. Системы с *однородной общей памятью* называются UMA-системами (Uniform Memory Access). Примером таких систем являются SMP-системы (симметричные многопроцессорные).

В системах с *неоднородной памятью* (NUMA – Non-Uniform Memory Access) также имеется единая для всех вычислителей память, однако физически она распределена по процессорам, т. е. каждый процессор имеет свою локальную память. Неоднородность доступа к памяти заключается в том, что появляются два варианта расположения нужных процессору данных – в локальной памяти этого процессора или в памяти другого процессора. В качестве примера можно привести CC-NUMA-системы (кэш-когерентный доступ к неоднородной памяти).

В *распределенных вычислительных системах* оперативная память распределена по вычислителям, т. е. каждый процессор имеет свою независимую память. В таких системах процессоры могут общаться между собой только путем передачи сообщений через коммуникационную сеть.

Различают два вида распределенных ВС с точки зрения стоимости и доступности используемого оборудования – системы с массивной параллельной обработкой (MPP) и кластеры.

Достоинством систем с общей памятью является то, что процессоры могут легко и быстро обмениваться данными; в то же время распределенные системы позволяют строить очень крупные вычислительные комплексы с большим числом процессоров – все системы из списка самых мощных суперкомпьютеров мира Top500¹ являются распределенными.

Рассмотрим каждый из представленных видов ВС более подробно.

6.2. Симметричные многопроцессорные системы (SMP)

Симметричные многопроцессорные системы (SMP – Symmetric Multiprocessor) можно охарактеризовать следующим образом:

- наличие двух и более процессоров, обладающих примерно одинаковыми характеристиками (отсюда название «симметричные»);
- процессоры совместно используют общую память;
- управление осуществляется единой операционной системой.

¹ TOP500 Supercomputer Sites: www.top500.org.

Кроме общей памяти каждый процессор может иметь собственную кэш-память (рис. 6.2).

Достоинства SMP-систем:

- простота и универсальность с точки зрения программирования;
- простота эксплуатации;
- относительно невысокая стоимость.

Недостатки:

- низкая отказоустойчивость (при отказе одного узла нарушается работа всей системы);
- плохая масштабируемость (сложность увеличения числа процессоров).

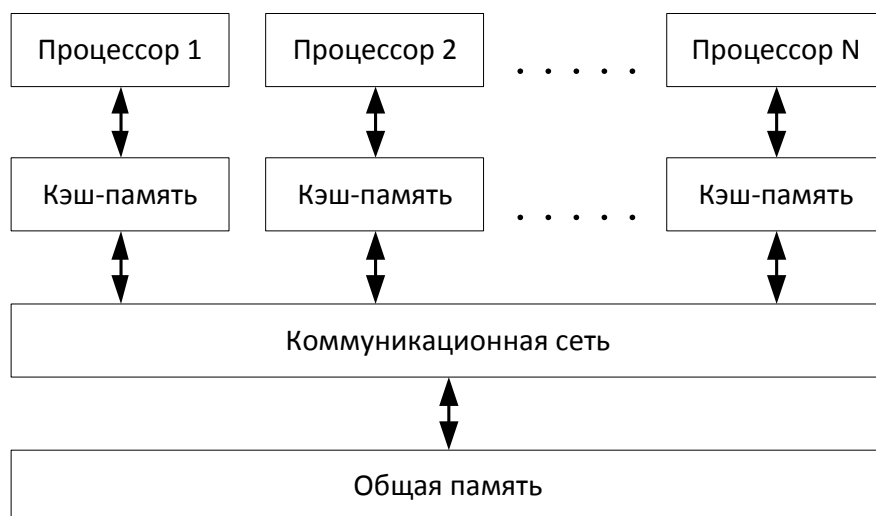


Рис. 6.2. Структура SMP-системы

Последний недостаток приводит к тому, что количество процессоров в SMP-системах не превышает нескольких сотен. Поэтому, если требуется увеличить это количество, используют NUMA-системы или кластеры на базе SMP.

6.3. Системы с неоднородным доступом к памяти (NUMA)

Увеличение числа процессоров в SMP-системах невозможно из-за того, что при этом коммуникационная сеть становится узким местом. Возможным выходом может служить структура, состоящая из нескольких узлов (узлом может быть либо отдельная ВМ, либо SMP-система), объединенных сетью. Такая структура по-прежнему относится к системам с общей памятью, так как каждый процессор считает, что работает с единой памятью, однако физически память распределена по узлам. Подобные системы называются *системами с неоднородной памятью* (NUMA – Non-Uniform Memory Access). Одним из распространенных видов NUMA являются *системы с кэш-когерентным доступом к неоднородной памяти* (CC-NUMA – Cache Coherent Non-Uniform Memory Access).

На рис. 6.3 показана структура CC-NUMA-системы, состоящей из N узлов, причем каждый узел включает процессор, кэш-память, системную шину и модуль памяти, который является частью общей памяти.

Таким образом, при необходимости обращения процессора к памяти, поиск нужной ячейки сначала осуществляется в кэш-памяти процессора, затем в общей локальной памяти узла, и только при отсутствии в ней происходит обращение по коммуникационной сети к другому узлу, то есть память является *трехуровневой*.

Кэш-когерентность (кэш-согласованность) означает, что данные в кэш-памяти каждого процессора должны быть согласованы между собой, причем это осуществляется на аппаратном уровне.

Основным достоинством CC-NUMA-систем является возможность масштабирования.

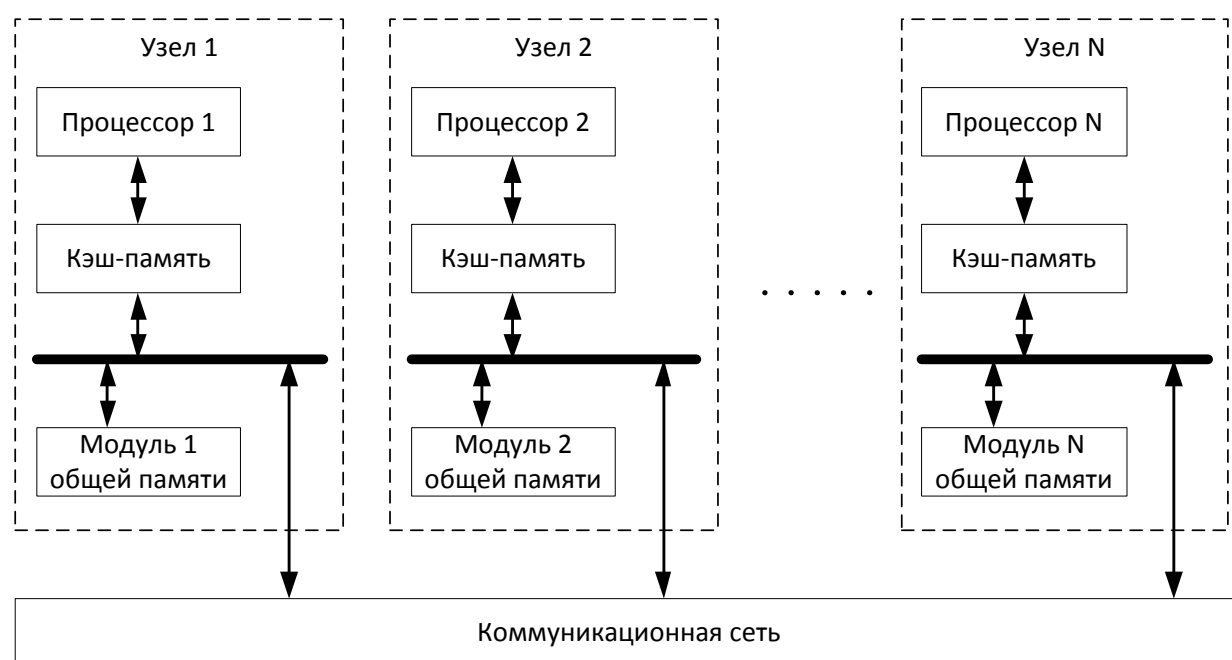


Рис. 6.3. Структура CC-NUMA-системы

6.4. Кластерные системы

Кластер – группа соединенных вычислительных узлов, работающих совместно и составляющих единый вычислительный ресурс. Узлом может быть обычная однопроцессорная ВМ или SMP-система, причем в случае соединения в кластер обычных ВМ используют термин *кластер рабочих станций* (workstation cluster).

В кластерах отсутствует общая для всех узлов память, т. е. вся память является распределенной (кроме внешней дисковой памяти, которая используется совместно). Структура кластера представлена на рис. 6.4.

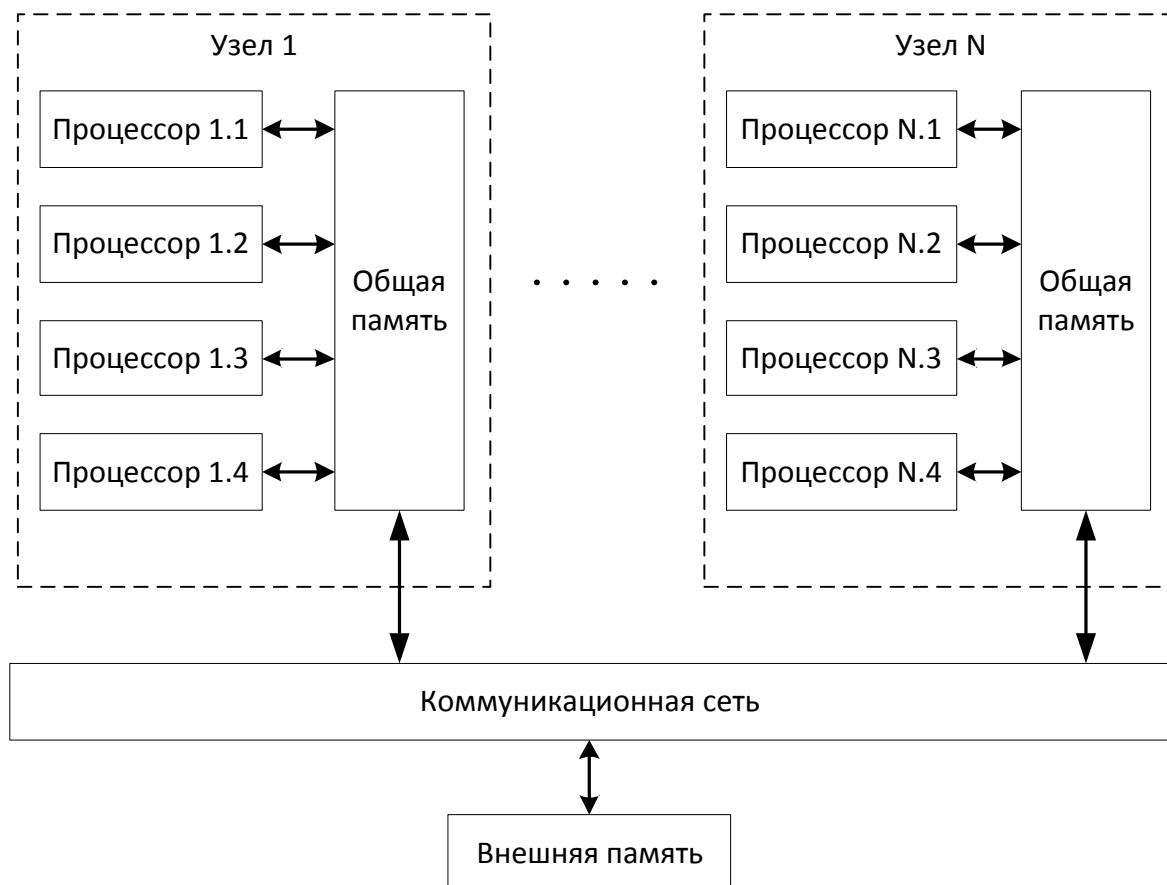


Рис. 6.4. Структура кластерной системы

Достоинства:

- хорошая масштабируемость;
- высокая надежность (отказ одного из узлов кластера не приводит к отказу всей системы);
- низкая стоимость (все компоненты кластера – узлы и коммуникационное оборудование – могут быть стандартными устройствами).

Недостатки:

- требует специального программного обеспечения;
- снижается производительность вследствие применения недорогих реализаций коммуникационной сети.

6.5. Системы с массовой параллельной обработкой (MPP)

Структура MPP-системы (MPP – Massively Parallel Processing) похожа на структуру кластера: это группа узлов, имеющих распределенную память и объединенных высокоскоростной сетью. MPP-системы отличаются от кластеров тем, что используют специфическое оборудование, вследствие чего оказываются дороже.

Основным достоинством MPP-систем является высокая масштабируемость. В качестве недостатка можно отметить высокую стоимость.

6.6. Измерение производительности вычислительных систем

Производительность вычислительных систем оценивается количеством операций, выполняемых за одну секунду. Единицами измерения являются следующие величины:

- MFLOPS, MegaFLOPS (Floating-Point Operations Per Second) – миллион (10^6) операций с плавающей точкой в секунду;
- GFLOPS, GigaFLOPS – миллиард (10^9) операций с плавающей точкой в секунду;
- TFLOPS, TeraFLOPS – триллион (10^{12}) операций с плавающей точкой в секунду;
- PFLOPS, PetaFLOPS – квадриллион (10^{15}) операций с плавающей точкой в секунду.

Для сравнения производительности ВС были разработаны фиксированные тесты. Наиболее известным из них является LINPACK – программа, предназначенная для решения системы линейных алгебраических уравнений с плотной матрицей с выбором главного элемента по строке. LINPACK используется для формирования списка Top500 – пятисот самых мощных компьютеров мира (www.top500.org). На ноябрь 2011 года самый мощный суперкомпьютер в мире имел производительность более 10 петафлопс.

Показатели MFLOPS, GFLOPS, TFLOPS или PFLOPS характеризуют максимальную производительность ВС. Однако на реальных задачах производительность может оказаться гораздо ниже, в связи с тем что далеко не все программы можно хорошо распараллелить, т. е. равномерно распределить по всем имеющимся процессорам.

Контрольные вопросы и задания

1. Чем понятие «вычислительная система» отличается от понятия «вычислительная машина»?
2. Какова цель создания вычислительных систем?
3. Объясните понятие «масштабирование».
4. Что такое «кэш-когерентность»?
5. * Проанализируйте рейтинг Top500 по различным данным: странам, мощности, областям применения, моделям процессоров и т. д.
6. * Приведите примеры алгоритмов, которые хорошо распараллеливаются.

ГЛАВА 7. КОМПЬЮТЕРНЫЕ СЕТИ

7.1. Введение в компьютерные сети

7.1.1. Основные понятия

Компьютерная сеть (computer network) – объединение связанных между собой компьютеров, которые могут обмениваться информацией.

Компьютеры объединяют в сеть ради *трех главных целей*: совместное использование ресурсов сети, общение между пользователями и предоставление услуг.

Цель 1: совместное использование ресурсов сети.

Ресурсами сети могут быть:

- данные и программы (файлы на дисках);
- устройства (принтеры, сканеры, модемы и т. д.);
- вычислительная мощность процессоров.

Пример. Если требуется решить сложную задачу, связанную с длительными вычислениями, например решить большую систему линейных уравнений, вполне вероятно, что такую задачу можно распараллелить и разделить между несколькими компьютерами в сети. В этом случае используется ресурс сети – вычислительная мощность процессоров.

Ресурсы, доступные в сети, называют *разделяемыми ресурсами* (shared resources).

Цель 2: общение между пользователями.

Общение между пользователями сети может происходить посредством специальных сайтов (социальные сети, блоги, форумы и т. д.), электронной почты (e-mail), чатов, видеоконференций, IP-телефонии и т. д.

Цель 3: предоставление услуг (сервисов) пользователям.

Этот способ использования компьютерных сетей стал очень востребованным в последние 10–15 лет в связи с повсеместным распространением глобальной сети – Интернета. Сюда можно отнести покупку-продажу всевозможных товаров (книг, подарков, ж.-д. и авиабилетов, электронной аппаратуры и т. п.), электронные платежные системы (PayPal, WebMoney, Яндекс.Деньги и т. д.), интернет-радио и телевидение, рекламу, онлайн-игры и др.

Все компьютеры в сети делятся на два вида – серверы и клиенты.

Сервер (server, от англ. *to serve* – служить) – компьютер или программа, предоставляющие ресурсы или услуги.

Клиент (client) – компьютер или программа, использующие ресурсы или услуги.

Иногда используется термин *peer*¹ – компьютер или программа, предоставляющие и использующие ресурсы.

Подчеркнем, что термины сервер и клиент могут использоваться по отношению и к компьютерам, и к программам. Например, *сервер базы данных* – мощный компьютер и/или специальная программа, управляющие доступом к базе данных.

Компьютеры, объединенные в сеть (как серверы, так и клиенты), а также другие сетевые устройства (концентраторы, коммутаторы, маршрутизаторы и т. д.) называются *узлами* (nodes) сети.

Канал связи, соединяющий узлы сети, называется *средой передачи данных* (media). Среда передачи может быть *проводной* (wire) и *беспроводной* (wireless).

Степень загруженности сети, определяемая потоком данных в ней, называется *трафиком* (traffic).

Виды передачи данных:

- *симплексная* (simplex) – передача может идти только в одну сторону (пример: телевидение, радиовещание);
- *полудуплексная* (half-duplex) – передача может идти в обе стороны, но попеременно (пример: две рации);
- *полнодуплексная* (full-duplex) – передача может идти в обе стороны одновременно (пример: телефония).

7.1.2. Классификация компьютерных сетей

Существует несколько способов классификации компьютерных сетей: по размеру, по способу взаимодействия, по топологии, по типу среды передачи и т. д. Рассмотрим некоторые из них.

Классификация по размеру

Компьютерные сети различаются по размеру (или занимаемой территории). Выделяют персональные, локальные, городские и глобальные сети.

Персональная сеть (PAN – Personal Area Network) – сеть одного человека. Назначение такой сети – объединение всех электронных устройств пользователя (настольный компьютер, ноутбук, принтер, коммуникатор, сотовый телефон и т. п.). Реализуются персональные сети на основе технологий Wi-Fi, Bluetooth, USB и иногда IrDA².

¹ От этого термина, например, произошло понятие *пиринговые файлообменные сети* – это сети, в которых каждый компьютер может как предоставлять свои файлы в общий доступ, так и скачивать файлы с других узлов сети.

² IrDA (Infrared Data Association) – стандарт связи по инфракрасному порту.

Локальная сеть (LAN – Local Area Network) расположена в пределах одного или нескольких близких зданий, и ее размер не превышает 1–2 километров. В LAN обычно используется ограниченный набор сетевых протоколов, например Ethernet + TCP/IP. Примеры: домашняя сеть, сеть факультета, университета, небольшого предприятия.

Недавно в связи с развитием беспроводных сетей появился термин *WLAN – Wireless LAN* (беспроводная локальная сеть).

Городская, или муниципальная, сеть (MAN – Metropolitan Area Network) является промежуточной между локальной и глобальной сетями как по размеру, так и по скорости передачи данных. Занимает территорию до нескольких десятков километров и расположена в пределах города. Набор используемых сетевых протоколов и оборудования обычно шире, чем в LAN. Примером является сеть провайдера, предоставляющего жителям одного города доступ в Интернет.

Глобальная сеть (WAN – Wide Area Network) – это сеть, которая связывает отдельные компьютеры, локальные и городские сети, находящиеся на очень большом расстоянии, например в разных городах или на разных континентах. В связи с этим спектр применяемых протоколов и оборудования наиболее велик. Самым известным примером глобальной сети является *Интернет*.

Классификация по типу взаимодействия

Компьютерные сети по типу взаимодействия можно разделить на два вида:

- *одноранговые* (peer-to-peer); в сетях Microsoft Windows называются *рабочими группами* (workgroup);
- *на основе сервера* (server based); в сетях Microsoft Windows называются *доменами*¹ (domain).

Компьютеры *одноранговой сети* могут быть и клиентами и серверами одновременно (*peer*). Ни один компьютер не имеет ни приоритета доступа, ни обязанности предоставлять ресурсы в совместное использование. Чаще всего в таких сетях находятся не более нескольких десятков компьютеров.

Управление доступом к ресурсам определенного компьютера должно осуществляться непосредственно на этом компьютере². Чтобы получить доступ к ресурсам компьютера, необходимо знать имя и пароль пользователя, учетная запись которого создана на этом компьютере (рис. 7.1). Таким образом, чтобы иметь доступ ко всем ресурсам сети, нужно создать свои учетные записи на каждом компьютере и знать пароли к ним.

¹ Термин *домен* может использоваться в разных контекстах (например, домен в Интернете). В любом случае под доменом понимают группу компьютеров, объединенных по какому-либо признаку.

² Конечно, если не используются специальные утилиты удаленного администрирования.

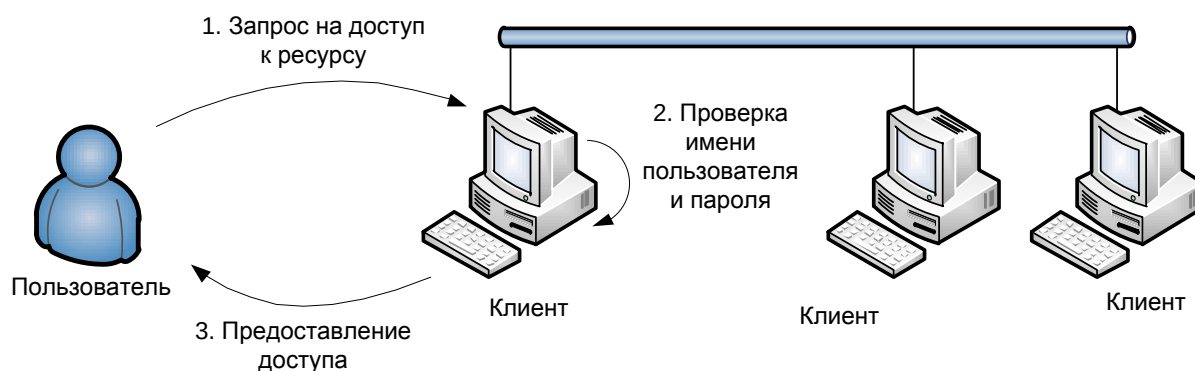


Рис. 7.1. Получение доступа к ресурсам в одноранговой сети

В сетях на основе сервера (доменах) существуют один или несколько выделенных серверов (мощных компьютеров с серверной операционной системой и со специальным программным обеспечением), которые предоставляют ресурсы. Кроме того, имеется один или несколько серверов, управляющих сетью¹ (в сетях Microsoft они называются *контроллерами домена*, *domain controller*).

Важное отличие от одноранговых сетей состоит в повышении уровня безопасности. Здесь существует централизованная проверка учетных записей пользователей, их прав и паролей. Прежде чем получить доступ к любому ресурсу, пользователь должен *войти в сеть* – на любом компьютере сети ввести свои имя и пароль, которые будут автоматически переданы контроллеру домена. Контроллер домена проверит пароль пользователя и, в зависимости от прав пользователя, разрешит или запретит доступ к ресурсу (рис. 7.2). При этом, выполнив однажды вход в сеть, пользователь может пользоваться всеми ресурсами сети, не вводя имя и пароль повторно (до завершения сеанса работы, т. е. до выхода из сети).

В табл. 7.1 приведена сравнительная характеристика рассмотренных видов сетей.

¹ Один и тот же сервер может и предоставлять ресурсы, и управлять сетью.



Рис. 7.2. Получение доступа к ресурсам в сети на основе сервера

Таблица 7.1. Сравнительная характеристика одноранговых сетей и сетей на основе сервера

Характеристика	Одноранговые сети	Сети на основе сервера
Стоимость	Низкая, так как не нужно дополнительных ресурсов (выделенного сервера, администратора, специального ПО)	Высокая, так как требуется специальное оборудование, ПО и персонал
Установка и настройка	Простая	Сложная
Администрирование (управление)	Раздельное, на каждом компьютере отдельно	Централизованное, на контроллере домена
Производительность	Низкая, поскольку нет выделенных серверов	Высокая, так как имеются выделенные серверы
Резервирование данных	Раздельное, на каждом компьютере отдельно	Централизованное, на серверах
Отказоустойчивость	Высокая – при отказе одного или нескольких компьютеров работоспособность сети сохраняется	При отказе контроллера домена вся сеть не работает; проблема решается дублированием контроллеров домена

В целом одноранговую модель можно использовать для небольших недорогих сетей, включающих не более 20–30 компьютеров, в которых вопросы безопасности не являются приоритетными. В остальных случаях следует предпочесть модель на основе сервера.

Классификация по топологии

Топология компьютерной сети – это способ организации связей между узлами сети.

Существуют следующие основные виды топологий: ячеистая, общая шина, кольцо, звезда, иерархическая звезда (или дерево), сотовая (рис. 7.3).

Можно выделить два подвида *ячеистой топологии* – *полносвязная* (full mesh), когда каждый узел связан со всеми другими, и *неполносвязная* (partial mesh), в которой между некоторыми узлами могут отсутствовать связи. Такая топология применяется редко, только если нужно обеспечить высокую надежность при малом числе узлов сети.

Топология общая шина (bus) позволяет создать сеть, используя кабель наименьшей длины. Особенностью шинной топологии является простота установки и изменения конфигурации сети. В качестве недостатков можно отметить то, что при повреждении шины отказывает вся сеть, причем достаточно сложно найти место разрыва; а также использование одного канала для всего трафика сети, что может стать узким местом при большом объеме трафика.

Кольцевая топология (ring) имеет существенное преимущество – наличие обратной связи. Это означает, что, если один компьютер посылает сообщение другому, получатель может включить в сообщение информацию об успешном (или неуспешном) приеме и отправить его дальше по кольцу источнику сообщения. С другой стороны, так же как в общей шине, при разрыве кабеля вся сеть перестает работать и бывает сложно определить место повреждения. Для повышения надежности сети часто используют *двойное кольцо*: в штатном режиме постоянно работает первое кольцо, а при его разрыве вступает в действие второе, резервное кольцо.

В *топологии звезда* (star) обязательно наличие центрального устройства, которое называется *концентратор* (hub). Каждый узел подключается к сети через отдельный кабель, таким образом, при разрыве линии связи от сети оказывается отрезанным только один компьютер, остальные работают как обычно. Только в случае неисправности концентратора вся сеть перестает работать.

Топология *дерево* (tree) или *иерархическая звезда* является объединением нескольких звезд и позволяет строить достаточно большие сети. На сегодняшний день топологии звезда и дерево – самые распространенные в локальных сетях.

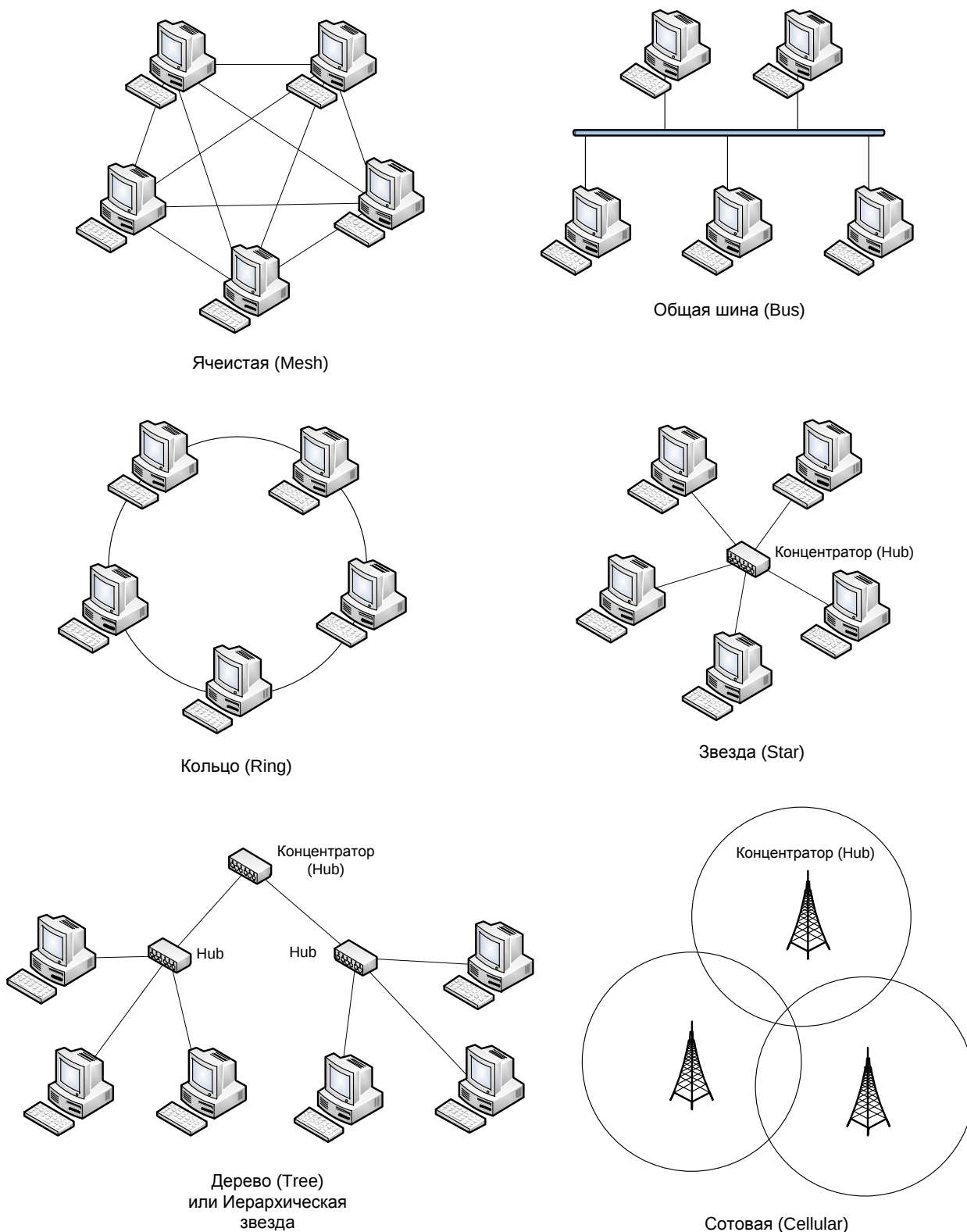


Рис. 7.3. Виды топологий

Сотовая топология (cellular) применяется в беспроводных сетях мобильной связи. Территория, охватываемая одним приемопередатчиком, называется *сотой* (cell). Информация может передаваться как в рамках одной соты, так и между несколькими сотами. Такая топология позволяет создавать огромные по размеру сети.

При выборе той или иной топологии необходимо оценивать сложность операций установки и нахождения неисправностей, а также принимать во внимание свойство отказоустойчивости сети. В общем, эти характеристики зависят от количества связей между узлами сети. Чем больше связей, тем сложнее установка, но тем проще находить поврежденный участок сети и тем сеть надежнее.

Например, ячеистая топология имеет наибольшее число связей и поэтому является самой надежной, но и очень сложна в установке. С другой стороны, шинная и кольцевая топологии обладают наименьшим количеством связей и, следовательно, просты в установке, но ненадежны и сложны в диагностике неисправностей.

7.1.3. Основные характеристики сети

Ключевые сетевые характеристики приведены на рис. 7.4.

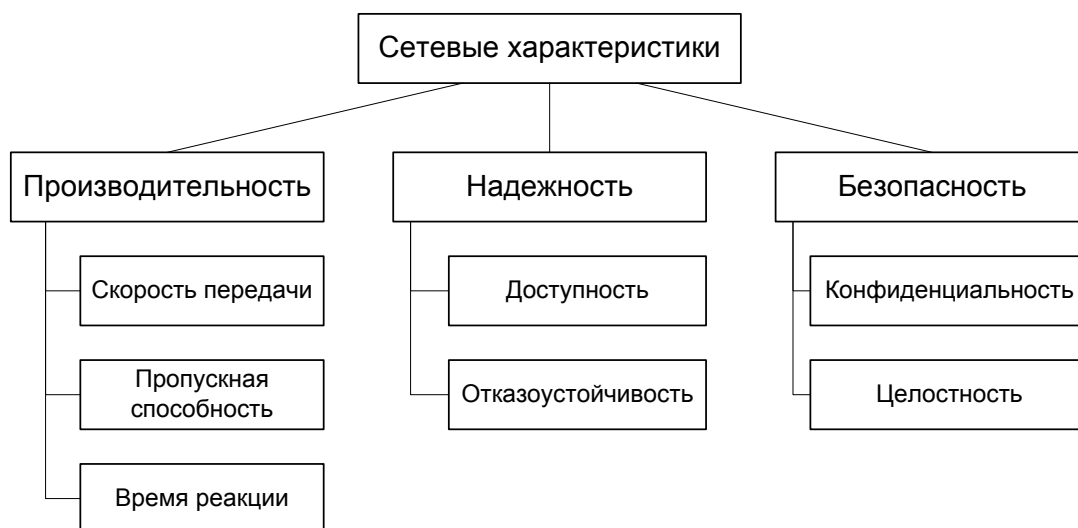


Рис. 7.4. Сетевые характеристики

Можно выделить три группы сетевых характеристик: производительность, надежность и безопасность.

Производительность (productivity) сети описывается тремя основными характеристиками – скоростью передачи данных, пропускной способностью и временем реакции.

Скорость передачи данных (data transfer rate) – количество информации, переданной за единицу времени. Измеряется в битах в секунду (бит/с). Чтобы найти скорость передачи данных, нужно разделить объем переданной информации на интервал времени, за который произошла передача.

Пропускная способность (throughput) – максимально возможная скорость передачи данных по линии связи. Измеряется также в бит/с.

Скорость передачи данных отражает реальное состояние сети в текущий момент времени и может изменяться. Это зависит от состояния канала

связи, типа передаваемых данных, внешних факторов (помех). Пропускная способность является теоретической характеристикой линии связи или сетевой технологии и не изменяется. Она устанавливает верхний предел для скорости передачи данных. Чаще всего скорость передачи составляет величину, меньшую пропускной способности; связано это с тем, что помимо полезных данных по сети передается служебная информация, кроме того, часть времени занимают процедуры доступа к сети.

Пример. В локальных сетях Fast Ethernet пропускная способность равна 100 Мбит/с, то есть теоретически за 1 секунду можно передать 100 Мбит данных (или 12,5 Мбайт). Однако на практике за 1 секунду передается файл размером 4 Мбайт, т. е. реальная скорость передачи данных оказывается 32 Мбит/с, или 32% от пропускной способности сети.

Ещё одной характеристикой производительности является *время реакции* (response time) – интервал времени между запросом пользователя к ресурсу сети и получением ответа на запрос. Время реакции складывается из интервалов времени выполнения следующих операций:

- 1) подготовка запроса на компьютере-клиенте;
- 2) передача запроса по сети;
- 3) подготовка ответа компьютером-сервером;
- 4) передача ответа по сети;
- 5) обработка ответа на компьютере-клиенте.

Как видно, время реакции является характеристикой не только сети, но и программного и аппаратного обеспечения, установленного на компьютерах.

Характеристика *надежности* (reliability) включает свойства доступности и отказоустойчивости.

Доступность (availability) – доля времени, в течение которого сеть готова к работе. Доступность является вероятностной величиной и измеряется на достаточно большом интервале времени (день, месяц, год). Обычное значение, например, 0,999 – означает, что, вероятно, сеть будет недоступна только около 8 с половиной часов в год.

Отказоустойчивость (fault tolerance) – это способность сети сохранять работоспособность при отказе отдельных элементов.

Например, ячеистая топология является отказоустойчивой, потому что при обрыве одной связи между узлами остается ещё несколько путей передачи сообщений, так что сеть остается работоспособной. А вот шинная топология не обладает этим свойством, поскольку при повреждении кабеля сеть перестает правильно работать.

Общим принципом, обеспечивающим высокую надежность, является *внесение избыточных связей в сетевую структуру* (сравните ячеистую и шинную топологии по этому критерию).

Безопасность (safety) сети определяется конфиденциальностью и целостностью её ресурсов.

Конфиденциальность (privacy) – способность сети обеспечивать доступ к ресурсам только тем пользователям, которые имеют на это право.

Целостность (integrity) – способность сети обеспечивать сохранность ресурсов.

7.2. Модель сетевого взаимодействия

На рынке сетевого оборудования и программного обеспечения существует множество фирм, и одной из важнейших является проблема совместимости, т. е. соответствия характеристик продукции разных производителей. Для того чтобы сетевые устройства правильно взаимодействовали между собой, требуется наличие общих для всех стандартов. К настоящему времени одним из наиболее распространенных сетевых стандартов, охватывающих весь процесс сетевого взаимодействия, является модель OSI¹.

7.2.1. Модель OSI

Модель взаимодействия открытых систем (OSI, Open Systems Interconnection model) разработана Международной организацией по стандартизации ISO² и Международным союзом электросвязи ITU³ в 1977–1984 годах и принята в качестве стандарта ISO 7498.

Процесс сетевого взаимодействия весьма сложен. Как его описать и реализовать? В модели OSI используется фундаментальный для информатики принцип борьбы со сложностью – «Разделяй и властвуй». Сетевое взаимодействие рассматривается на семи уровнях, организованных в иерархию⁴ (рис. 7.5). Каждый уровень имеет определенные функции и должен быть реализован на всех сетевых узлах.

Уровень модели OSI может взаимодействовать с соседними уровнями в иерархии, причем нижележащий уровень предоставляет набор услуг вышележащему уровню в соответствии со своим интерфейсом.

Интерфейс (interface) – это правила, определяющие формат сообщений, которыми обмениваются сетевые компоненты, находящиеся на соседних уровнях в одном узле.

Между одноименными уровнями, реализованными на разных узлах, также осуществляется взаимодействие по соответствующему протоколу.

Протокол (protocol) – это правила, определяющие формат сообщений, которыми обмениваются сетевые компоненты, лежащие на одном уровне, но в разных узлах.

¹ Также распространена модель DoD (Department of Defense – министерство обороны США), в соответствии с которой построен стек протоколов TCP/IP, лежащий в основе Интернета.

² ISO – International Organization for Standardization.

³ ITU – International Telecommunications Union.

⁴ Иерархия – система уровней, в которой строго определено подчинение нижележащего уровня вышележащему.

Стек протоколов – это иерархически организованный набор протоколов, достаточный для организации взаимодействия узлов в сети.

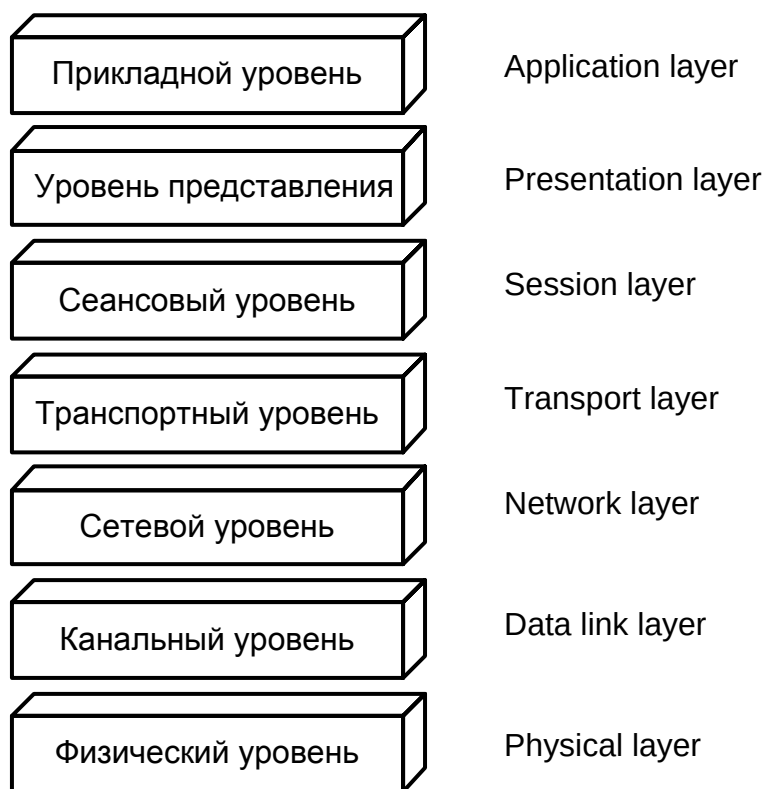


Рис. 7.5. Уровни модели OSI

В процессе коммуникации узлы обмениваются *сообщениями* (*message*). Передачу сообщения начинает сетевое приложение (программа, которой нужны услуги сети). При передаче сообщения сверху вниз каждый уровень добавляет к нему свой заголовок, содержащий информацию для одноименного уровня на другом узле. Непосредственно по каналам связи сообщение передается на узел-получатель в соответствии с протоколом физического уровня. Затем сообщение поднимается по иерархии снизу вверх и каждый уровень считывает «свой» заголовок. Иллюстрация этого процесса приведена на рис. 7.6.

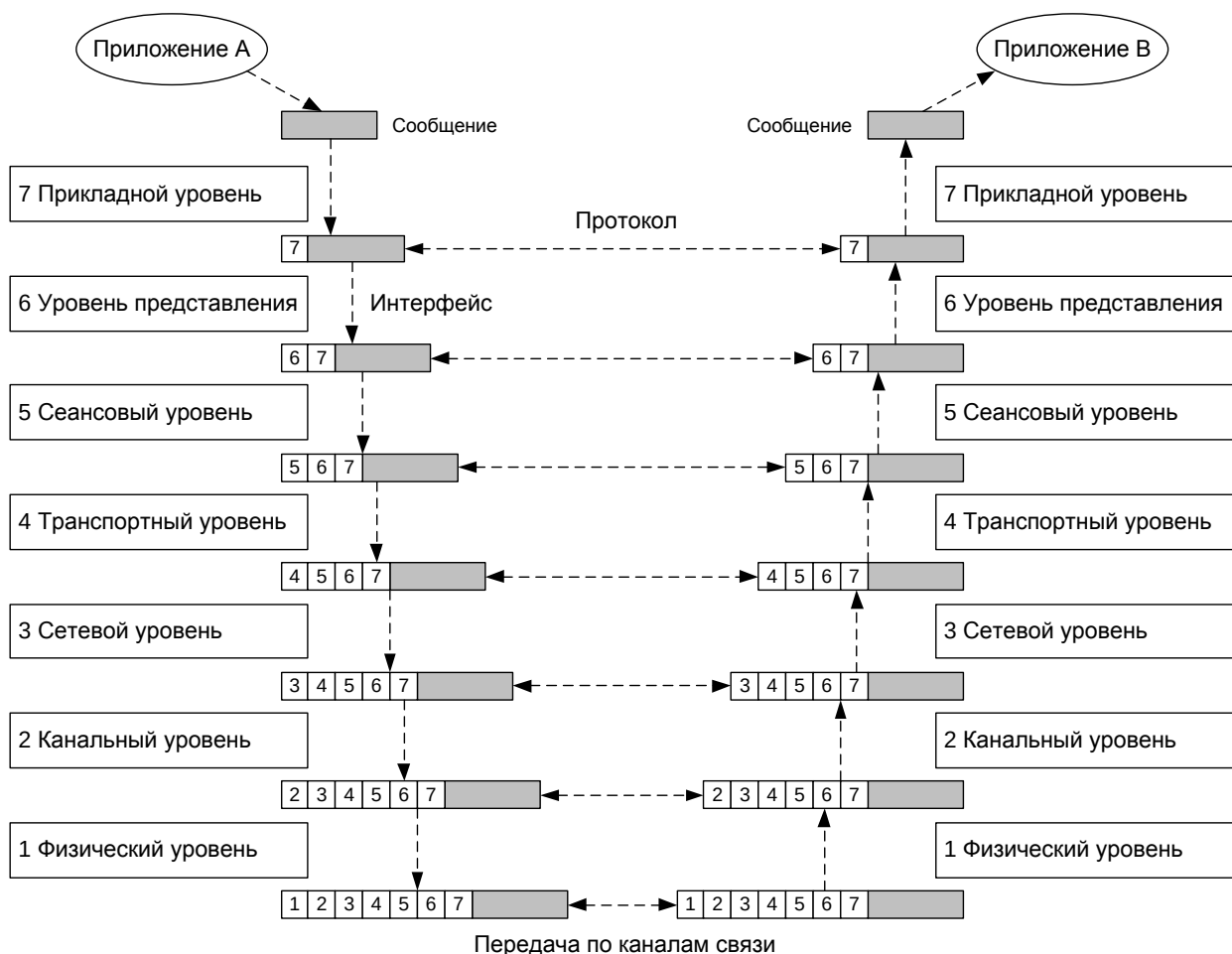


Рис. 7.6. Схема процесса взаимодействия в модели OSI

Чтобы лучше понять принцип работы модели OSI, можно воспользоваться следующим примером. Представим себе президентов двух стран и попробуем смоделировать процесс их общения как иерархическую систему (рис. 7.7).

В этой системе три уровня – уровень президента, уровень секретаря и уровень почты. Секретарь предоставляет президенту услуги по передаче сообщений (интерфейс). Интерфейс может быть реализован, например, посредством телефонной связи или непосредственного общения (директор вызывает секретаря и передает ему все необходимую информацию). В системе существуют три протокола – между президентами, между секретарями и почтовый протокол: стиль общения президентов отличается от стиля общения секретарей, а почта работает по своим особым правилам.

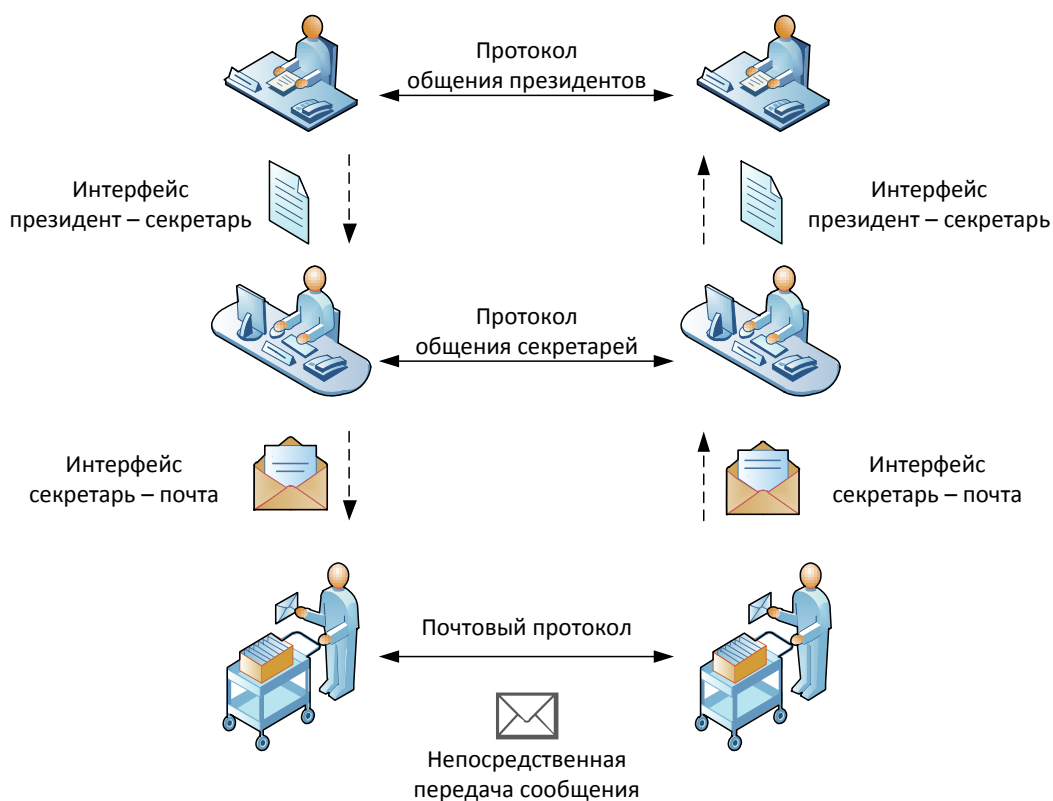


Рис. 7.7. Пример иерархической системы

Процесс передачи сообщения в такой системе происходит следующим образом. Одному президенту требуется передать сообщение другому президенту. Для этого он вызывает своего секретаря и передает ему сообщение, допустим, на листе бумаги (работает интерфейс президент – секретарь). Секретарь переводит сообщение на иностранный язык, вкладывает сообщение в конверт, пишет на нем адрес и лицо, кому предназначено сообщение (добавляет свой заголовок), и отправляет его по почте (передача сообщения по каналам связи). Секретарь другого президента, получив письмо, читает информацию на конверте (считывает свой заголовок) и отправляет сообщение президенту (работает интерфейс президент-секретарь).

7.2.2. Функции уровней модели OSI

Физический уровень

Физический уровень должен передать по линии связи данные, приходящие от канального уровня. Данные передаются в виде потока бит.

Функции физического уровня:

- 1) представить последовательность бит данных в виде электрических сигналов – описать форму сигналов, их величину и продолжительность;
- 2) передать получившиеся электрические сигналы по определенным каналам связи – описать характеристики используемых кабелей, разъемов, беспроводных сред;

3) принять электрические сигналы и декодировать их в исходную последовательность бит.

Перечисленные функции реализуются с помощью аппаратных средств узлов сети – микросхем, проводников, транзисторов, конденсаторов и др.

Канальный уровень

Канальный уровень предназначен для передачи данных сетевого уровня в рамках простой сети с определенной топологией. Данные пересылаются уже не битами, а наборами бит, которые называются *кадры* (frame). В кадр может входить от нескольких десятков до нескольких тысяч байт.

Функции канального уровня:

1) проверка доступности среды передачи – в случае, если несколько сетевых узлов используют один канал связи, для предотвращения конфликтов требуются правила, определяющие порядок доступа узлов к среде;

2) адресация в простых сетях с определенной топологией – чтобы кадр дошел до нужного узла сети, в нем должна быть информация о том, кому он предназначен. Адресация в сложных составных сетях с разными топологиями возлагается на сетевой уровень;

3) обнаружение и коррекция ошибок – для решения этой задачи в кадр добавляется служебная информация, называемая *контрольной суммой*. Отметим, что проблема надежной передачи может решаться на всех уровнях, от физического до прикладного.

Канальный уровень реализуется совместно аппаратными средствами и программными компонентами (драйверами). Устройства, работающие на канальном уровне, – сетевые карты, повторители, концентраторы, мосты, коммутаторы.

Сетевой уровень

Сетевой уровень используется для объединения простых сетей в единую составную сеть. Простые сети в этом случае называются *подсетями* (subnet) и могут иметь разные топологии, типы сред передачи, технологии реализации. На сетевом уровне данные, приходящие от транспортного уровня, пересылаются в виде *пакетов* (packet).

Функции сетевого уровня:

1) *маршрутизация* (routing) – выбор оптимального пути передачи сообщений. Термин «оптимальность» подразумевает определенный *критерий оптимальности*. В данном случае критериями могут быть время передачи, надежность и безопасность (см. параграф 7.1.3 «Основные характеристики сети»);

2) *адресация* в составных сетях – способ адресации должен включать адрес подсети и адрес узла в данной подсети, в отличие от канального уровня, где может быть только адрес узла.

Сетевой уровень реализуется на узлах сети с помощью программных компонентов операционных систем.

Подсети объединяются в единую сеть при помощи специальных устройств – *маршрутизаторов* (router). Маршрутизаторы решают задачу выбора оптимального пути на основании информации о текущем состоянии сети. Пример построения составной сети приведен на рис. 7.8.

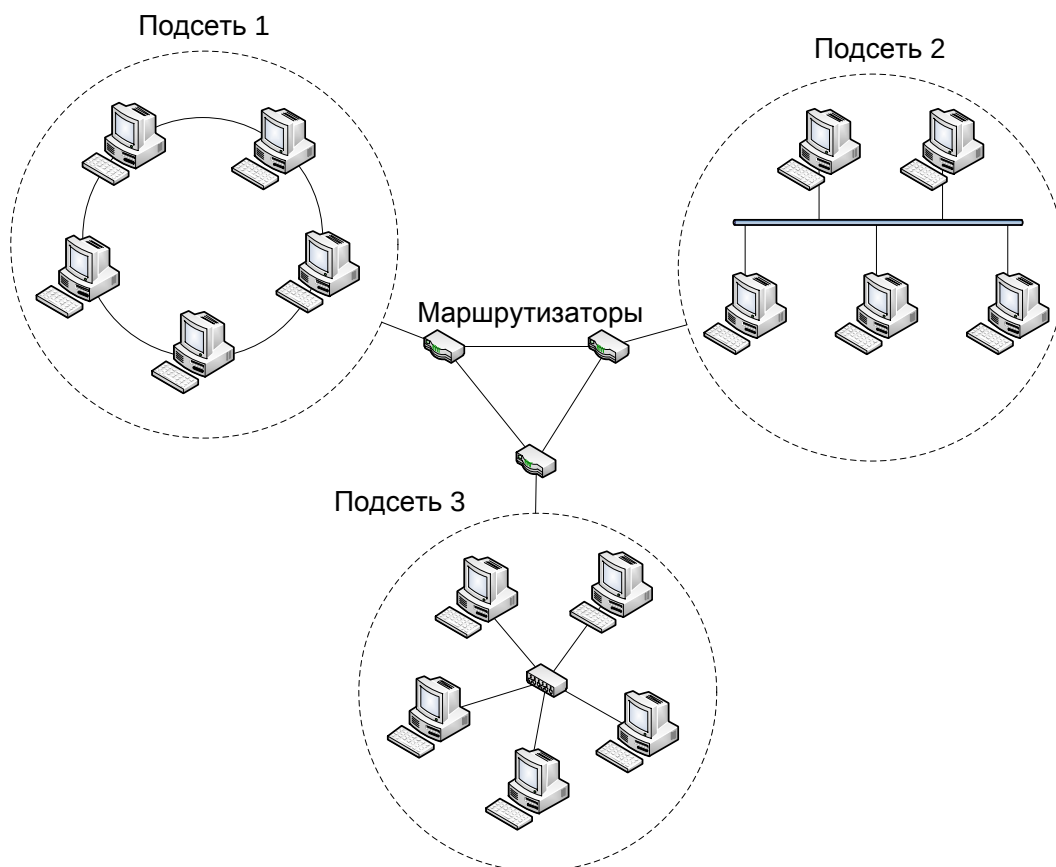


Рис. 7.8. Пример составной сети

Транспортный уровень

Транспортный уровень обеспечивает передачу данных прикладного уровня с необходимой степенью надежности и с учетом требований сетевого уровня к размеру пакетов.

Функции транспортного уровня:

1) передача данных верхних уровней с той степенью надежности, которая им требуется – существуют два основных уровня надежности: *гарантированная доставка* и *негарантированная* (рассмотрены ниже);

2) разбивка сообщений верхних уровней на части – размеры пакетов и кадров обычно не превышают нескольких килобайт; в то же время сообщения прикладного уровня могут быть намного больше. Для решения проблемы несоответствия размеров сообщения разбиваются на части, которые в

случае гарантированной доставки называются *сегменты* (*segment*), а в случае негарантированной – *дейтаграммы* (*datagram*);

3) сборка переданных сегментов или дейтаграмм в исходное сообщение – части сообщения могут приходить получателю не в том порядке, в каком они были отправлены, так как состояние сети меняется и маршрутизаторы могут выбирать для пакетов разные пути. Чтобы на узле-приемнике собрать сегменты в исходное сообщение, каждому сегменту (дейтаграмме) присваиваются идентификатор сообщения и номер сегмента в этом сообщении. На основе этой информации исходное сообщение собирается из отдельных сегментов (дейтаграмм).

В случае *гарантированной доставки* между отправителем и получателем устанавливается *соединение* (*виртуальный канал*), по которому они могут обмениваться информацией о ходе доставки сообщения. Сообщение на транспортном уровне разбивается на сегменты, которые передаются на нижележащий уровень. Каждый сегмент на сетевом уровне помещается в пакет и отправляется по сети с помощью канального и физического уровней. В процессе передачи получатель обязан на каждый полученный сегмент отправить подтверждение (положительную квитанцию) о доставке. В случае отсутствия подтверждения или отрицательной квитанции отправитель заново посылает тот же сегмент.

При *негарантированной доставке* соединение не устанавливается, сообщение разбивается на дейтаграммы, получение которых приемником не сопровождается отправлением подтверждающих квитанций.

Таким образом, гарантированная доставка обеспечивает надежную передачу сообщений, но является более медленной за счет обмена служебной информацией (установление соединения и квитанции). Напротив, негарантированная доставка является быстрой, но ненадежной.

Функции транспортного уровня, как и сетевого, реализуются программными компонентами операционных систем.

Сеансовый уровень

Сеансовый уровень позволяет организовать сеанс связи между узлами сети. В рамках сеанса можно управлять последовательностью передачи сообщений и в случае возникновения ошибки после её устранения продолжать передачу с прерванного места.

Уровень представления

Уровень представления занимается преобразованием данных из одного формата в другой для обеспечения совместимости узлов, работающих с разными формами представления, например преобразование между различными кодировками символов. К функциям этого уровня относится также шифрование и дешифрование передаваемых данных.

Прикладной уровень

Прикладной уровень отвечает за непосредственную реализацию основных целей создания компьютерной сети (см. параграф 7.1.1 «Основные понятия»), т. е. за предоставление пользователю доступа к ресурсам и услугам сети. На прикладном уровне работает самое большое число протоколов, что объясняется большим спектром ресурсов и услуг.

Перечислим некоторые важнейшие функции прикладного уровня:

- 1) передача файлов;
- 2) доступ к сетевым устройствам (принтерам, сканерам, модемам, ...);
- 3) доступ к веб-сайтам;
- 4) электронная почта;
- 5) телеконференции;
- 6) IP-телефония;
- 7) организация чатов и т. д.

Единица данных прикладного уровня называется *сообщением* (message).

В табл. 7.2 сведены основные функции, устройства и единицы данных уровней модели OSI.

Таблица 7.2. Функции, устройства и единицы данных уровней модели OSI

Уровень	Функции	Устройства	Единица данных
Прикладной	– передача файлов; – доступ к сетевым устройствам; – электронная почта; – ...		– сообщение (message)
Транспортный	– надежная передача данных; – разбиение и сборка сообщений		– сегмент (segment) – дейтаграмма (datagram)
Сетевой	– маршрутизация; – адресация	– маршрутизатор (router); – шлюз (gateway)	– пакет (packet)
Канальный	– доступ к среде передачи; – адресация; – обнаружение и коррекция ошибок	– мост (bridge) – коммутатор (switch)	– кадр (frame)
Физический	– кодирование и декодирование бит в сигналы; – характеристики каналов связи	– повторитель (repeater) – концентратор (hub)	– бит (bit)

7.3. Линии связи

7.3.1. Классификация линий связи

Рассмотрим два способа классификации линий связи – по типу используемой среды передачи данных и по виду передаваемых сигналов.

Классификация по типу используемой среды передачи данных (рис. 7.9).

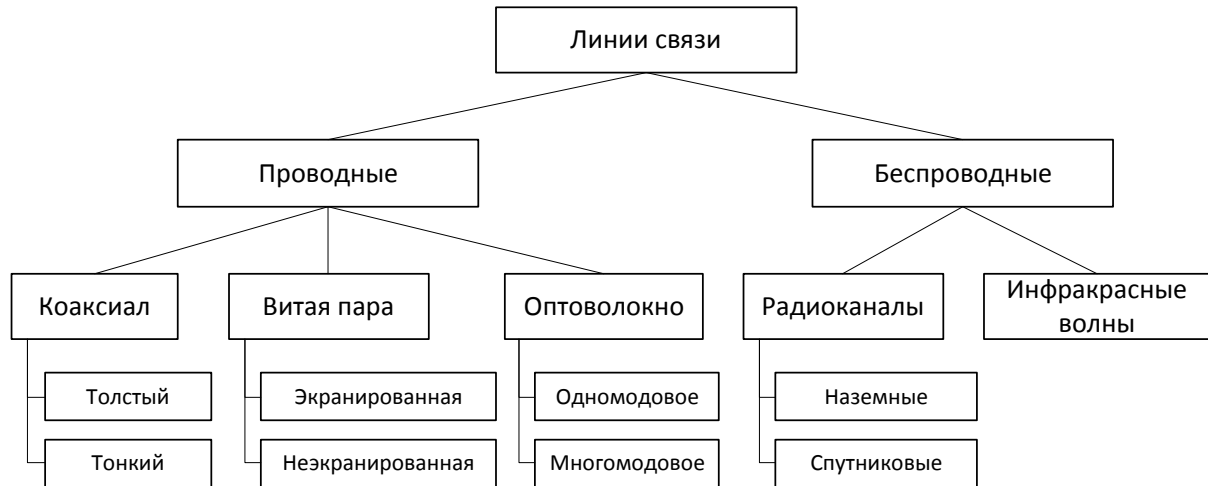


Рис. 7.9. Классификация по типу используемой среды передачи данных

Информация в компьютерных сетях передается с применением *электромагнитных волн*. Электрический ток, свет, радиосигналы, инфракрасное излучение – все это виды электромагнитных волн. Они могут распространяться по проводам или без проводов, отсюда две основные группы линий связи – проводные и беспроводные.

Классификация по виду передаваемых сигналов (рис. 7.10).

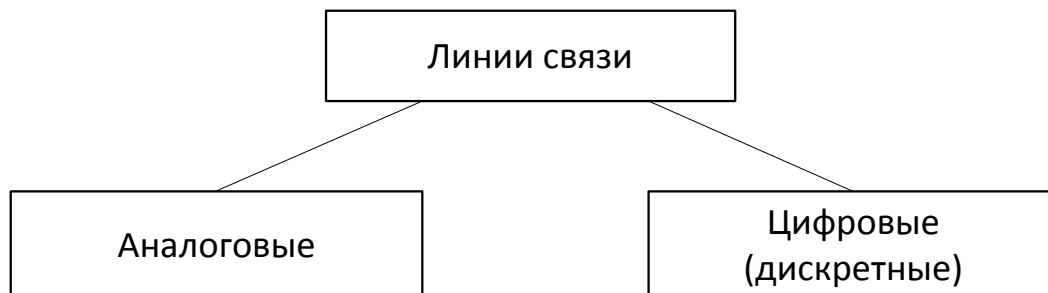


Рис. 7.10. Классификация по виду передаваемых сигналов

Аналоговые и дискретные сигналы рассматривались в параграфе 3.1.

7.3.2. Коаксиальный кабель

Название «*коаксиальный*» (coaxial) произошло от латинских слов со – «совместно» и axis – «ось», что означает два проводника с общей осью (рис. 7.11).

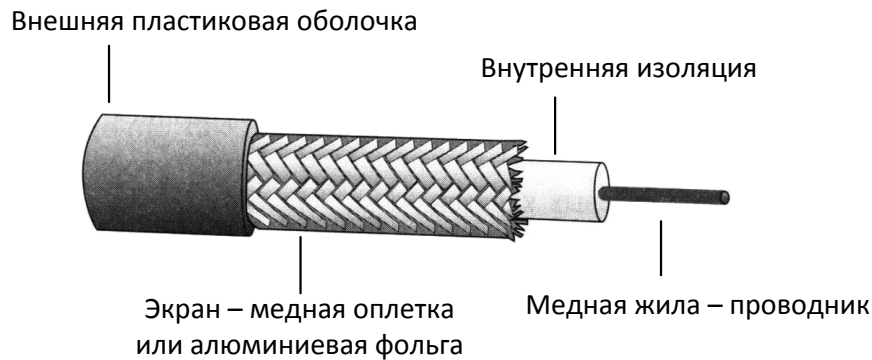


Рис. 7.11. Структура коаксиального кабеля

Одним из этих проводников является *медная жила*, находящаяся в центре кабеля; именно по ней проходят информационные сигналы. Другой проводник – так называемый *экран* (shield), который представляет собой оплетку из тонких медных проводников или алюминиевую фольгу. Назначение экрана – защита информационных сигналов от внешних электромагнитных помех. Чтобы проводники не замыкались, между ними прокладывается слой внутренней пластиковой изоляции. Вся эта конструкция помещается в прочную пластиковую оболочку, которая предотвращает механическое повреждение кабеля (например, при сгибе или кручении), а также изолирует экран снаружи.

Некоторые виды коаксиального кабеля:

- RG-8 – «толстый» коаксиальный кабель (thick coaxial cable) для сетей Ethernet 10Base-5, диаметром примерно 9,5 мм. Максимальная длина сегмента¹ – до 500 м, сопротивление кабеля – 50 Ом. Сложен в установке (плохо гнется);
- RG-58 – «тонкий» коаксиальный кабель (thin coaxial cable) для сетей Ethernet 10Base-2, диаметром около 5 мм. Максимальная длина сегмента – до 185 м, сопротивление кабеля – 50 Ом. Установка проще, чем толстого коаксиала;
- RG-6, RG-59 – применяются в кабельном телевидении, сопротивление 75 Ом.

В настоящее время компьютерные сети на основе коаксиала (Ethernet 10Base-5 и 10Base-2) практически не применяются; в то же время коаксиаль-

¹ Сегмент – участок сети без повторителей – устройств, усиливающих сигнал.

ный кабель широко используется в кабельном телевидении и спутниковых системах.

7.3.3. Витая пара

В кабеле *витая пара* (*twisted pair*) для передачи данных используется пара медных проводников, перевитых между собой. Обычно в один кабель помещают четыре такие пары. Перевивают проводники для уменьшения внешних и перекрестных¹ помех.

Существуют два основных вида витой пары:

- *экранированная* (*shielded twisted pair, STP*) – у каждой пары имеется собственный экран; применяется этот вид редко, в тех случаях, когда кабелю требуется дополнительная защита от помех;
- *неэкранированная* (*unshielded twisted pair, UTP*) – применяется в большинстве случаев (рис. 7.12).

Оба вида кабеля могут обладать общим внешним экраном; в этом случае используется аббревиатура S/STP (Screened STP) или S/UTP (Screened UTP).

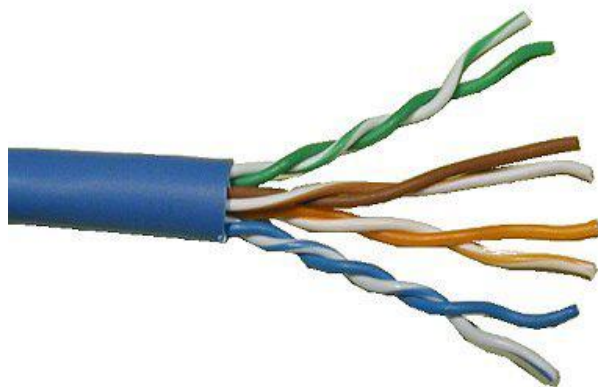


Рис. 7.12. Кабель UTP

Существуют несколько категорий неэкранированной витой пары. Они описаны в стандарте EIA/TIA 568² и обозначаются от CAT1 до CAT7 (табл. 7.3). Чем выше категория кабеля, тем шире у него полоса пропускания³ и больше количество витков на единицу длины.

¹ Перекрестные помехи – помехи в паре проводников, возникающие в одном проводнике от создаваемого другим проводником электромагнитного поля и наоборот.

² EIA – Electronics Industries Alliance – Альянс отраслей электронной промышленности;
TIA – Telecommunication Industry Association – Ассоциация телекоммуникационной промышленности.

³ Полоса пропускания – диапазон частот, которые линия связи передает без искажений. Чем шире полоса пропускания, тем больше теоретически достижимая пропускная способность линии связи.

Таблица 7.3. Категории неэкранированной витой пары UTP

Категория	Полоса пропускания, МГц	Кол-во пар	Сетевая технология	Примечание
CAT-1	0,1	1	–	Телефонный кабель
CAT-2	1	2	Token Ring, ARCNet	Может встречаться в телефонных сетях
CAT-3	16	4	10BASE-T, 100BASE-T4, Token Ring	Может встречаться в телефонных сетях
CAT-4	20	4	10BASE-T, 100BASE-T4, Token Ring	В настоящее время не применяется
CAT-5	100	4	100BASE-TX	
CAT-5e	100	4	100BASE-TX, 1000BASE-T	Усовершенствованная CAT5
CAT-6	250	4	100BASE-TX, 1000BASE-T, 10GBASE-T	
CAT-6A (Augmented)	500	4	100BASE-TX, 1000BASE-T, 10GBASE-T	Дополненная CAT6
CAT-7	600–700	4	Ethernet 10G, 40G, 100G	Основа будущих сетей

В настоящее время наиболее распространенными видами витой пары являются кабели категорий 5е, 6 и 6А, причем эти кабели могут быть как без общего экрана, так и с экраном (т. е. S/UTP).

Для соединения кабелей с оборудованием используются вилки и розетки RJ-45¹, представляющие собой 8-контактные разъемы, похожие на обычные телефонные розетки RJ-11, только шире. При использовании кабеля UTP компьютер подключается к розетке через специальный короткий кабель (1–1,5 м), называемый *патч-корд* (patch cord).

¹ RJ-45 (Registered jack – зарегистрированный разъем) – это ошибочное, но широко распространенное название разъема 8P8C (8 Position 8 Contact).

7.3.4. Оптоволоконный кабель

В *оптоволоконном кабеле* (fiber-optic cable) информация передается при помощи световых сигналов. Средой передачи служит специальное стеклянное волокно. Структура оптоволоконного кабеля показана на рис. 7.13.

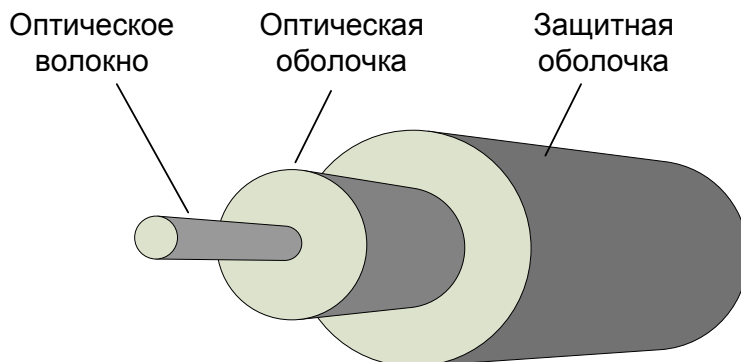


Рис. 7.13. Структура оптоволоконного кабеля

Информационные световые сигналы проходят по оптическому волокну, выполненному из стекла или пластика. В оптоволоконном кабеле может быть до нескольких сот волокон, но не менее двух — для передачи данных одновременно в противоположных направлениях.

Большинство волоконно-оптических систем используют инфракрасный свет¹ с длиной волны между 800 и 1600 нм (нм — *нанометр*, 1 нм = 10^{-9} м), поскольку стекло является более прозрачным для инфракрасного излучения, чем для видимого света.

Оптическая оболочка также изготавливается либо из стекла, либо из пластика и имеет коэффициент преломления, немного меньший, чем у оптического волокна (например, волокно — 1,47, оболочка — 1,46). Такая разность коэффициентов преломления обеспечивает эффект полного отражения света, проходящего по волокну, от оптической оболочки так, что свет не рассеивается.

Пластиковая защитная оболочка предназначена для защиты от механических воздействий на кабель (изгиб, кручение).

Существуют стандартные диаметры стекловолокна и оптической оболочки, приведенные в табл. 7.4. Размер указывается в *микронах*² (обозначается *мкм*, 1 мкм = 10^{-6} м).

¹ *Инфракрасным светом* называют световые волны с длиной волны больше, чем у видимого света. Видимый свет имеет длину волны от 380 нм (темно-фиолетовый) до 750 нм (темно-красный).

² Для сравнения: диаметр человеческого волоса примерно 100 мкм.

Таблица 7.4. Стандартные диаметры стекловолокна и оптической оболочки

Диаметр оптического волокна, мкм	Диаметр оптической оболочки, мкм
8	125
50	125
62,5	125
100	140

Диаметры оболочек кабеля обозначаются следующим образом: 8/125.

Различают два основных вида оптоволоконных кабелей: многомодовые и одномодовые (рис. 7.14). *Мода* – это вид траектории, по которой движется свет.

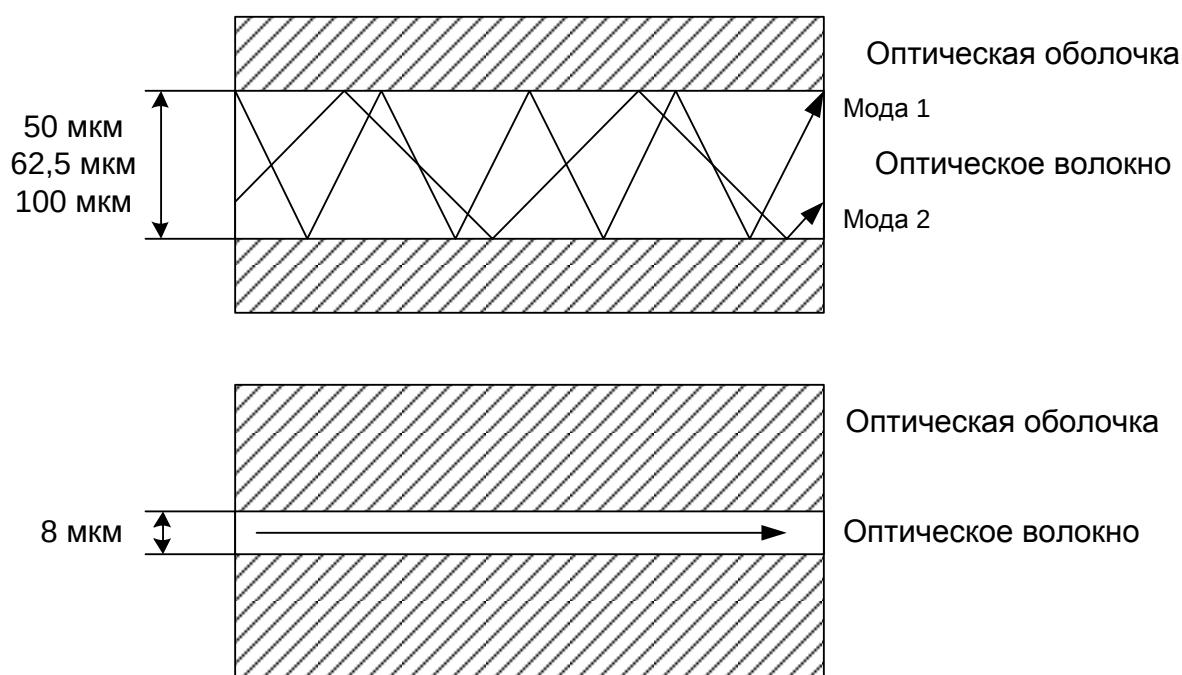


Рис. 7.14. Многомодовые и одномодовые оптоволоконные кабели

В *многомодовых* кабелях диаметр оптоволокна настолько велик, что свет распространяется по нескольким траекториям (модам), а это приводит к искажениям и затуханию передаваемого сигнала. *Одномодовые* кабели имеют очень малый диаметр оптоволокна, поэтому свет проходит только по одному пути, за счет чего устраняются искажения сигнала, а затухание существенно уменьшается.

Таким образом, одномодовые кабели вследствие малого диаметра волокна сложнее изготавливать и устанавливать, но максимальная длина сегмента и пропускная способность у них выше, чем у многомодовых кабелей.

Сравнительная характеристика проводных линий связи приведена в табл. 7.5.

7.4. Технология Ethernet

Самая распространенная на сегодняшний день технология локальных сетей называется *Ethernet*. В соответствии с моделью OSI Ethernet отвечает за физический и канальный уровни.

7.4.1. История Ethernet

Технология Ethernet появилась в 1973 году – первый документ датируется 22 мая 1973 года. Это докладная записка, которую Роберт Меткалф (Robert Metcalfe) подал главе исследовательского центра Xerox в калифорнийском городке Пало-Альто (Xerox Palo Alto Research Center).

Название *Ethernet* произошло от двух слов: *ether* (эфир) и *net* (сеть), т. е. дословный перевод – «эфирная сеть». Дело в том, что эта технология основана на разработках в области компьютерных радиосетей (сеть ALO-HAnet).

В 1975 году был получен патент, а в 1980 году усилиями трех фирм – DEC, Intel и Xerox – был создан первый стандарт Ethernet, который получил название DIX (по первым буквам названий фирм-разработчиков). В 1983 году международная ассоциация IEEE¹ публикует стандарт IEEE 802.3, разработанный на основе Ethernet DIX, в котором устанавливалась пропускная способность 10 Мбит/с.

80-е годы прошли под знаком соперничества Ethernet с другими распространенными технологиями локальных сетей: Token Ring (IBM) и ARCNET. В 1995 году появляется стандарт IEEE 802.3u Fast Ethernet («быстрый Ethernet») со скоростью передачи 100 Мбит/с, а в 1998 году – стандарт IEEE 802.3z Gigabit Ethernet (1 Гбит/с). Эти новые стандарты, а также простота, надежность и низкая стоимость сетей позволили Ethernet победить конкурентов и занять лидирующее положение на рынке LAN.

В 2003 году опубликован стандарт IEEE 802.3ae с пропускной способностью 10 Гбит/с, а в 2010 году – стандарт IEEE 802.3ba с пропускными способностями 40 и 100 Гбит/с.

7.4.2. Формат кадра

Чтобы сетевые узлы могли обмениваться информацией, должны быть согласованы форматы единиц передачи данных. Ethernet отвечает за физический и канальный уровни, следовательно, в стандартах Ethernet должны быть определены способы кодирования бит и представления кадров. Рассмотрим формат кадров Ethernet. Считаем, что кадр передается от *узла-источника кадра* (source) к *узлу-приемнику* (destination).

¹ IEEE (Institute of Electrical and Electronics Engineers) – Институт инженеров по электронике и электротехнике, некоммерческая международная ассоциация, одной из функций которой является разработка стандартов в области информатики.

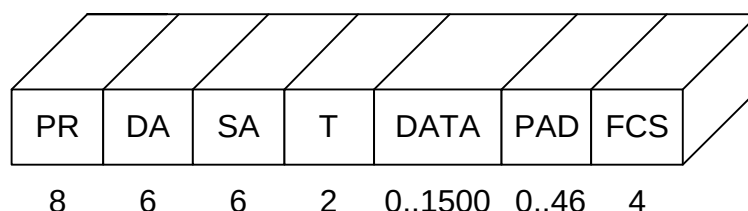


Рис. 7.15. Формат кадров Ethernet

Кадр состоит из набора полей, размер которых указан на рис. 7.15.

- 1) PR – *Preamble* (преамбула): последовательность восьми байтов вида 10101010, которая служит для синхронизации приемника и источника.
- 2) DA – *Destination Address* (адрес приемника).
- 3) SA – *Source Address* (адрес источника).

На канальном уровне для идентификации узлов сети используются MAC-адреса¹, состоящие из 6 байт. MAC-адрес записывается в виде последовательности шестнадцатеричных цифр, разделенных дефисами, например:

01-98-E5-67-DA-F0.

MAC-адреса являются атрибутом сетевой карты, поэтому называются ещё *физическими* (physical address), или *аппаратными*, адресами (hardware address).

4) T – *Type* (тип): указывает протокол верхнего (сетевого) уровня, которому следует передать данный кадр (см. рис. 7.6).

5) DATA – данные: полезная информация, которую нужно передать. Чаще всего это пакет, пришедший от сетевого уровня. Размер данных может быть от 0 до 1500 байт.

6) PAD – *Padding* (заполнение): если данных в кадре менее 46 байт, используется поле *Padding* для дополнения до 46 байт. Заполнение требуется для обеспечения минимального размера кадра в 72 байта. Если кадр будет меньшего размера, существует вероятность его неправильной передачи.

7) FCS – *Frame Check Sequence* (контрольная сумма): источник перед отправкой кадра вычисляет значение определенной функции² от передаваемых данных и записывает это значение (которое называется *контрольная сумма*) в поле FCS. Приемник, получив кадр, вычисляет значение той же самой функции от полученных данных и сравнивает его со значением в поле FCS. Если они равны, значит кадр передан без искажений, иначе делается вывод об ошибке передачи.

¹ MAC – Media Access Control – управление доступом к среде передачи данных.

² Например, в качестве FCS используется остаток от деления по модулю 2 двоичной строки данных на определенное число. Деление по модулю 2 осуществляется с помощью логической операции XOR и происходит очень быстро.

7.4.3. Fast Ethernet

Рассмотрим один из стандартов Ethernet – Fast Ethernet с пропускной способностью 100 Мбит/с. В этом стандарте используется топология звезда и существует несколько спецификаций¹, которые обозначаются по формуле, общей для всех стандартов Ethernet:

<Пропускная способность> BASE – <Тип среды передачи>.

Например, 10BASE-T обозначает спецификацию Ethernet с пропускной способностью 10 Мбит/с и витой парой в качестве среды передачи данных (Т – Twisted). Слово BASE (*базовый*) обозначает, что передача данных ведется на одной, базовой, частоте, в отличие от сетей, где информация по одному кабелю может передаваться с разными частотами.

Спецификации Fast Ethernet и их характеристики приведены в табл. 7.6.

Таблица 7.6. Спецификации Fast Ethernet

Спецификация	Среда передачи	Длина сегмента	Примечание
100BASE-TX	2 витые пары UTP категории 5 или выше	100 м	Самая распространенная в LAN спецификация
100BASE-T4	4 витые пары UTP категории 3 или выше	100 м	Не используется
100BASE-T2	2 витые пары UTP категории 3 или выше	100 м	Не используется
100BASE-FX	многомодовое оптоволокно, длина волны – 1300 нм	2 км	
100BASE-SX	многомодовое оптоволокно, длина волны – 850 нм	300 м	Дешевый вариант оптоволоконной сети
100BASE-LX	двухжильное одномодовое оптоволокно	10 км	
100BASE-BX	одножильное одномодовое оптоволокно	40 км	

¹ Спецификация – подробный перечень технических характеристик какого-либо продукта.

7.5. Сетевые устройства

В современных сетях присутствует большое количество оборудования всевозможных типов и производителей. Рассмотрим самые распространенные виды сетевых устройств: сетевые карты, модемы, концентраторы, коммутаторы, маршрутизаторы.

7.5.1. Сетевые карты

Сетевая карта, сетевая плата, сетевой адаптер (Network Interface Card, NIC) – периферийное устройство, позволяющее подключать компьютер к компьютерной сети. Выпускаются в виде отдельных плат (рис. 7.16) или встраиваются в материнскую плату.



Рис. 7.16. Сетевая карта

Сетевая карта под управлением своего драйвера реализует физический и канальный уровни модели OSI и выполняет две основные функции – передачу и прием кадра.

При передаче кадра сетевая карта принимает от вышележащего сетевого уровня модели OSI пакет, оформляет его в кадр соответствующей технологии, например Ethernet (см. рис. 7.15), т. е. вставляет MAC-адреса источника и приемника, вычисляет контрольную сумму. После этого в кабель, подключенный к разъему карты, выдается последовательность электрических сигналов, обозначающих двоичные ноли и единицы.

Прием кадра происходит следующим образом: карта считывает пришедшие по кабелю сигналы, преобразует их в двоичные ноли и единицы, определяет MAC-адрес приемника и сравнивает его со своим. Если MAC-адреса не совпадают, кадр просто отбрасывается. Иначе вычисляется контрольная сумма и сравнивается с полученной в кадре. Если контрольные суммы не совпадают, на сетевой уровень выдается сообщение об ошибке передачи. Если все в порядке, из кадра извлекается поле данных (пакет) и передается на сетевой уровень.

7.5.2. Концентраторы

Концентратор, хаб, повторитель (hub) – сетевое устройство, которое объединяет несколько сегментов сети и повторяет сигналы, полученные из одного сегмента на все другие сегменты. Для подключения каждого сегмента используются разъемы, которые называются *портами*. Портов может быть от 4 до 48 и более.

Концентраторы работают на физическом уровне модели OSI, т. е. обрабатывают электрические сигналы, но не MAC-адреса. При использовании концентраторов возникают проблемы низкой производительности и безопасности – каждый кадр, отправленный в сеть, попадает всем узлам сети, даже тем, которым кадр не предназначен.

В настоящее время концентраторы вытеснены более совершенными устройствами – коммутаторами.

7.5.3. Коммутаторы

Коммутатор, свитч (switch) – сетевое устройство, объединяющее сегменты сети и передающее кадры только в необходимые сегменты (рис. 7.17).

Таким образом, кадр, попавший на один из портов коммутатора, будет передан только на тот порт, к которому присоединен сегмент сети с находящимся в нем узлом-приемником данного кадра. В этом заключается основное отличие коммутаторов от концентраторов.



Рис. 7.17. 16-портовый коммутатор

Коммутатор работает на физическом и канальном уровнях модели OSI и умеет считывать MAC-адреса кадров. Коммутатор не нужно отдельно настраивать и сообщать ему, в каком сегменте какой компьютер находится, обнаружение узлов происходит автоматически.

Каким образом коммутатор узнает о расположении компьютеров сети? В нем хранится таблица соответствия «порт – MAC-адрес» и работает «алгоритм прозрачного моста»: коммутатор отслеживает прохождение всех кадров через свои порты и считывает MAC-адреса источников кадров. Если на какой-то порт пришел кадр с определенным MAC-адресом, значит, этот MAC-адрес записывается в таблице как принадлежащий данному порту.

7.5.4. Маршрутизаторы

Маршрутизатор (router) – сетевое устройство, объединяющее подсети составной сети и решающее задачу маршрутизации на сетевом уровне модели OSI (см. параграф 7.2.2 «Функции уровней модели OSI»).

У маршрутизатора также есть таблица соответствия «порт – адрес», которая называется *таблицей маршрутизации*, но используются не MAC-адреса, а IP-адреса.

IP-адрес¹ представляет собой 4-байтовое двоичное число, которое обычно записывается в виде 4 десятичных чисел, разделенных точками, например:

в десятичном виде: 192.168.0.1,

в двоичном виде: 11000000.10101000.00000000.00000001.

В отличие от MAC-адреса у IP-адреса можно выделить два уровня адресации – адрес подсети и адрес узла в этой подсети. Для этого в паре с IP-адресом часто указывают *маску подсети* – также 4-байтовое двоичное число, в определенном количестве старших разрядах которого (слева) находятся всегда единицы, а в остальных младших разрядах (справа) находятся ноли. Единицы в двоичной записи маски означают, что соответствующий разряд в IP-адресе должен интерпретироваться как адрес подсети, а остальные разряды – как адрес узла в этой подсети.

Пример. Даны IP-адрес 192.168.0.1 и маска подсети 255.255.255.0. Требуется найти адрес подсети и адрес узла в подсети.

Запишем IP-адрес и маску подсети в двоичном виде:

IP-адрес: 11000000.10101000.00000000.00000001,

маска подсети: 11111111.11111111.11111111.00000000.

Единицы в маске содержатся в первых 24 битах, следовательно, в IP-адресе соответствующие биты будут отвечать за адрес подсети (в остальные биты ставим ноли):

адрес подсети:

двоичный 11000000.10101000.00000000.00000000,

десятичный 192.168.0.0.

Ноли в маске находятся в младших 8 битах, соответствующие биты в IP-адресе обозначают адрес узла:

¹ IP – Internet Protocol (межсетевой протокол) – один из основных протоколов стека TCP/IP, на котором основана работа Интернета.

адрес узла:

двоичный	00000000.00000000.00000000.00000001,
десятичный	0.0.0.1.

Таблица маршрутизации может заполняться вручную администратором сети или автоматически самим маршрутизатором, который при этом не следит за проходящим трафиком, а рассылает запросы другим маршрутизаторам об имеющейся у них информации о конфигурации сети.

7.6. Стек протоколов TCP/IP

Стек TCP/IP – это набор иерархически упорядоченных сетевых протоколов. Название стек получил по двум важнейшим протоколам – TCP (Transmission Control Protocol) и IP (Internet Protocol). Помимо них в стек входят ещё несколько десятков различных протоколов. В настоящее время протоколы TCP/IP являются основными для Интернета, а также для большинства корпоративных и локальных сетей.

Стек протоколов TCP/IP обладает двумя важными свойствами:

- *платформонезависимостью*, т. е. возможностью его реализации на самых разных операционных системах и процессорах;
- *открытостью*, т. е. доступностью стандартов, по которым строится стек TCP/IP, любому желающему.

7.6.1. История создания TCP/IP

В 1967 году Агентство по перспективным исследовательским проектам министерства обороны США (ARPA – Advanced Research Projects Agency) инициировало разработку компьютерной сети, которая должна была связать ряд университетов и научно-исследовательских центров, выполнявших заказы Агентства. Проект получил название ARPANET. К 1972 году сеть соединяла 30 узлов.

В рамках проекта ARPANET были разработаны и в 1980–1981 годах опубликованы основные протоколы стека TCP/IP – IP, TCP и UDP. Важным фактором распространения TCP/IP стала реализация этого стека в операционной системе UNIX 4.2 BSD (1983 год).

К концу 80-х годов значительно расширившаяся сеть ARPANET стала называться Интернет (Interconnected networks – связанные сети) и объединила университеты и научные центры США, Канады и Европы.

В 1992 году появился новый сервис Интернет – WWW (World Wide Web – всемирная паутина), основанный на протоколе HTTP. Во многом благодаря WWW Интернет, а с ним и протоколы TCP/IP, получил в 90-е годы бурное развитие.

В начале XXI века стек TCP/IP приобретает ведущую роль в средствах коммуникации не только глобальных, но и локальных сетей.

7.6.2. Структура TCP/IP

В основе TCP/IP лежит не модель OSI, а собственная модель, называемая по-разному: *DoD* (Department of Defense – министерство обороны США), *DARPA* (Defense ARPA – новое название Агентства по перспективным исследовательским проектам) или просто *модель TCP/IP*. В этой модели всего четыре уровня. Соответствие модели OSI модели DoD, а также основным протоколам стека TCP/IP показано на рис. 7.18.

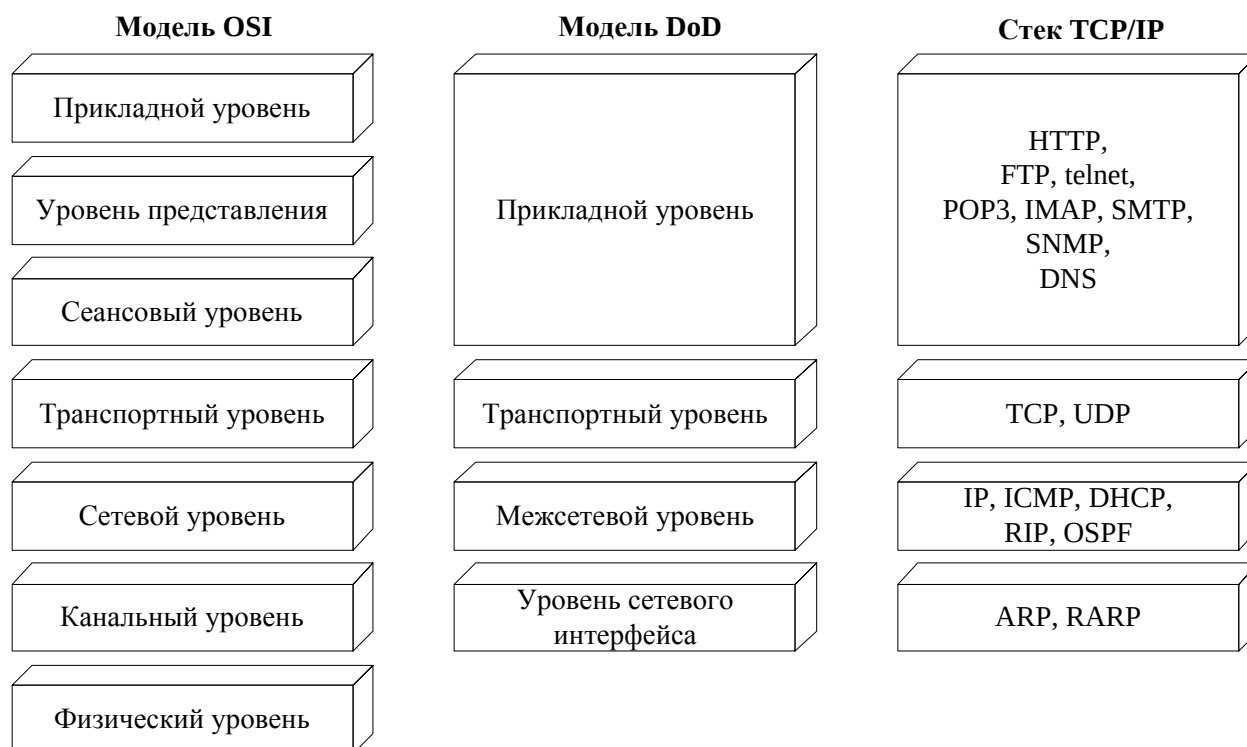


Рис. 7.18. Соответствие протоколов TCP/IP моделям OSI и DoD

Следует заметить, что нижний уровень модели DoD – уровень сетевого интерфейса, строго говоря, не выполняет всех функций канального уровня, а лишь обеспечивает связь (интерфейс) верхних уровней DoD с технологиями сетей, входящих в составную сеть (например, Ethernet, FDDI, ATM).

Все протоколы, входящие в стек TCP/IP, стандартизованы в документах RFC.

7.6.3. Документы RFC

Утвержденные официальные стандарты Интернета и TCP/IP публикуются в виде документов RFC (Request for Comments – рабочее предложение). Стандарты разрабатываются всем сообществом ISOC (Internet Society – Сообщество Интернет, международная общественная организация). Любой член ISOC может представить на рассмотрение документ для его публика-

ции в RFC. Далее документ рассматривается техническими экспертами, группами разработчиков и редактором RFC и проходит в соответствии с RFC 2026 следующие этапы, называемые *уровнями готовности* (maturity levels):

1) *черновик* (Internet Draft) – на этом этапе с документом знакомятся эксперты, вносятся дополнения и изменения;

2) *предложенный стандарт* (Proposed Standard) – документу присваивается номер RFC, эксперты подтвердили жизнеспособность предлагаемых решений, документ считается перспективным, желательно, чтобы он был опробован на практике;

3) *черновой стандарт* (Draft Standard) – документ становится черновым стандартом, если не менее двух независимых разработчиков реализовали и успешно применили предлагаемые спецификации. На этом этапе ещё допускаются незначительные исправления и усовершенствования;

4) *стандарт Интернета* (Internet Standard) – наивысший этап утверждения стандарта: спецификации документа получили широкое распространение и хорошо зарекомендовали себя на практике. Список стандартов Интернета приведен в RFC 5000. Из тысяч RFC только несколько десятков являются документами в статусе «стандарт Интернета».

Кроме стандартов документами RFC могут быть также описания новых сетевых концепций и идей, руководства, результаты экспериментальных исследований, представленных для информации, и т. д. Таким документам RFC может быть присвоен один из следующих статусов:

- *экспериментальный* (Experimental) – документ, содержащий сведения о научных исследованиях и разработках, которые могут заинтересовать членов ISOC;

- *информационный* (Informational) – документ, опубликованный для предоставления информации и не требующий одобрения сообщества ISOC;

- *лучший современный опыт* (Best Current Practice) – документ, предназначенный для передачи опыта конкретных разработок, например реализаций протоколов.

Статус указывается в заголовке документа RFC после слова *Category* (категория). Для документов в статусе стандартов (Proposed Standard, Draft Standard, Internet Standard) указывается название *Standards Track*, так как уровень готовности может меняться.

Номера RFC присваиваются последовательно и никогда не выдаются повторно. Первоначальный вариант RFC никогда не обновляется. Обновленная версия публикуется под новым номером. Устаревший и замененный документ RFC получает статус *исторический* (Historic).

Все существующие на сегодня документы RFC можно посмотреть, например, на сайте <http://www.rfc-editor.org>. В конце 2011 года их насчитывалось около шести с половиной тысяч.

7.6.4. Обзор основных протоколов

Протокол IP (Internet Protocol) – это основной протокол сетевого уровня, отвечающий за адресацию в составных сетях и передачу пакета между сетями. Протокол IP является *дейтаграммным* протоколом, т. е. не гарантирует доставку пакетов до узла назначения. Обеспечением гарантий занимается протокол транспортного уровня TCP.

Протоколы RIP (Routing Information Protocol – протокол маршрутной информации) и *OSPF* (Open Shortest Path First – «первыми открываются кратчайшие маршруты») – протоколы маршрутизации в IP-сетях.

Протокол ICMP (Internet Control Message Protocol – протокол управляющих сообщений в составных сетях) предназначен для обмена информацией об ошибках между маршрутизаторами сети и узлом-источником пакета. С помощью специальных пакетов сообщает о невозможности доставки пакета, о продолжительности сборки пакета из фрагментов, об аномальных величинах параметров, об изменении маршрута пересылки и типа обслуживания, о состоянии системы и т. п.

Протокол ARP (Address Resolution Protocol – протокол преобразования адресов) преобразует IP-адреса в аппаратные MAC-адреса. Обратное преобразование осуществляется с помощью протокола *RARP* (Reverse ARP).

Протокол TCP (Transmission Control Protocol – протокол управления передачей) обеспечивает надежную передачу сообщений между удаленными узлами сети за счет образования логических соединений. TCP позволяет без ошибок доставить сформированный на одном из компьютеров поток байтов на любой другой компьютер, входящий в составную сеть. TCP делит поток байтов на части – *сегменты* – и передает их сетевому уровню. После того как эти сегменты будут доставлены в пункт назначения, протокол TCP снова соберет их в непрерывный поток байтов.

Протокол UDP (User Datagram Protocol – протокол дейтаграмм пользователя) обеспечивает передачу данных дейтаграммным способом.

Далее рассматриваются протоколы прикладного уровня.

Протокол HTTP (HyperText Transfer Protocol – протокол передачи гипертекста) – протокол доставки web-документов, основной протокол службы WWW.

Протокол FTP (File Transfer Protocol – протокол передачи файлов) – протокол для пересылки информации, хранящейся в файлах.

Протоколы POP3 (Post Office Protocol version 3 – протокол почтового офиса), *IMAP* (Internet Message Access Protocol – протокол для доступа к электронной почте) и *SMTP* (Simple Mail Transfer Protocol – простой протокол пересылки почты) – протоколы для доставки входящей электронной почты (POP3 и IMAP) и отправки исходящей (SMTP).

Протоколы Telnet и *SSH* (Secure Shell – безопасная оболочка) – протоколы эмуляции терминала¹, позволяющие пользователю подключаться к

¹ Терминал – это сочетание устройства ввода и устройства вывода, например клавиатура и дисплей.

другим удаленным станциям и работать с ними со своей машины, как если бы она была их удаленным терминалом. Протокол SSH, в отличие от Telnet, обеспечивает безопасное соединение за счет шифрования.

Протокол SNMP (Simple Network Management Protocol – простой протокол управления сетью) предназначен для диагностики работоспособности различных устройств сети.

7.6.5. Утилиты диагностики TCP/IP

В состав операционных систем семейства Windows входит ряд утилит (небольших служебных программ), предназначенных для диагностики функционирования стека TCP/IP (аналогичные утилиты есть и UNIX/Linux-системах).

Для запуска этих утилит следует в меню *Пуск* выбрать пункт *Выполнить*, в появившемся окне набрать *cmd* и нажать *Enter*¹. Появится интерфейс командной строки с приглашением для ввода команд (рис. 7.19).

Информацию о любой утилите можно вывести, набрав в командной строке имя утилиты с ключом «/?», например: *IPconfig /?*

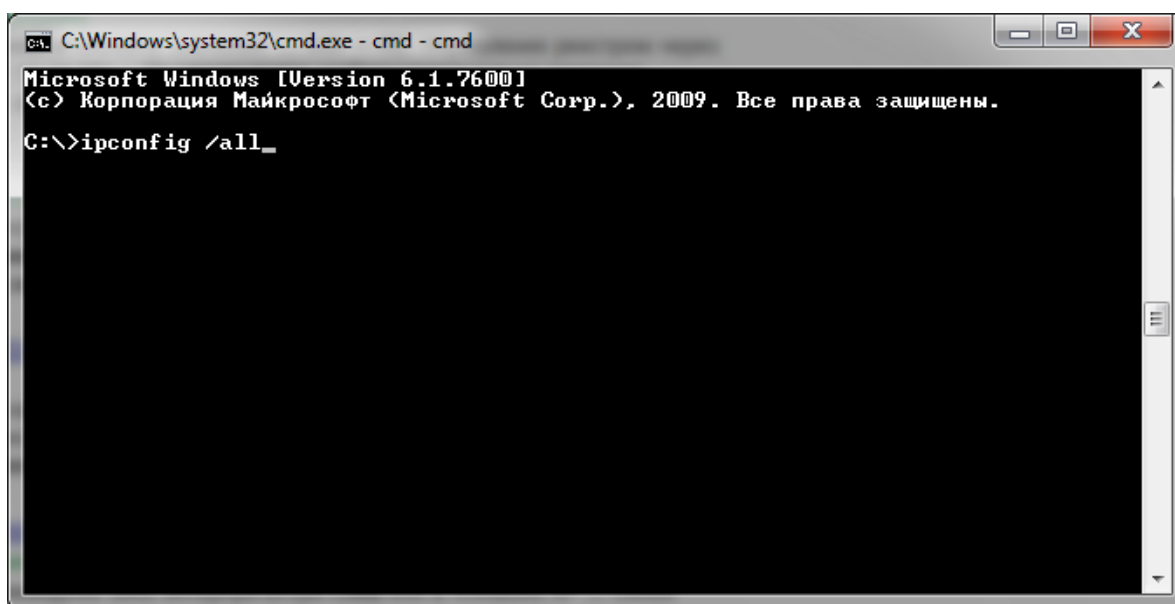


Рис. 7.19. Интерфейс командной строки

Hostname

Это самая простая утилита – она выводит на экран имя компьютера.

IPconfig

Утилита предназначена, во-первых, для вывода информации о конфигурации стека TCP/IP, во-вторых, для выполнения некоторых действий по настройке стека.

¹ В Windows 7 нужно в меню *Пуск* в текстовом окне «Найти программы и файлы» набрать *cmd* и нажать *Enter*.

При вводе названия утилиты в командной строке без параметров на экране отобразится информация об основных настройках TCP/IP, например, IP-адрес (IP Address) и маска подсети (Subnet Mask).

Использование утилиты с ключом **/all** позволит узнать полную информацию о настройке стека TCP/IP на данном компьютере. Следует отметить, что при наличии нескольких сетевых карт выводятся данные по каждой карте отдельно, например можно узнать MAC-адрес сетевой карты (физический адрес).

Ping

Основная цель этой утилиты – выяснение возможности установления соединения с удаленным узлом.

Принцип работы: утилита отправляет на удаленный узел несколько пакетов (число пакетов определяется ключом **-n**, по умолчанию четыре) по протоколу ICMP. Такие пакеты называются *эхо-пакетами*, т. е. требуют ответа. Если удаленный узел доступен, он отвечает на каждый эхо-пакет своим пакетом, а утилита измеряет интервал между отправкой эхо-пакета и приходом ответа.

Нужно отметить, что отсутствие ответа может быть связано не с физической недоступностью удаленного компьютера, а с тем, что на нем установлена специальная программа для обеспечения сетевой безопасности – *брандмауэр* (другое название *firewall*), запрещающая отправку ответов на эхо-пакеты.

Tracert

Название утилиты произошло от Trace Route – отслеживание маршрута. Утилита позволяет решить следующие задачи:

- проследить путь прохождения пакета от данного компьютера до удаленного узла (отображаются промежуточные узлы-маршрутизаторы);
- выявить участки задержки пакетов;
- выявить места потери пакетов.

Принцип работы: утилита отправляет эхо-пакеты на заданный удаленный узел. Отличие между эхо-пакетами заключается в параметре, который называется *время жизни* (TTL – Time To Live). Этот параметр обозначает количество маршрутизаторов (процесс перехода пакета через маршрутизатор называется *hop* – прыжок), которое может пройти пакет, прежде чем попадет на заданный узел. Каждый маршрутизатор уменьшает время жизни на единицу. Если на каком-то маршрутизаторе TTL станет равным нулю, тот отбрасывает пакет и отправляет служебное сообщение на узел-источник.

Первый эхо-пакет посылается с временем жизни, равным единице. Первый маршрутизатор отбрасывает эхо-пакет и отправляет служебное сообщение, в котором содержится информация об имени и адресе маршрутизатора. Следующий эхо-пакет имеет TTL = 2 и отбрасывается уже на втором маршрутизаторе. Таким образом, эхо-пакеты отправляются с увеличением

времени жизни на единицу, пока не придет ответ от заданного удаленного узла или время ожидания не будет превышено.

Arp

Эта утилита работает с протоколами преобразования IP-адресов в MAC-адреса и обратно ARP и RARP. С её помощью можно выводить на экран таблицу соответствия IP-адресов и MAC-адресов (ARP-кэш), добавлять и удалять записи в ней.

Основные ключи:

- **/a** – отображение таблицы ARP или, если указан IP-адрес, запись только для этого адреса;
- **/s** – добавление записи в таблицу;
- **/d** – удаление записи из таблицы.

Контрольные вопросы и задания

1. Перечислите цели объединения компьютеров в сеть и придумайте примеры, иллюстрирующие каждую цель.
2. Составьте таблицу, в которой перечислите достоинства и недостатки рассмотренных в пособии топологий.
3. Сравните известные топологии с точки зрения наличия избыточности.
4. Каким образом скорость передачи данных влияет на время реакции сети?
5. Классифицируйте известные вам сети по размеру, типу взаимодействия, топологии.
6. Опишите характеристики известной вам сети (например, факультетскую или домашнюю): производительность, надежность, безопасность.
7. * Вам предложили создать локальную сеть факультета. Составьте проект такой сети, включающий обоснование выбранных топологии, типа взаимодействия компьютеров, среды передачи данных.
8. Перечислите уровни модели OSI и кратко опишите функции каждого уровня.
9. Какая задача сложнее: обнаружение ошибок или коррекция ошибок?
10. В пособии рассмотрены два вида сетевых адресов – MAC-адреса и IP-адреса. Существует ещё третий вид, который называется символьные доменные имена. За преобразование IP-адреса в доменное имя и обратно отвечает служба DNS (Domain Name System). Самостоятельно найдите информацию о доменных именах – из каких частей они состоят, приведите примеры.
11. Сколько всего существует MAC-адресов? IP-адресов?
12. Как Вы думаете, почему поле DA (Destination Address) в кадре Ethernet идет раньше поля SA (Source Address)?
13. Какая технология Ethernet (10 Мбит/с, Fast Ethernet или Gigabit Ethernet) используется в известных вам локальных сетях?

14. Перечислите сетевые устройства, которые присутствуют в известных вам сетях.

15. Дан IP-адрес 172.16.235.57, маска подсети 255.255.0.0. Укажите адрес подсети и адрес узла.

16. Зайдите на сайт <http://www.rfc-editor.org> и посмотрите там несколько документов RFC, например:

- самый первый RFC – RFC1,
- описание протокола IP – RFC791,
- описание доменных имен – RFC1034 и RFC1035,
- почтовый протокол POP3 – RFC1939,
- список стандартов – RFC5000.

17. При помощи утилиты *hostname* узнайте имя своего компьютера.

18. Определите свой IP-адрес, маску подсети и MAC-адрес с помощью утилиты *IPconfig*.

19. Узнайте, достижимы ли с Вашего компьютера узлы www.google.ru, www.yandex.ru, а также узлы локальной сети (если она есть), используя утилиту *ping*.

20. Определите маршрут до узлов www.google.ru, www.yandex.ru при помощи утилиты *tracert*.

21. Выведите на дисплей таблицу соответствия IP адресов и MAC адресов (ARP кэш) при помощи утилиты *arp*.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Таненбаум Э. Архитектура компьютера. – 5-е изд. – СПб.: Питер, 2012.
2. Орлов С. А., Цилькер Б. Я. Организация ЭВМ и систем: учебник для вузов. – 2-е изд. – СПб.: Питер, 2011.
3. Хамахер К., Вранешич З., Заки С. Организация ЭВМ. – 5-е изд. – СПб.: Питер; Киев: Издательская группа BHV, 2003.
4. Таненбаум Э. Компьютерные сети. – 5-е изд. – СПб.: Питер, 2012.
5. Олифер В. Г., Олифер Н. А. Компьютерные сети. Принципы, технологии, протоколы: учебник для вузов. – 4-е изд. – СПб.: Питер, 2012.
6. Жмакин А. П. Архитектура ЭВМ. – 2-е изд. – СПб.: БХВ-Петербург, 2010.
7. Пятибратов А. П., Гудыно Л. П., Кириченко А. А. Вычислительные системы, сети и телекоммуникации. – 4-е изд. – М.: Финансы и статистика, 2008.
8. Бройдо В. Л. Вычислительные системы, сети и телекоммуникации: учебник для вузов. – 4-е изд. СПб.: Питер, 2011.
9. Бройдо В. Л. Архитектура ЭВМ и систем: учебник для вузов. – 2-е изд. – СПб.: Питер, 2009.
10. Интернет-университет информационных технологий (<http://www.intuit.ru>).
11. Голубь Н. Г. Искусство программирования на Ассемблере. – 3-е изд. – СПб.: Питер, 2006.
12. Гук М. Ю. Аппаратные средства IBM PC: энциклопедия. – 3-е изд. – СПб.: Питер, 2008.
13. Страбыкин Д. А. Организация ЭВМ: Лабораторный практикум на компьютерах: учебное пособие. – 2-е изд. – Киров: Изд-во ВятГУ, 2005.
14. Андреева Е. В., Босова Л. Л., Фалина И. Н. Математические основы информатики. Элективный курс: учебное пособие. – М.: БИНОМ. Лаборатория знаний, 2005.
15. Петцольд Ч. Код. – М.: Издательско-торговый дом «Русская Редакция», 2004.
16. Еремин Е. А. Популярные лекции об устройстве компьютера. – СПб.: БХВ-Петербург, 2003.
17. Каган Б. М. Электронные вычислительные машины и системы: Учебное пособие для вузов. – 3-е изд. – М.: Энергоатомиздат, 1991.
18. Столлинкс У. Структурная организация и архитектура компьютерных систем. – М.: Вильямс, 2002.
19. Полунов Ю. Л. От абака до компьютера: судьбы людей и машин: в 2 т. М.: Издательско-торговый дом «Русская Редакция», 2004.
20. Бабищ Н. П., Жуков И. А. Основы цифровой схемотехники. – М.: Додэка XXI; МК-Пресс, 2007.
21. Угрюмов Е. П. Цифровая схемотехника. – СПб.: БХВ-Петербург, 2007.

Комментарии к библиографическому списку

[1] – Эндрю Таненбаум, известный голландский профессор, является автором многих учебников по вычислительным машинам и системам, компьютерным сетям, операционным системам. Книга характерна доступным изложением материала, изобилием примеров.

[2] – учебник Сергея Александровича Орлова и Бориса Яковлевича Цилькера можно рекомендовать как фундаментальное пособие по принципам и тонкостям архитектур современных ВМ и ВС. Материал рассчитан на глубокое и вдумчивое чтение и насыщен важными техническими подробностями.

[3] – учебник канадских авторов более сложен в чтении, чем книга Э. Таненбаума, однако некоторые разделы изложены подробнее, другие рассмотрены с иных позиций, а третьи отсутствуют в учебнике Таненбаума. Так, например, можно порекомендовать разделы, посвященные арифметике (глава 6), конвейерной обработке команд (глава 8), логическим схемам (приложение А).

[4] – ещё один отличный учебник Эндрю Таненбаума, на этот раз по компьютерным сетям.

[5] – учебник по компьютерным сетям Виктора Григорьевича Олифер и Натальи Алексеевны Олифер является одним из наиболее полных изложений теории и практики компьютерных сетей. Достаточно сказать, что учебник выдержал уже четыре издания и переведен на несколько языков.

[6] – книга Анатолия Петровича Жмакина рассчитана на студентов технических вузов, поэтому её можно рекомендовать для углубленного изучения материала. Подробно представлены арифметические основы ВМ, принципы проектирования операционного и управляющего автоматов. Полезно познакомиться с лабораторным практикумом и с учебной ЭВМ, демонстрирующими принципы работы компьютера.

[7] – учебник Александра Петровича Пятибротова, Льва Петровича Гудыно, Александра Аполлоновича Кириченко предназначен для студентов специальности «Прикладная информатика».

[8], [9] – учебники Владимира Львовича Бройдо предназначены для студентов специальностей «Прикладная информатика» и «Информационные системы».

[10] – это электронный образовательный ресурс, на котором можно пройти обучение многим дисциплинам, связанным с информатикой. Рекомендуется обратить внимание на разделы «Архитектура ЭВМ», «Аппаратное обеспечение», «Сетевые технологии».

[11] – замечательная книга Надежды Григорьевны Голубь, посвященная языку Ассемблера. Один из лучших учебников для введения в архитектуру ВМ и практику программирования на Ассемблере. Изложение материала сопровождается многочисленными примерами на Ассемблере, а также программами на Паскале и Си, взаимодействующими с Ассемблером.

[12] – энциклопедия Михаила Юрьевича Гука очень популярна в среде профессионалов и любителей в области компьютерной техники. Несмотря на то что некоторые сведения устарели, многие приводимые в ней данные сложно где-то ещё найти. В любом случае этот справочник поможет разрешить сложные технические вопросы, связанные с компьютерным «железом».

[13] – учебное пособие профессора Вятского государственного университета Дмитрия Алексеевича Страбыкина содержит ценный теоретический и практический материал по проведению лабораторных работ на программном имитаторе процессора.

[14] – в учебном пособии Елены Владимировны Андреевой, Людмилы Леонидовны Босовой, Ирины Николаевны Фалиной подробно рассматриваются вопросы, связанные с системами счисления, представлением информации в компьютере и алгеброй логики.

[15] – книга Чарльза Петцольда представляет собой, пожалуй, самое доступное и одно из самых интересных введений в принципы организации и функционирования вычислительных машин. Как пишет сам автор, «*Код* – это книга о том, как работают компьютеры».

[16] – ещё одно интересное и доступное введение в предмет. Евгений Александрович Еремин описывает фундаментальные принципы построения ВМ, иллюстрируя их на простых примерах и приводя любопытные эксперименты. Один из разделов посвящен обзору и сравнению учебных моделей ЭВМ.

[17] – по книге Бориса Моисеевича Кагана изучали архитектуру ВМ несколько поколений студентов технических вузов. Современные пособия во многом основаны на этом учебнике. Фундаментальные теоретические положения, приведенные в книге, сохранили актуальность и сегодня.

[18] – книга Уильяма Столлинга насыщена подробностями, которых лишены другие учебники. Можно рекомендовать обращаться к этой книге для разрешения неясных или плохо освещенных в других пособиях тем.

[19] – книга целиком посвящена истории развития вычислительной техники и людям, творившим эту историю.

[20]–[21] – из этих книг можно почерпнуть подробные сведения из области микроэлектроники, разрешить неясные или сложные вопросы по разделу «Цифровой логический уровень».

Учебное издание

Котельников Евгений Вячеславович

Вычислительные машины, системы и сети

Подписано в печать 26.01.2012 г.

Формат 60х84/16.

Бумага офсетная.

Усл. п. л. 13,75.

Тираж 100 экз.

Заказ № 059.

Издательство

Вятского государственного гуманитарного университета,
610002, г. Киров, ул. Красноармейская, 26, т. (8332) 673-674
www.vggu.ru

Отпечатано

в полиграфическом цехе Издательства ВятГГУ,
610002, г. Киров, ул. Ленина, 111, т. (8332) 673-674

